▼ **Paxos and Raft**

- 1 ✦ Paxos and Raft
- 2 ✦ The topics of today's lecture
- ▼ Single-decree Paxos
  - 3 ✦ The consensus problem solved by Paxos is very important
  - 4 ✦ Motivating case: A parliament must agree on one decree
  - 5 ✦ Why achieving consensus is challenging
  - 6 ✦ A computer system is similar to the Paxos Parliament
  - 7 ✦ What are Paxos's design goals and assumptions?
  - 8 ✦ Observation: The problem is trivial with a single writer
  - 9 ✦ Paxos' idea: grant the write to the largest number
  - 10 ✦ The three roles in Paxos
  - 11 ✦ Round one: the proposer acquires the right to write
  - 12 ✦ Round two: replicate the value to a majority
  - 13 ✦ What if there are concurrent writers?
  - 14 ✦ Question: what if a value has been written?
  - 15 ✦ Solution: old value must be carried back
  - 16 ✦ The complete picture: can S3 write "bar" here?
  - 17 ✦ Answer: S2's promise carries foo
  - 18 ✦ Put it all together: Roles and state on each node
  - 19 ✦ Paxos pseudo-code: prepare
  - 20 ✦ Paxos pseudo-code: accept and decide
  - 21 ✦ Question: what if two proposers pick the same number?
  - 22 ✦ Some quiz questions about Paxos
  - 23 ✦ Question: what about liveness?
- ▼ Multi-Paxos
  - 24 ✦ Case study: Using Paxos to implement a view server (VS)
  - 25 ✦ Single-decree Paxos vs. Multi-Paxos
  - 26 ✦ Basic Multi-Paxos: Suppose we want to append a new log entry
  - 27 ✦ Basic Multi-Paxos: What if the instance has been taken?
  - 28 ✦ Basic Multi-Paxos: retry with a larger instance
  - 29 ✦ The efficiency problem of Multi-Paxos
  - 30 ✦ Benefits of a leader: first, without batching
  - 31 ✦ Benefits of a leader: Prepare message batching
  - 32 ✦ Summary: Why Multi-Paxos uses a distinguished proposer
  - 33 ✦ Paxos is challenging to implement
  - 34 ✦ Two case studies of why implementing Paxos is challenging
- ▼ Raft: decompose the problem, elect a leader first
  - 35 ✦ Raft: a different approach for consensus
  - 36 ✦ Why learn Raft? (Instead of Paxos)
  - 37 ✦ Raft's abstraction: a replicated log
  - 38 ✦ Raft's high-level approach: problem decomposition
  - 39 ✦ The corresponding Raft server states
  - 40 ✦ Simplification (and decomposition): followers are passive
  - 41 ✦ Questions: how to determine the one and only leader? Terms
  - 42 ✦ Heartbeats & timeouts: Detect leader failures for starting a new term
  - 43 ✦ Election basics: For at most one leader per term
  - 44 ✦ Election requirement #1: safety
  - 45 ✦ Election requirement #2: liveness
  - 46 ✦ Put it all together: the Raft election process
  - 47 ✦ The code: RequestVote RPC
- ▼ Log replication in Raft
  - 48 ✦ Part two of the decomposition: log replication
  - 49 ✦ Normal operation: the path of one log entry replication
  - 50 ✦ Normal operation: crashed or slow followers
  - 51 ✦ The problem: crashes and partitions make logs diverge
  - 52 ✦ The coherency Raft maintains
  - 53 ✦ Realizing coherence: AppendEntries consistency check
  - 54 ✦ Handling rejections
  - 55 ✦ nextIndex: step back one entry per rejection
  - 56 ✦ AppendEntries RPC
- ▼ When can we commit in Raft
  - 57 ✦ Overwriting repairs the log — but when can we commit?
  - 58 ✦ Case study: an entry on a majority can be overwritten
  - 59 ✦ Safety requirement for commit
  - 60 ✦ Case #1/2: committing an entry from the current term
  - 61 ✦ Picking the best leader
  - 62 ✦ Case #1/2: current-term entries are safe
  - 63 ✦ The commit rule so far
  - 64 ✦ Case study: an old entry on a majority still gets overwritten
  - 65 ✦ Raft's commit rules
- 66 ✦ Paxos and Raft
- 67 ✦ 参考文献

IPADS
INSTITUTE OF PARALLEL AND DISTRIBUTED SYSTEMS

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Paxos and Raft

IPADS, Shanghai Jiao Tong University

SE3331-1 (2025 Fall)

The topics of today's lecture

How to solve the consensus problem with Paxos and Raft

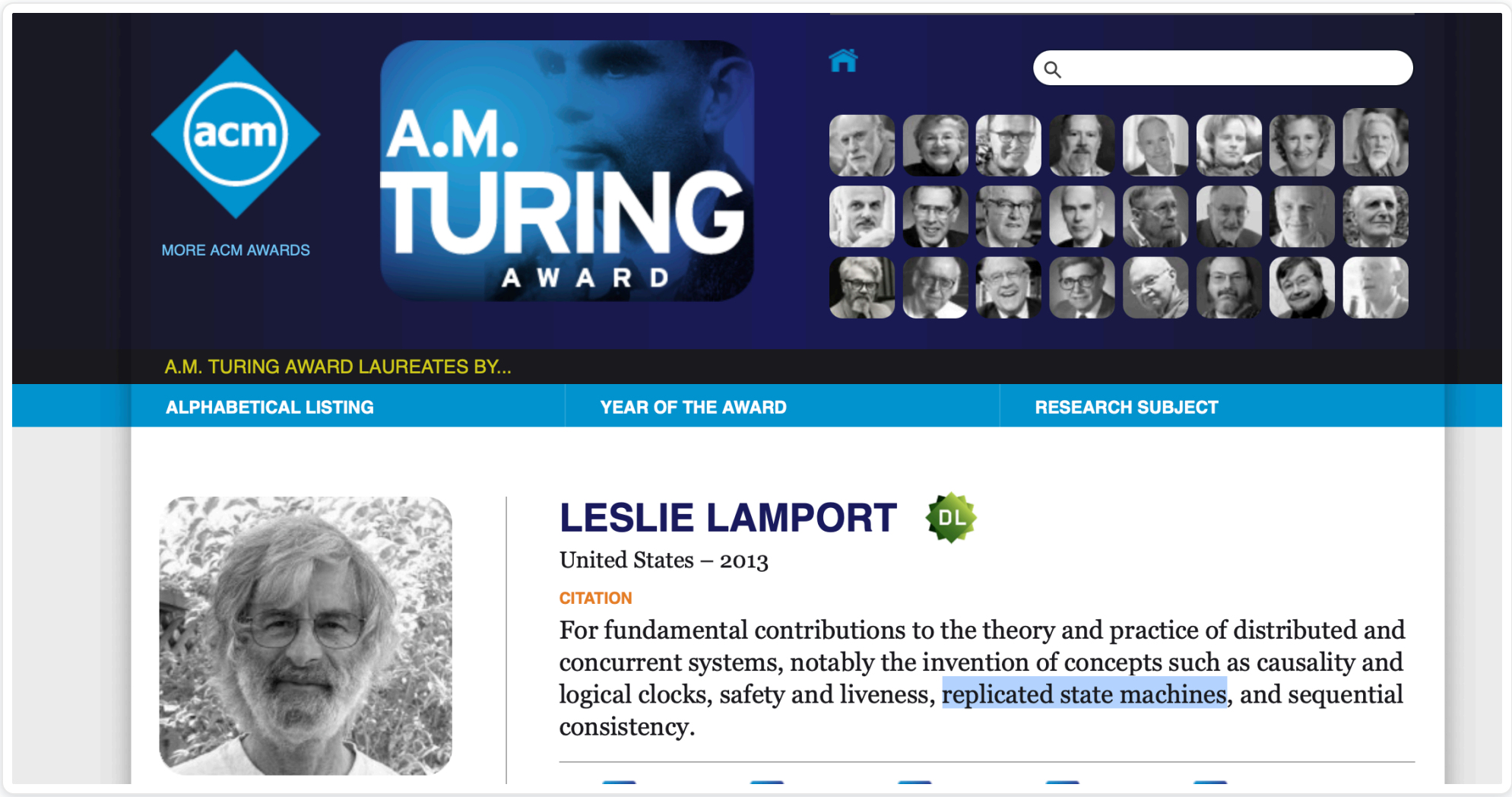
Paxos: a bottom-up approach

- Single-decree Paxos: first solve the problem of consensus on a single value
- Multi-Paxos: replicate a **sequence** of values based on Single-decree Paxos

Raft: the other way around — solve the problem directly (consensus on the log of a replicated state machine)

- leader election
- log replication
- when consensus has been reached on a log entry

The consensus problem solved by Paxos is very important



[1]

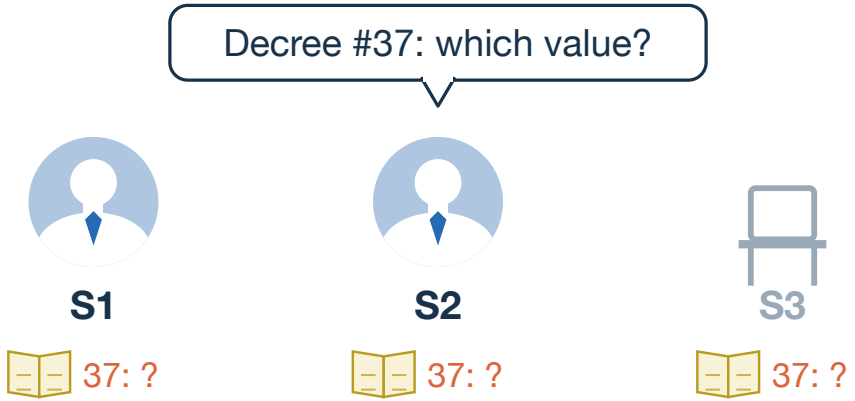
[1] Source: ACM A.M. Turing Award

Motivating case: A parliament must agree on one decree

The parliament passes decree #37: every ledger must end up saying the same thing

- Legislators are part-time and walk out mid-session — like servers can go up and down

> 📖 Lamport's story： *Recent archaeological discoveries on the island of Paxos reveal that the parliament functioned despite the peripatetic propensity of its part-time legislators. The legislators maintained consistent copies of the parliamentary record, despite their frequent forays from the chamber and the forgetfulness of their messengers. The Paxon parliament's protocol provides a new way of implementing the state-machine approach to the design of distributed systems.* (abstract)



Why achieving consensus is challenging

Try the naive rule first: each legislator records the first decree they hear on a ledger

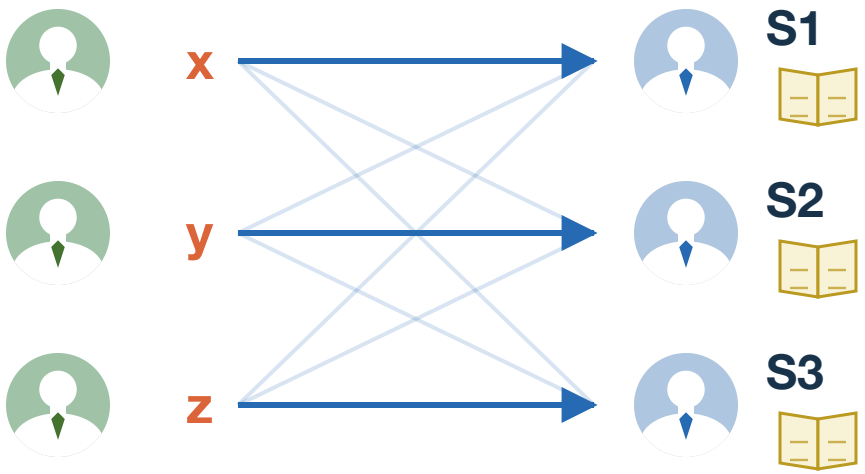
- What would happen if each legislator passes a different decree without knowing the others?



Why achieving consensus is challenging

Try the naive rule first: each legislator records the first decree they hear on a ledger

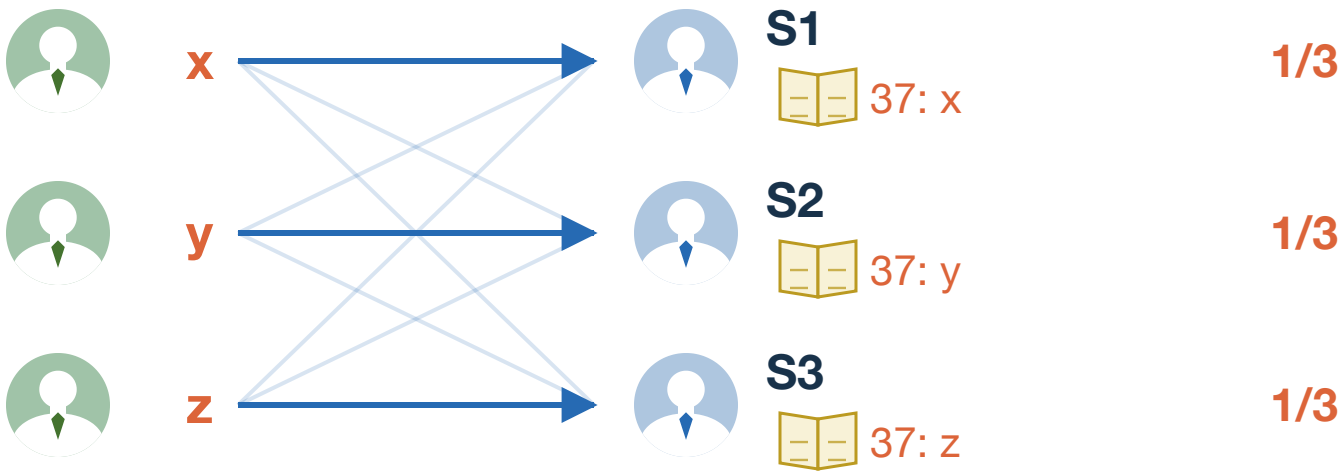
- What would happen if each legislator passes a different decree without knowing the others?



Why achieving consensus is challenging

Try the naive rule first: each legislator records the first decree they hear on a ledger

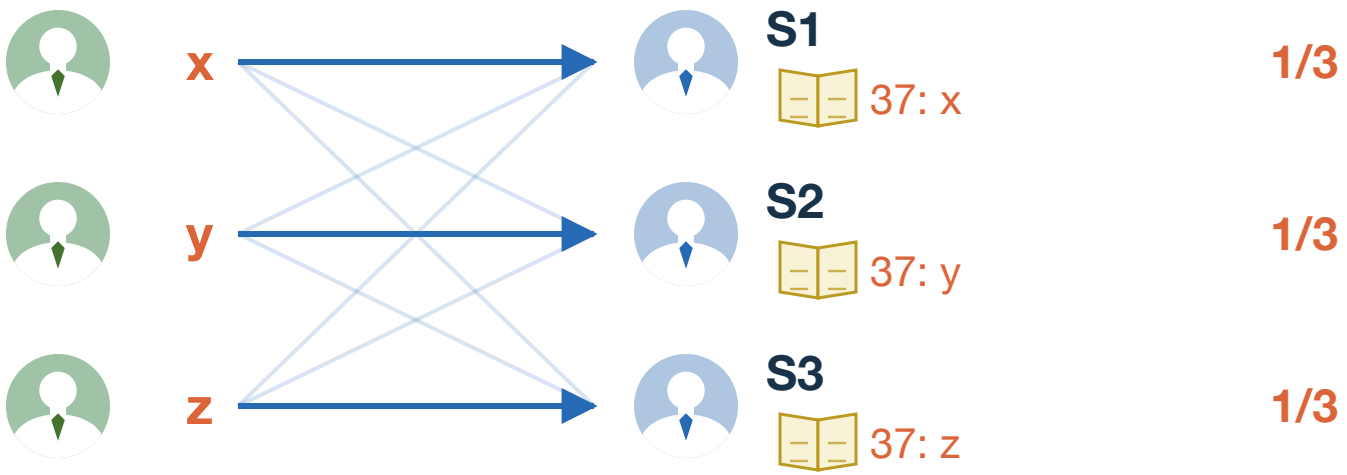
- What would happen if each legislator passes a different decree without knowing the others?



📖 From Lamport's story： *Consistency would be lost if one group of legislators passed the decree 37: Painting on temple walls is forbidden and then left for a banquet, whereupon a different group of legislators entered the Chamber and, knowing nothing about what had just happened, passed the conflicting decree 37: Freedom of artistic expression is guaranteed.* (§1.2)

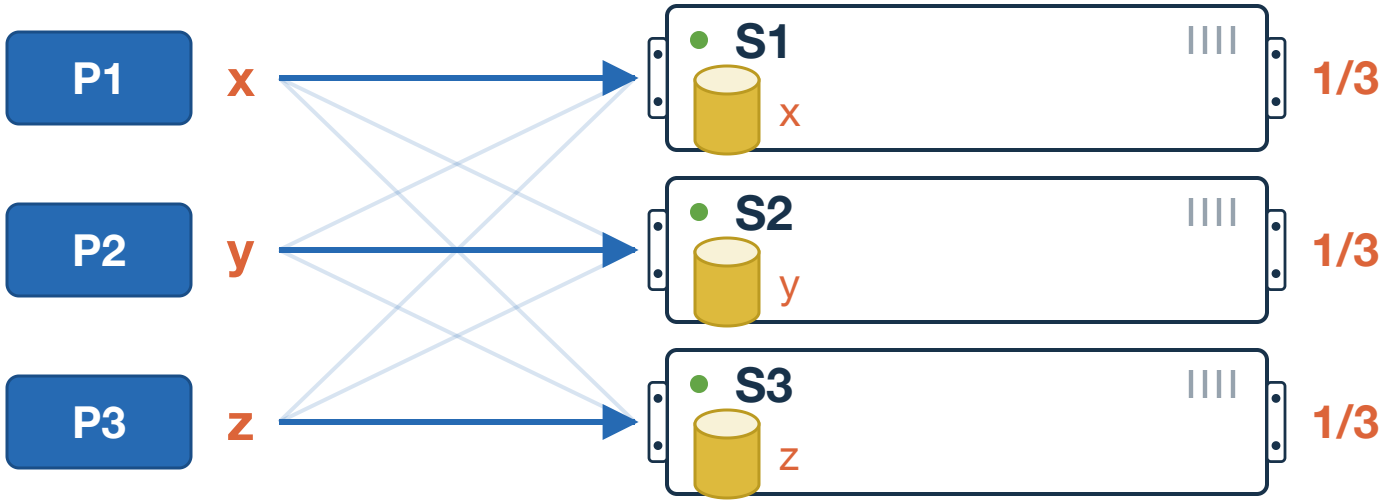
A computer system is similar to the Paxos Parliament

Parliament	Machine
legislator	acceptor
decree	the value to agree on
ledger	disk
majority present	quorum



A computer system is similar to the Paxos Parliament

Parliament	Machine
legislator	acceptor
decree	the value to agree on
ledger	disk
majority present	quorum



What are Paxos's design goals and assumptions?

Assumption:

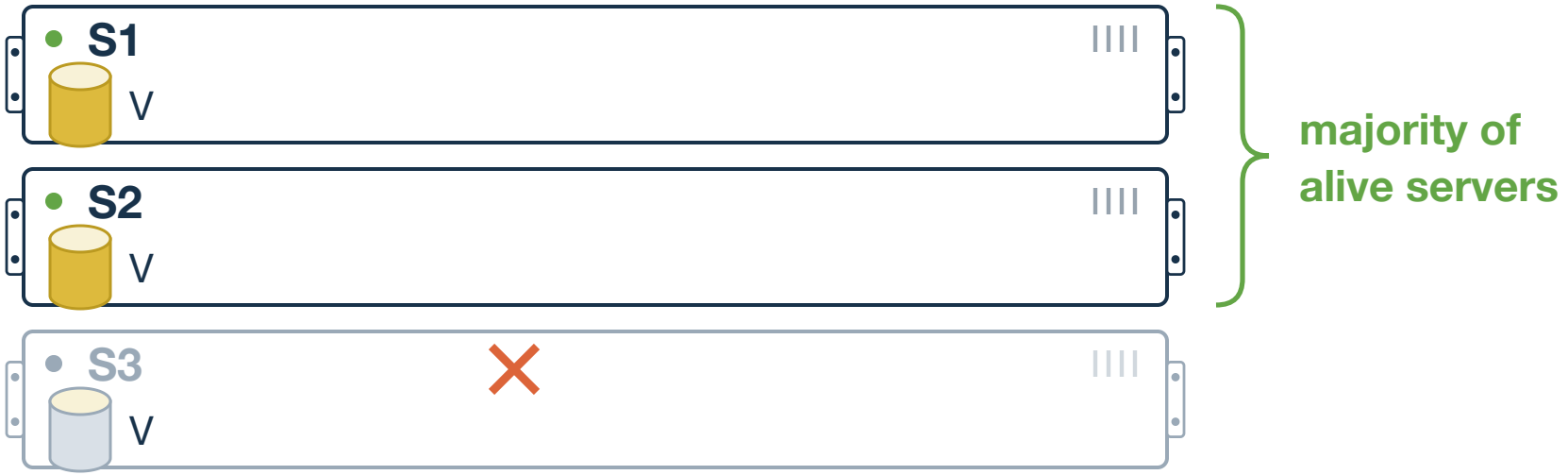
- Servers can go down or come back up, and the network can become partitioned

Goal (Consistency)

- All acceptors agree on one value V sent by arbitrary clients
- Once a value is agreed on, it never changes, and it is persisted

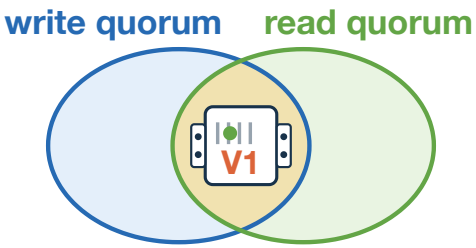
Goal (Availability)

- As long as a majority of servers are alive, the sent value can be accepted within some time

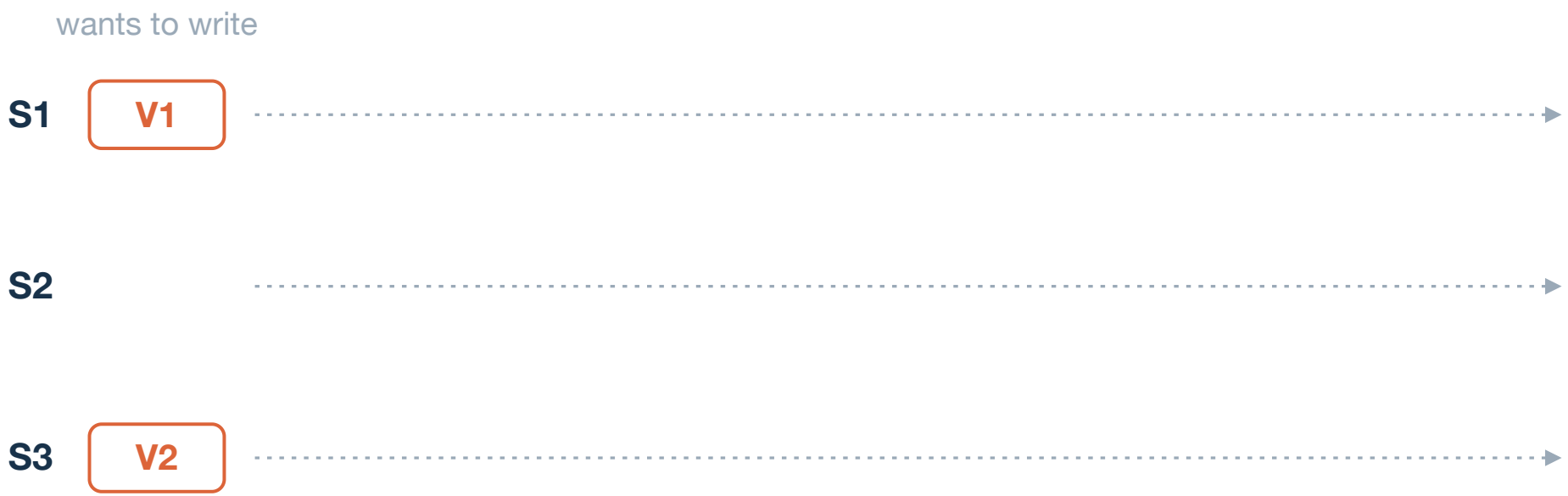


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

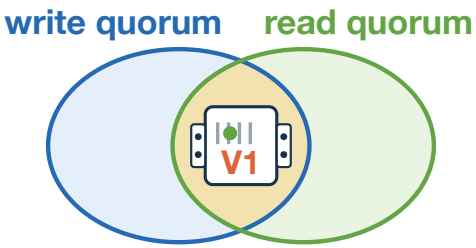


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

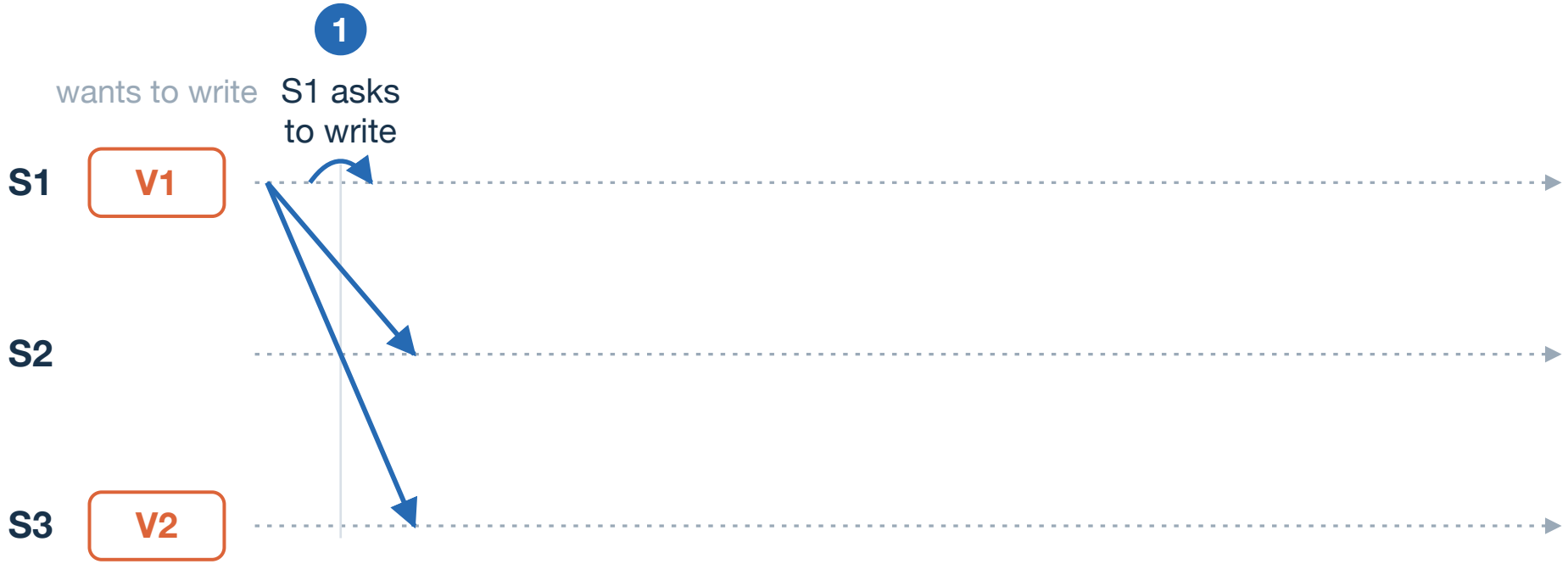


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

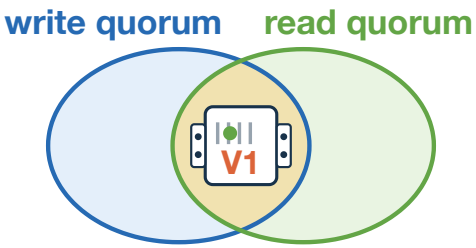


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

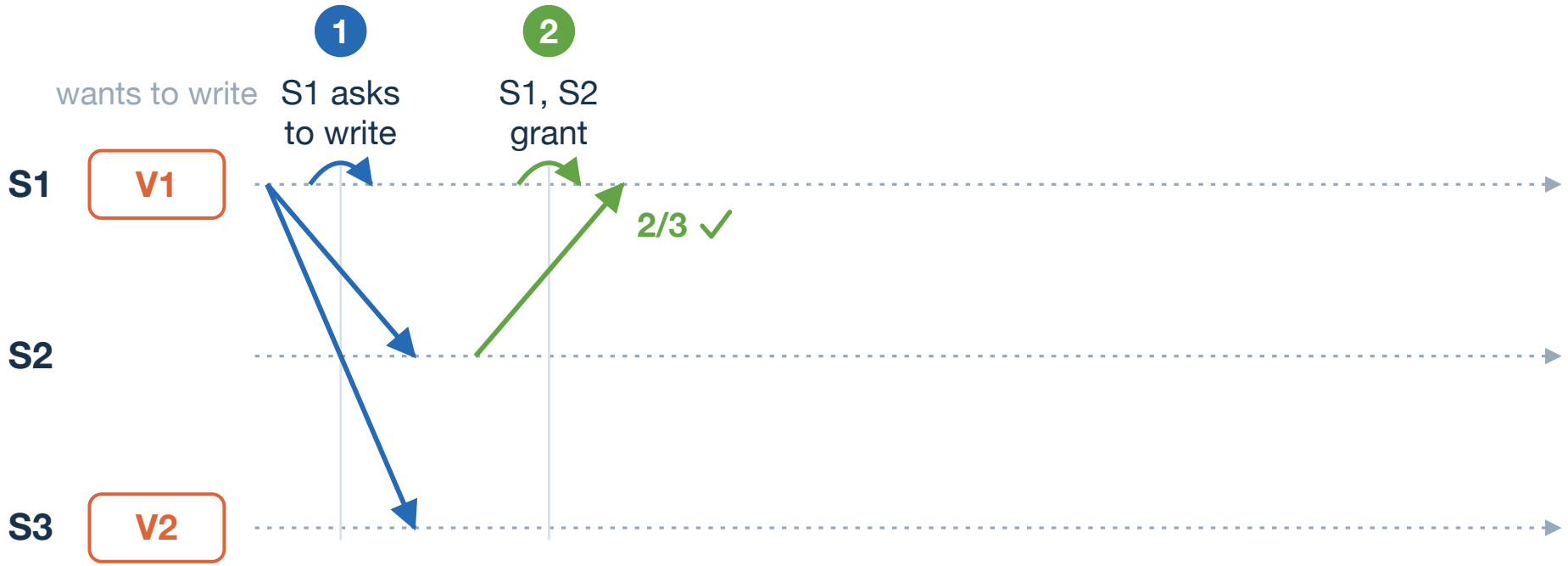


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

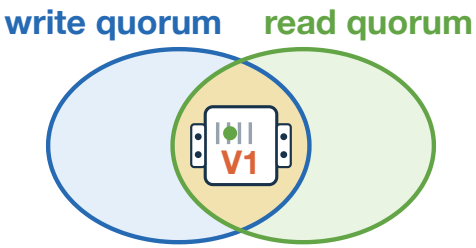


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

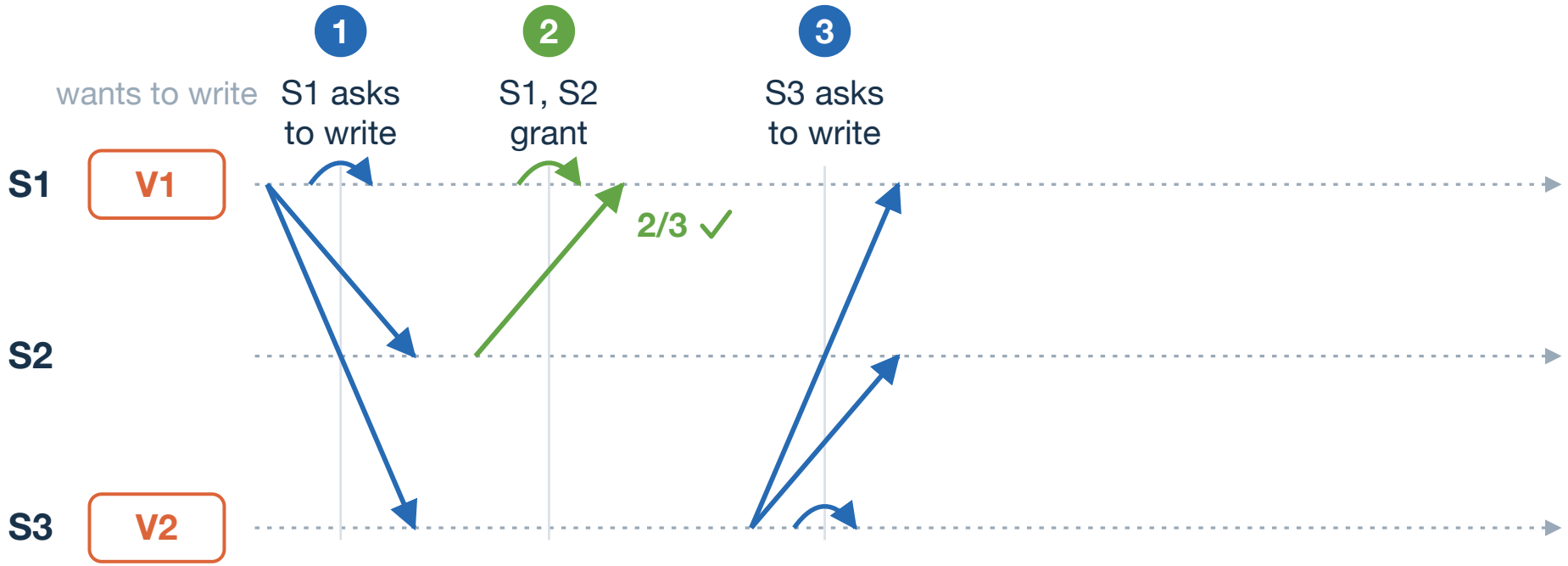


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

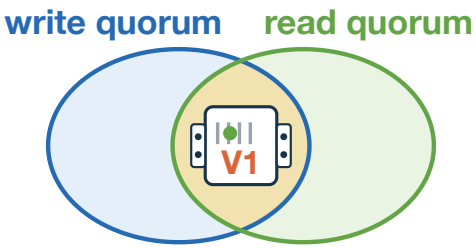


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

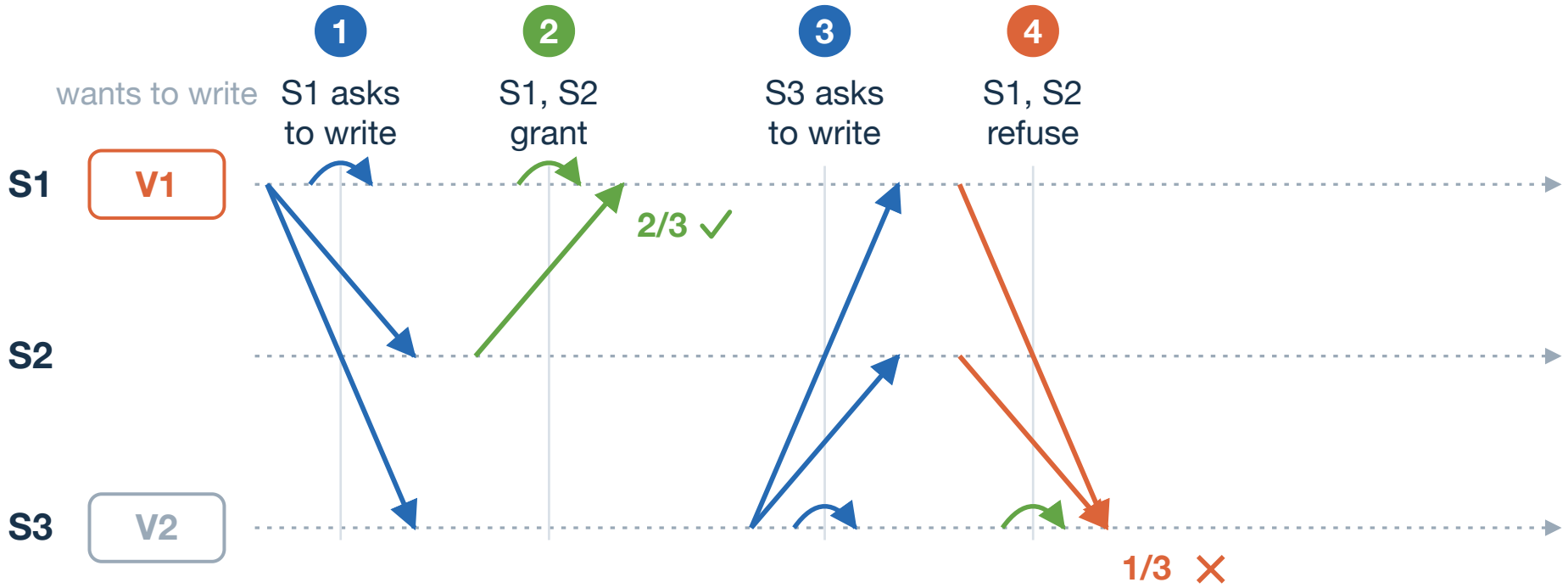


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

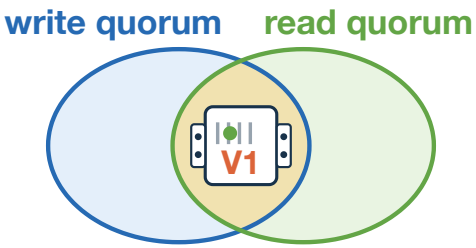


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

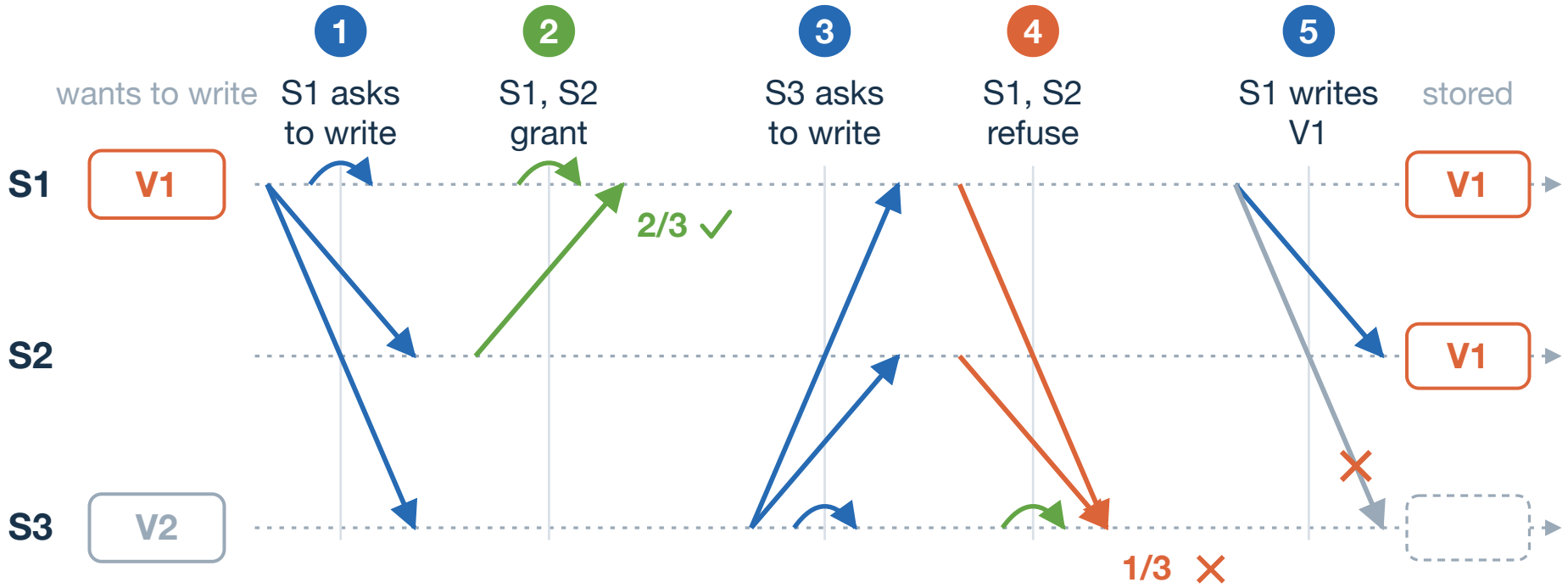


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

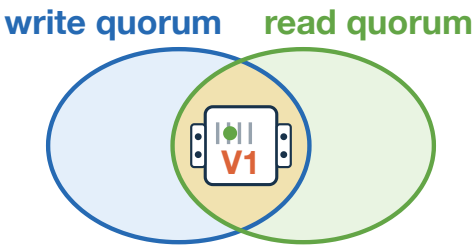


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

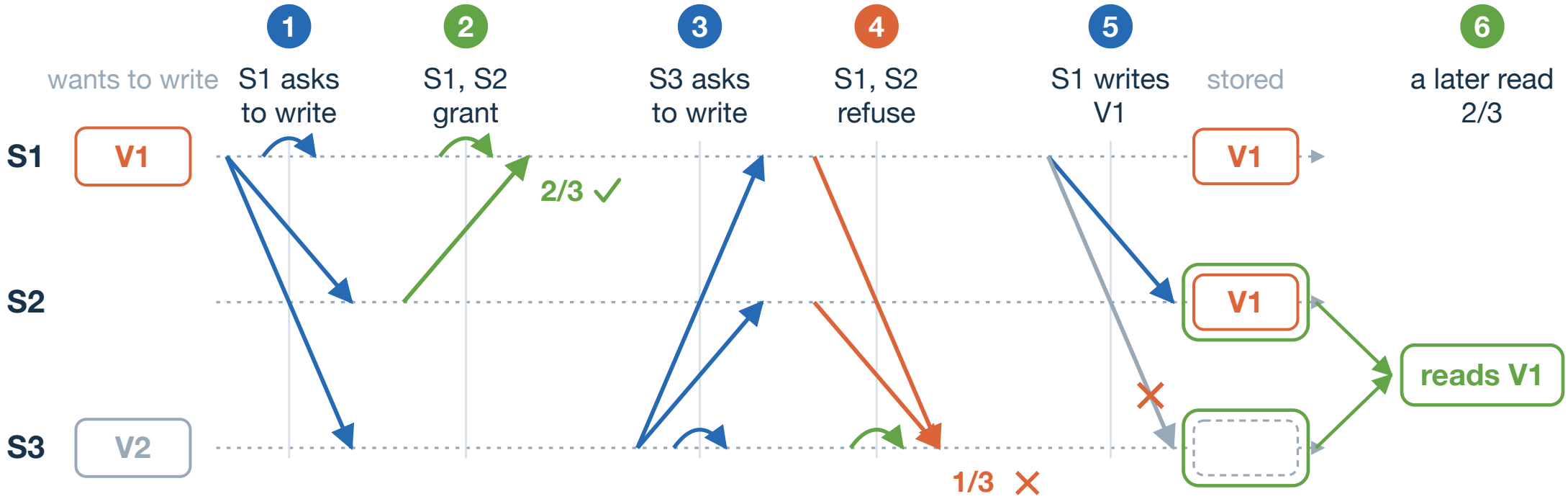


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say

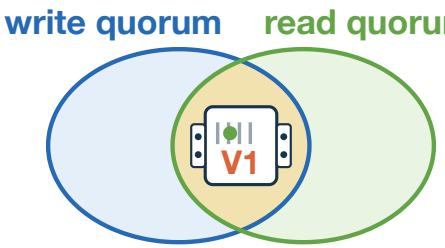


- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

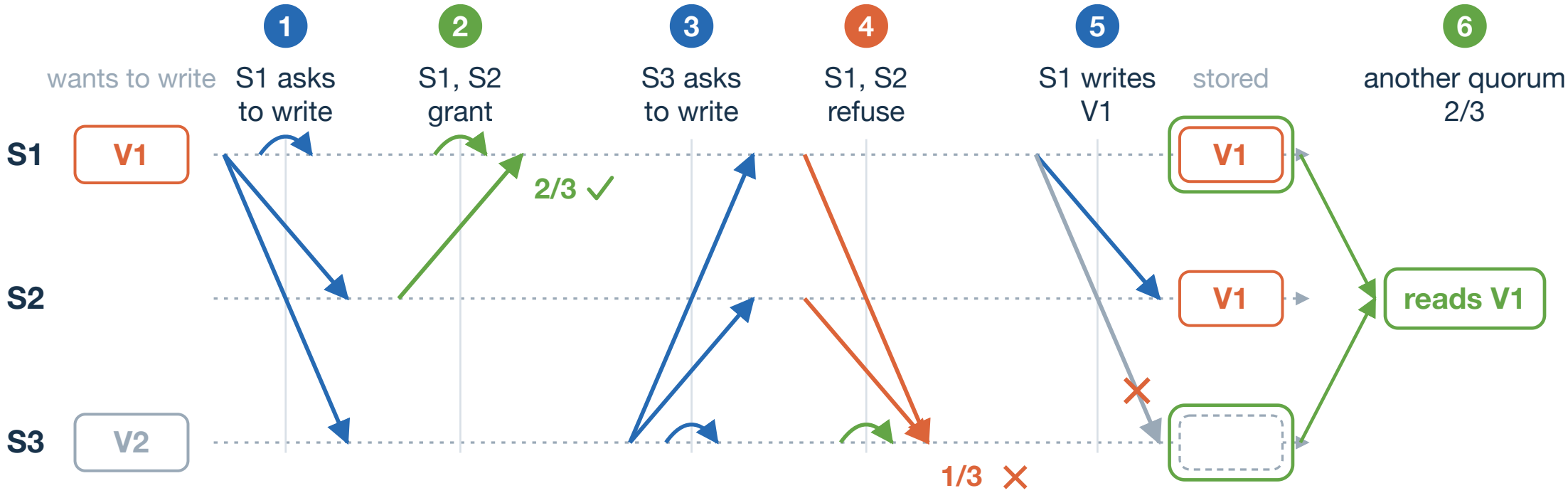


Observation: The problem is trivial with a single writer

One writer (e.g., the primary) has the final say



- The challenge is how to establish a single writer in a distributed system. That is, who gets to be the single writer?
- Insight: a grant approved by a majority is settled (quorum), because no others can get the same grant

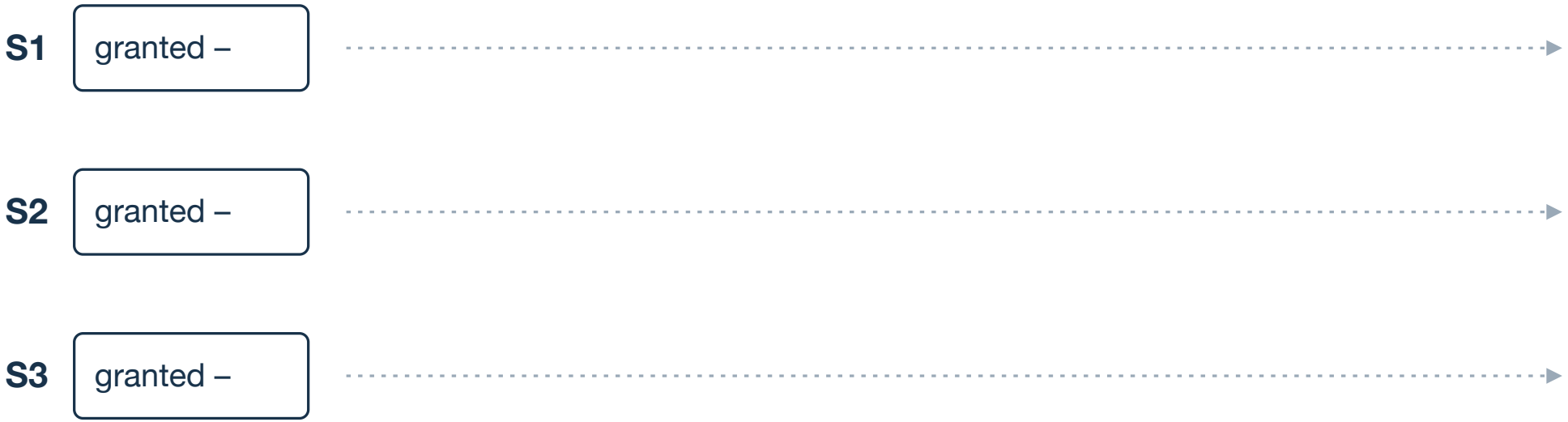


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on

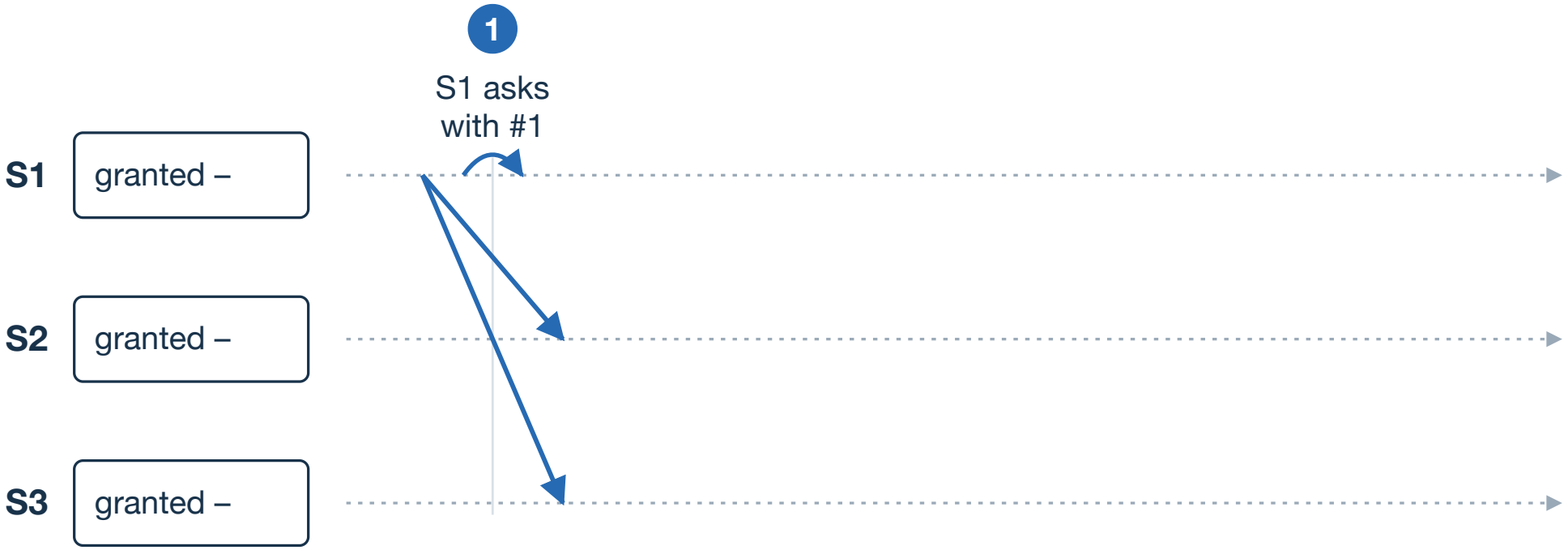


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on

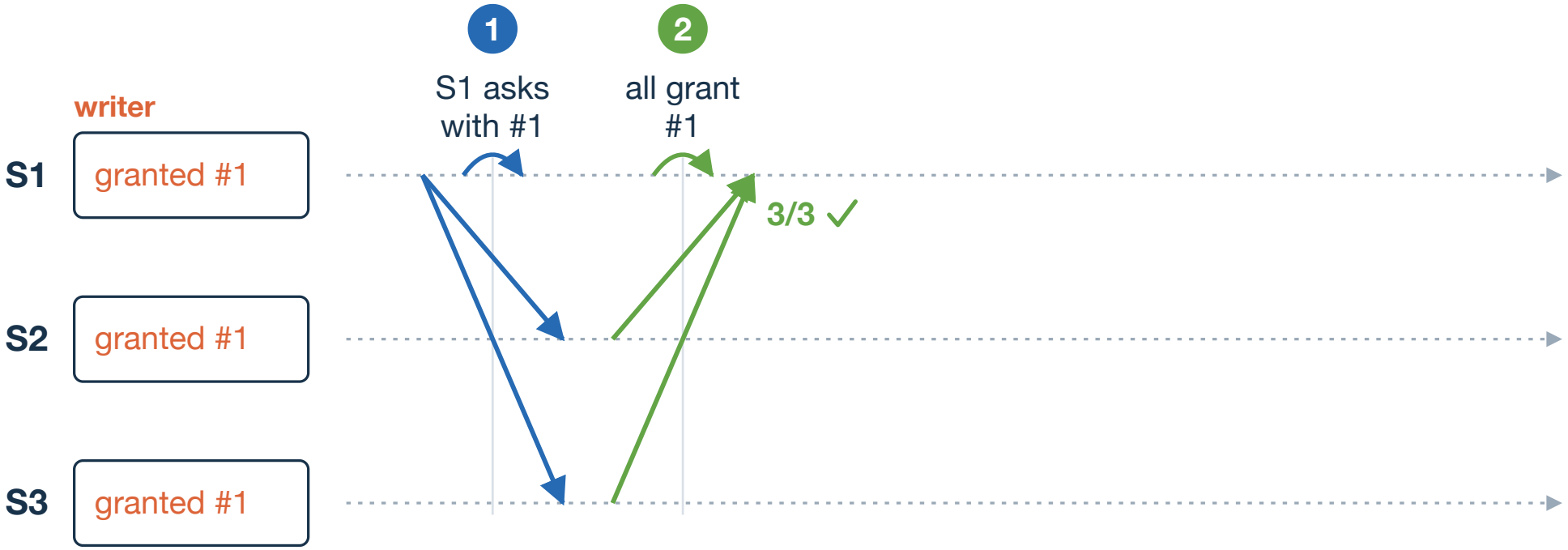


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on

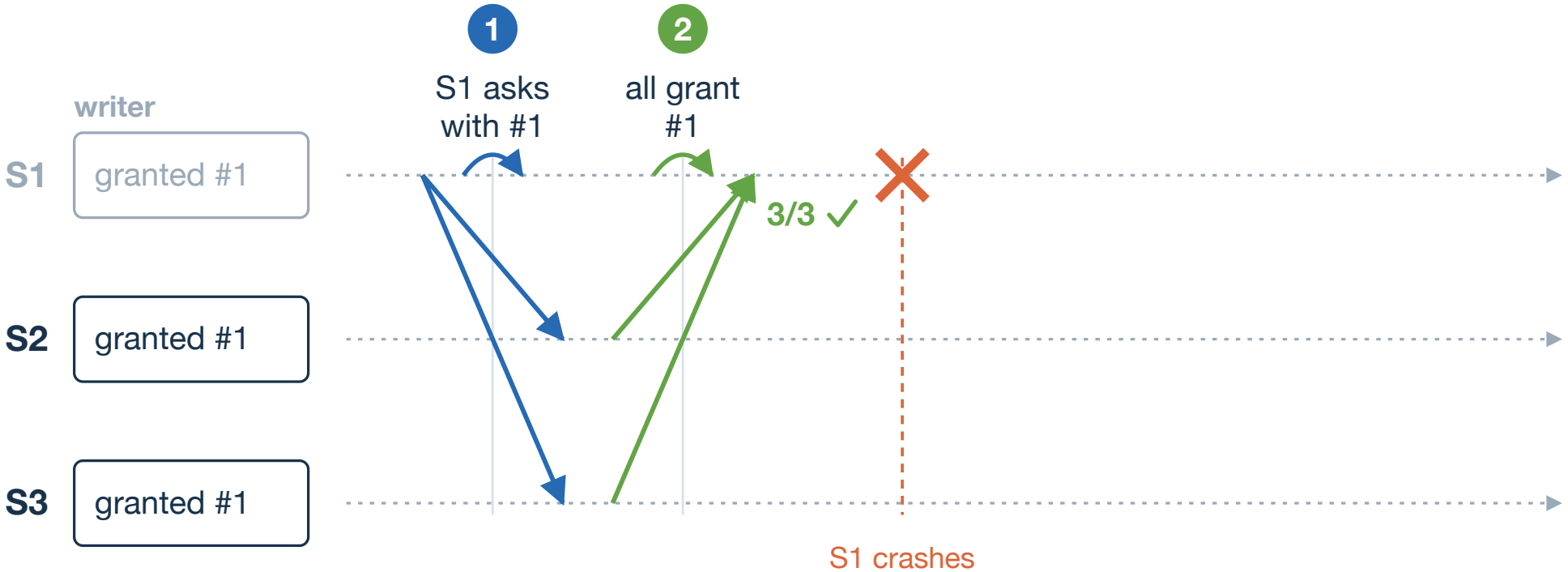


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on

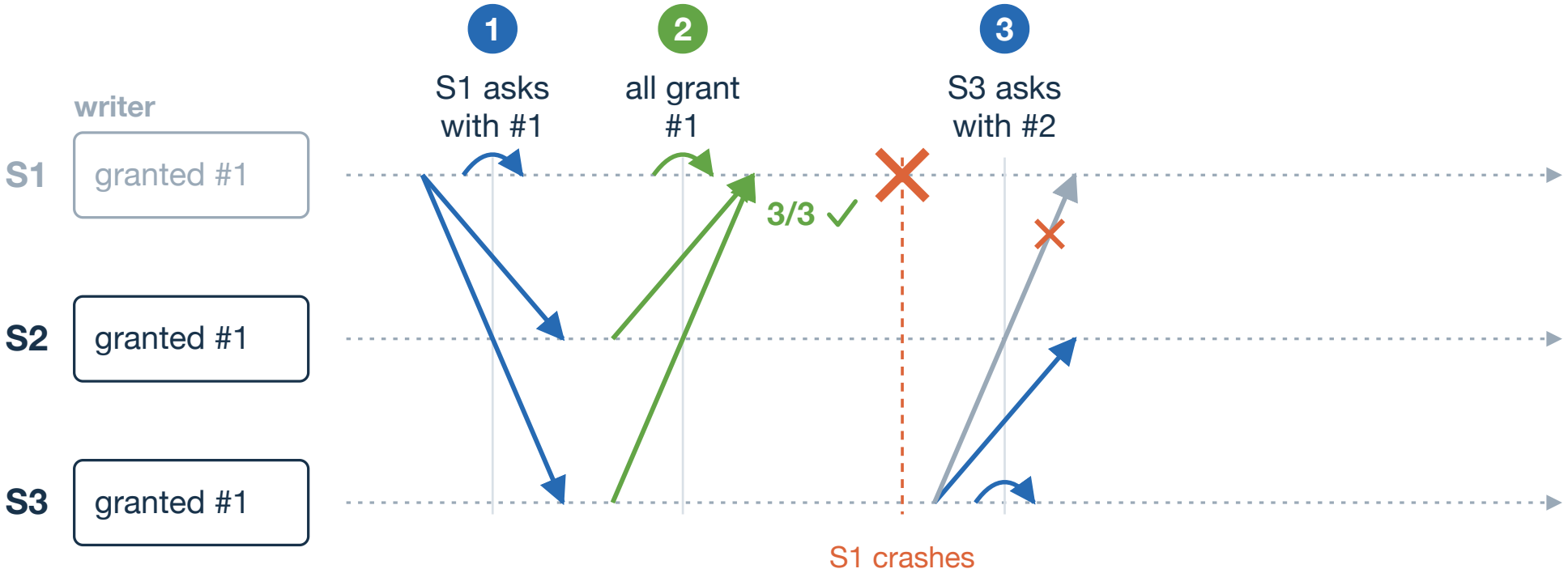


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on

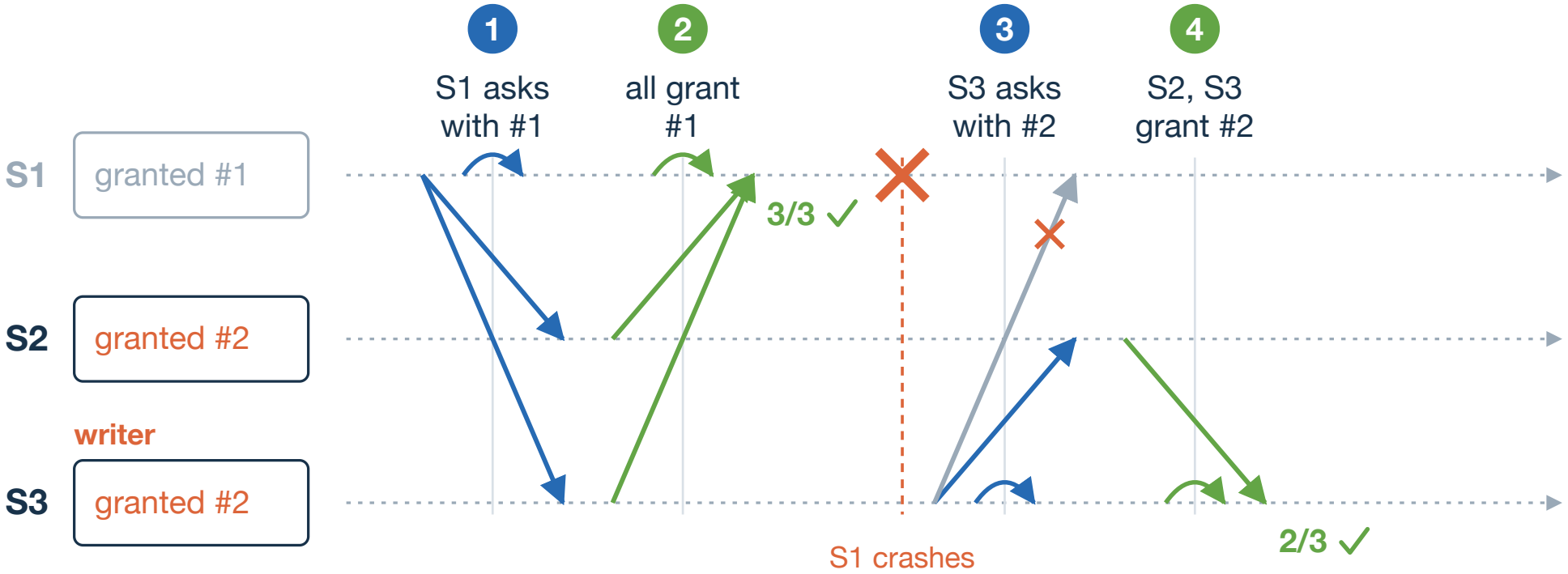


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on

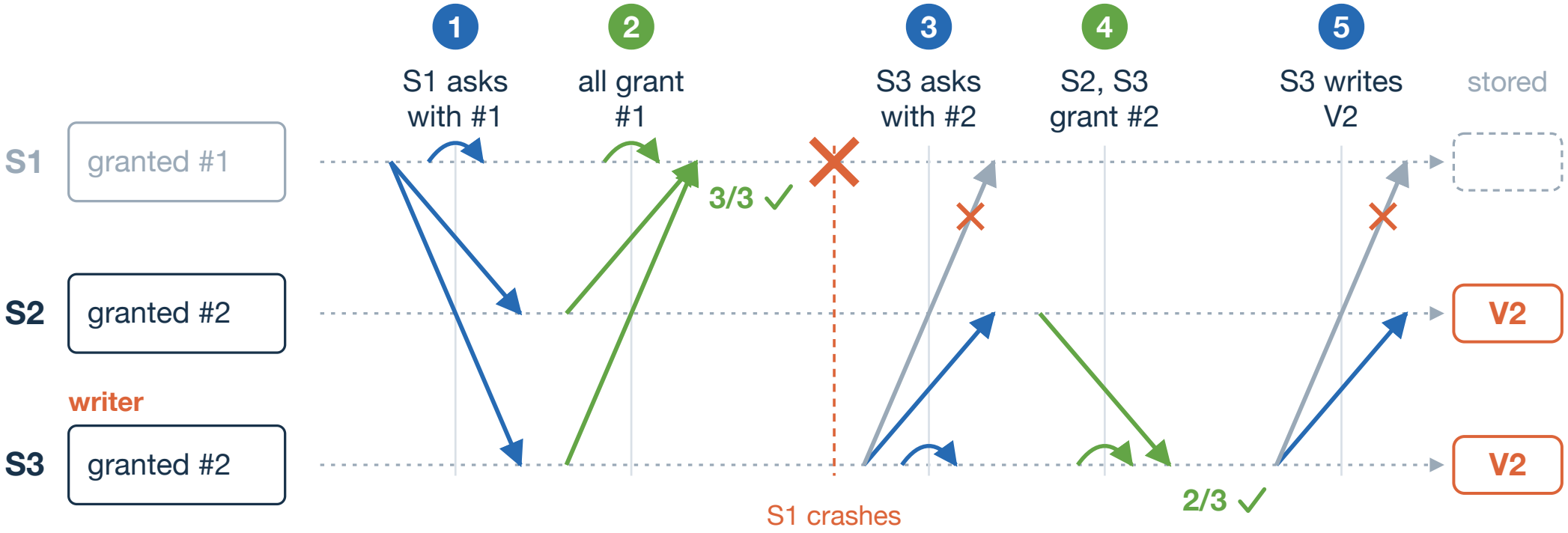


Paxos' idea: grant the write to the largest number

Problem: what if the writer holding the grant crashes? No one else can ever write (no availability)

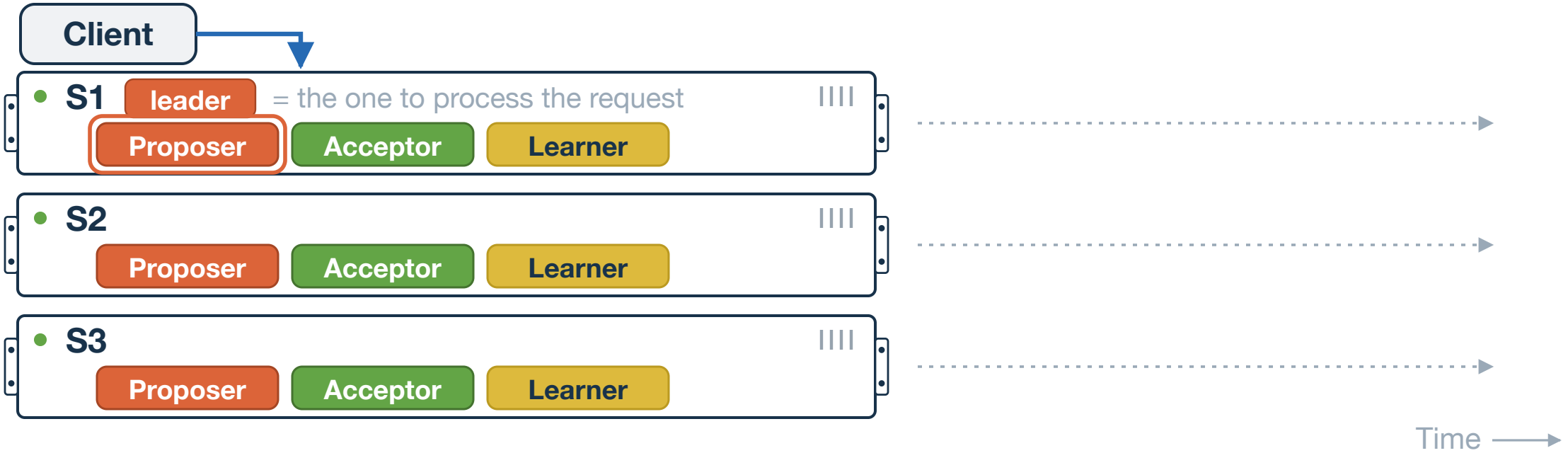
Paxos: an ask carries a number; an acceptor grants the largest number it has seen

- The writer is the one whose number is the largest granted by a majority
- A crashed writer blocks nothing: ask again with a larger number, and the write moves on



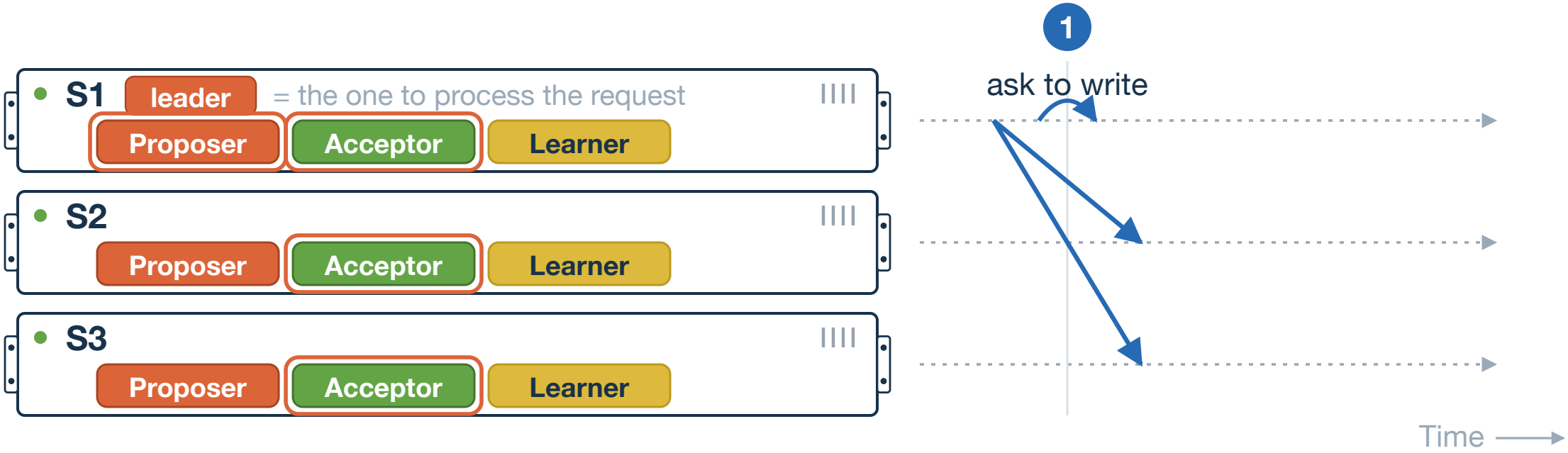
The three roles in Paxos

- **proposer**: receives a request, runs the protocol to become the writer, and writes the requested value
- **acceptor**: remembers the state of the protocol
- **learner**: once the value is chosen, executes the request and replies to the client



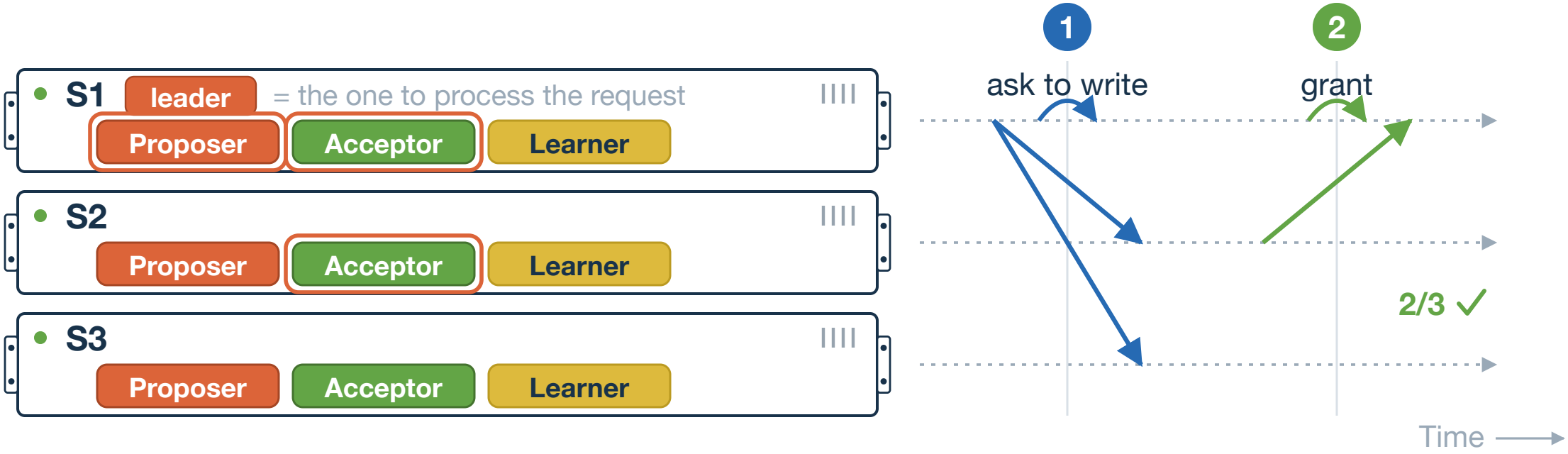
The three roles in Paxos

- **proposer**: receives a request, runs the protocol to become the writer, and writes the requested value
- **acceptor**: remembers the state of the protocol
- **learner**: once the value is chosen, executes the request and replies to the client



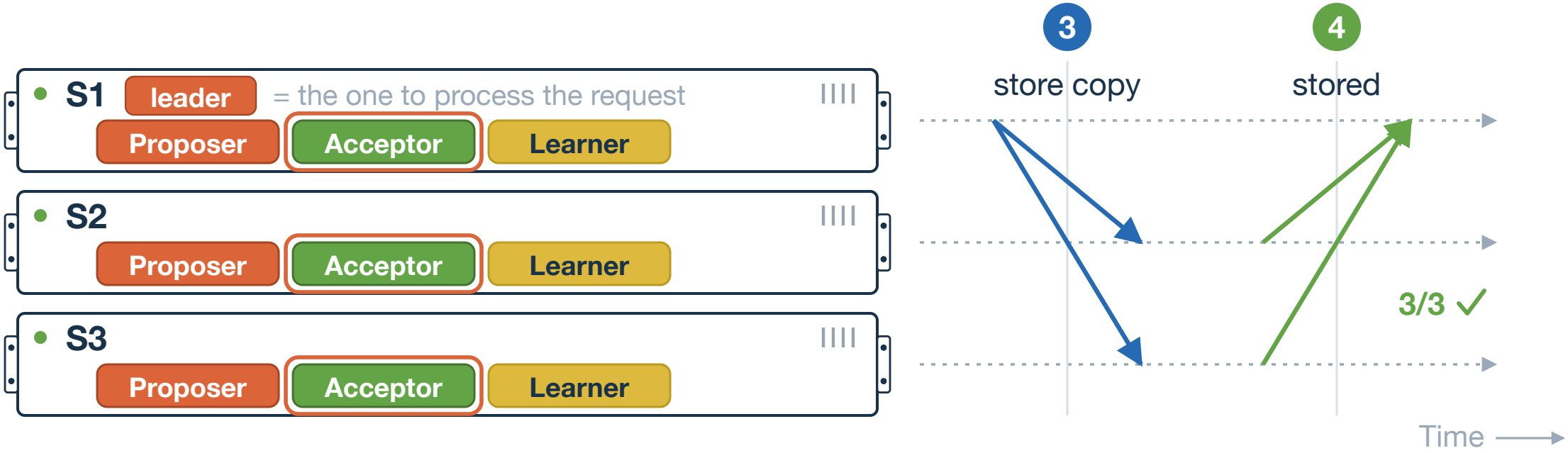
The three roles in Paxos

- **proposer**: receives a request, runs the protocol to become the writer, and writes the requested value
- **acceptor**: remembers the state of the protocol
- **learner**: once the value is chosen, executes the request and replies to the client



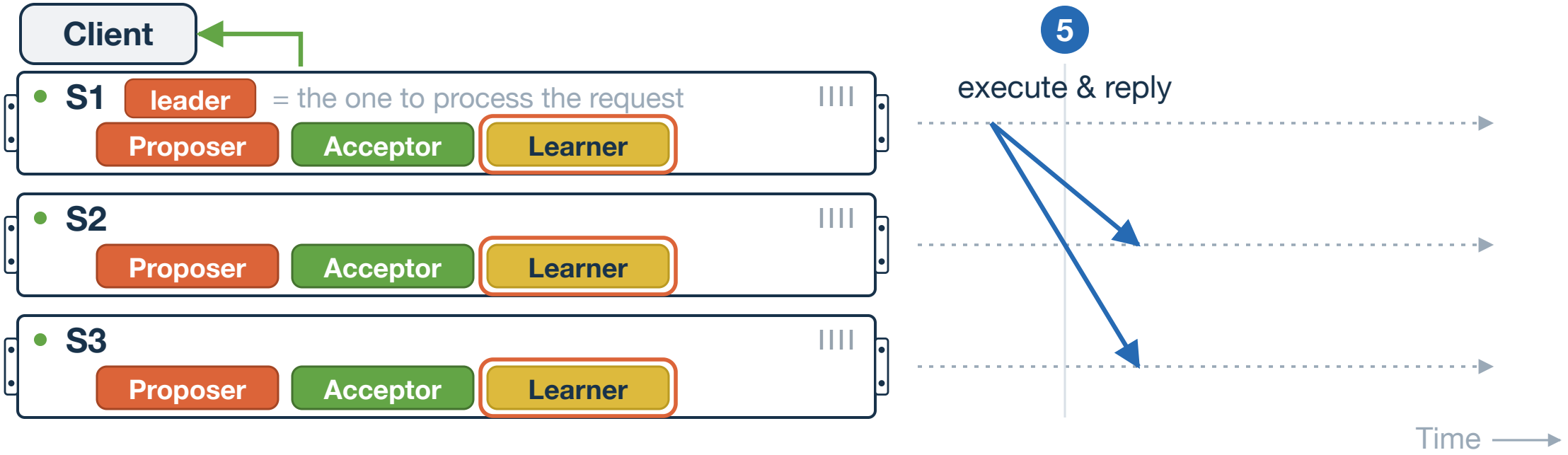
The three roles in Paxos

- **proposer**: receives a request, runs the protocol to become the writer, and writes the requested value
- **acceptor**: remembers the state of the protocol
- **learner**: once the value is chosen, executes the request and replies to the client



The three roles in Paxos

- **proposer**: receives a request, runs the protocol to become the writer, and writes the requested value
- **acceptor**: remembers the state of the protocol
- **learner**: once the value is chosen, executes the request and replies to the client



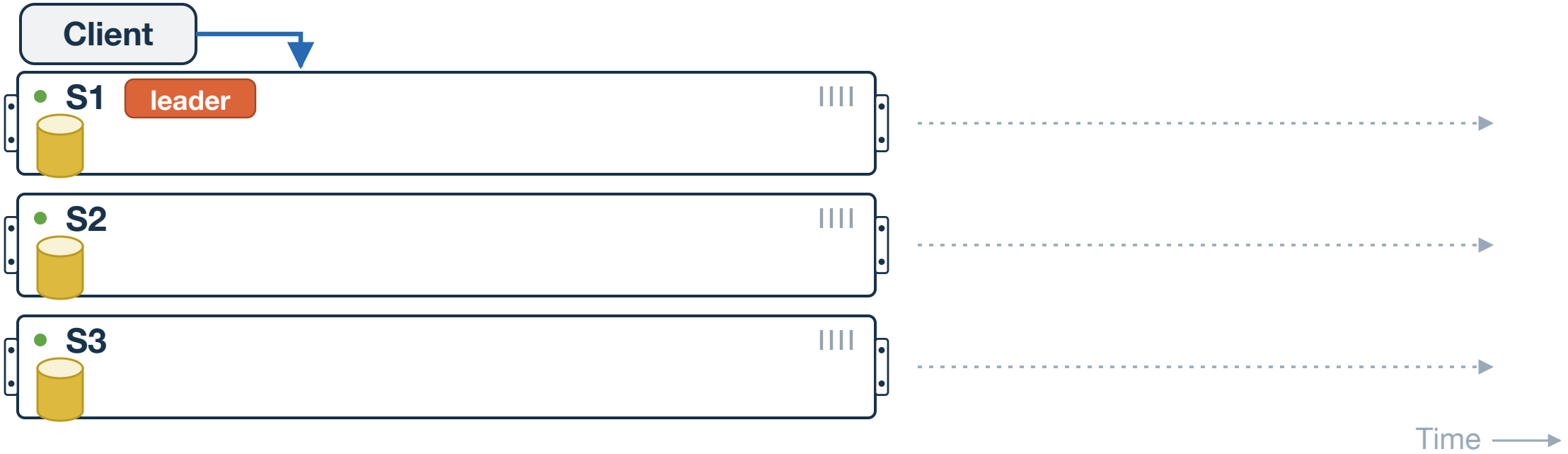
Round one: the proposer acquires the right to write

The leader chooses a proposal number N and sends it to a quorum

- N must exceed every number it has issued, so it tracks where it is: M_n

How does an acceptor decide? By the highest number it has seen: N_h

- If $N > N_h$: record $N_h = N$, and promise to ignore any proposal below N



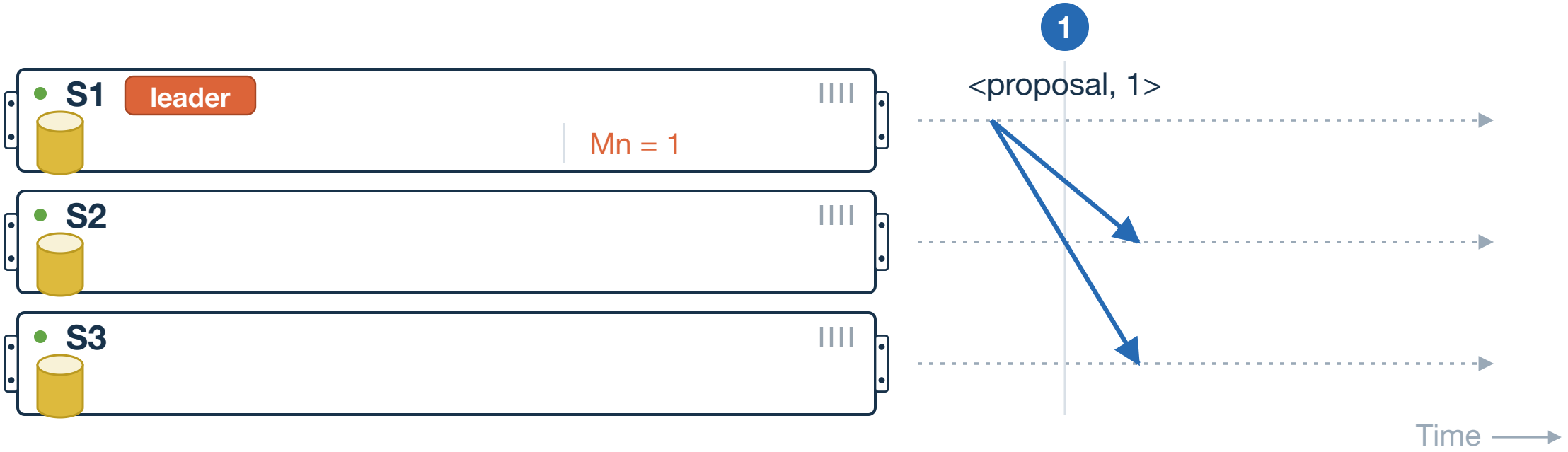
Round one: the proposer acquires the right to write

The leader chooses a proposal number N and sends it to a quorum

- N must exceed every number it has issued, so it tracks where it is: M_n

How does an acceptor decide? By the highest number it has seen: N_h

- If $N > N_h$: record $N_h = N$, and promise to ignore any proposal below N



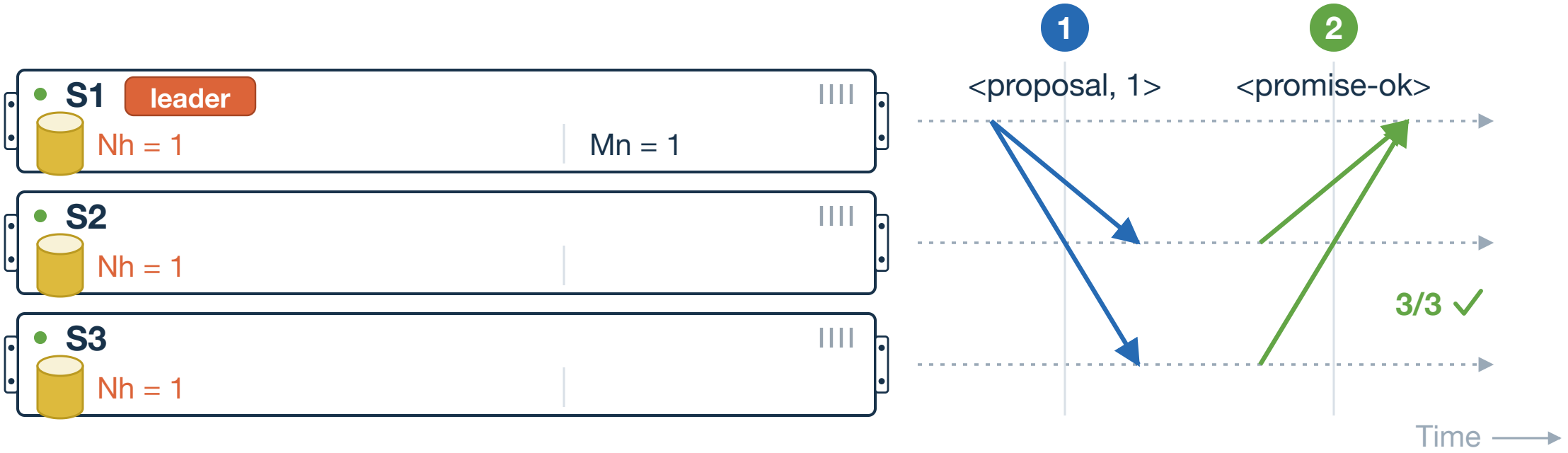
Round one: the proposer acquires the right to write

The leader chooses a proposal number N and sends it to a quorum

- N must exceed every number it has issued, so it tracks where it is: M_n

How does an acceptor decide? By the highest number it has seen: N_h

- If $N > N_h$: record $N_h = N$, and promise to ignore any proposal below N

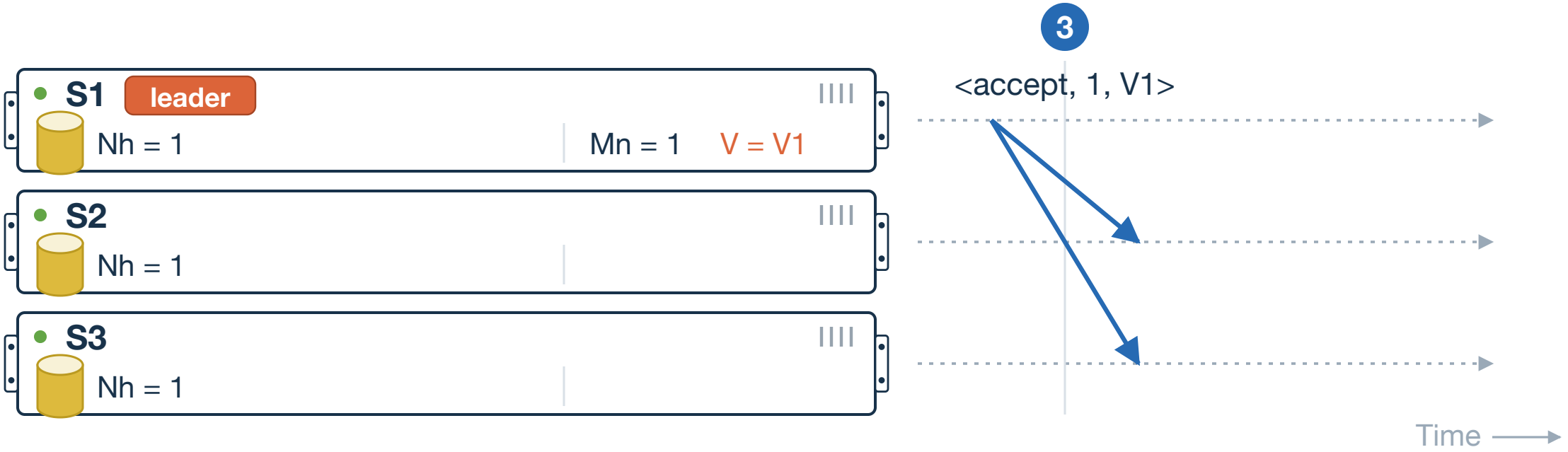


Round two: replicate the value to a majority

With promises from a majority, the proposer picks a value V and sends accept requests

So far the value is volatile — if the proposal dies, nobody knows what was chosen

- So an acceptor taking the accept writes the value to disk: V_a
- Once a majority of acceptors take it, learners execute and reply to the client

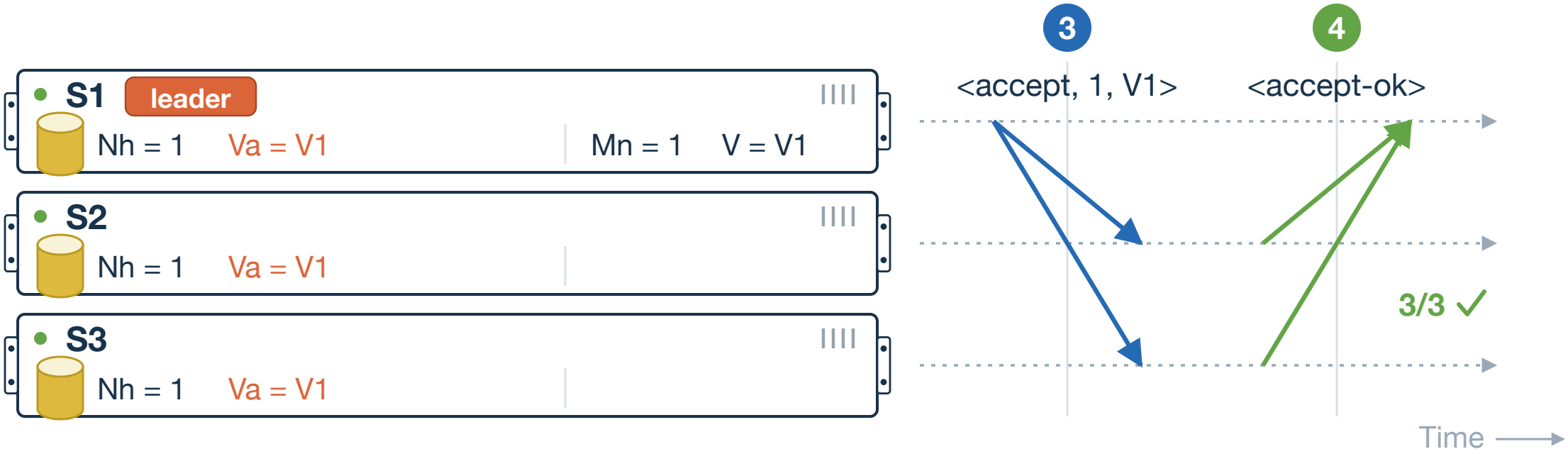


Round two: replicate the value to a majority

With promises from a majority, the proposer picks a value V and sends accept requests

So far the value is volatile — if the proposal dies, nobody knows what was chosen

- So an acceptor taking the accept writes the value to disk: V_a
- Once a majority of acceptors take it, learners execute and reply to the client

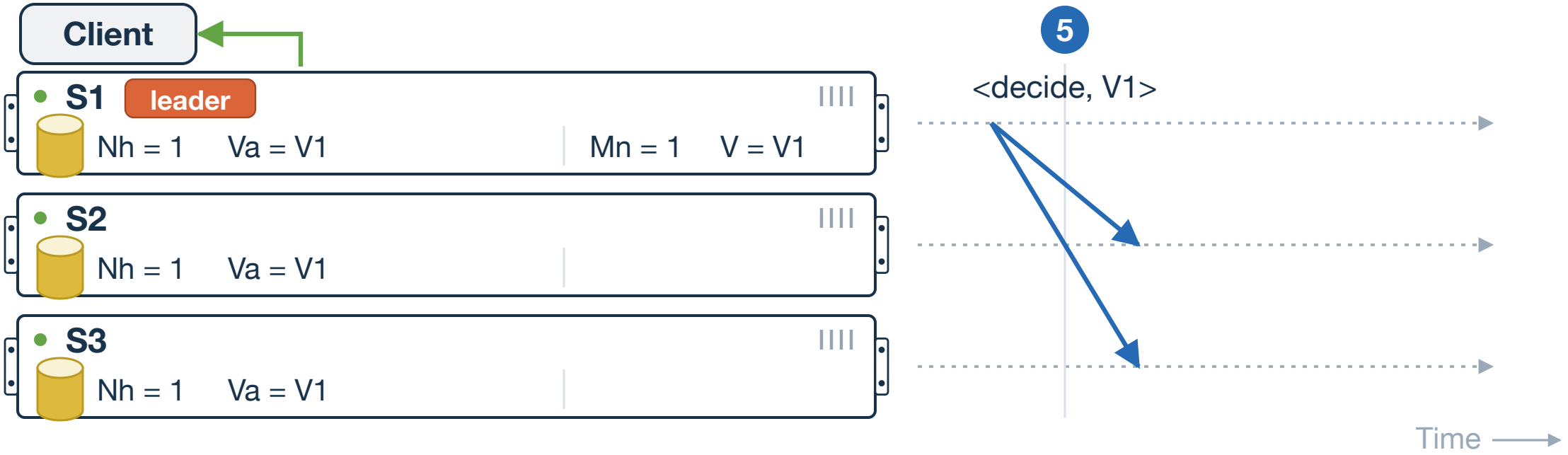


Round two: replicate the value to a majority

With promises from a majority, the proposer picks a value V and sends accept requests

So far the value is volatile — if the proposal dies, nobody knows what was chosen

- So an acceptor taking the accept writes the value to disk: V_a
- Once a majority of acceptors take it, learners execute and reply to the client

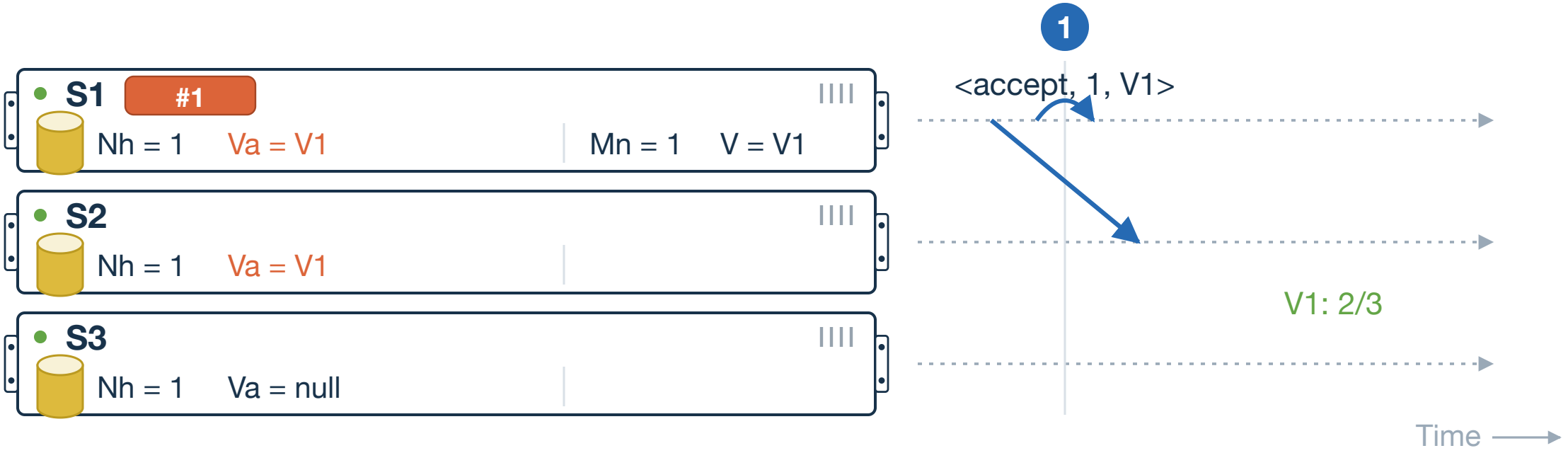


What if there are concurrent writers?

S1 and S2 have accepted V1: it has been chosen by a majority

- Incomplete protocol: no rule yet to carry forward the old value

S3 obtains promises from S2 and S3 with number 2, then proposes its own V2

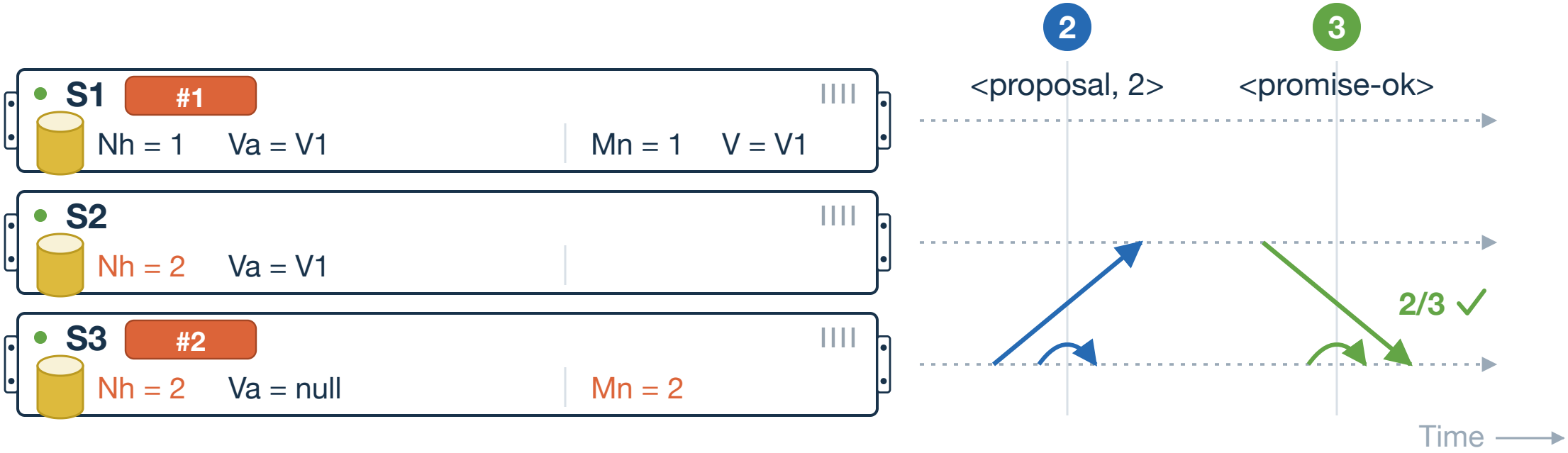


What if there are concurrent writers?

S1 and S2 have accepted V1: it has been chosen by a majority

- Incomplete protocol: no rule yet to carry forward the old value

S3 obtains promises from S2 and S3 with number 2, then proposes its own V2

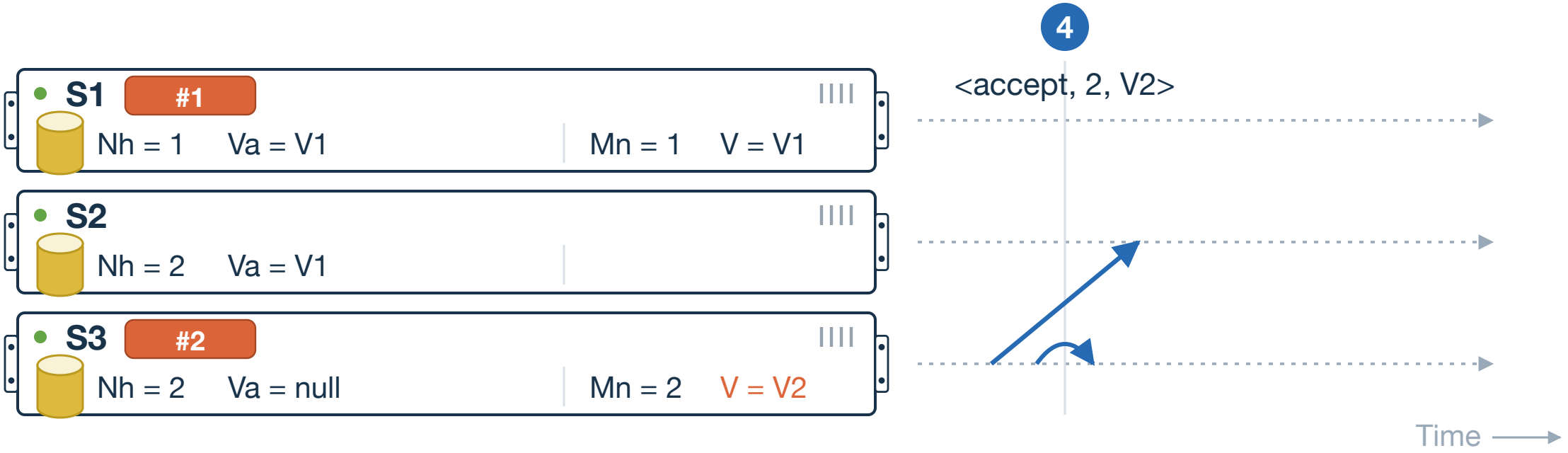


What if there are concurrent writers?

S1 and S2 have accepted V1: it has been chosen by a majority

- Incomplete protocol: no rule yet to carry forward the old value

S3 obtains promises from S2 and S3 with number 2, then proposes its own V2

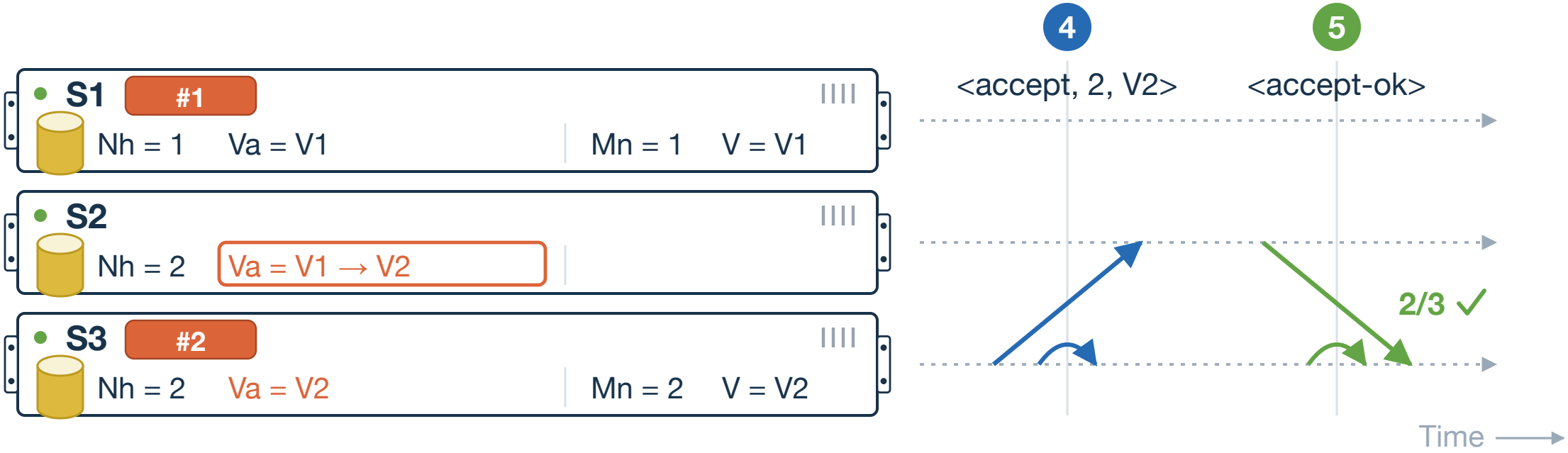


What if there are concurrent writers?

S1 and S2 have accepted V1: it has been chosen by a majority

- Incomplete protocol: no rule yet to carry forward the old value

S3 obtains promises from S2 and S3 with number 2, then proposes its own V2



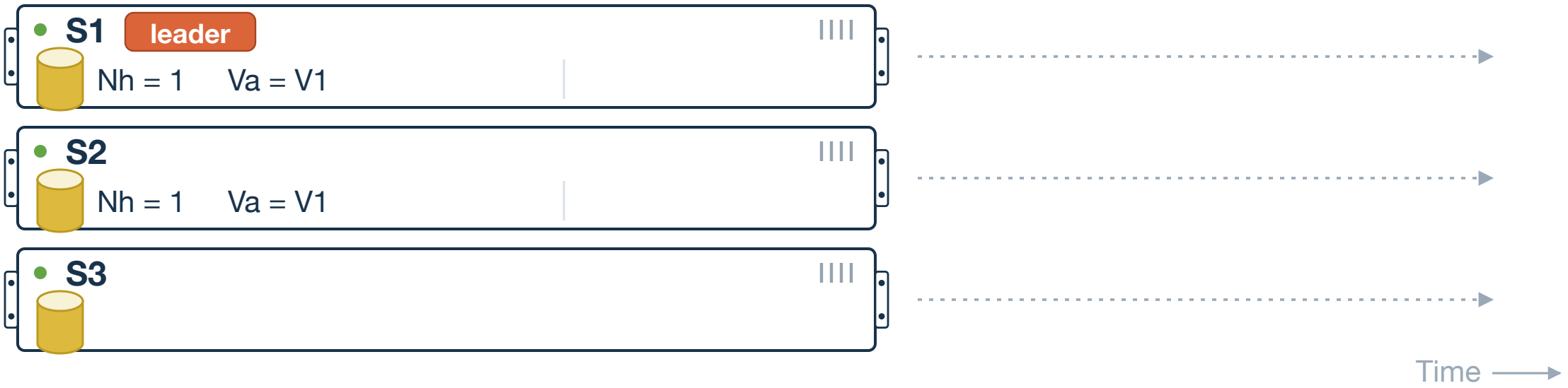
S2's V1 is overwritten: V2 also reaches a majority — safety is violated

Question: what if a value has been written?

S1's `<accept, 1, V1>` was already taken by a majority

- What must prevent S3 from choosing its own V2 with number 2?

Once V1 is chosen, later proposals must preserve V1

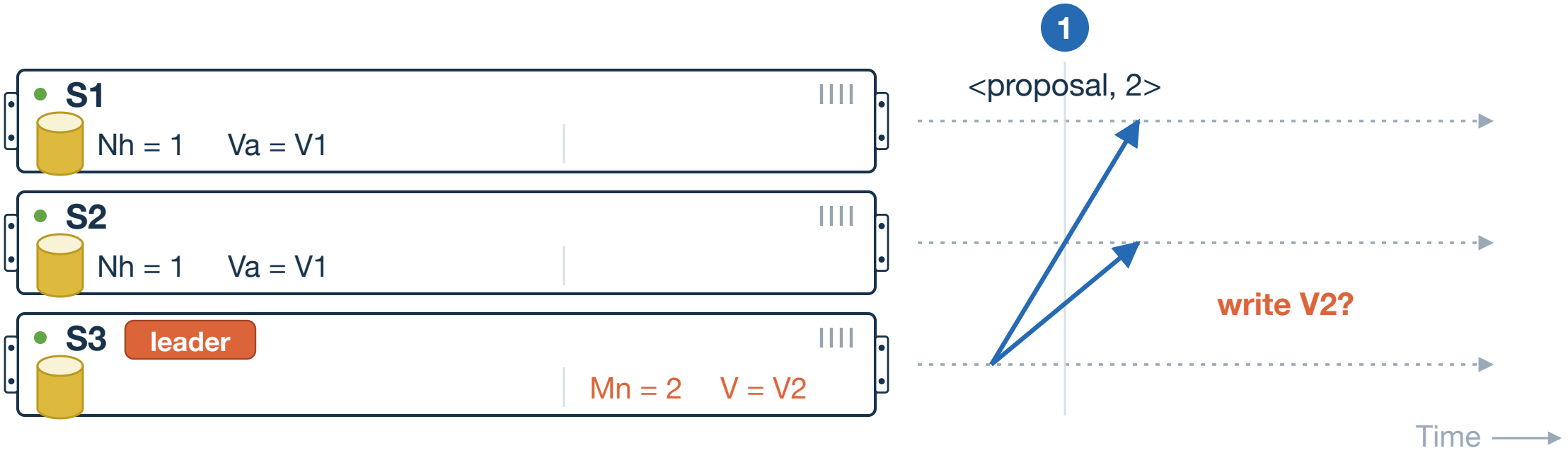


Question: what if a value has been written?

S1's `<accept, 1, V1>` was already taken by a majority

- What must prevent S3 from choosing its own V2 with number 2?

Once V1 is chosen, later proposals must preserve V1



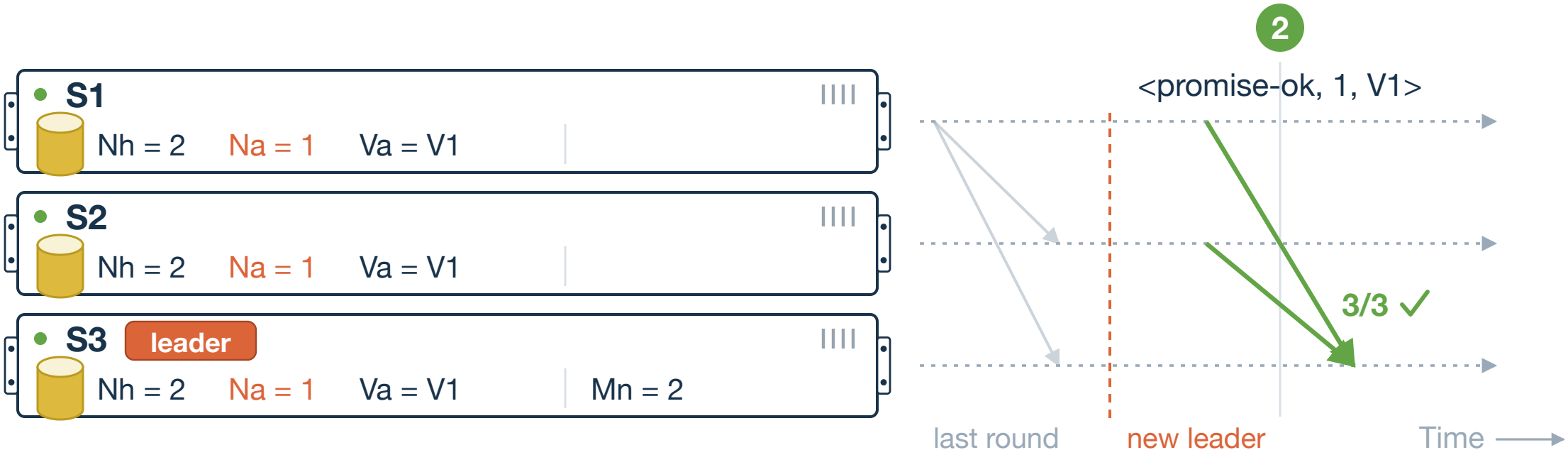
Solution: old value must be carried back

When an acceptor replies with a promise, it sends back the value it has accepted so far

- The proposer must use the old value in its accept request

What if the leader receives multiple values?

- The proposer must choose the value associated with the highest N_a as V



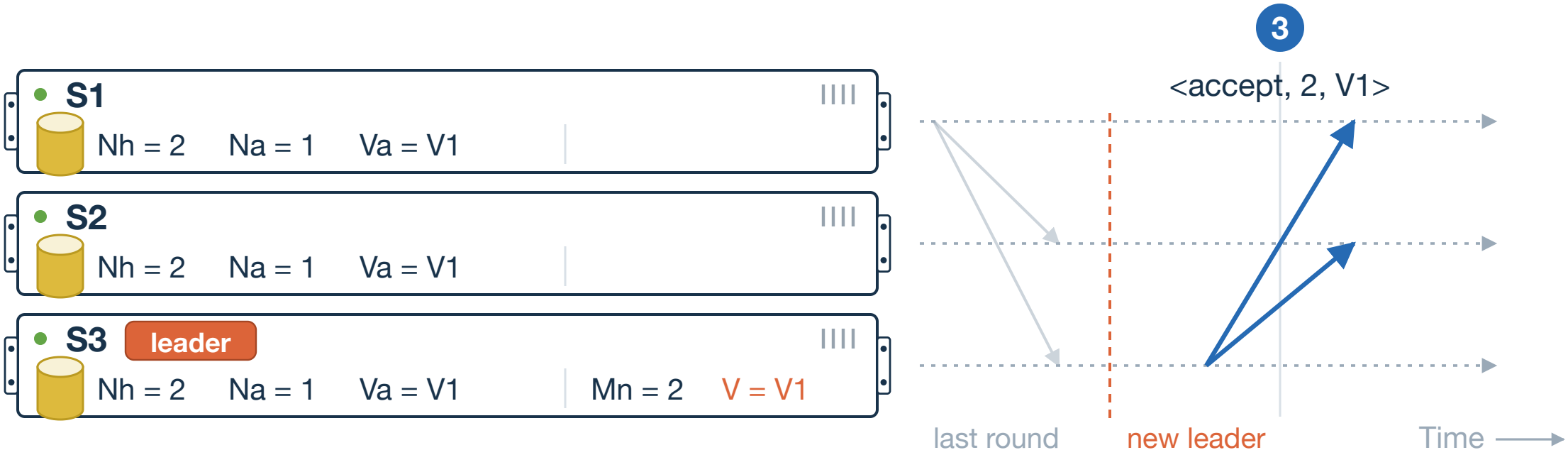
Solution: old value must be carried back

When an acceptor replies with a promise, it sends back the value it has accepted so far

- The proposer must use the old value in its accept request

What if the leader receives multiple values?

- The proposer must choose the value associated with the highest N_a as V



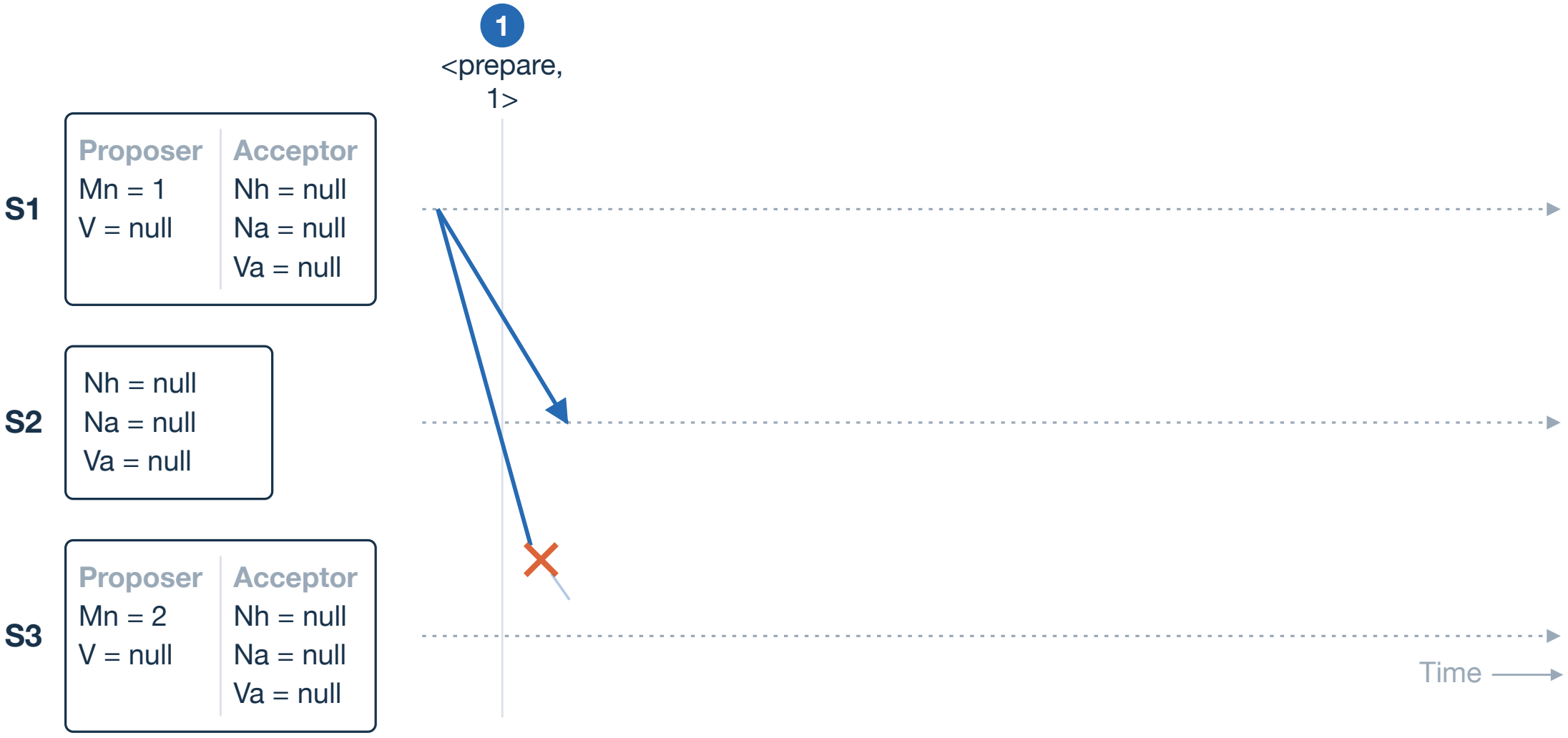
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



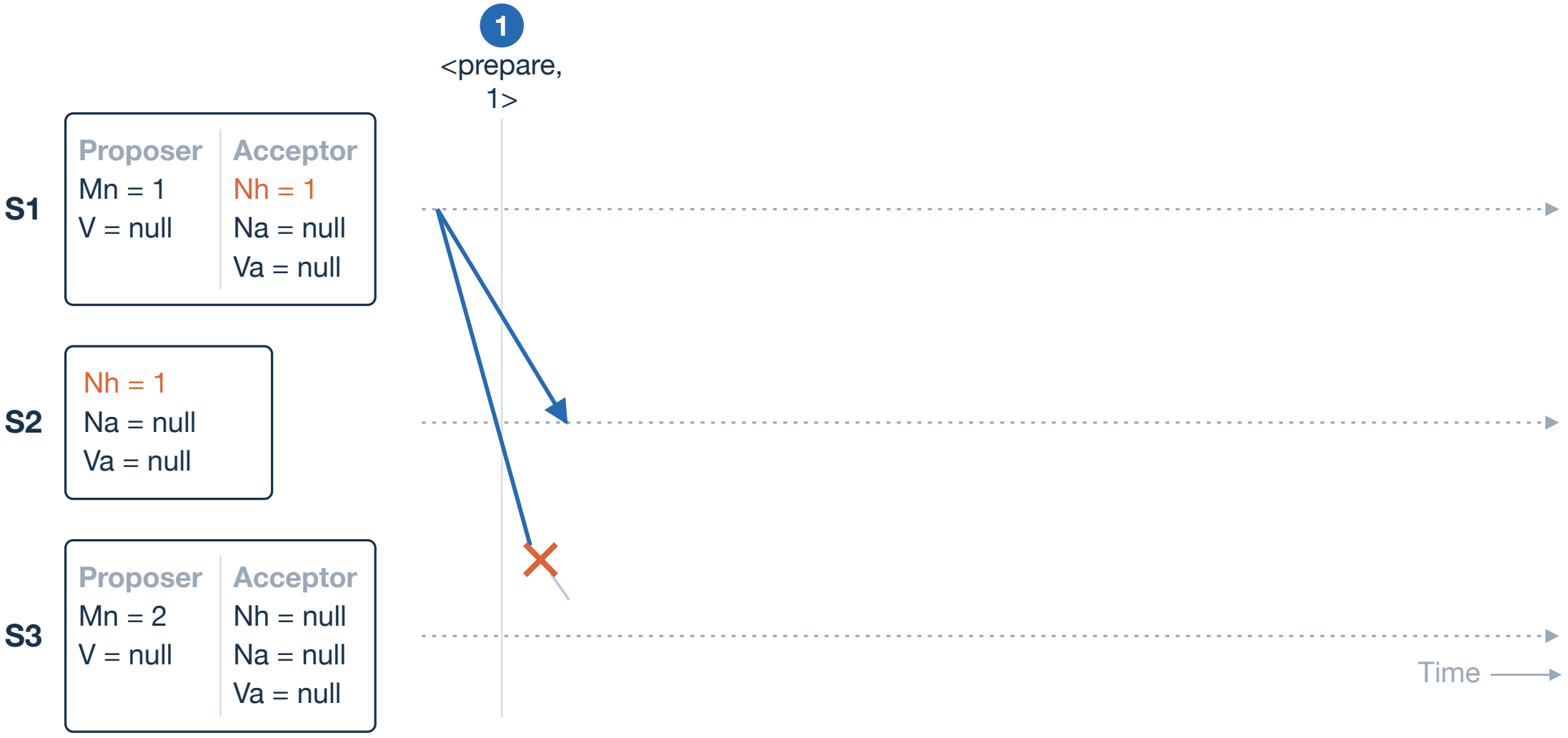
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



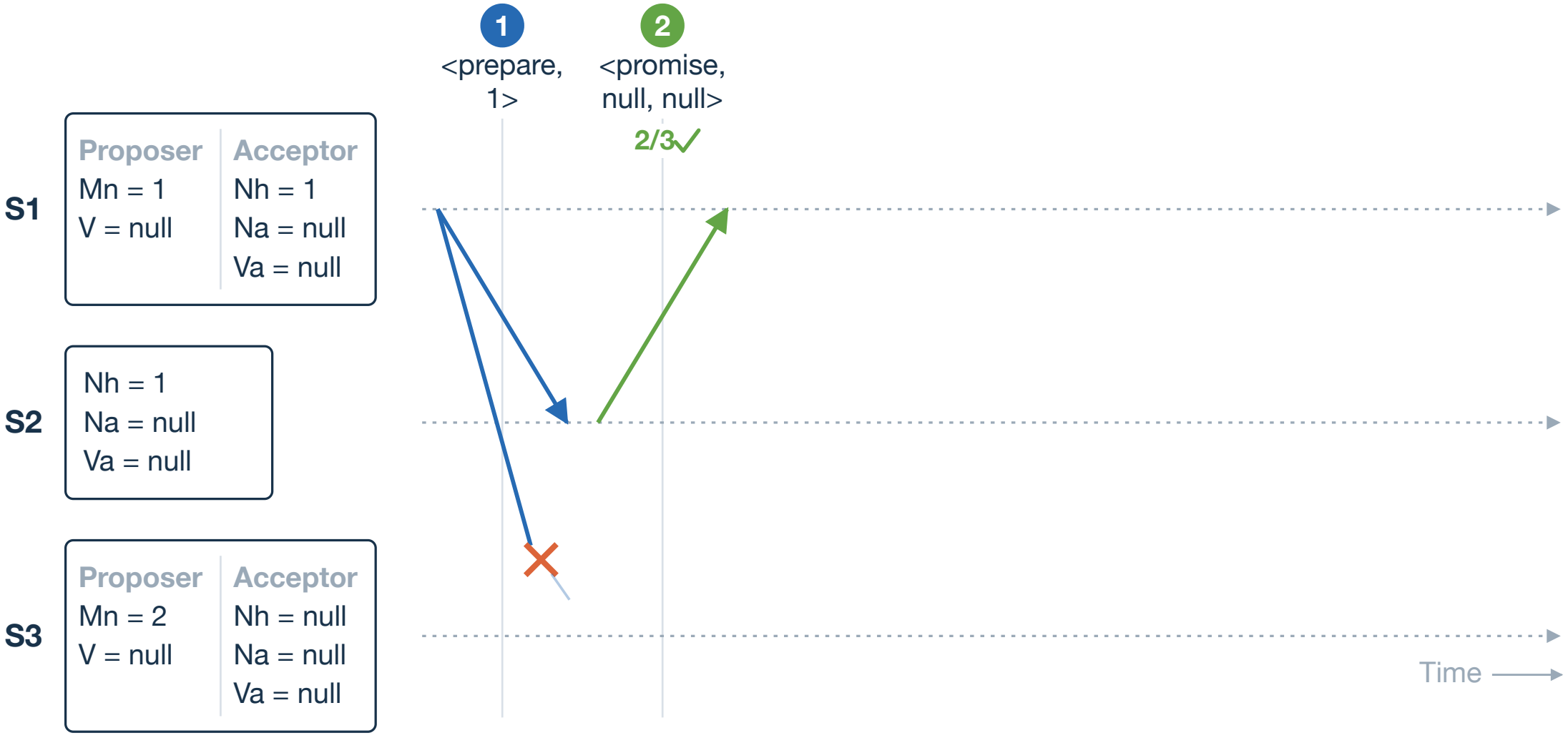
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



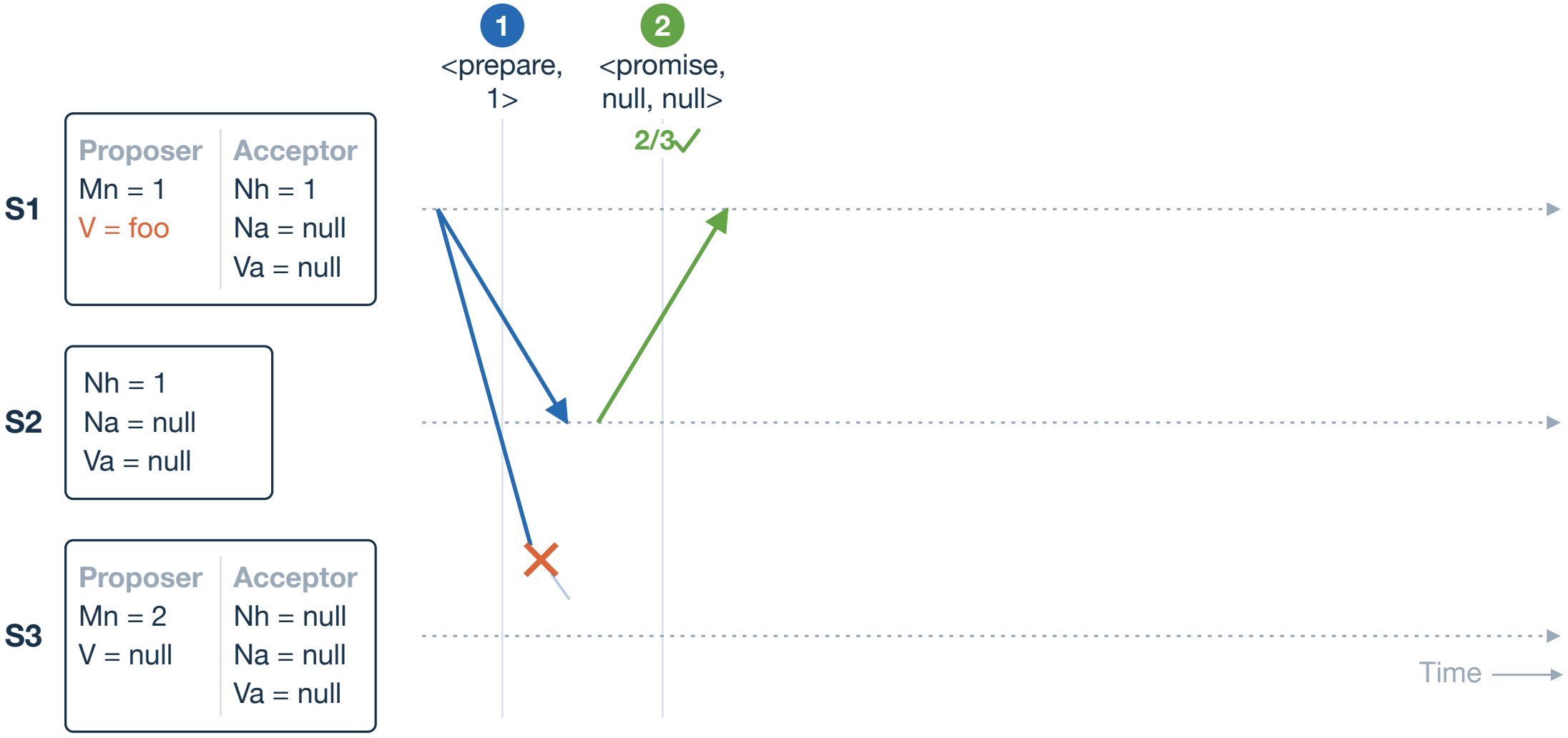
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



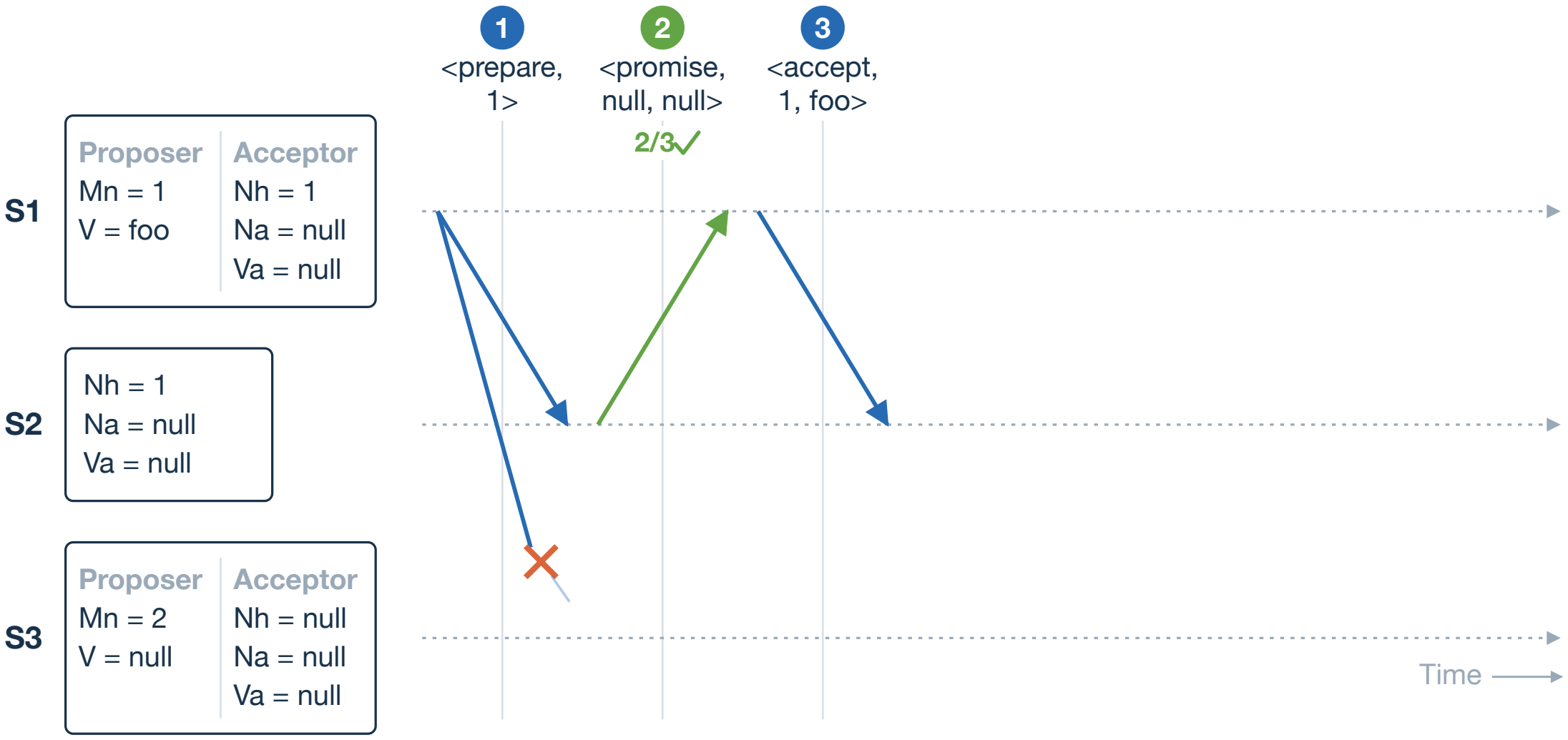
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



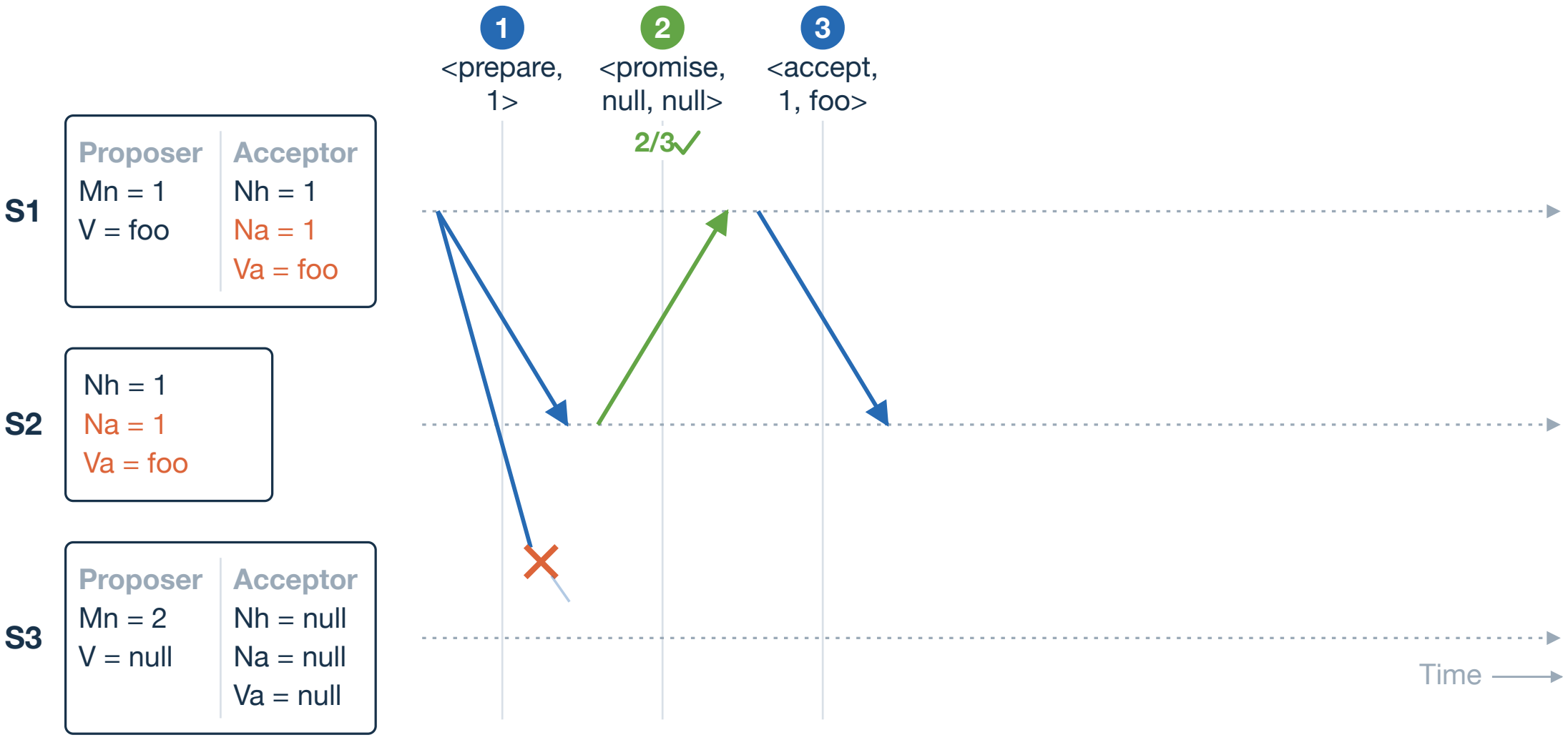
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



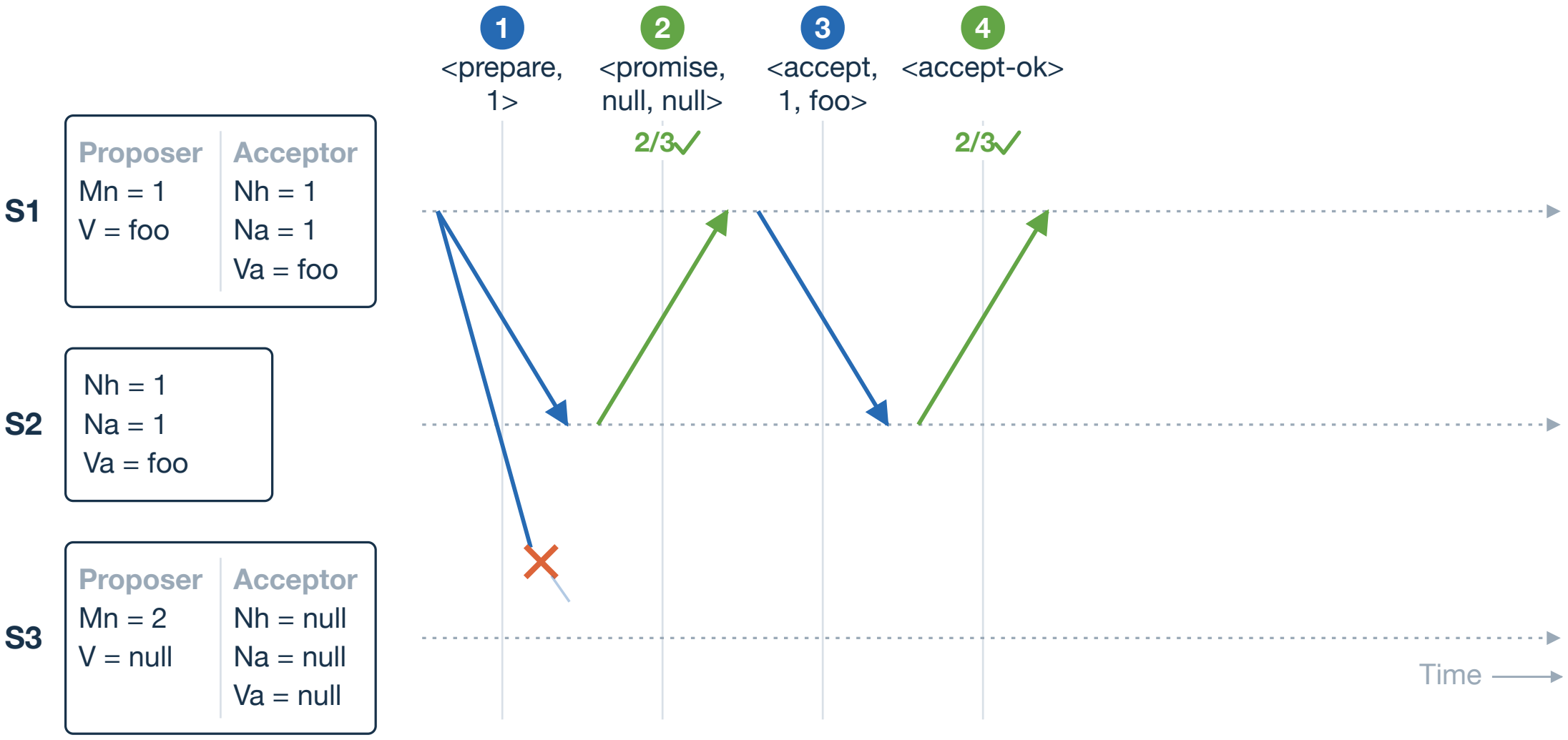
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



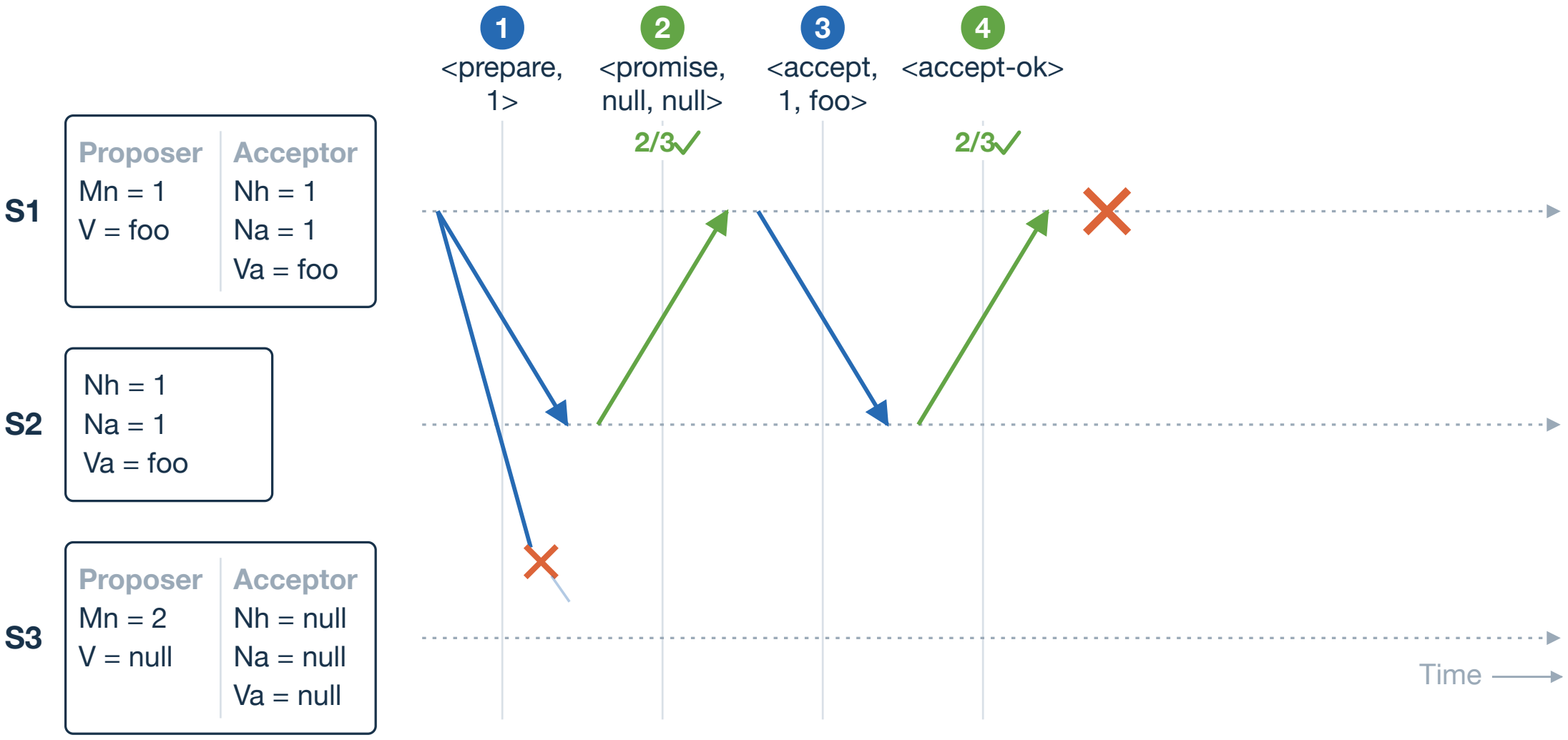
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



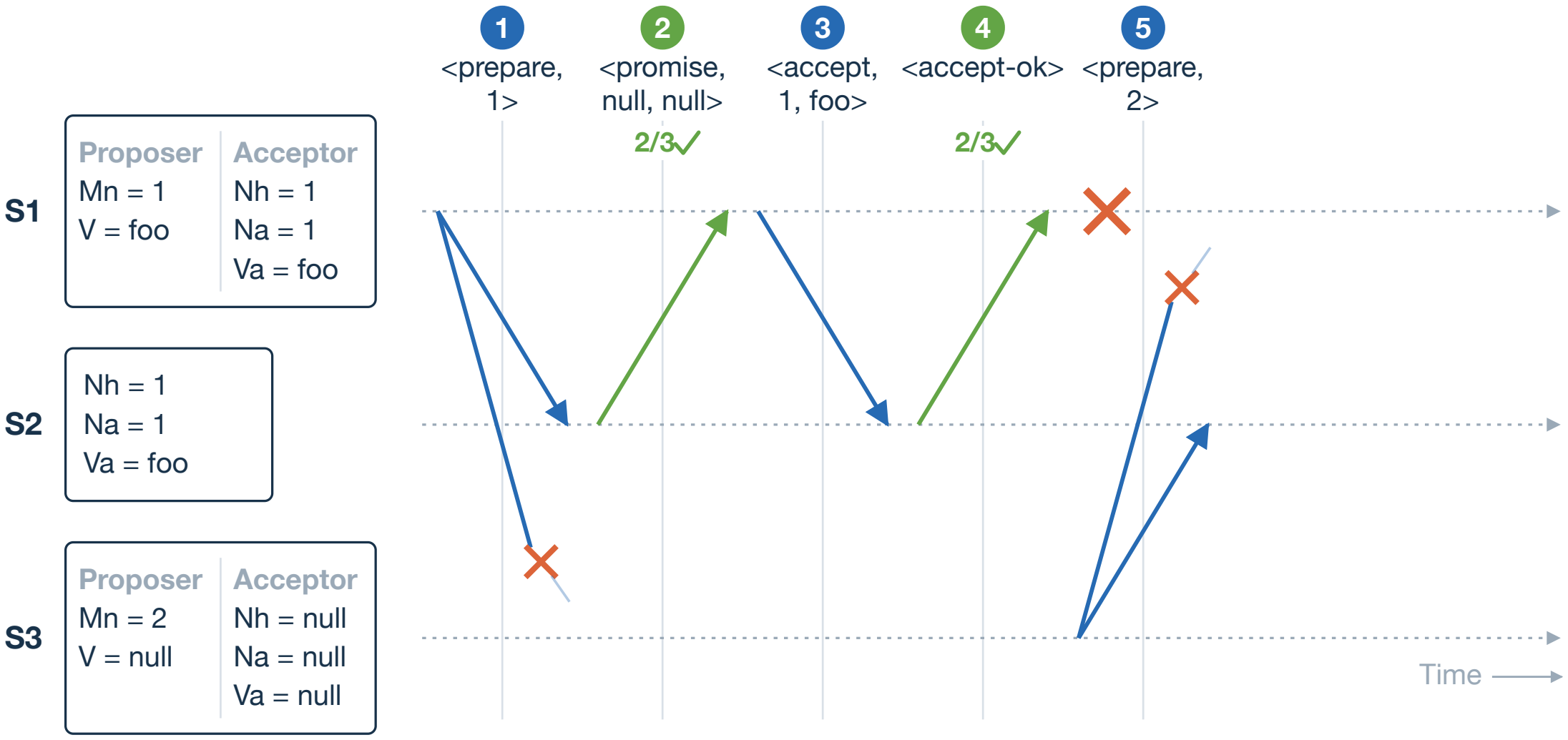
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



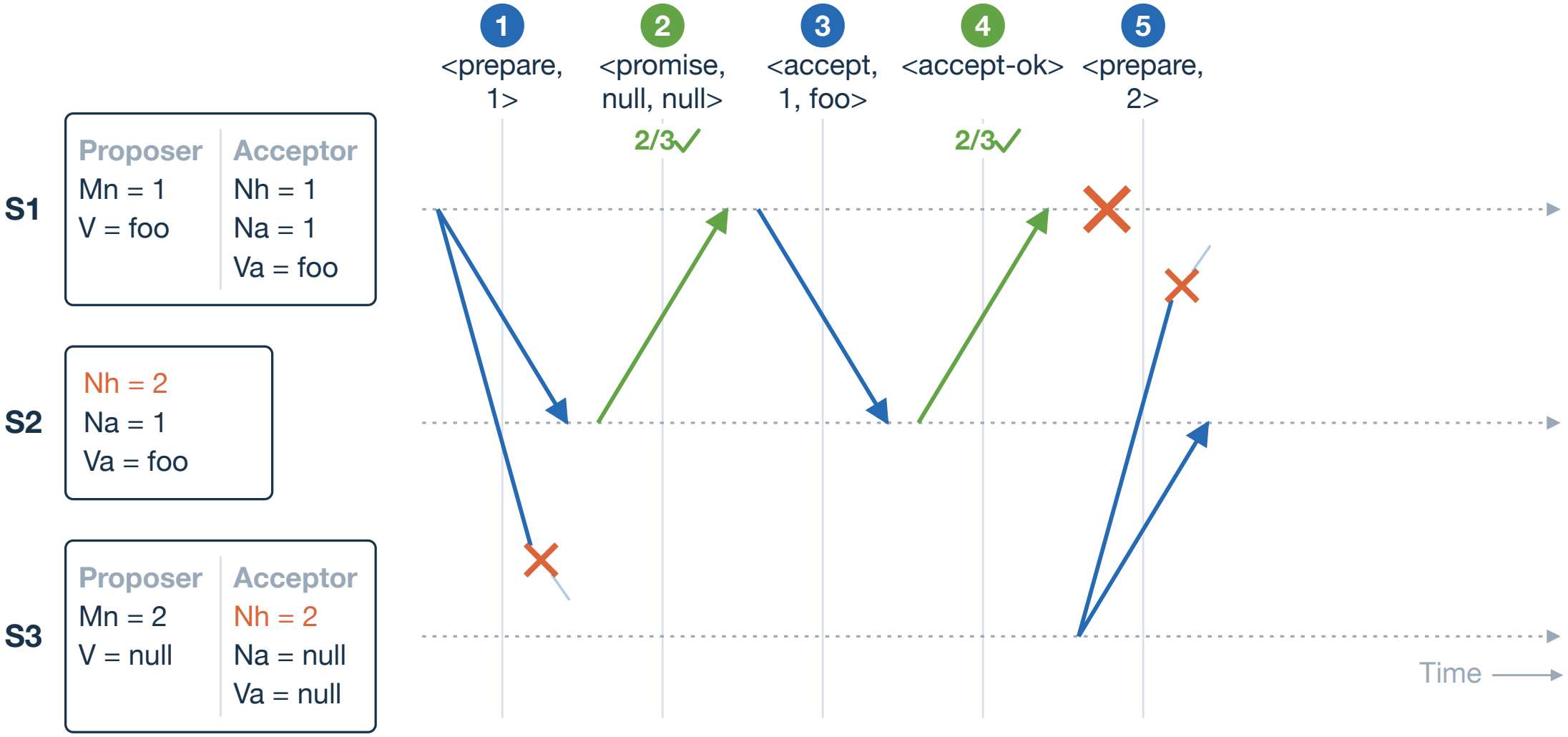
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



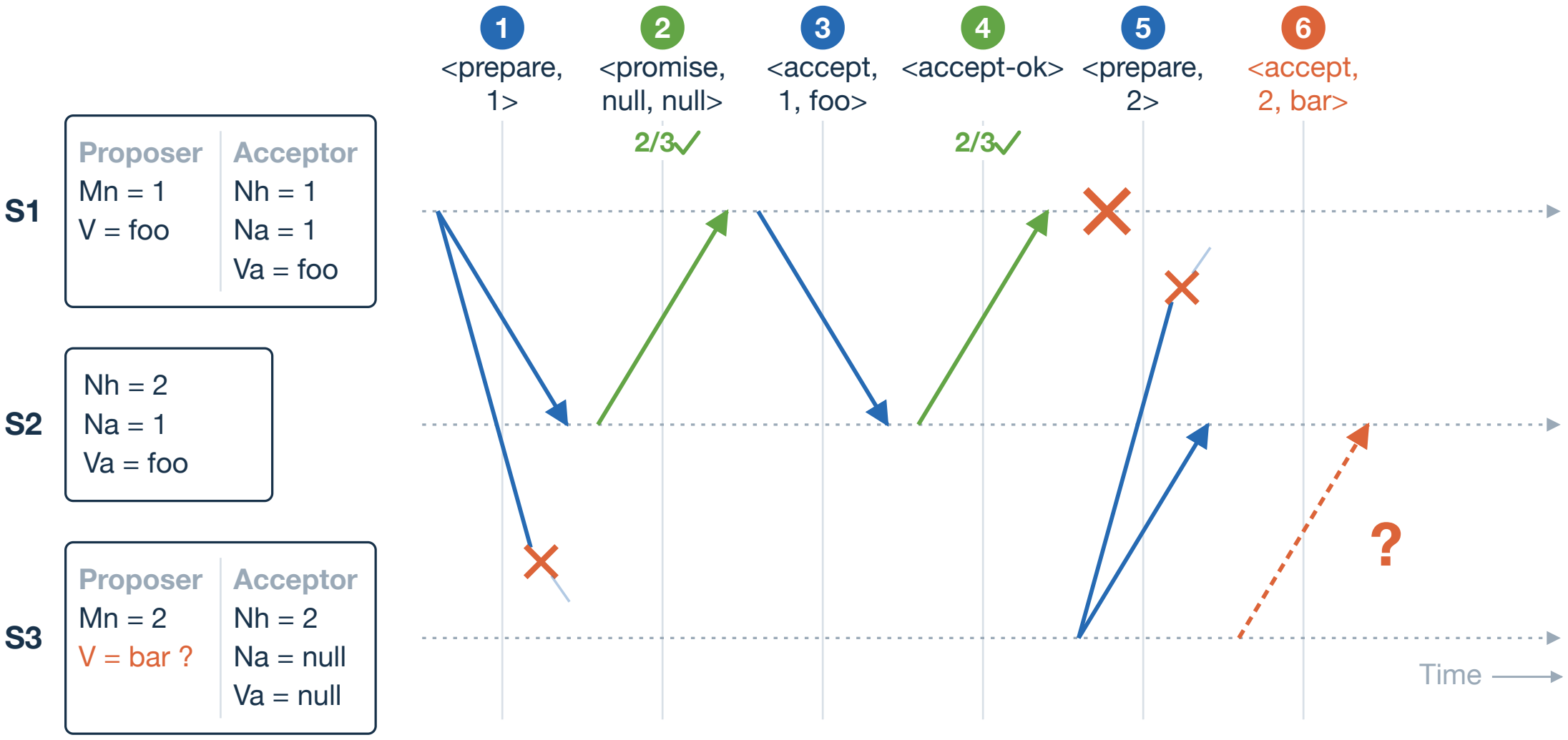
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



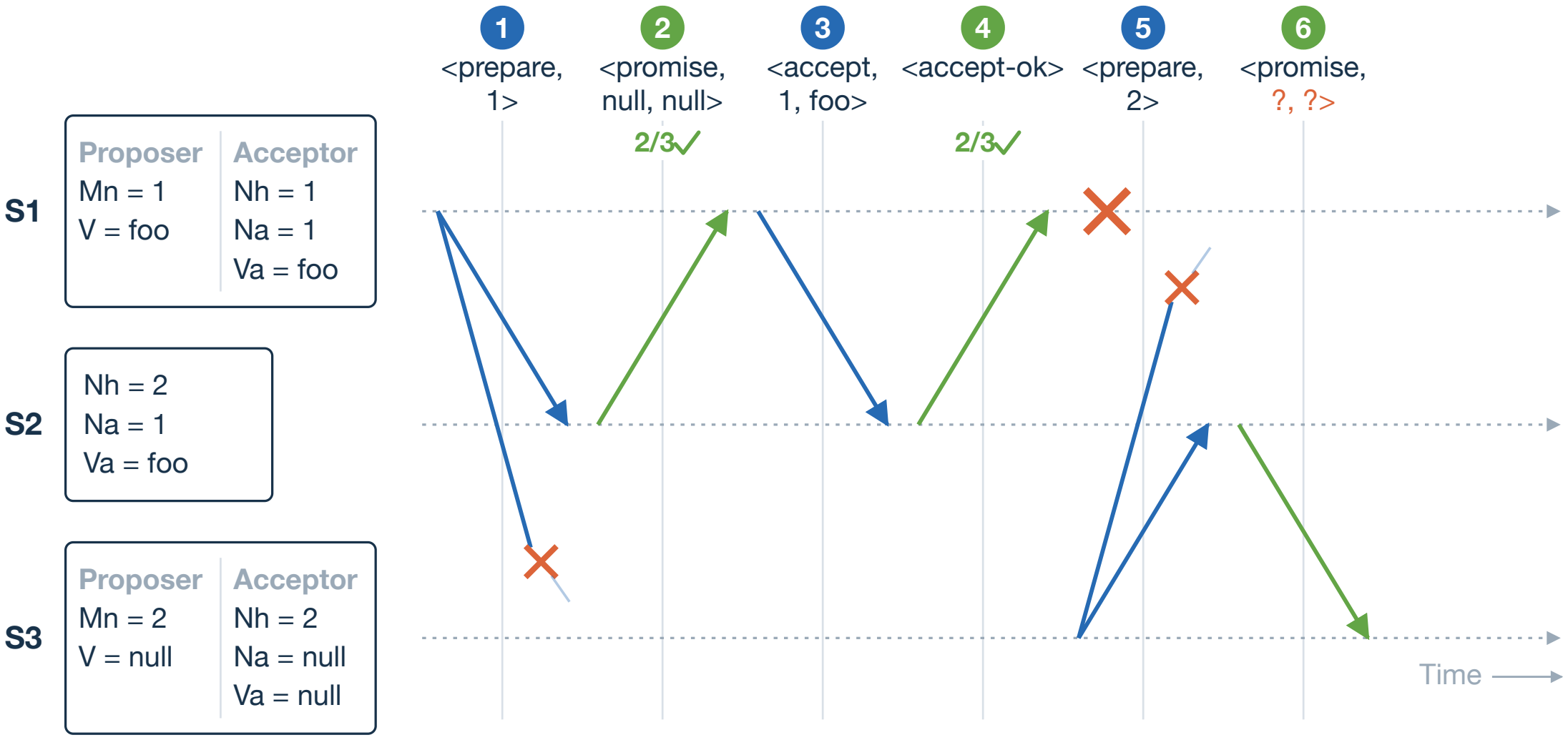
The complete picture: can S3 write "bar" here?

Two proposers: S1 writes `foo` and crashes, then S3 comes in with number 2



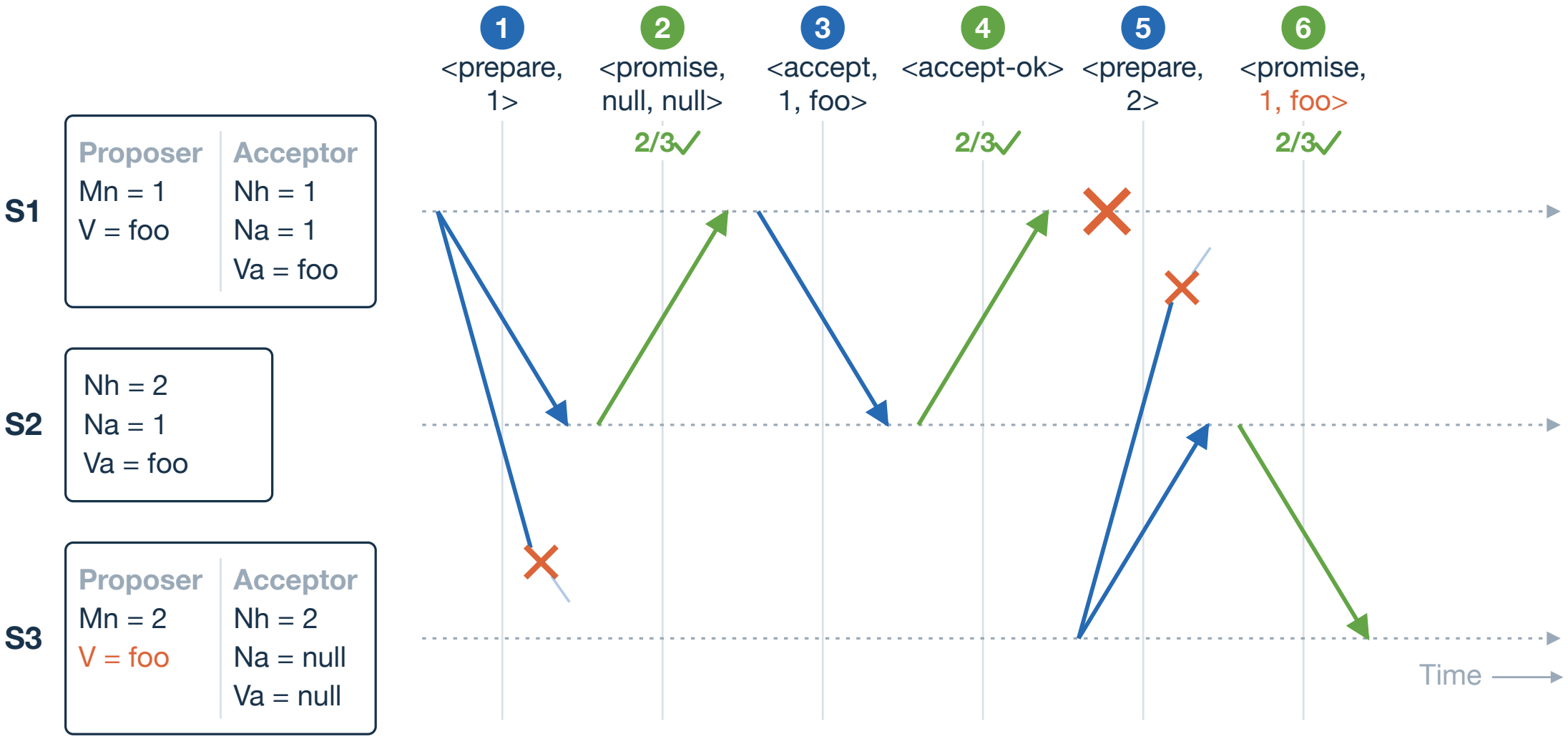
Answer: S2's promise carries foo

S2 has already accepted `foo` — so what can it put in its promise?



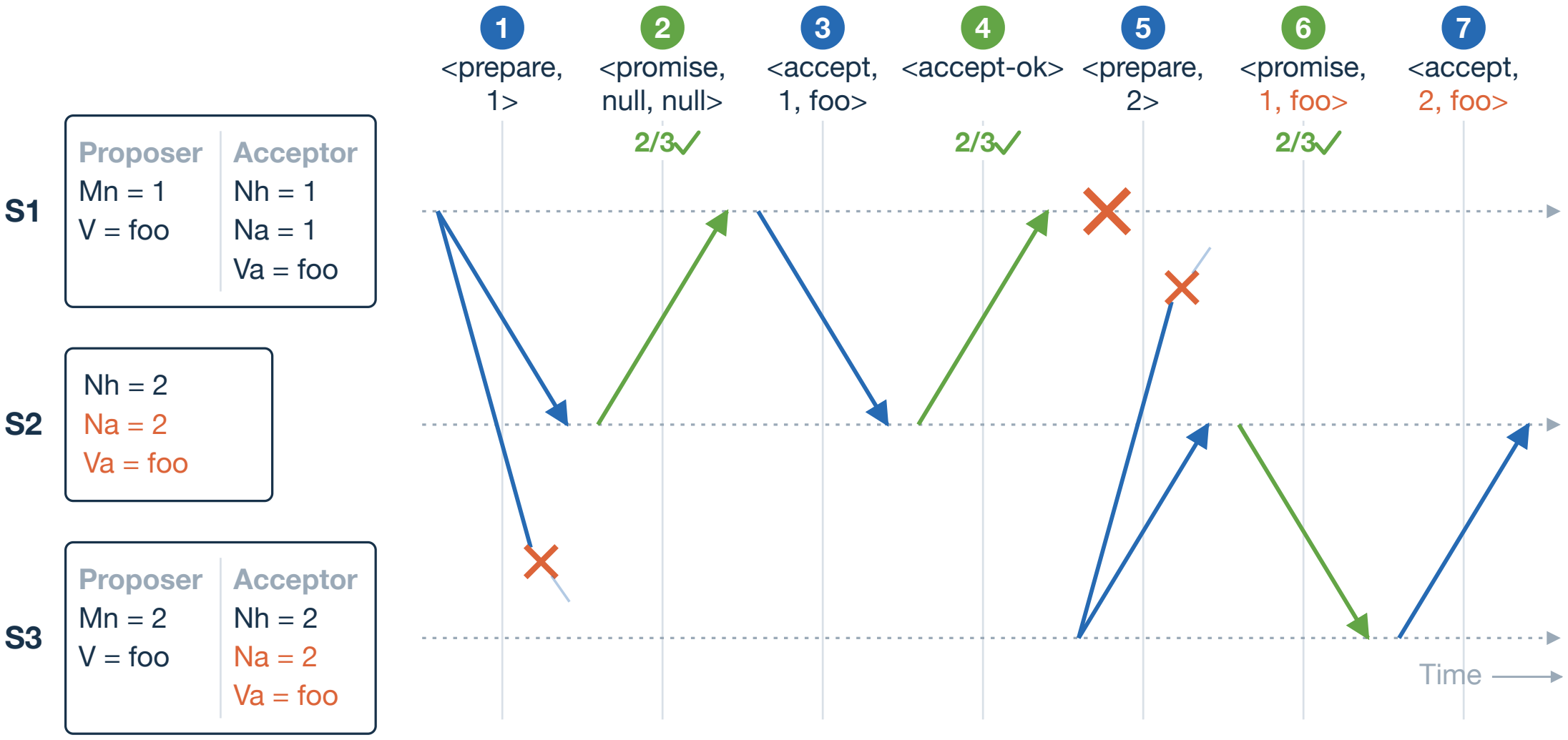
Answer: S2's promise carries foo

S2 has already accepted `foo` — so what can it put in its promise?



Answer: S2's promise carries foo

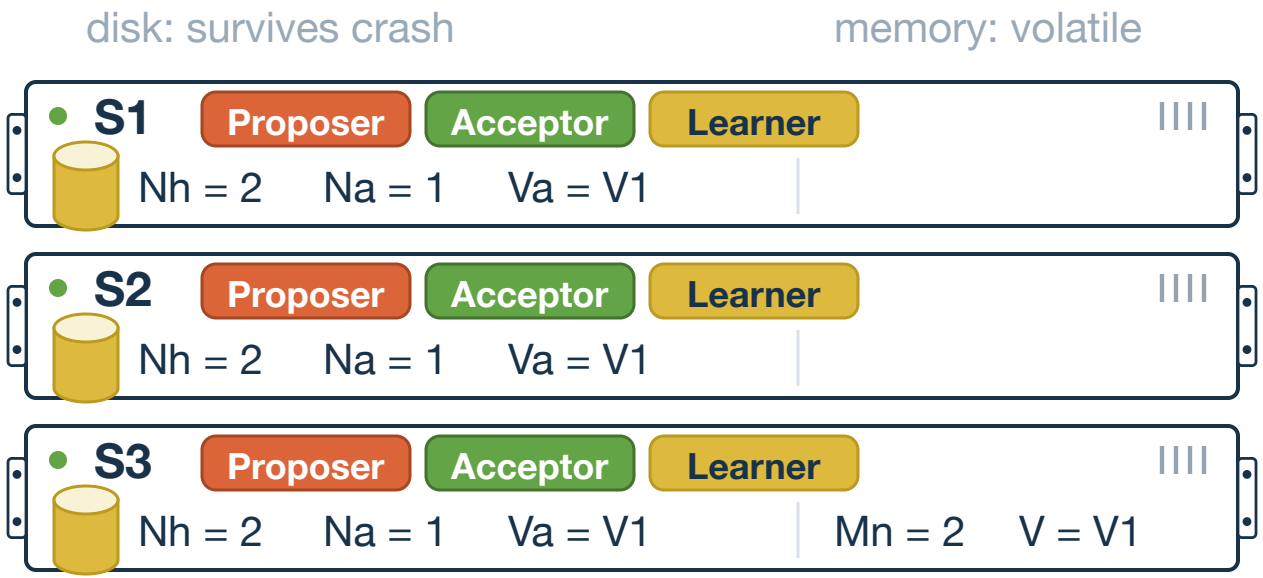
S2 has already accepted `foo` — so what can it put in its promise?



Put it all together: Roles and state on each node

Every node can run all three roles

- On disk (durable): N_h highest proposal number seen; N_a / V_a highest number accepted, and its value
- In memory (gone on a crash): M_n my proposal number; V the value to propose this round

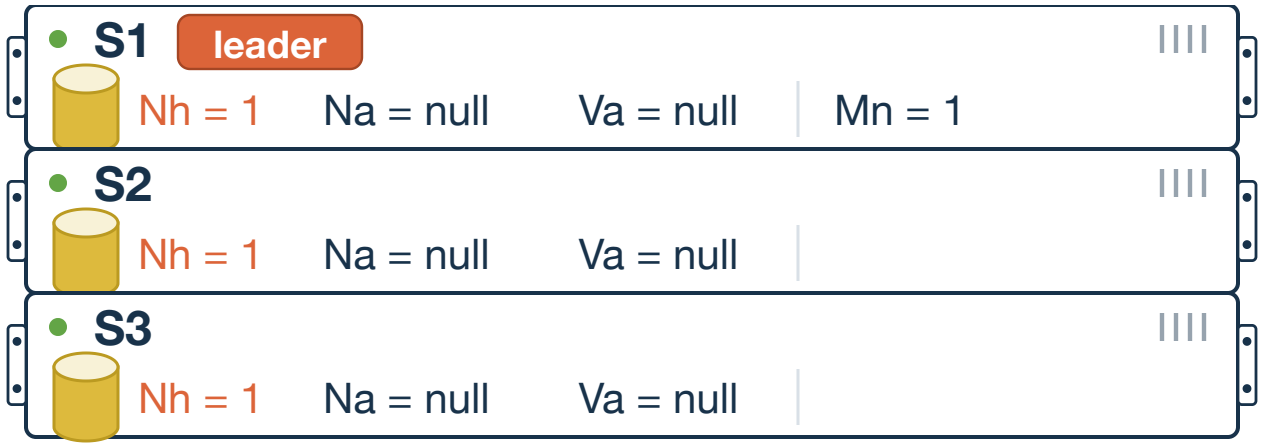


Paxos pseudo-code: prepare

```
A node decides to be Leader
Leader chooses Mn = a number greater than Nh
Leader sends <proposal, Mn> to all nodes

Acceptor receives <proposal, N>
    if  N < Nh
        reply <promise-reject>
    else
        Nh = N
        reply <promise-ok, Na, Va>
```

Ignore all proposals < Nh



Paxos pseudo-code: accept and decide

```
If Leader gets promise-ok from a majority
    if  V != null, V = the value of the highest Na received
    if  V = null, then Leader can pick any V
    send <accept, Mn, V> to all nodes

If Leader fails to get majority promise-ok
    delay and restart Paxos

Upon receiving <accept, N, V>
    if  N < Nh
        reply <accept-reject>
    else
        Na = N;  Va = V;  Nh = N;
        reply <accept-ok>

If Leader gets accept-ok from a majority
    send <decide, Va> to all nodes

If Leader fails to get majority accept-ok
    delay and restart Paxos
```

Question: what if two proposers pick the same number?

S1 and S3 both choose `Mn = 1`. An acceptor rejects only when `N < Nh`, and `1 < 1` is false

So a proposal number must be unique: use a `(counter, node id)` pair, e.g. `1:S1`

Some quiz questions about Paxos

When is the value V chosen?

- Leader receives a majority of `<promise-ok, …>`
- A majority of acceptors receive `<accept, N, V>`
- Leader receives a majority of `<accept-ok, …>`

What if an acceptor fails after sending `<promise-ok>`?

What if an acceptor fails after receiving `<accept, N, V>`?

What if the leader fails while sending `<accept, N, V>`?

Question: what about liveness?

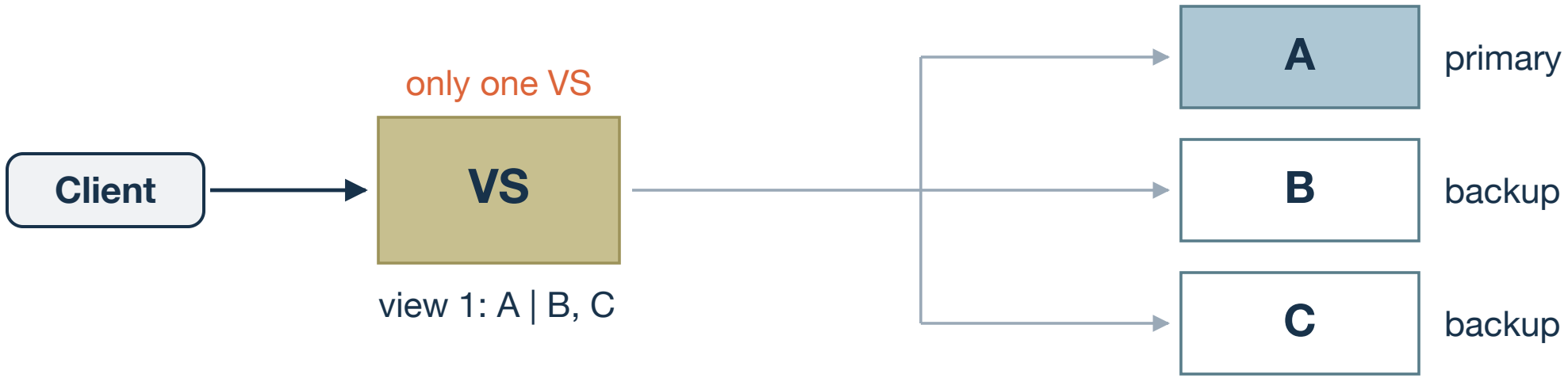
Liveness: is Paxos guaranteed to decide a value? [1]

[1] In Lamport's own words: *it was impossible to ensure that any decree would ever be passed* (The Part-Time Parliament §1.2)

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

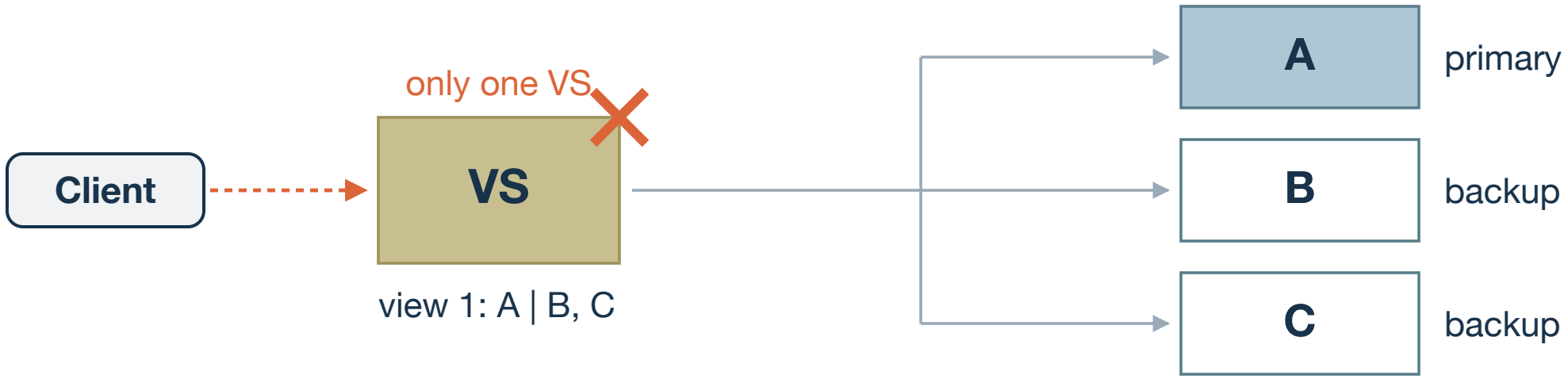


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

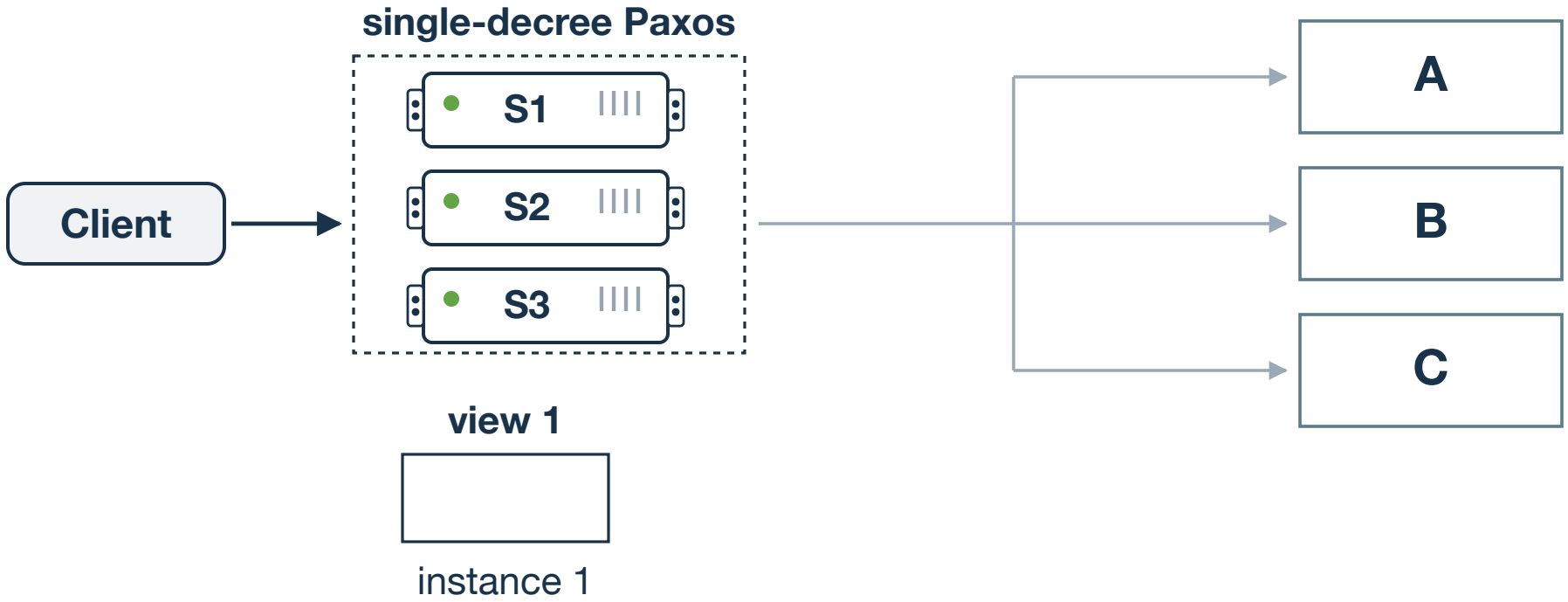


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

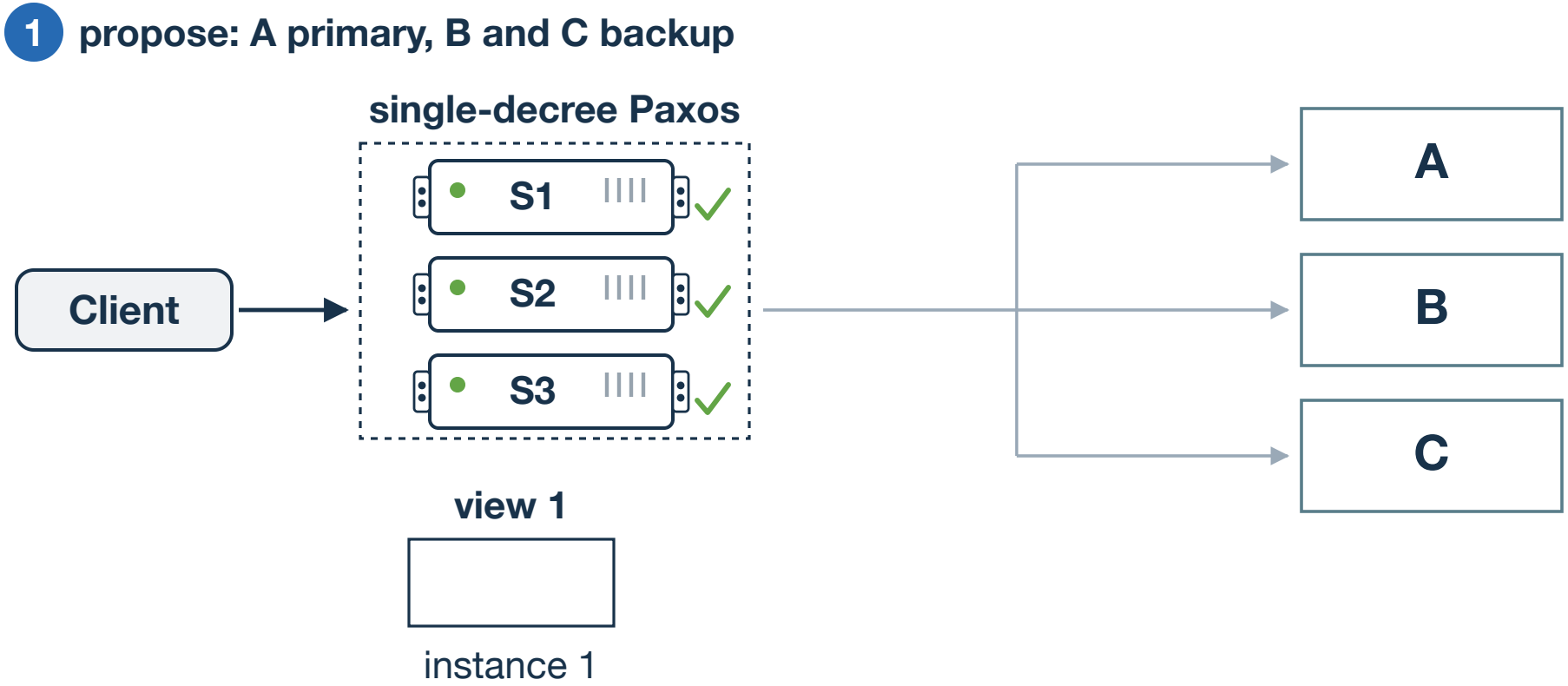


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

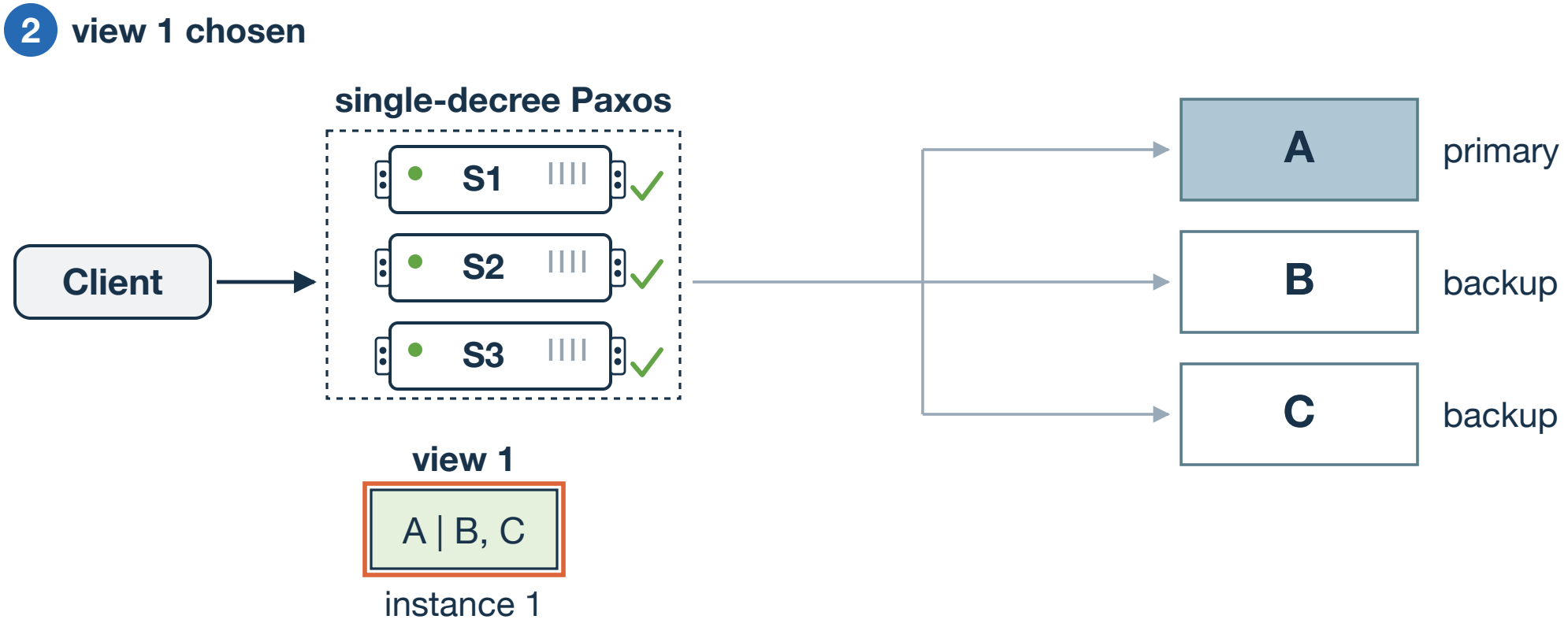


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

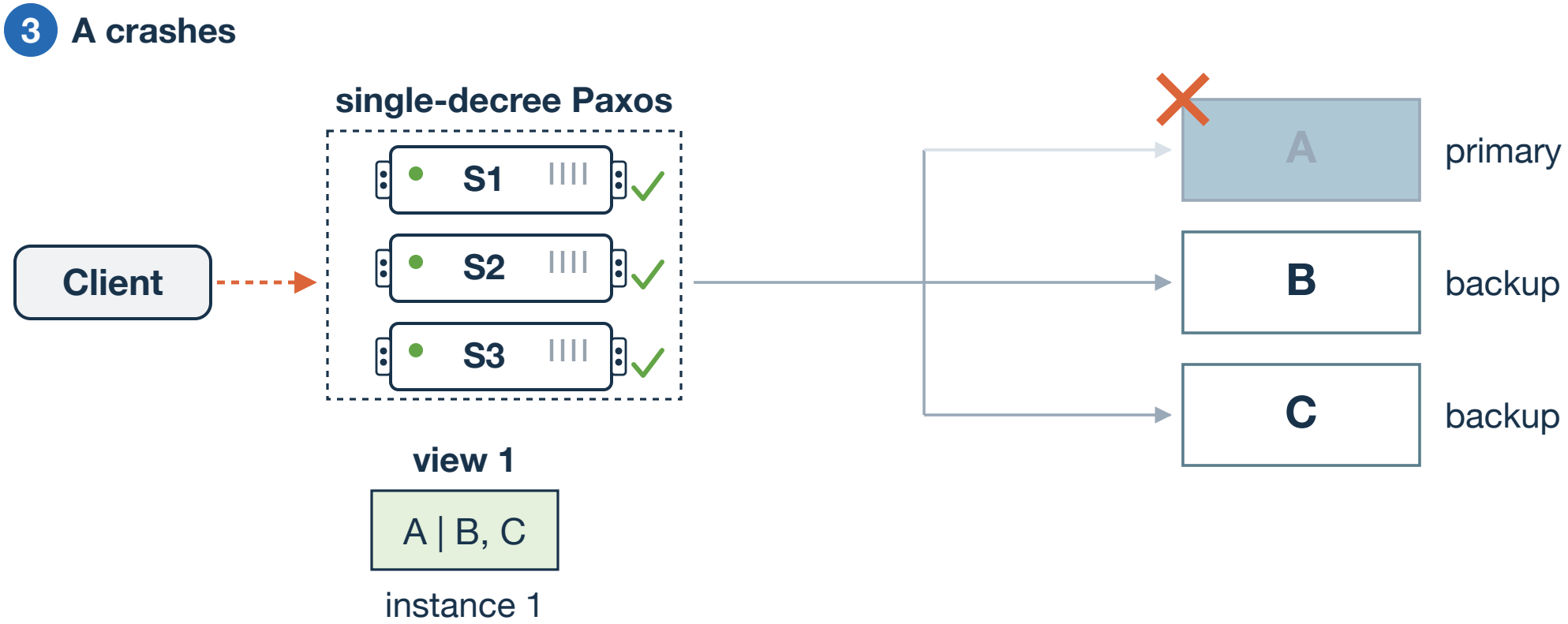


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

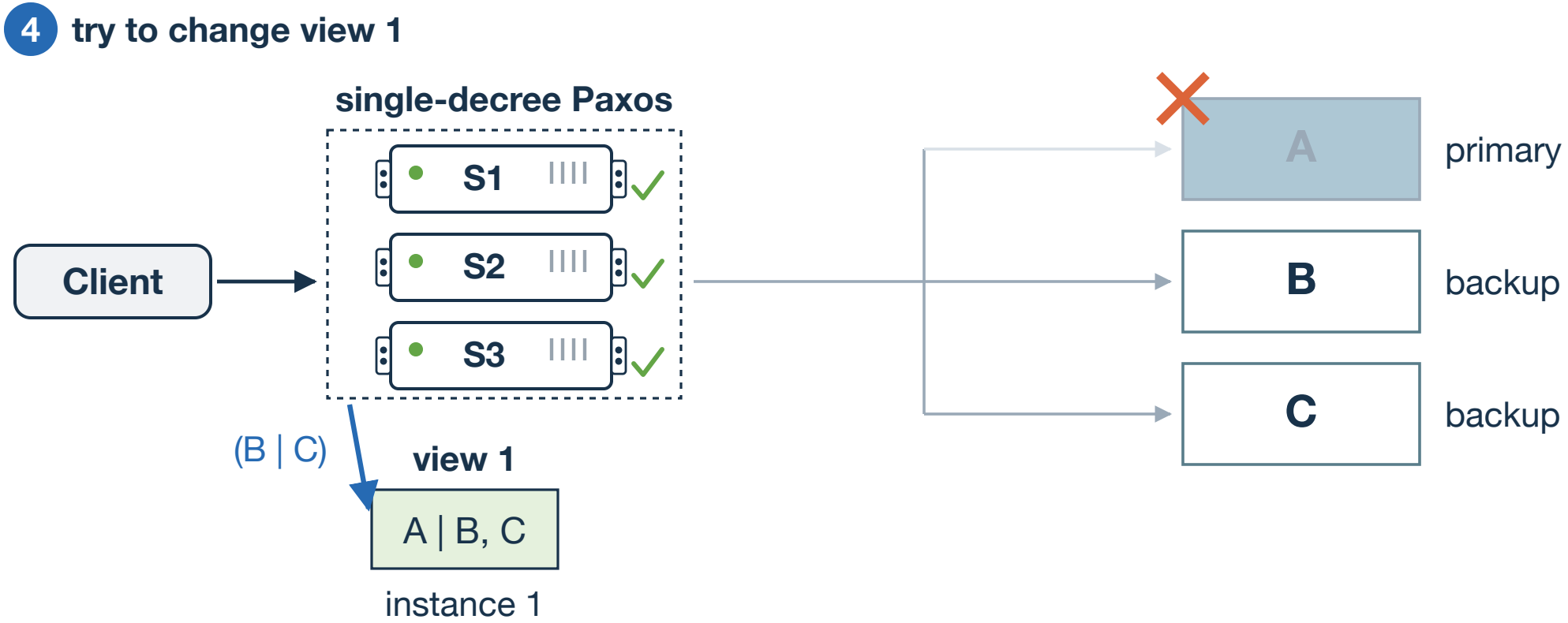


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

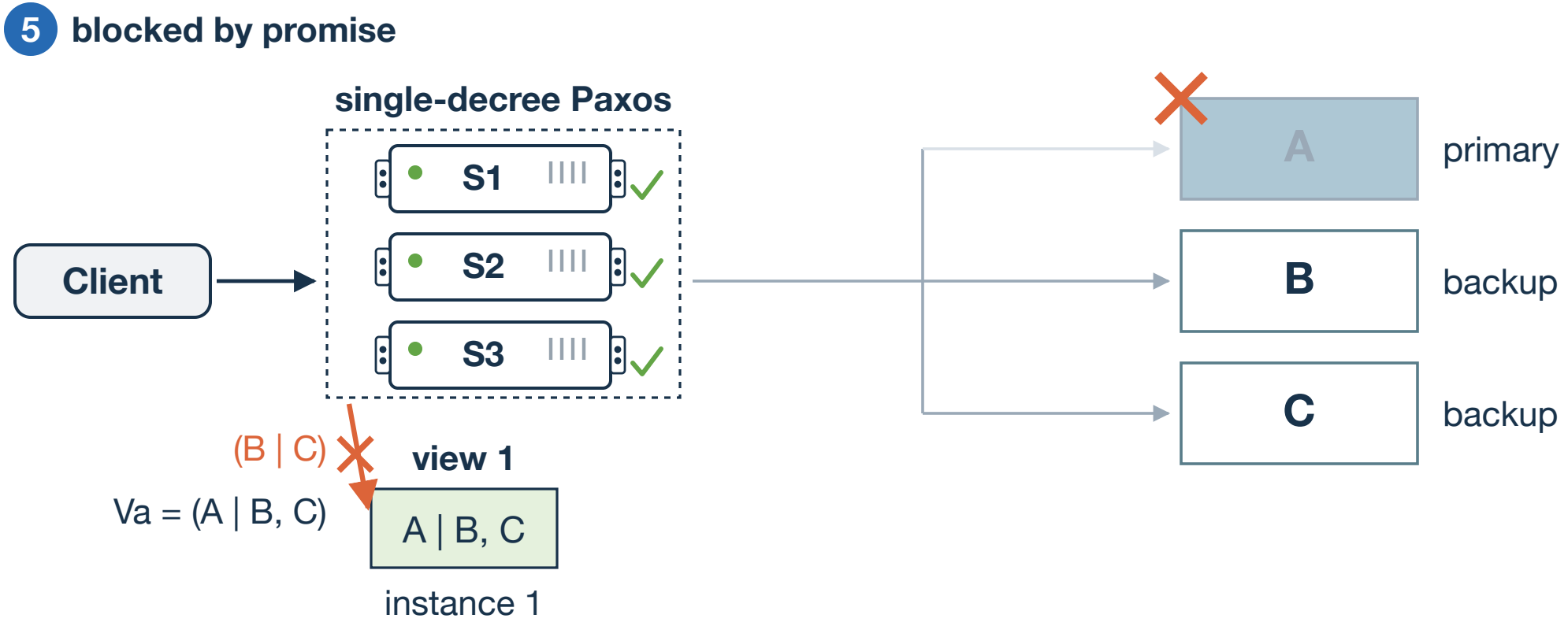


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

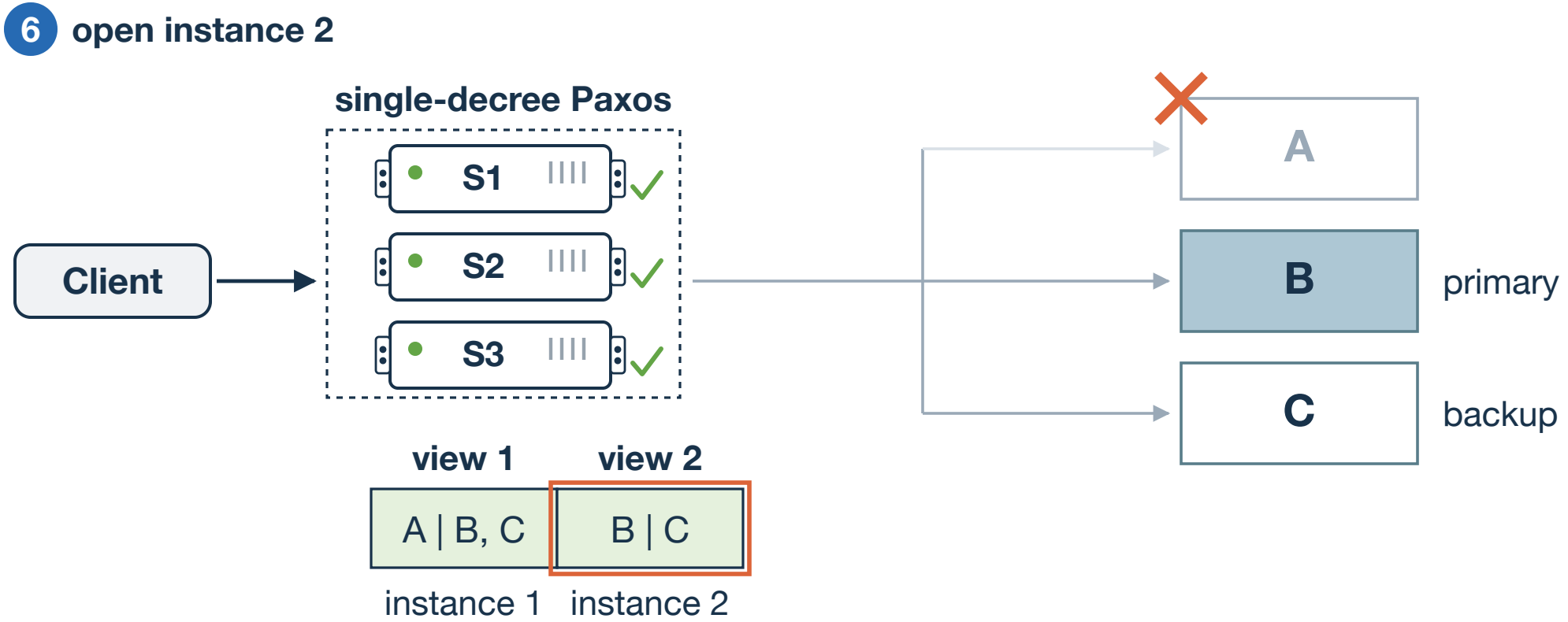


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

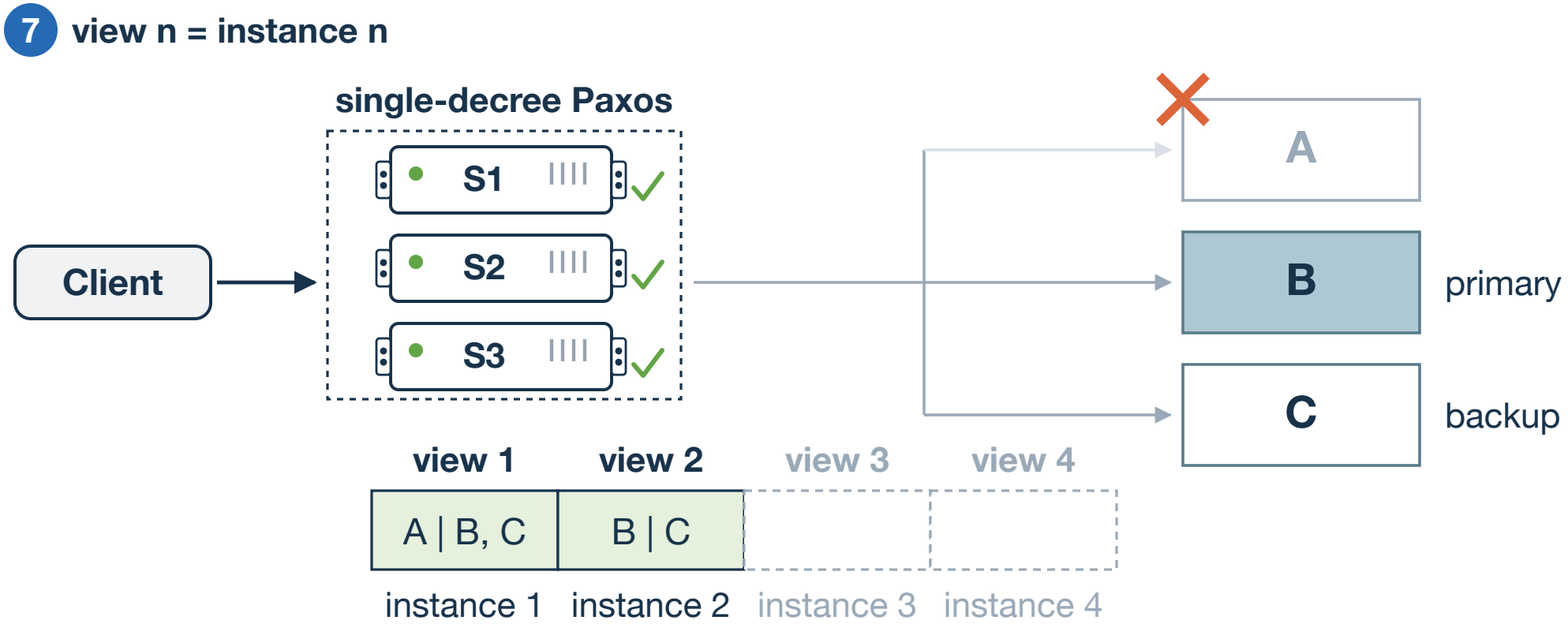


A view is written as primary | backups

Case study: Using Paxos to implement a view server (VS)

The VS decides who is primary in the primary-backup replication:

- Upon a primary/backup failure, it needs to decide the new primary/backup
- Question: is the current Paxos able to implement it?

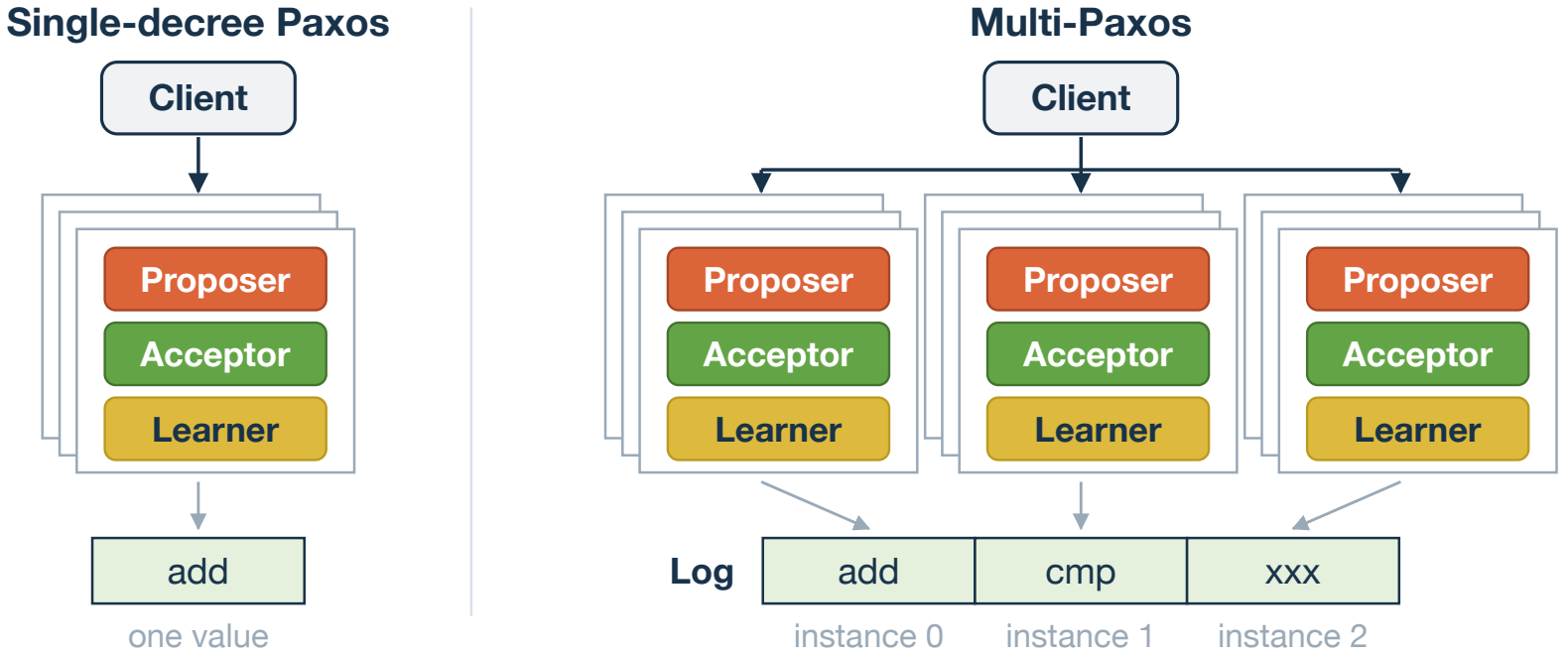


A view is written as primary | backups

Single-decree Paxos vs. Multi-Paxos

Multi-Paxos: agree on a sequence of values

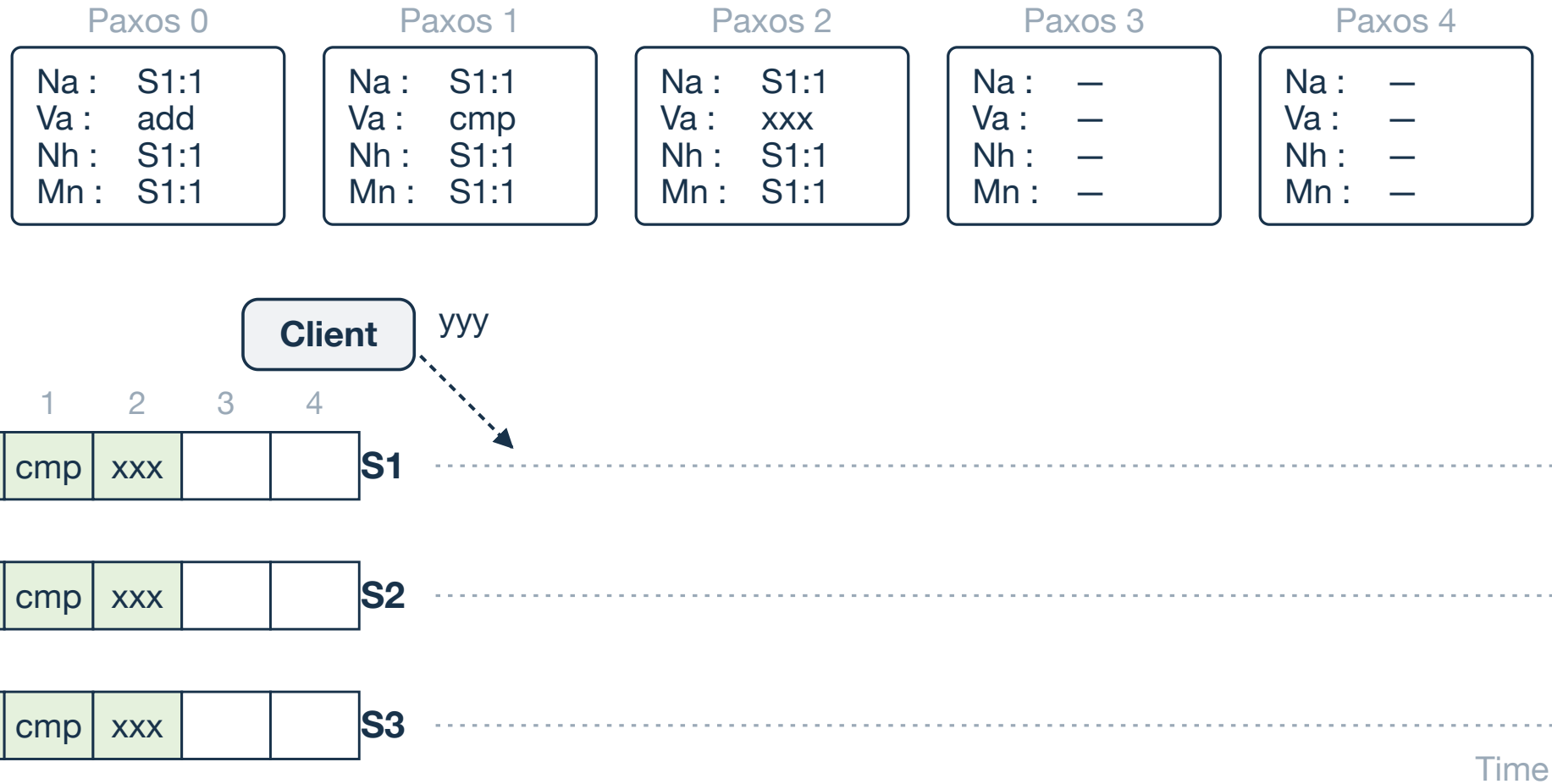
- In our previous example, one view = one Paxos instance, and the view number is the instance number
- Each instance is a log entry in a replicated state machine, so Multi-Paxos is able to support arbitrary replicated computation



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

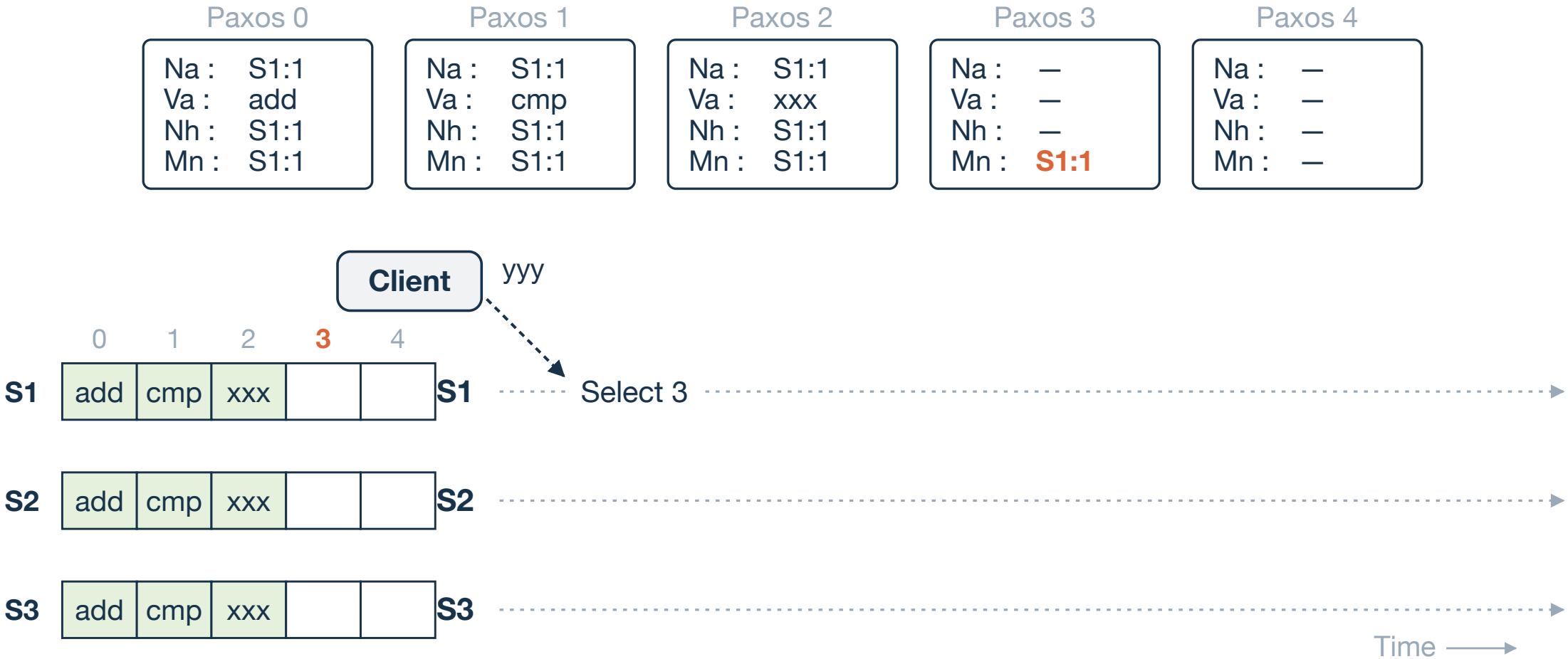
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

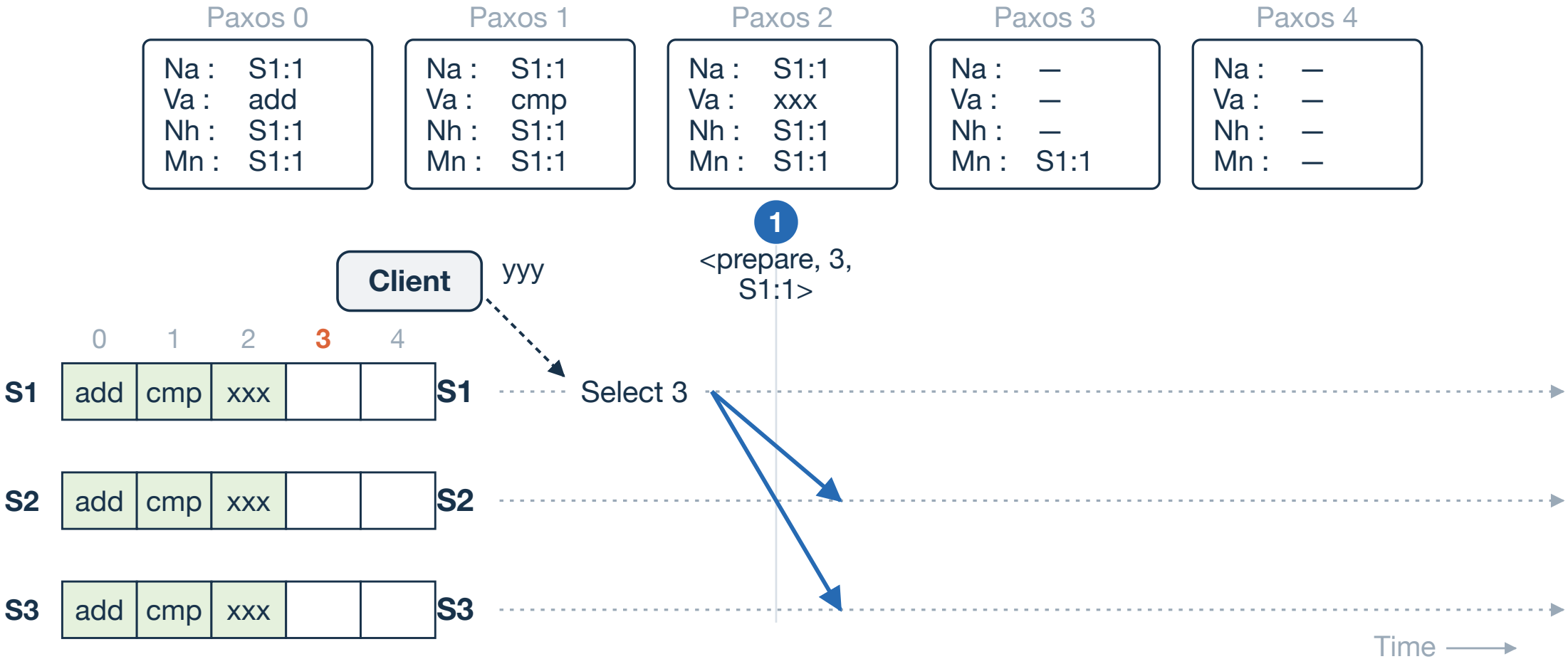
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

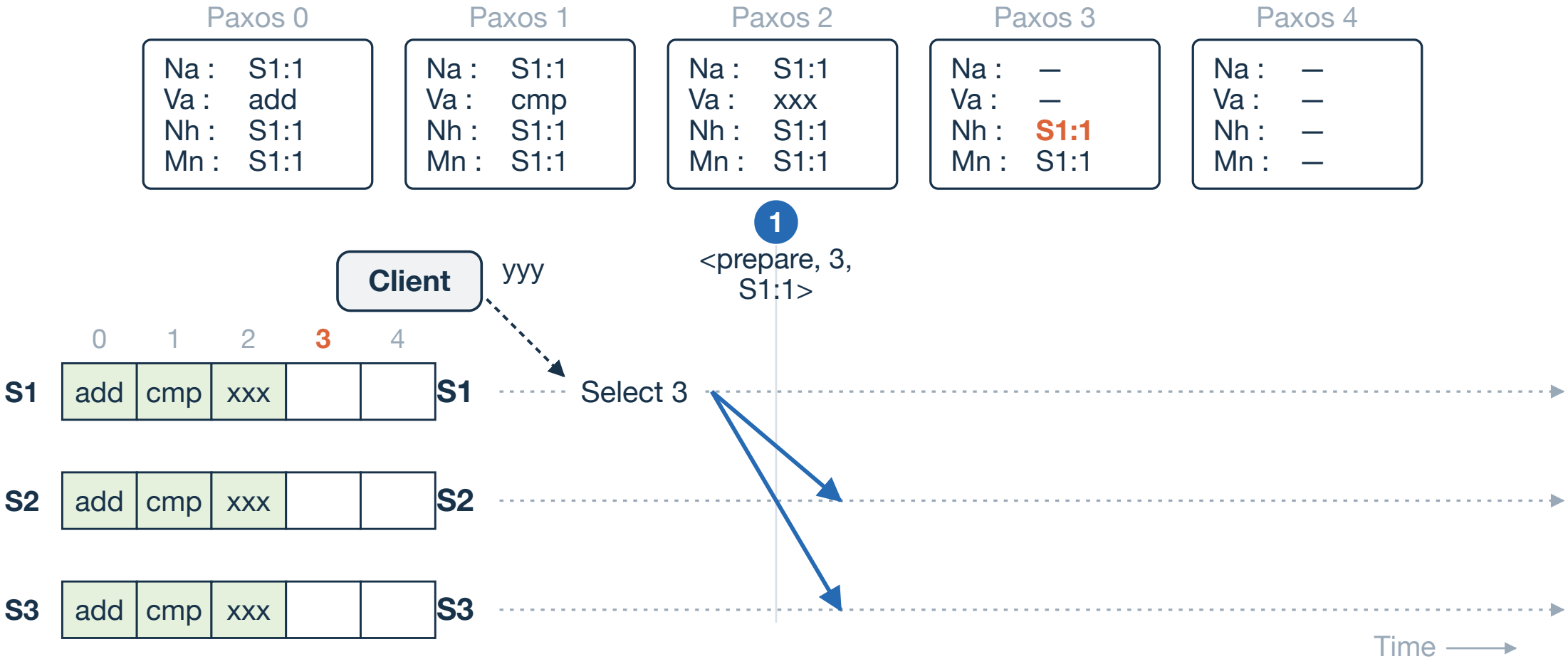
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

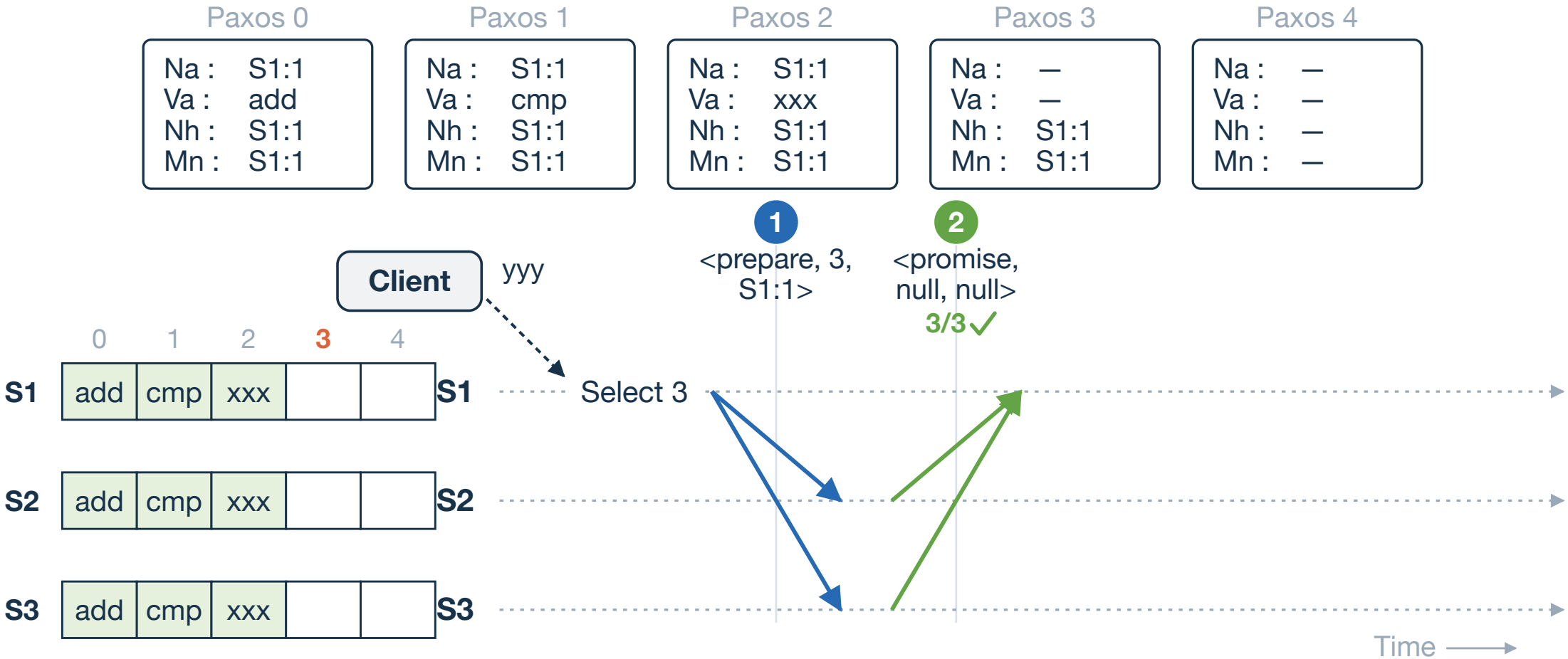
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

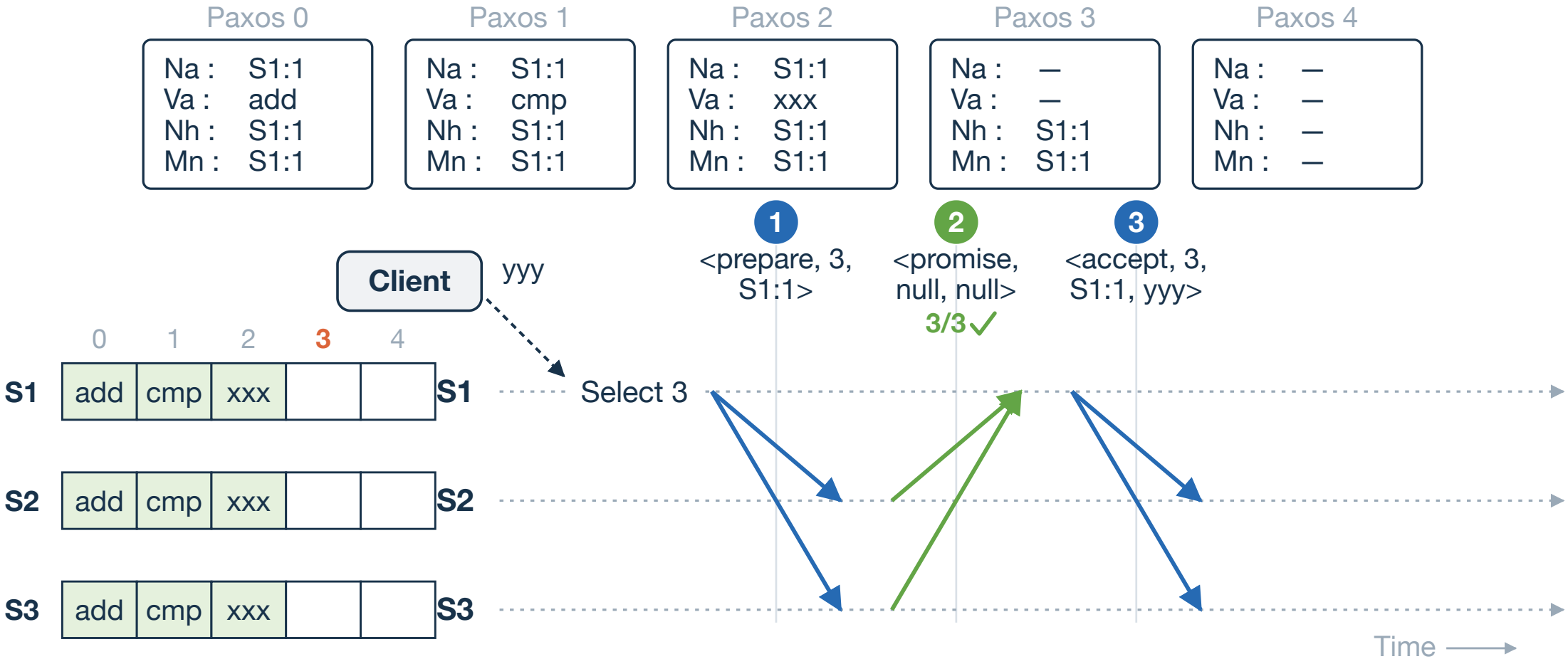
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

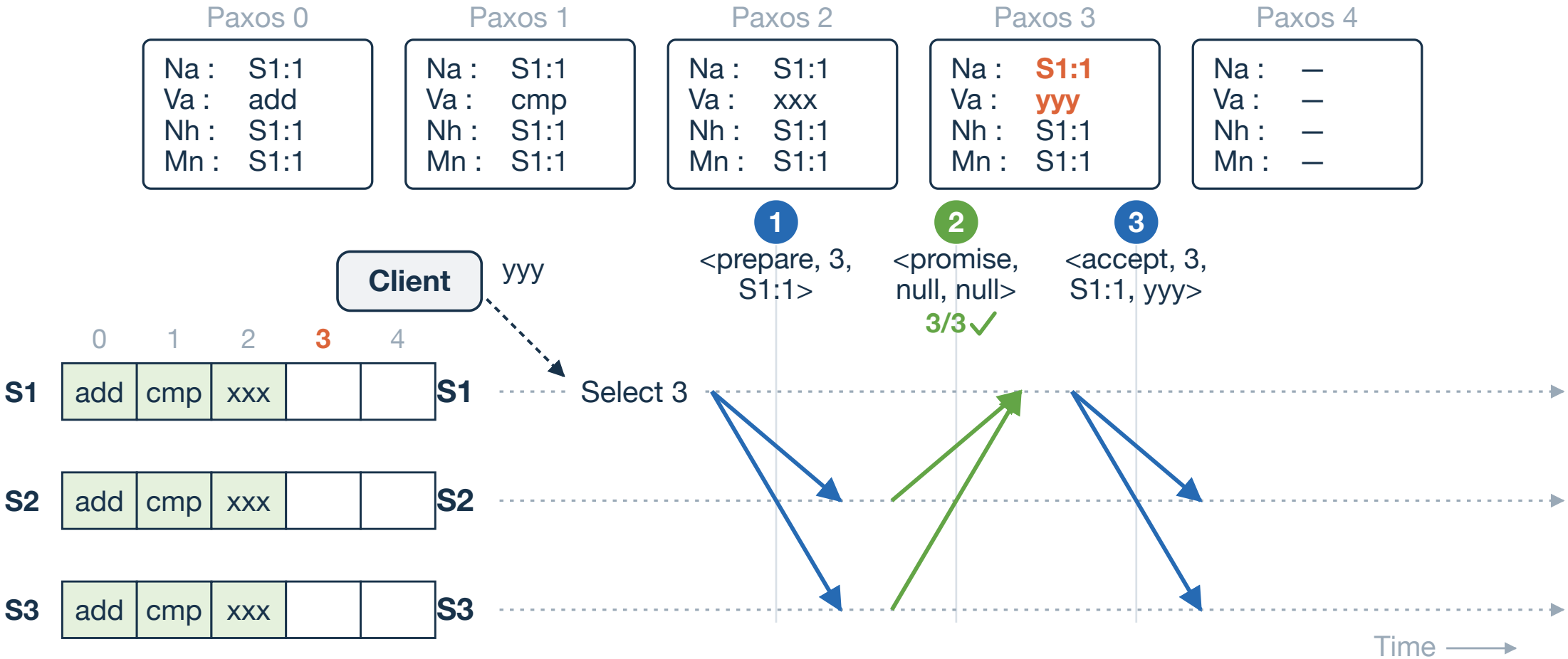
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

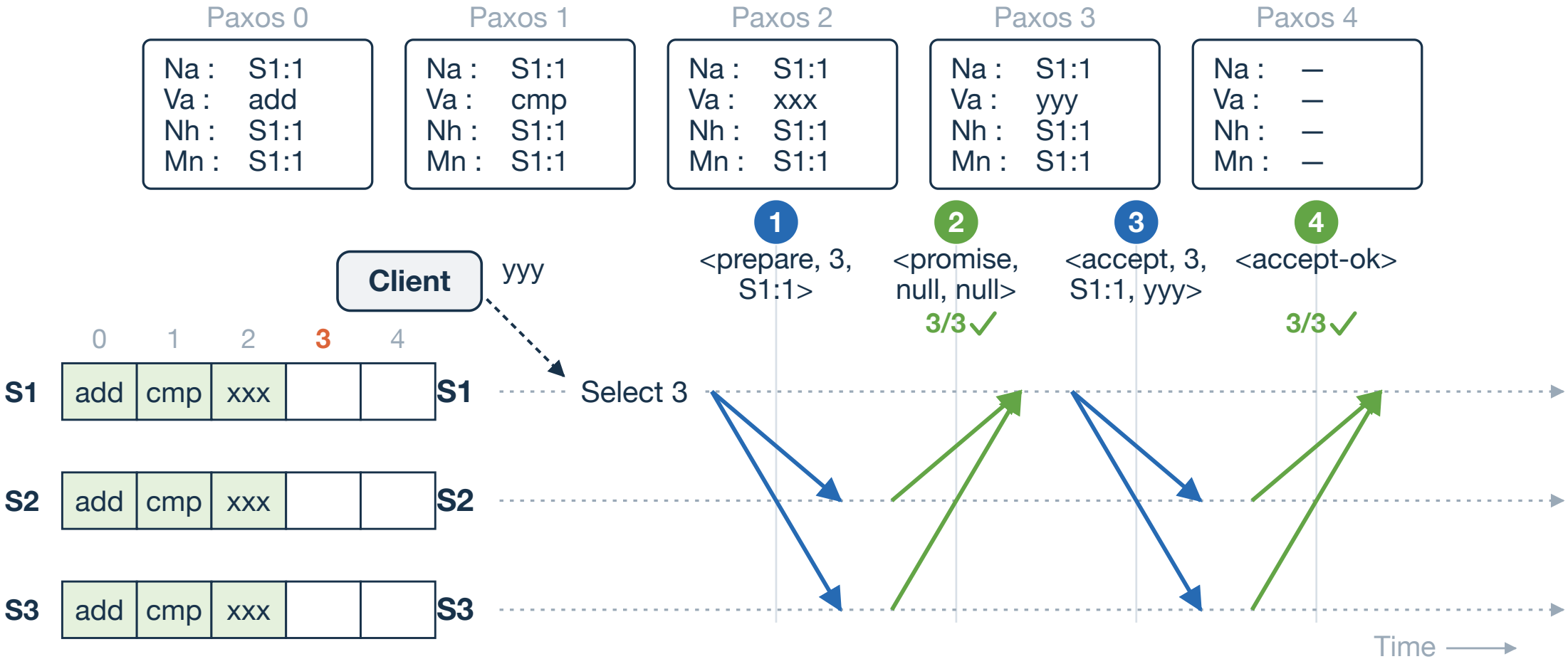
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

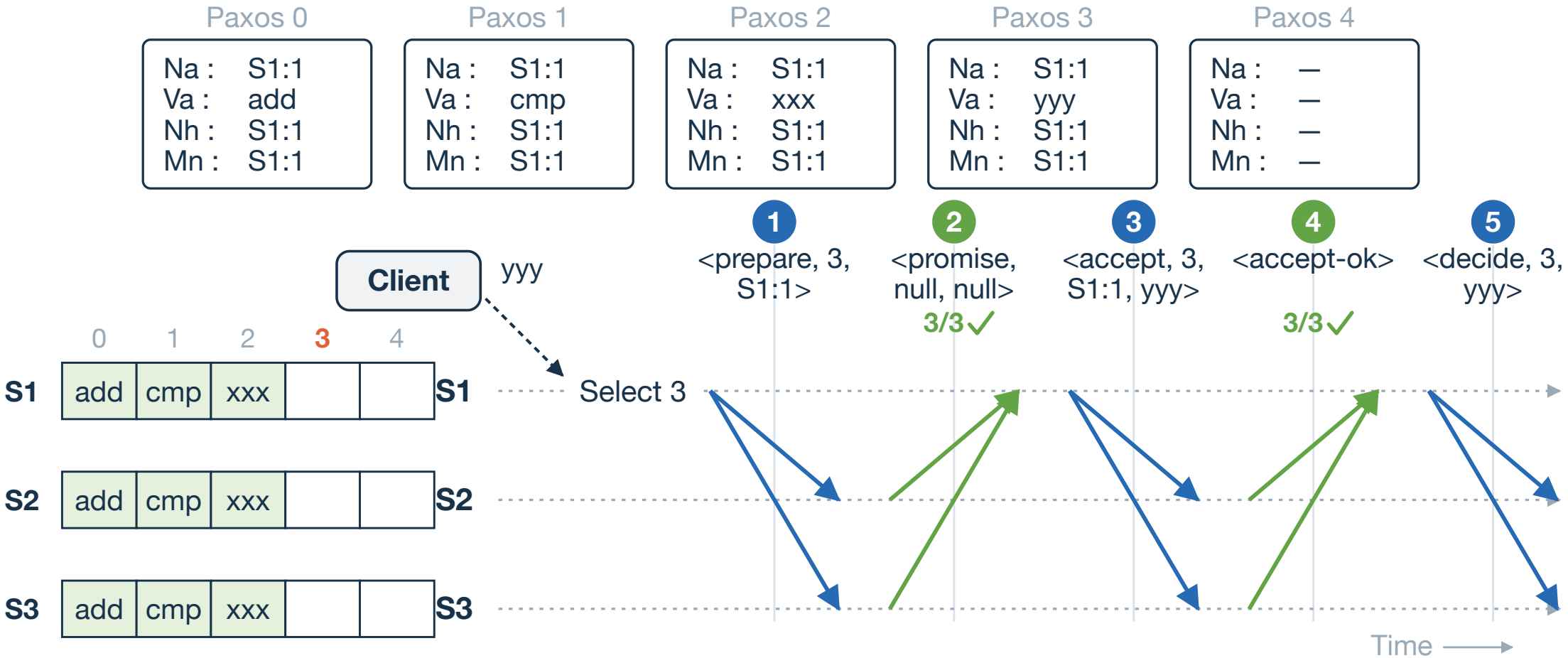
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

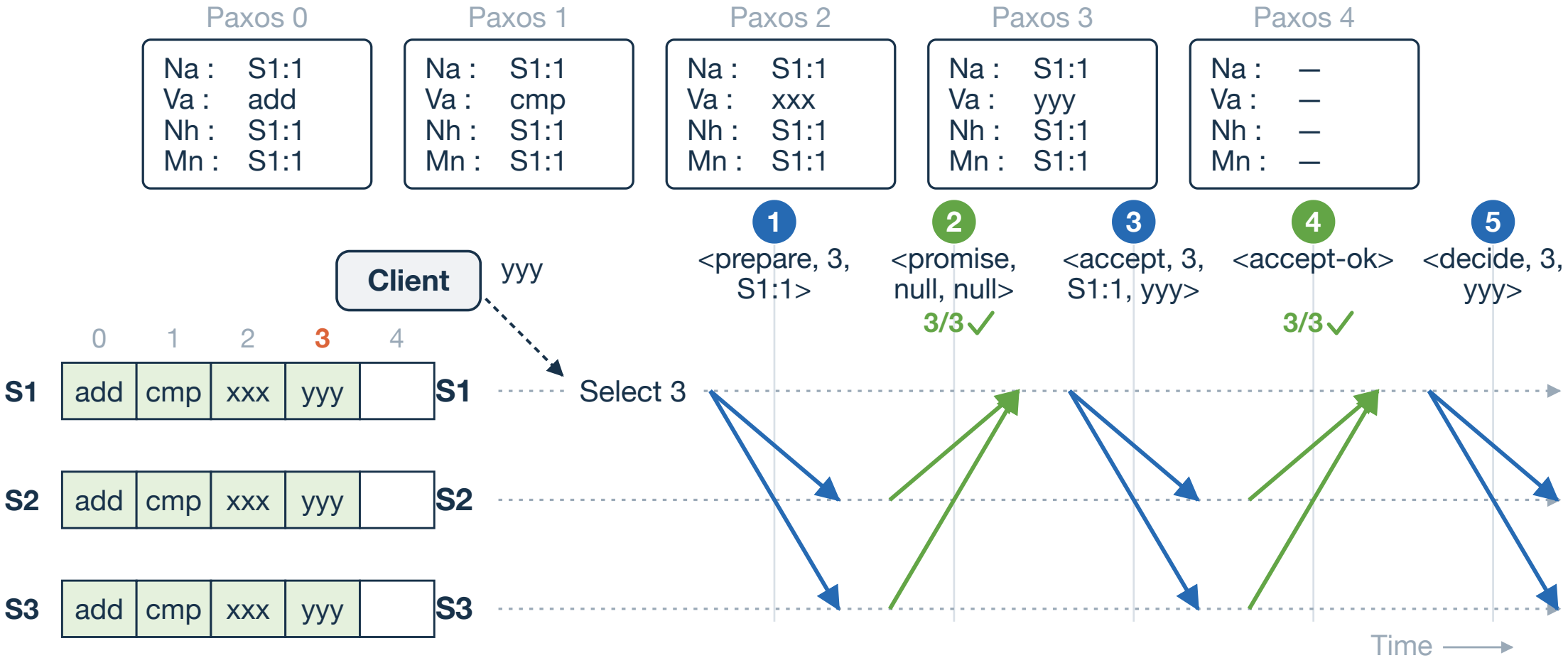
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: Suppose we want to append a new log entry

S1 first decides which instance the entry goes to, then runs single-decree Paxos on it

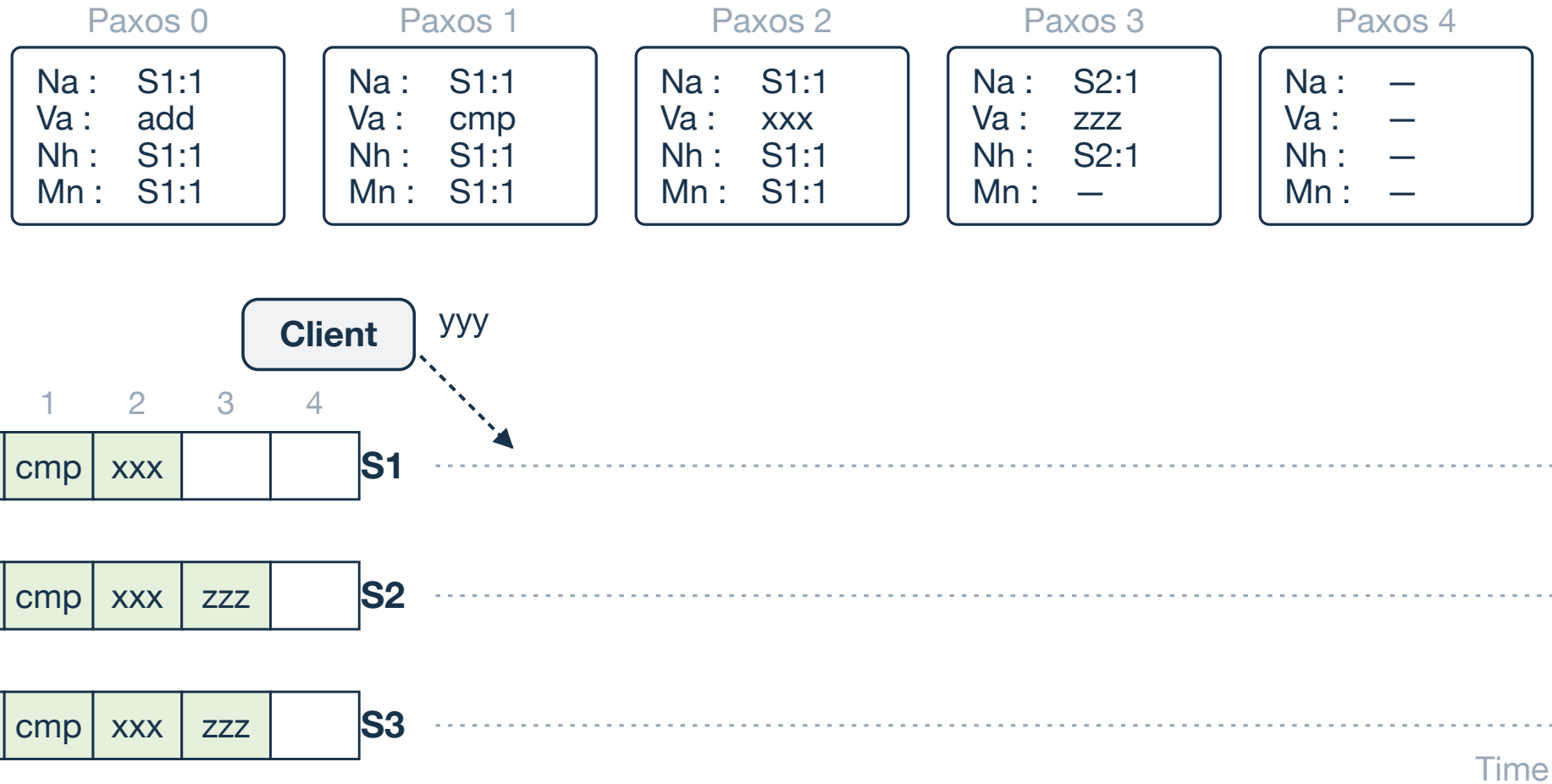
- Question: what could happen if instance 3 has already gotten a value?



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

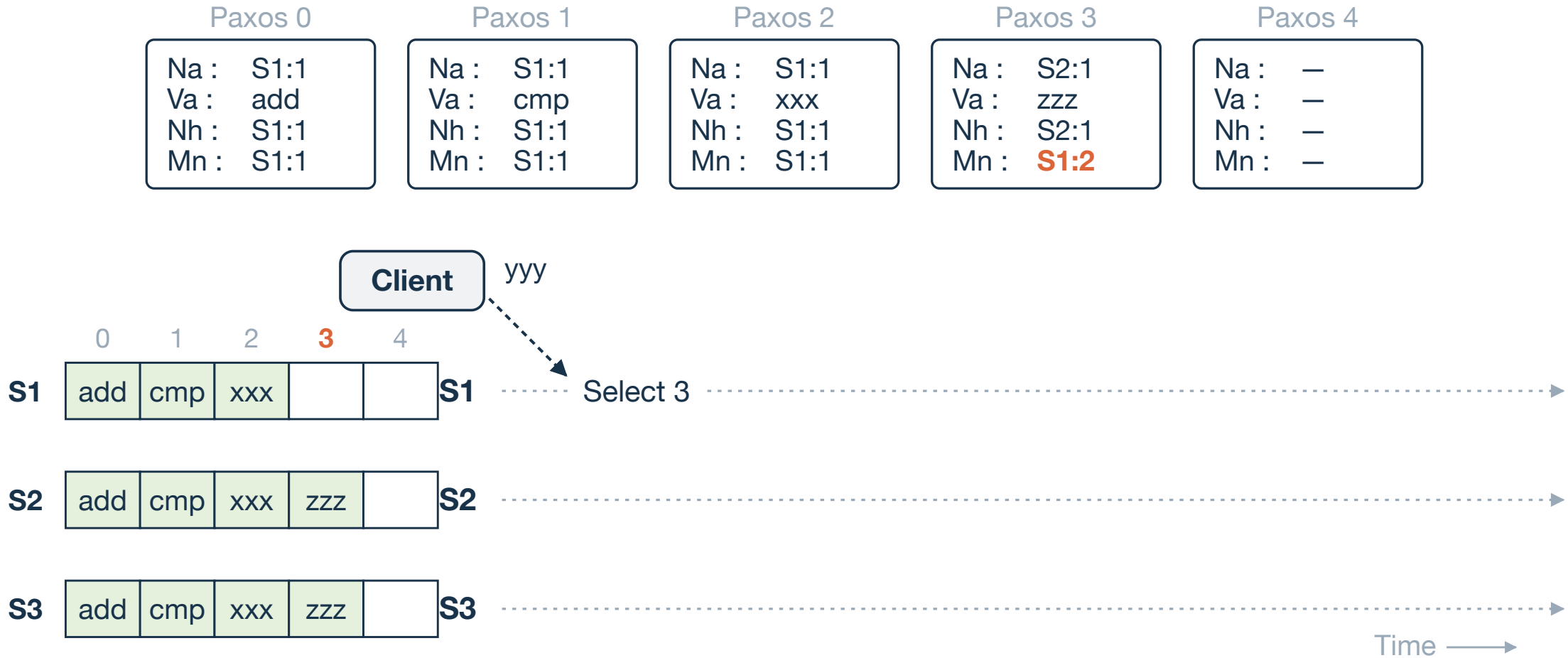
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

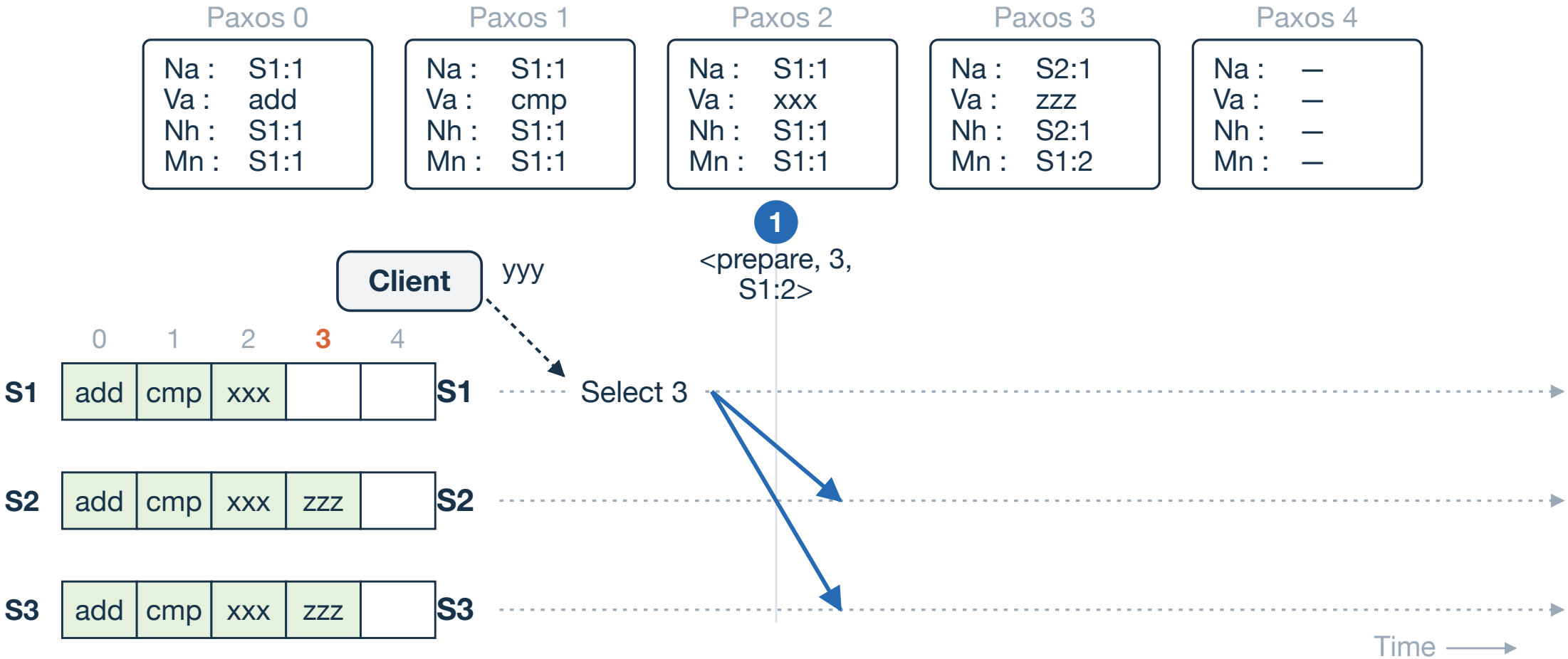
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

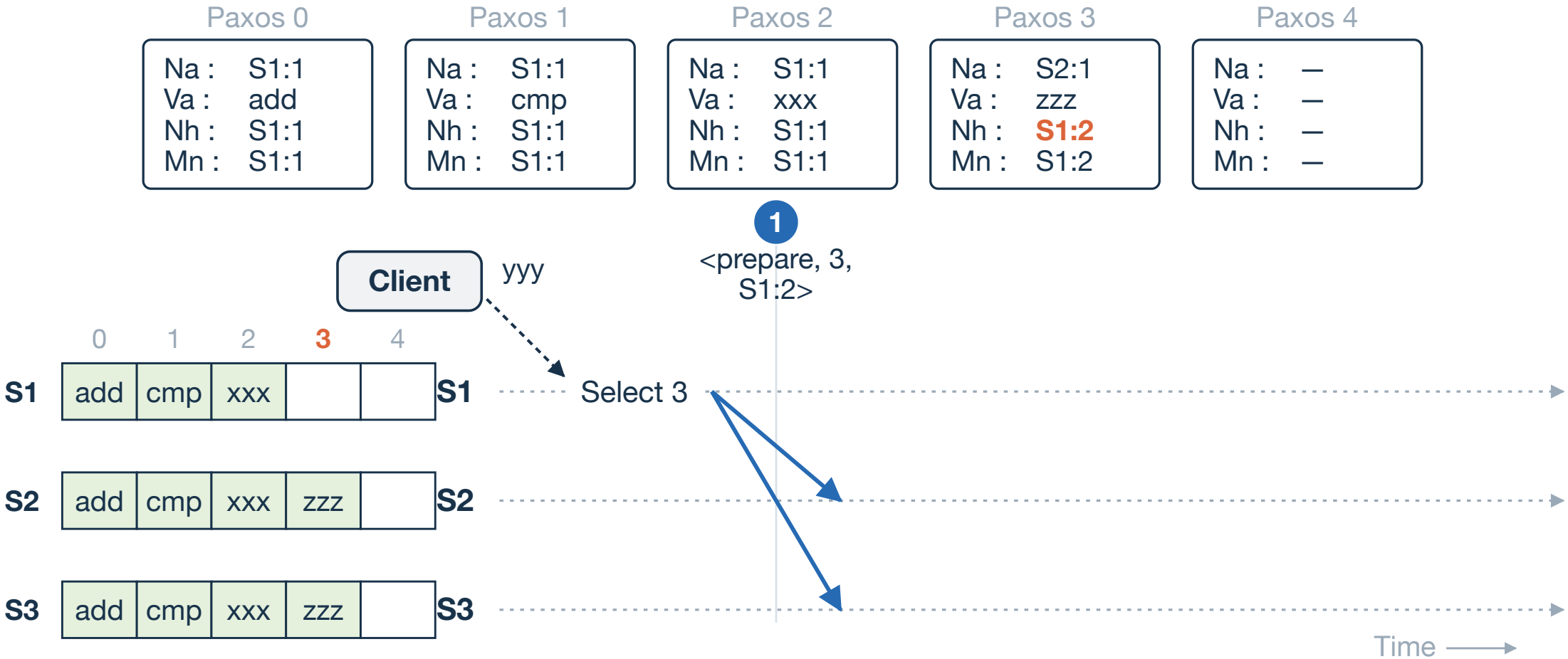
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

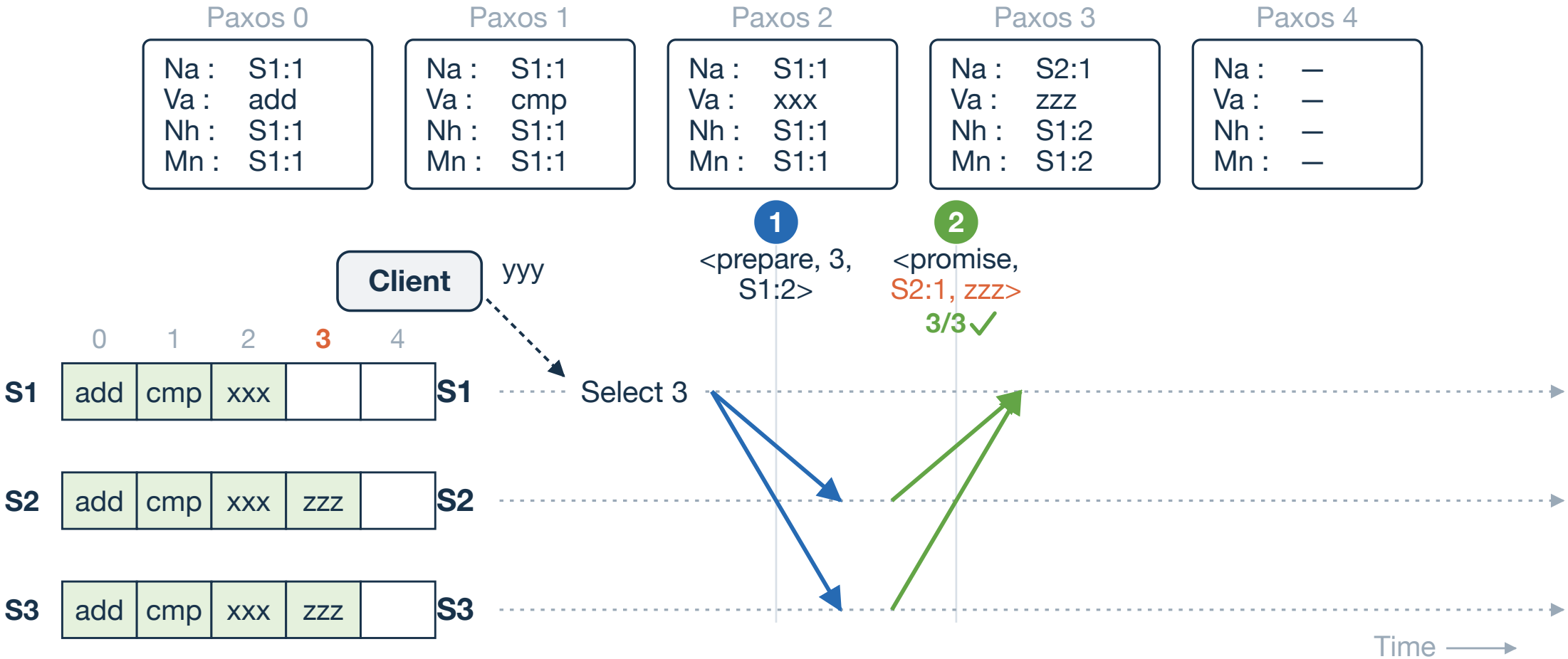
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

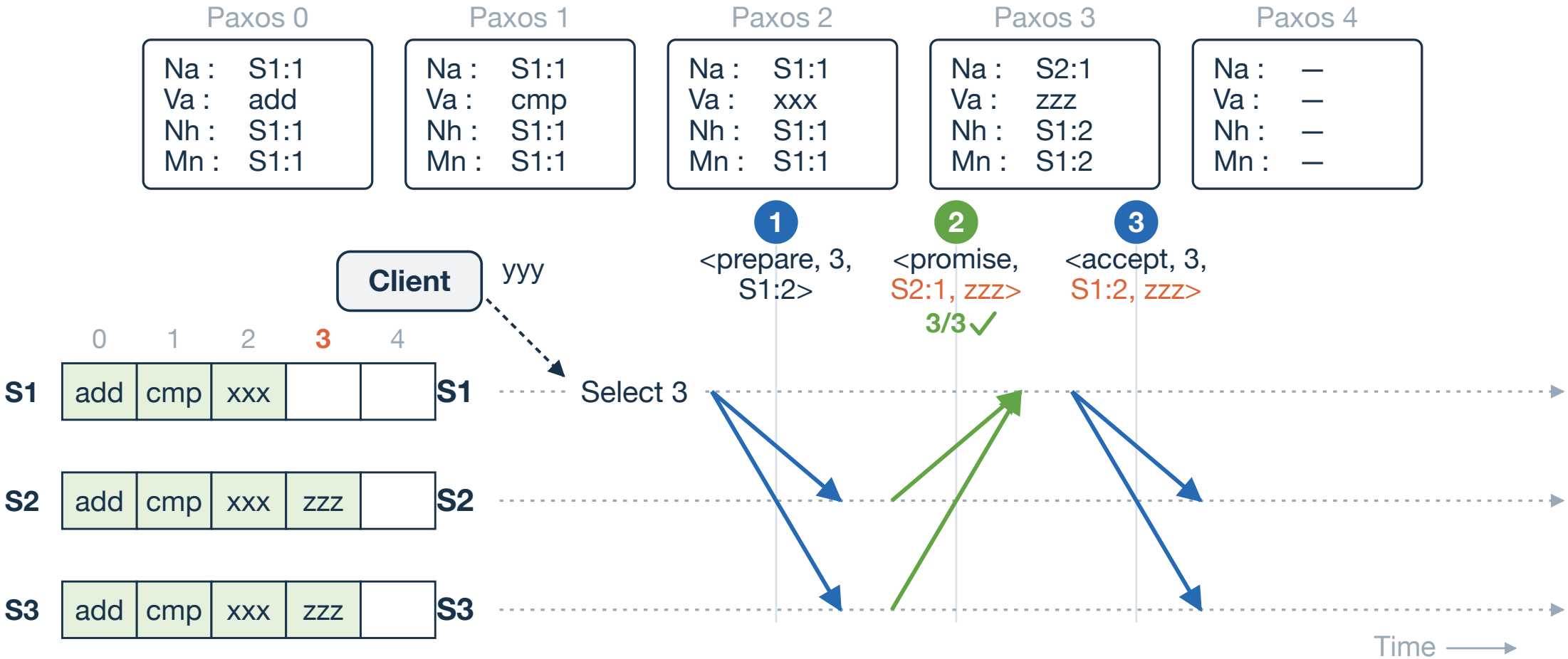
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

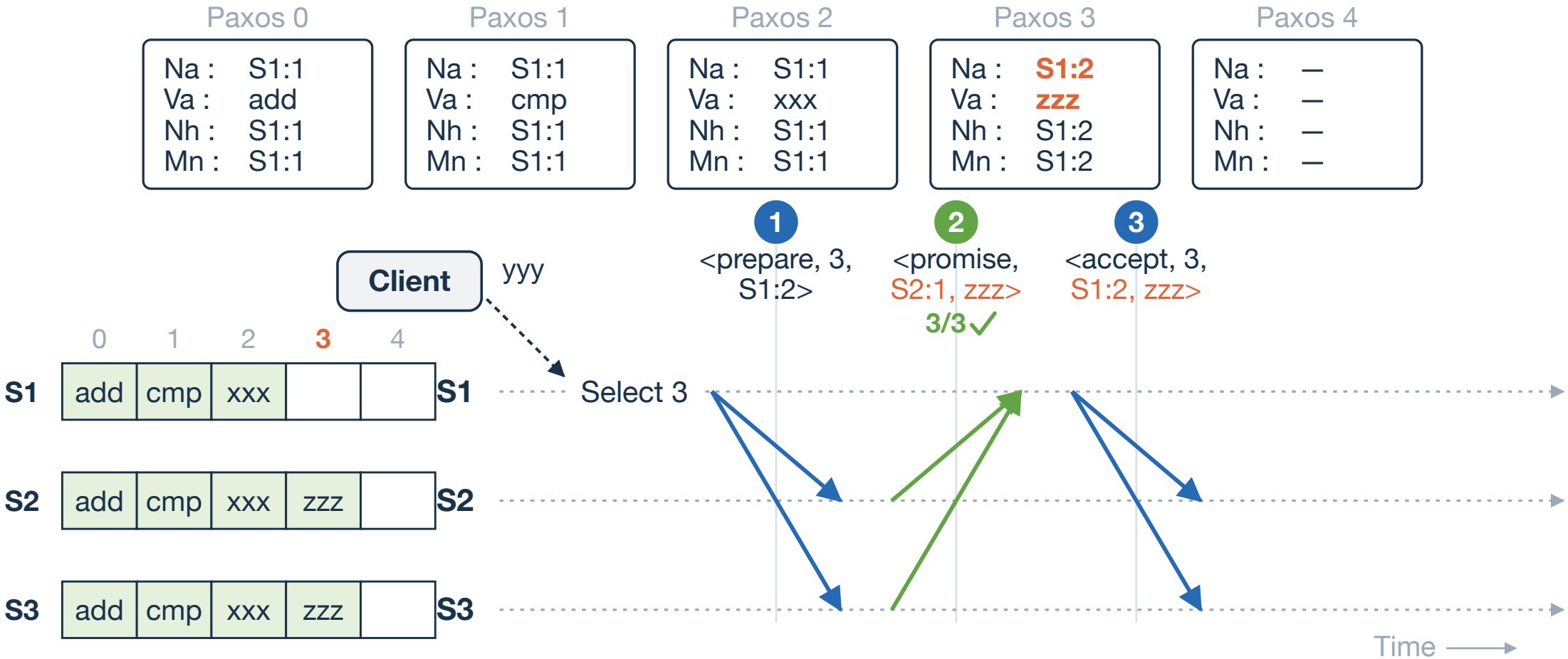
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

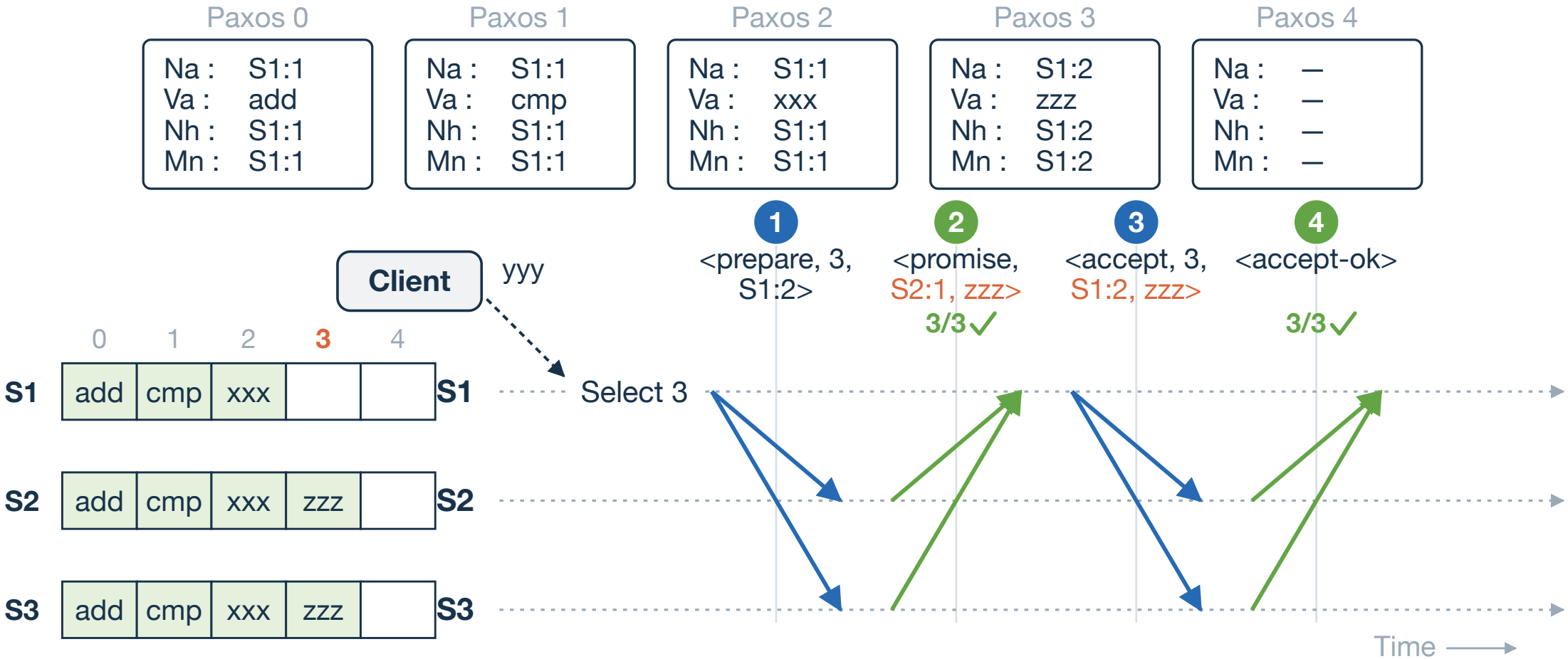
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

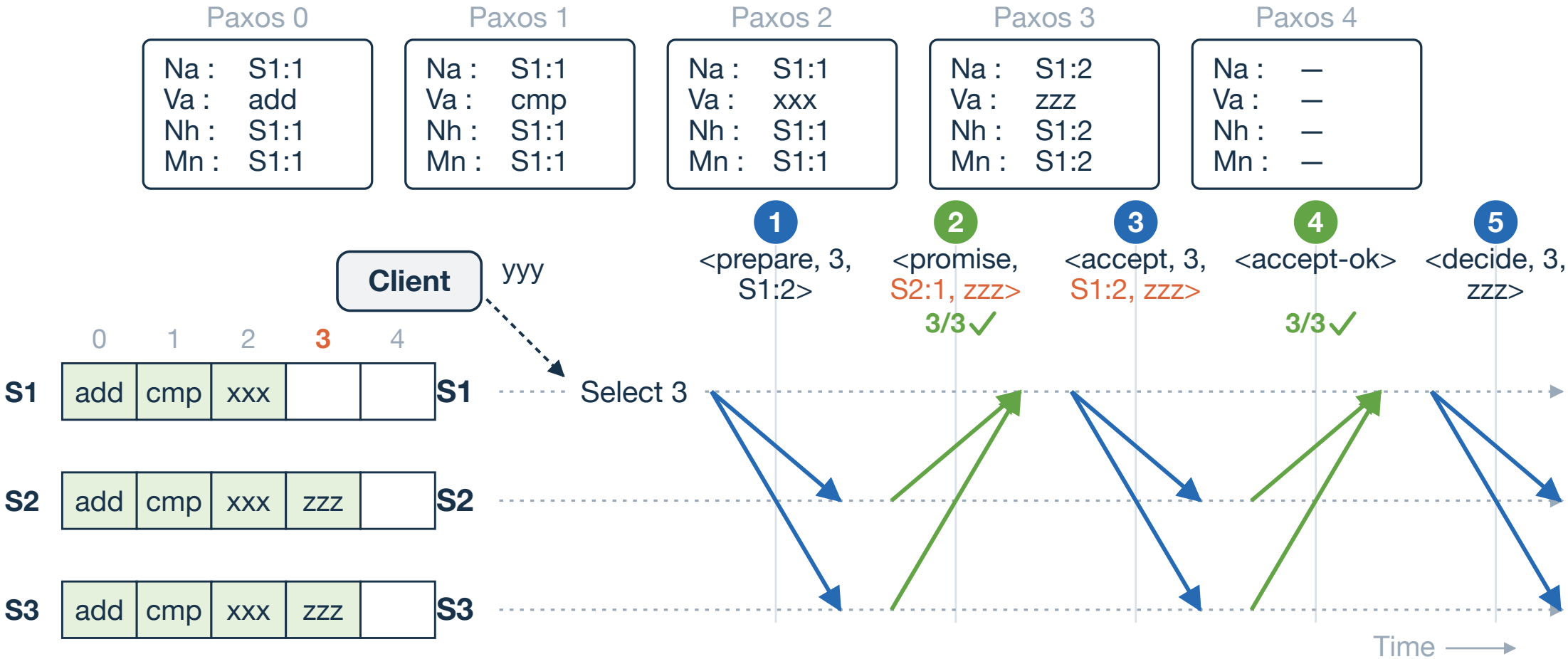
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

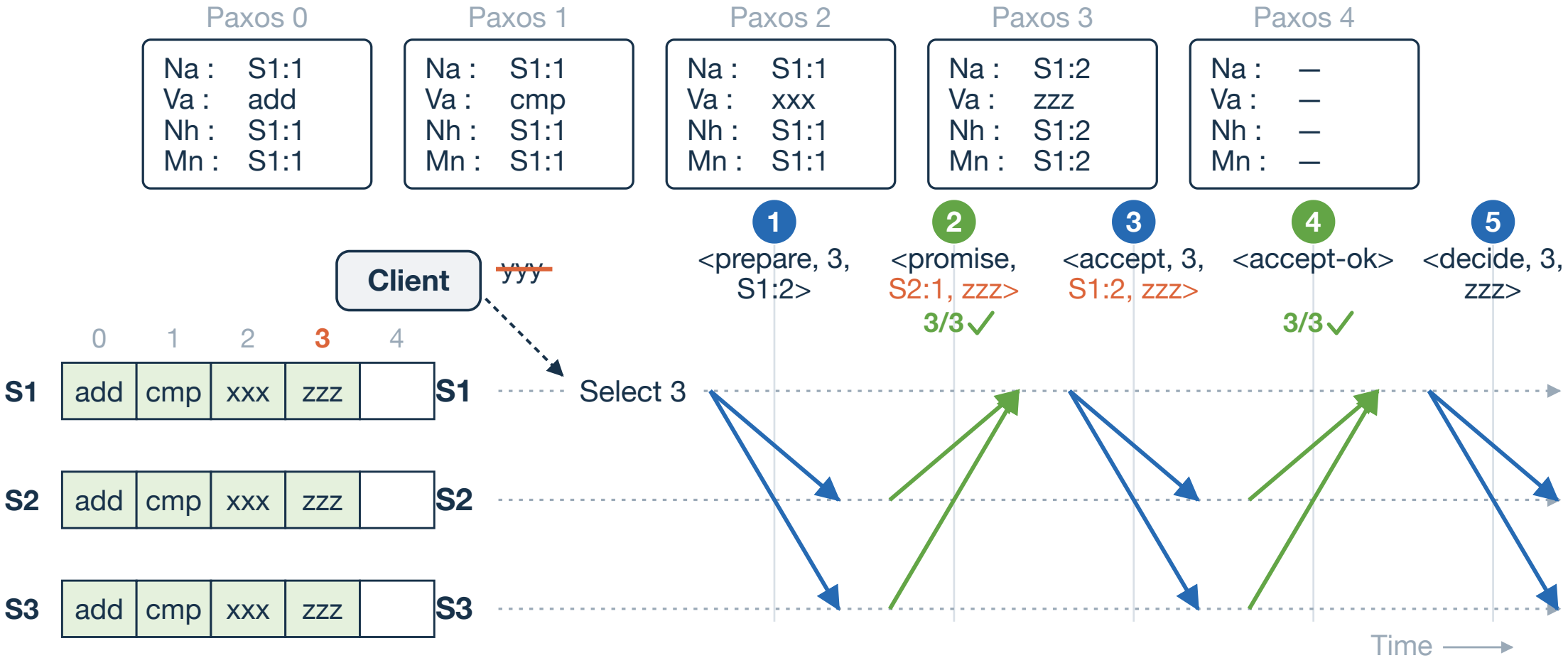
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: What if the instance has been taken?

S2 and S3 have already accepted another value zzz for instance 3

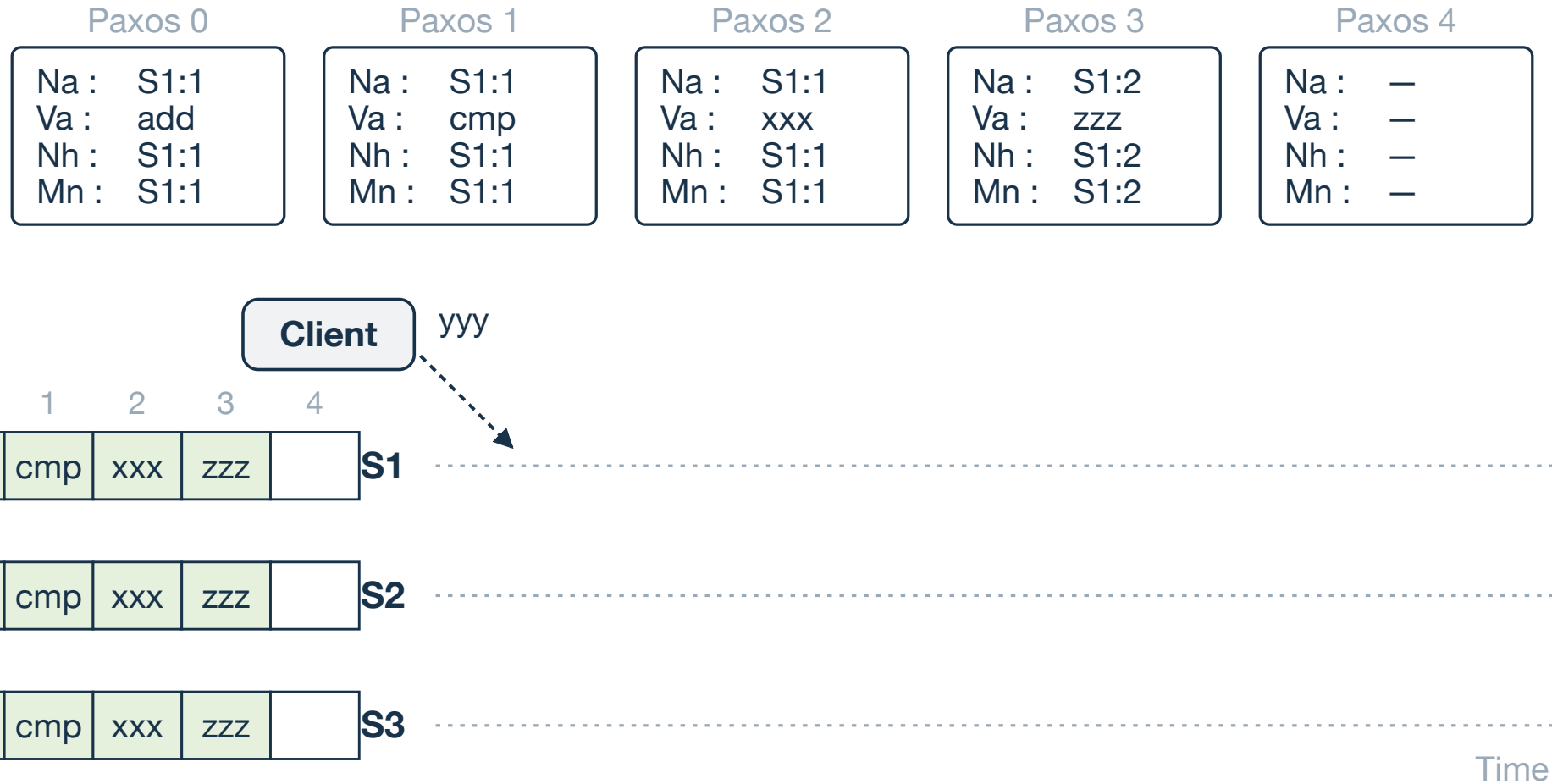
- zzz must be accepted when S1 tries to append a value to instance 3



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

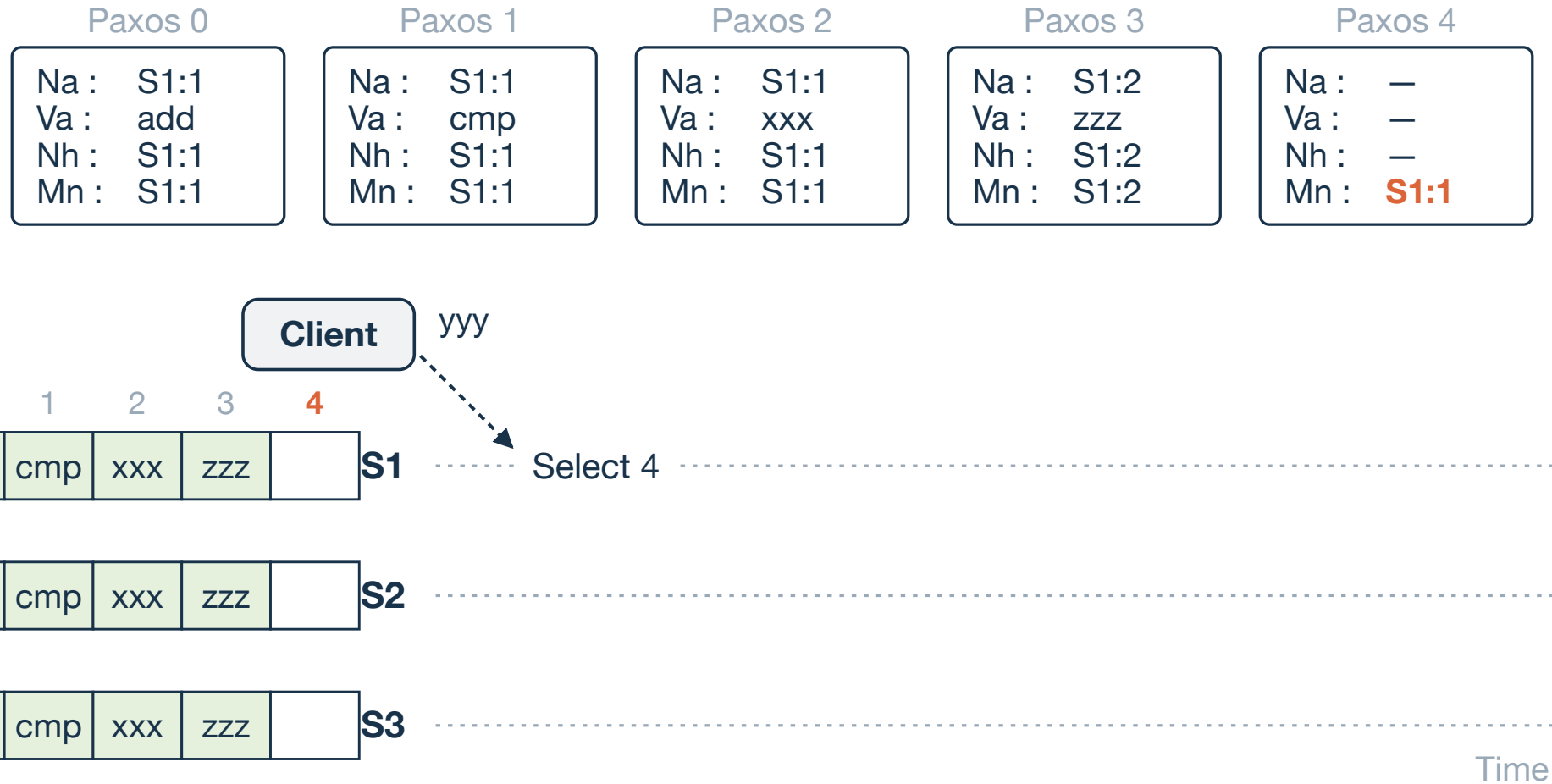
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

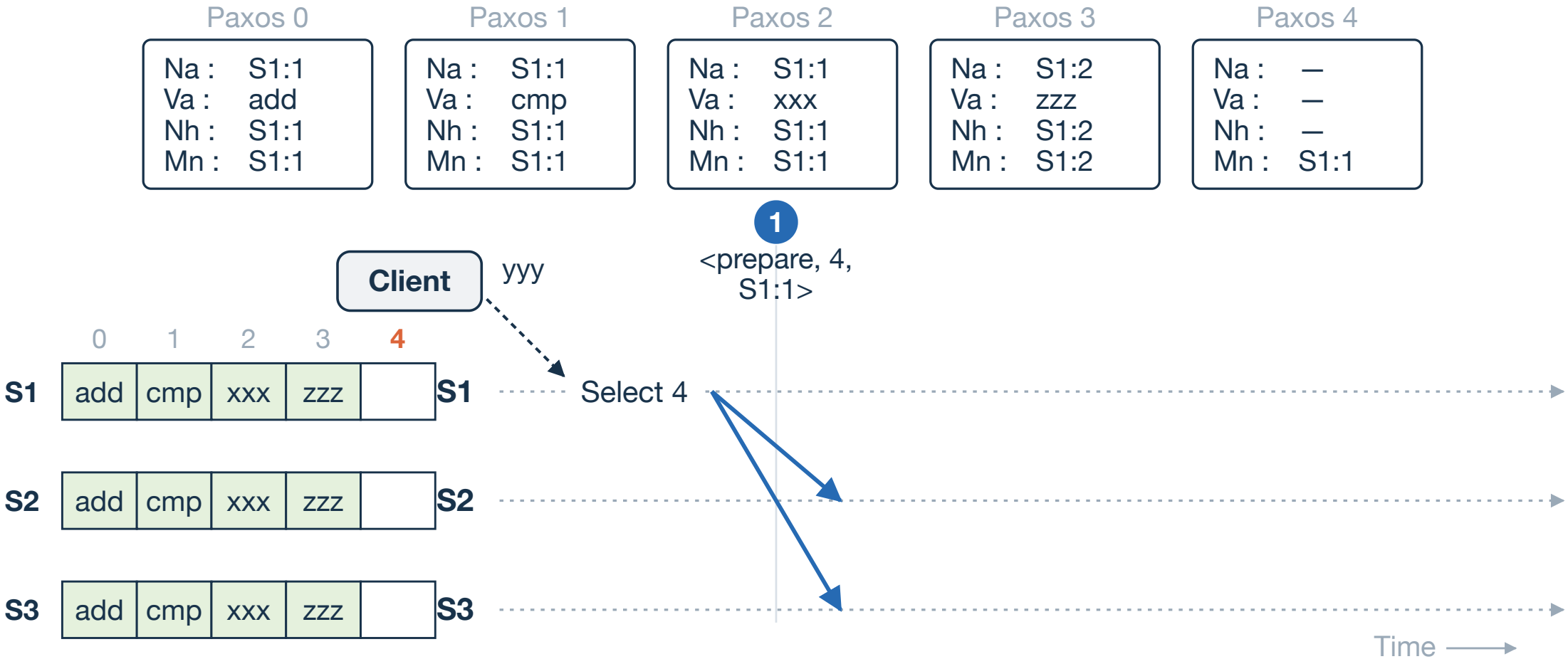
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

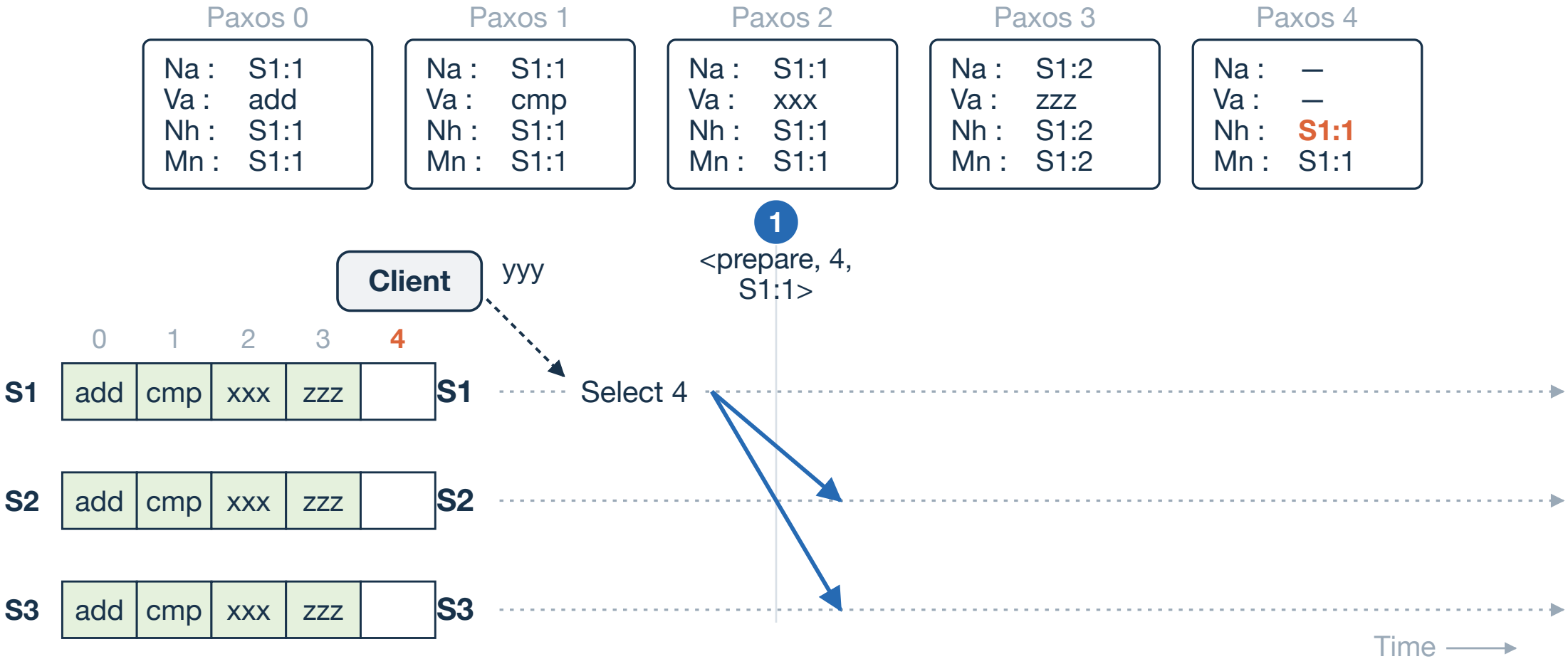
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

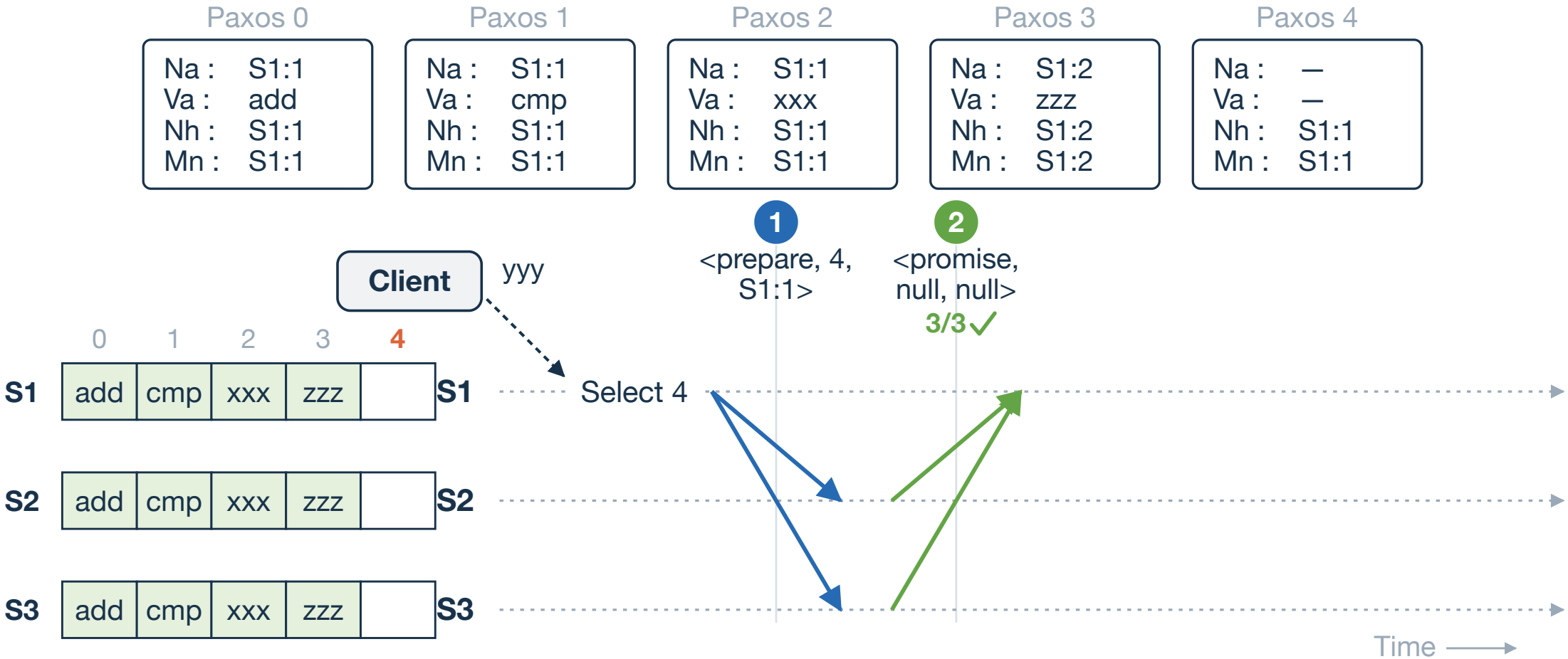
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

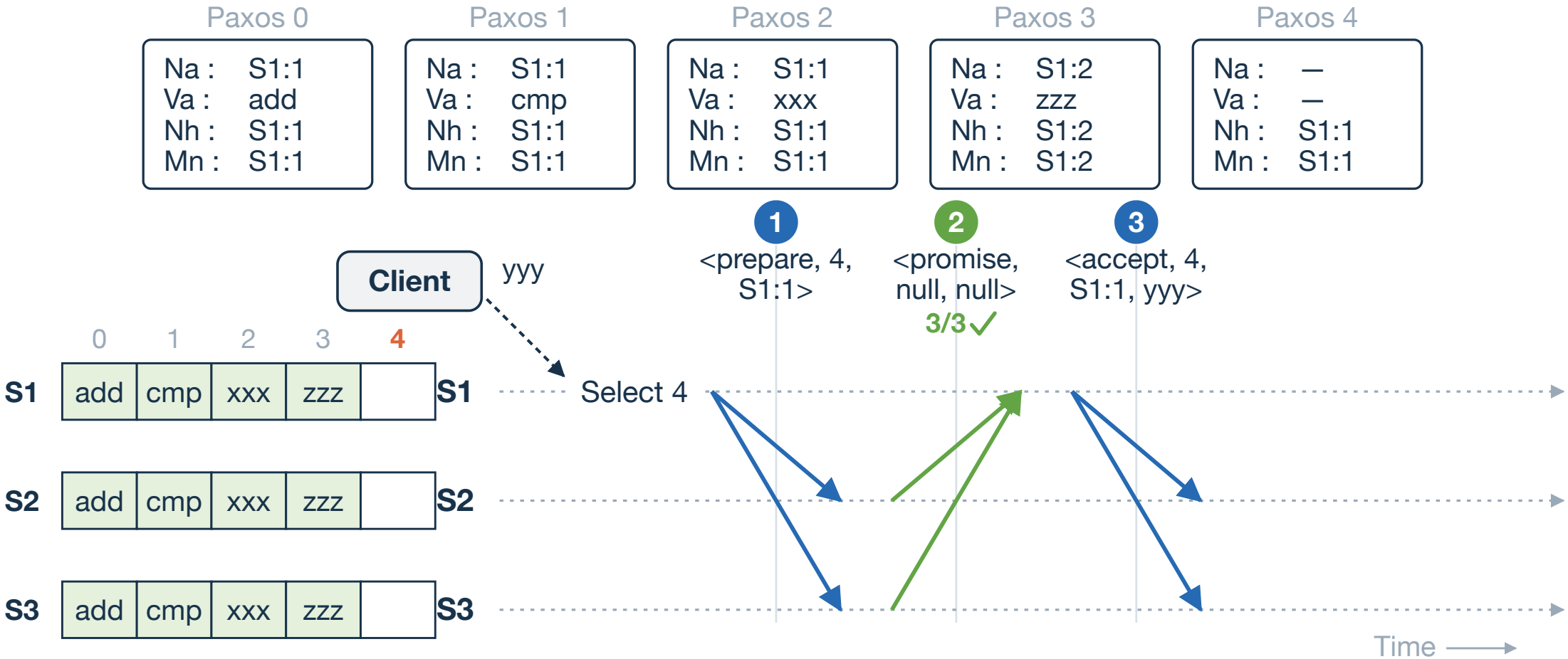
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

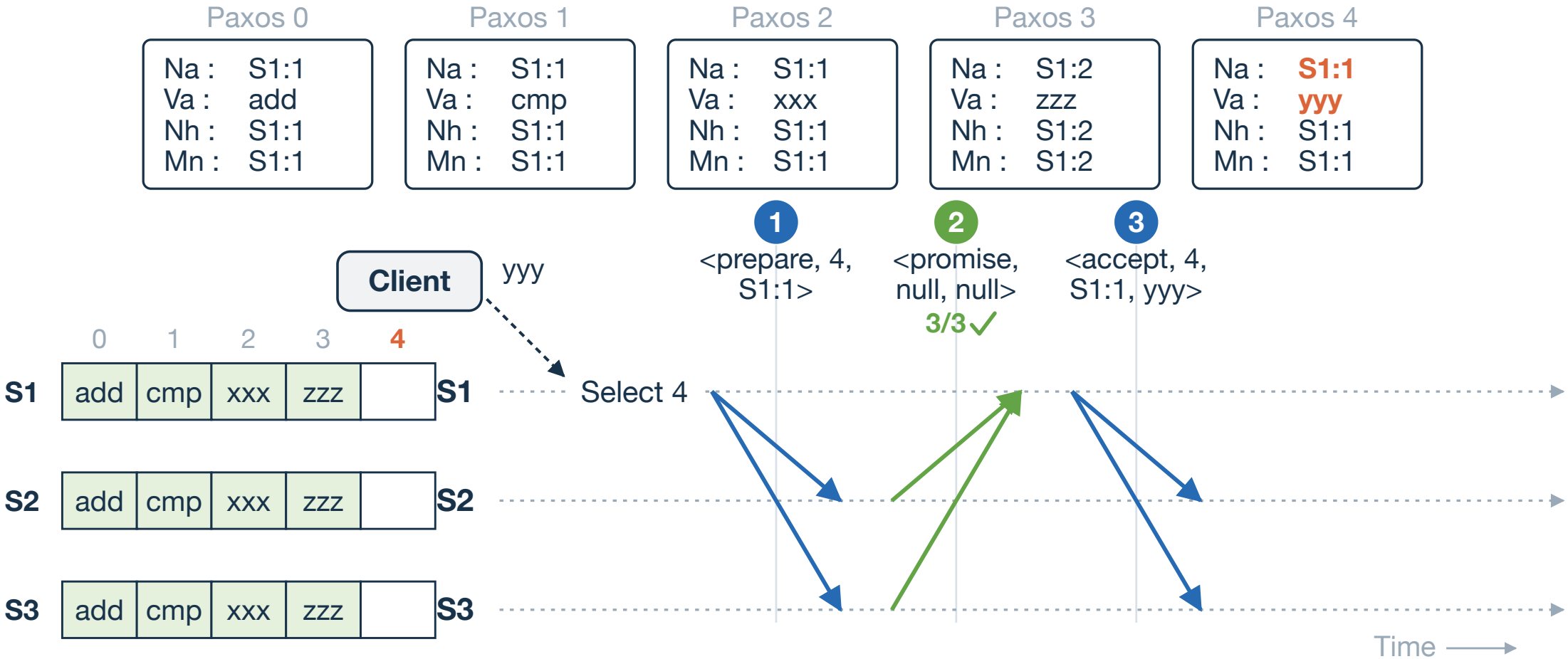
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

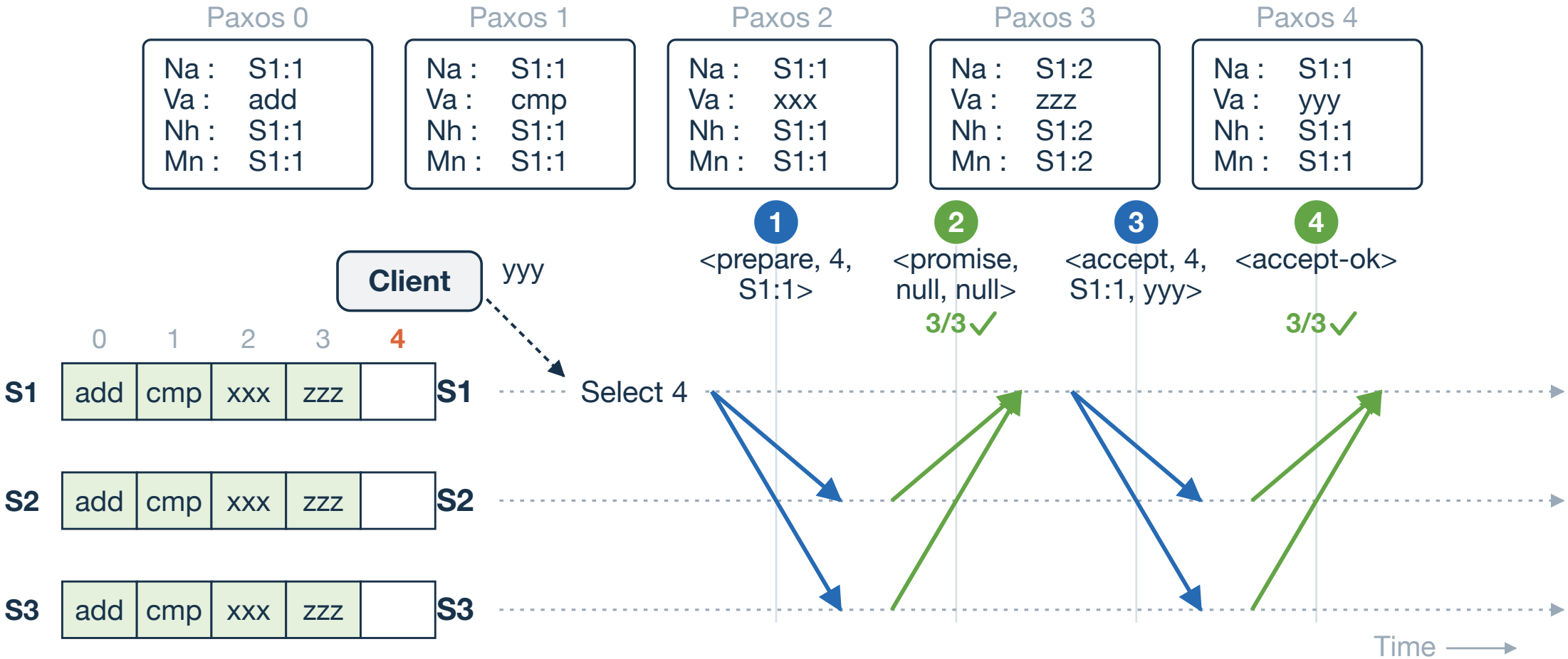
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

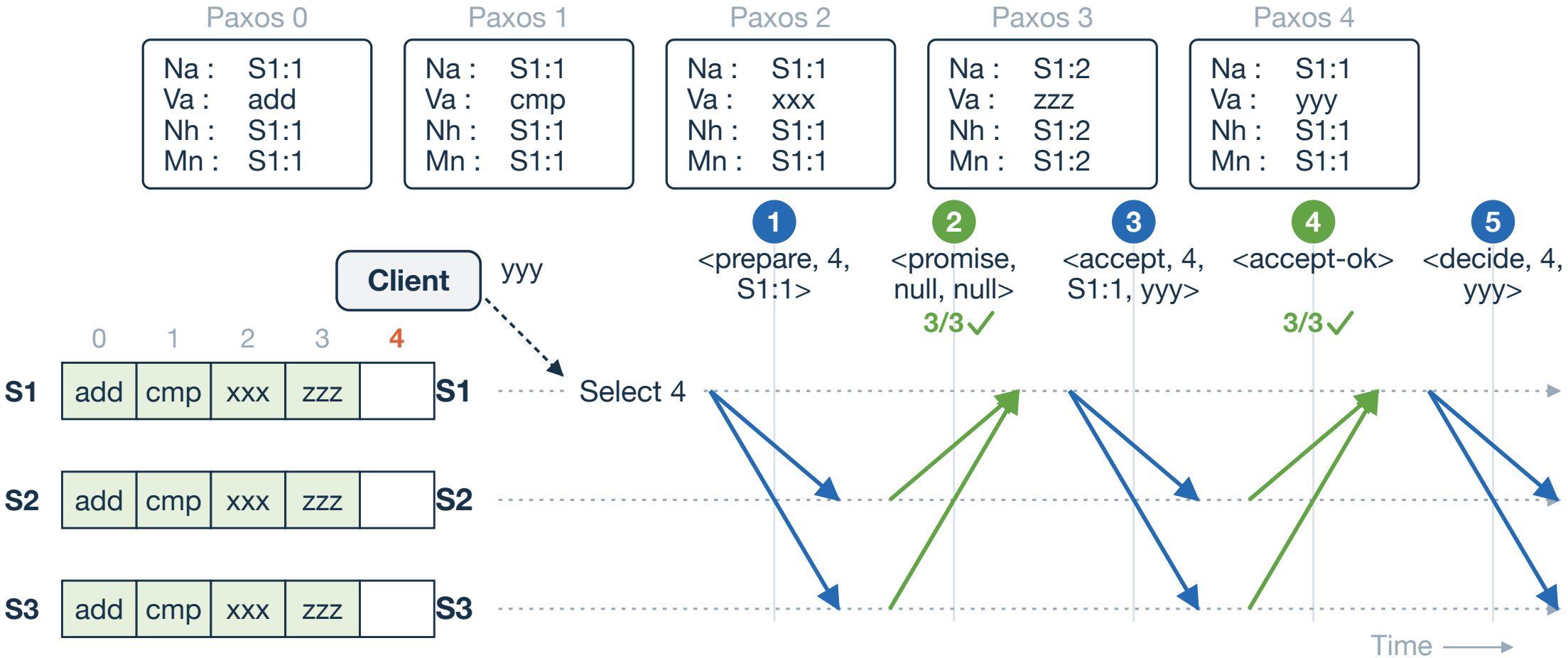
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

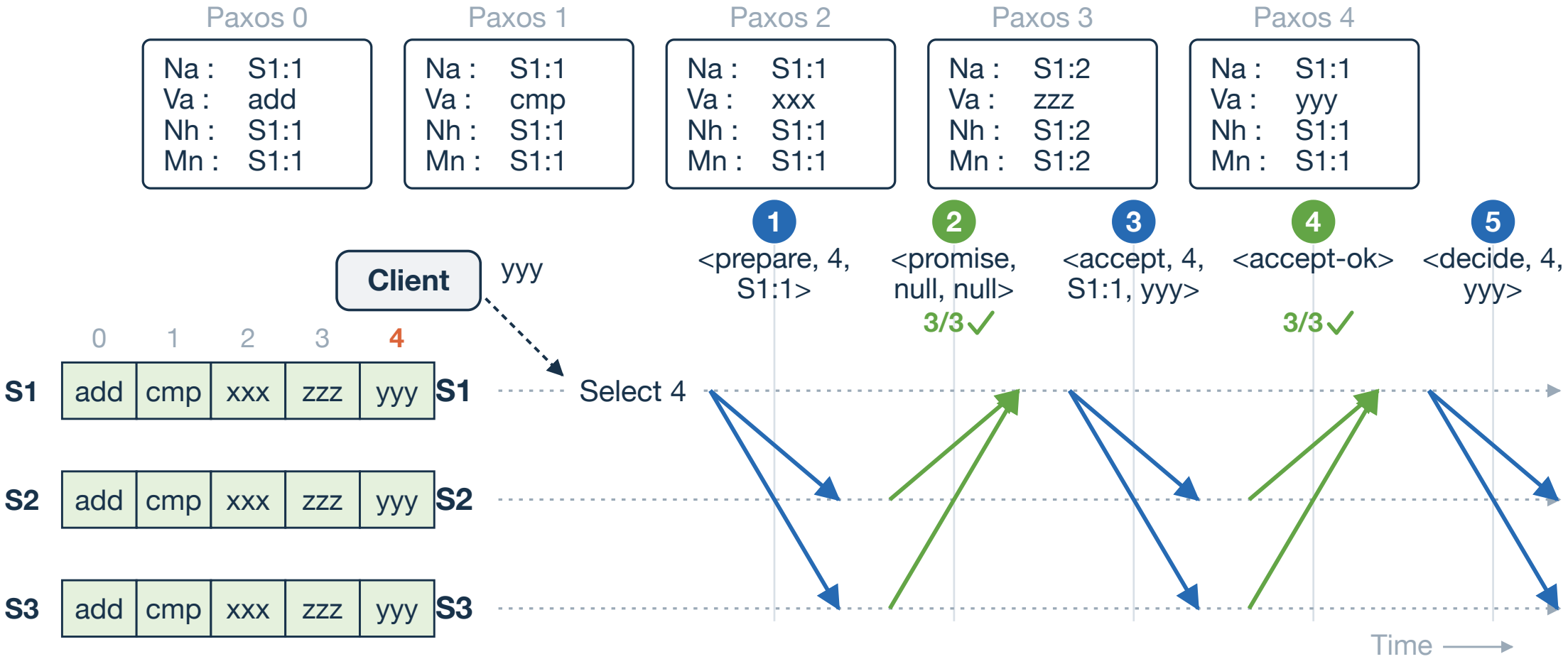
- It keeps trying until it finds an unoccupied slot — possibly many times



Basic Multi-Paxos: retry with a larger instance

In case of conflict, S1 chooses a larger instance (4 here) and retries

- It keeps trying until it finds an unoccupied slot — possibly many times



The efficiency problem of Multi-Paxos

2 roundtrips (RTTs) of RPCs for each value chosen, in the optimal case

- Prepare, Accept

With multiple concurrent proposers, conflicts and restarts are likely

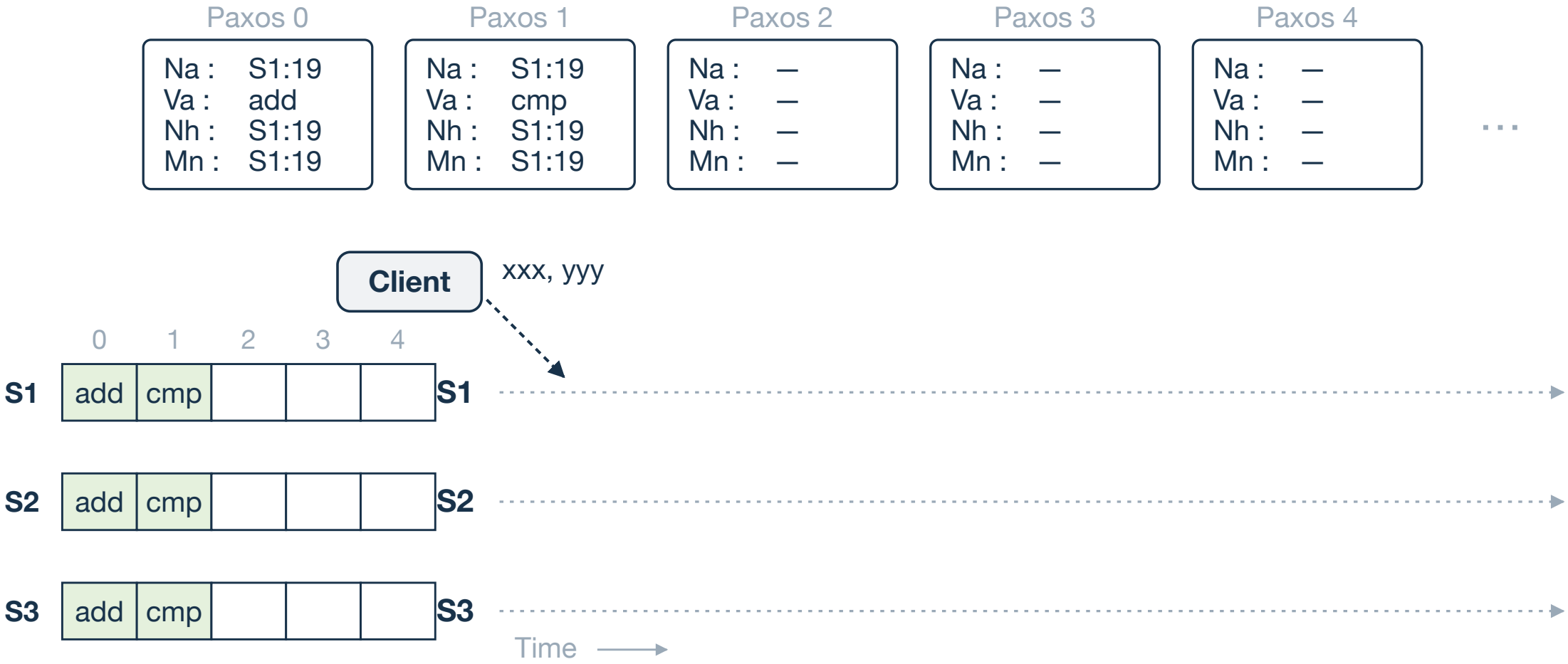
- 1. Servers may conflict on the position of the new log entry (our previous example)
- 2. Even in the same Paxos instance, different proposers may conflict

Solution

- Select a leader: only the leader can propose
- Leader can batch multiple `prepare`s to reduce RTTs per request

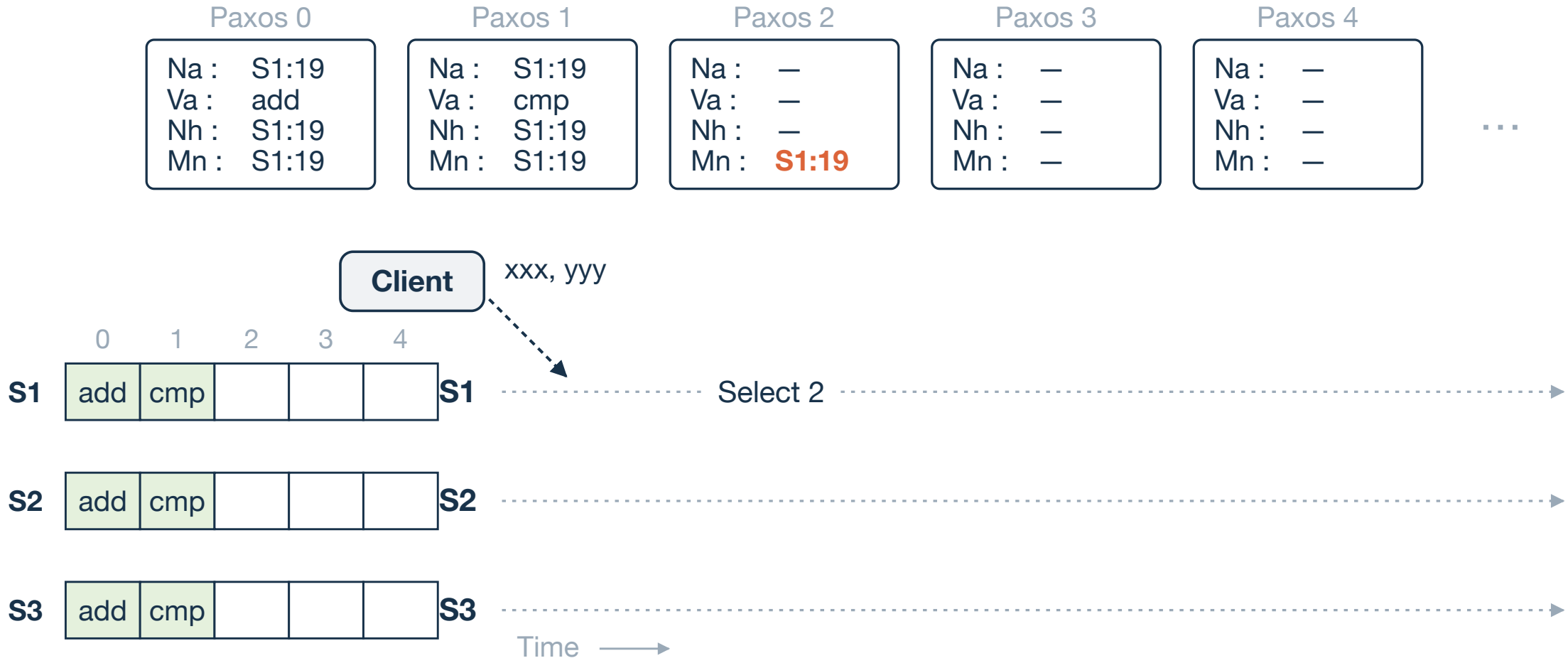
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



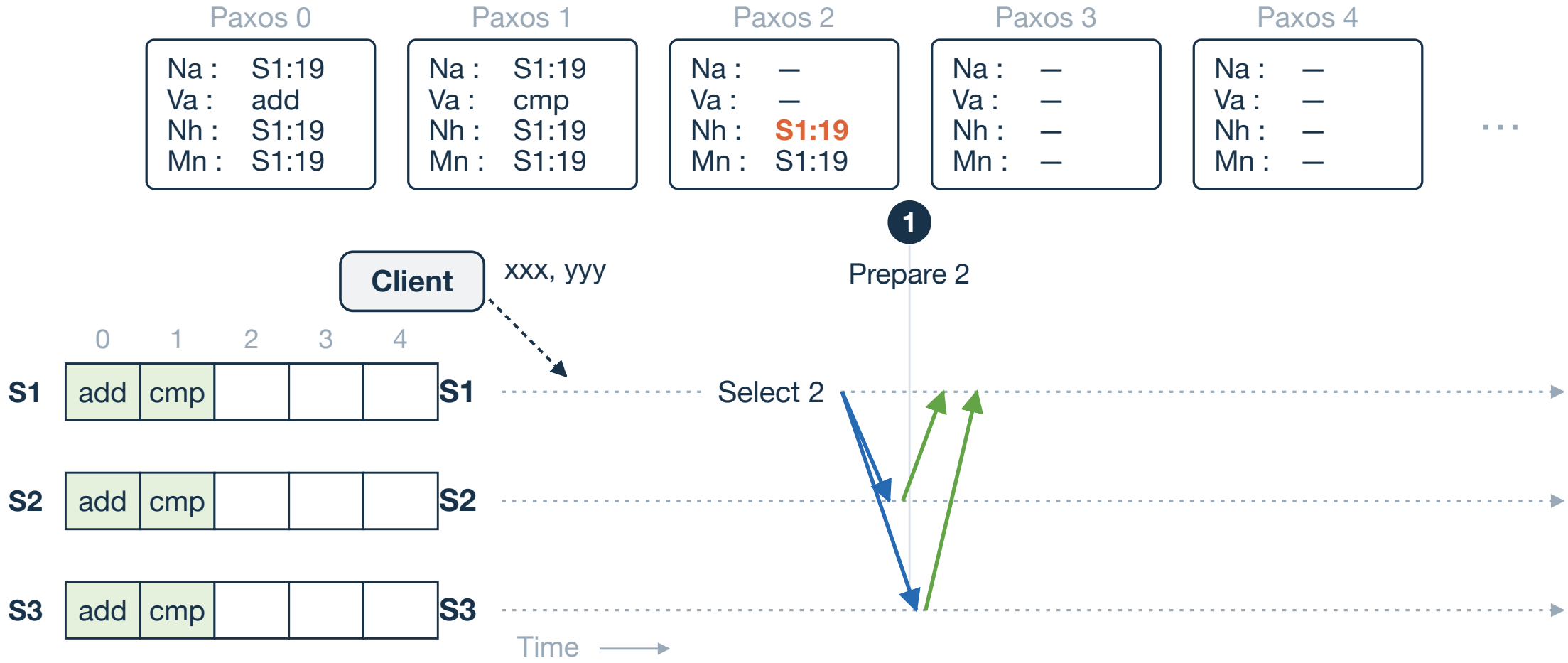
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



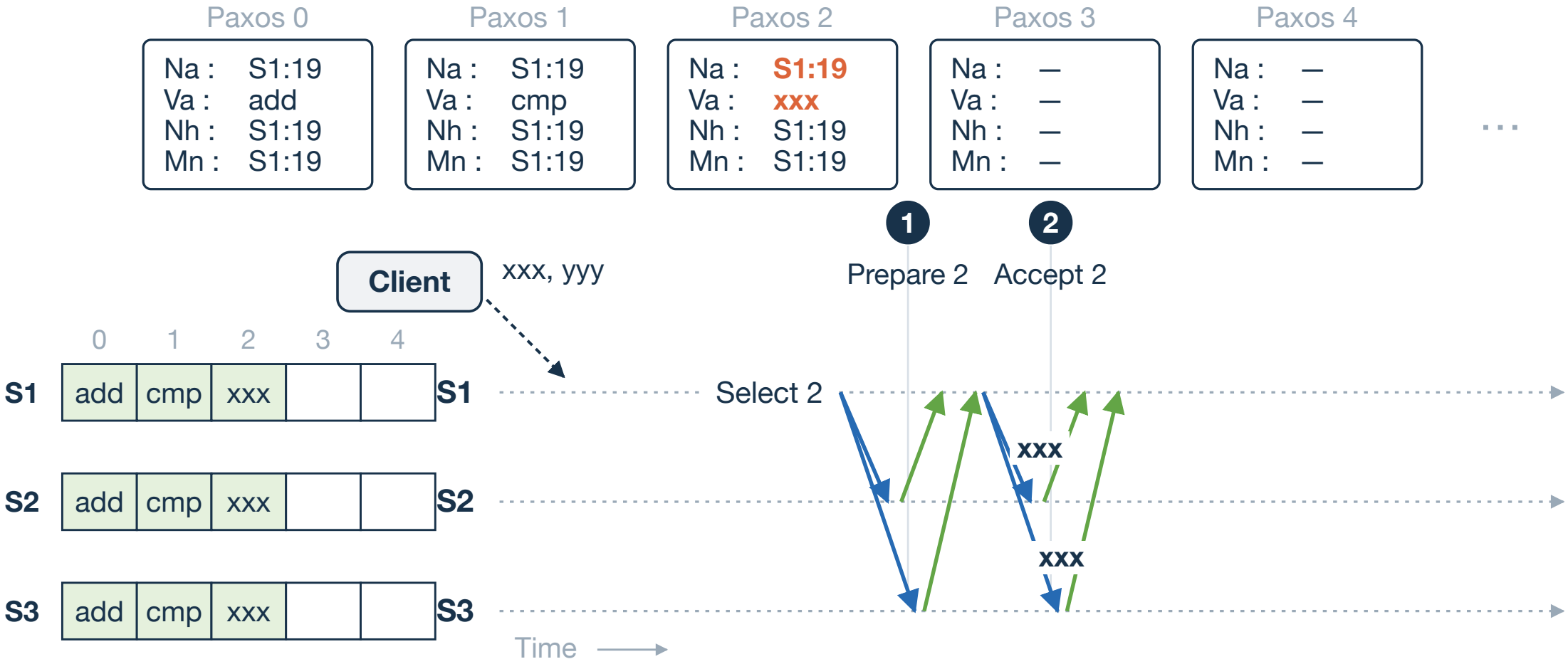
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



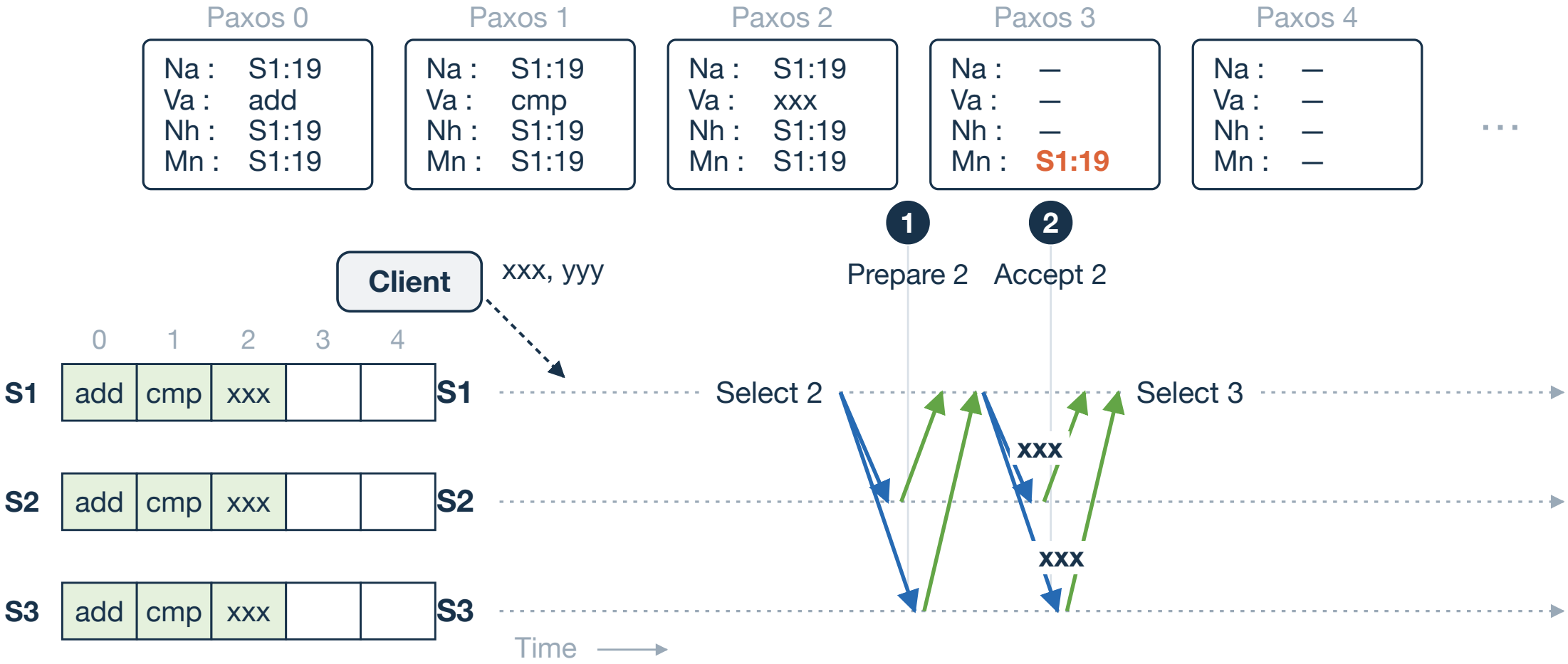
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



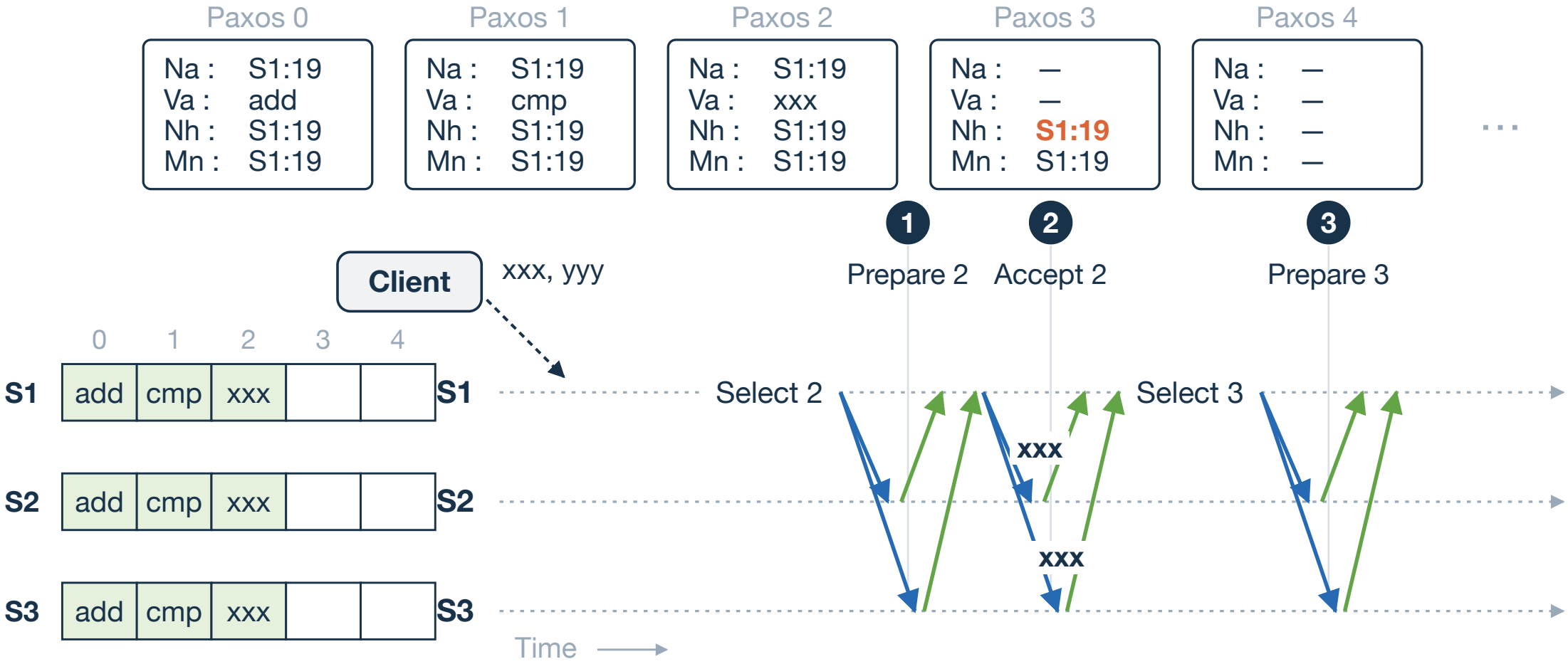
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



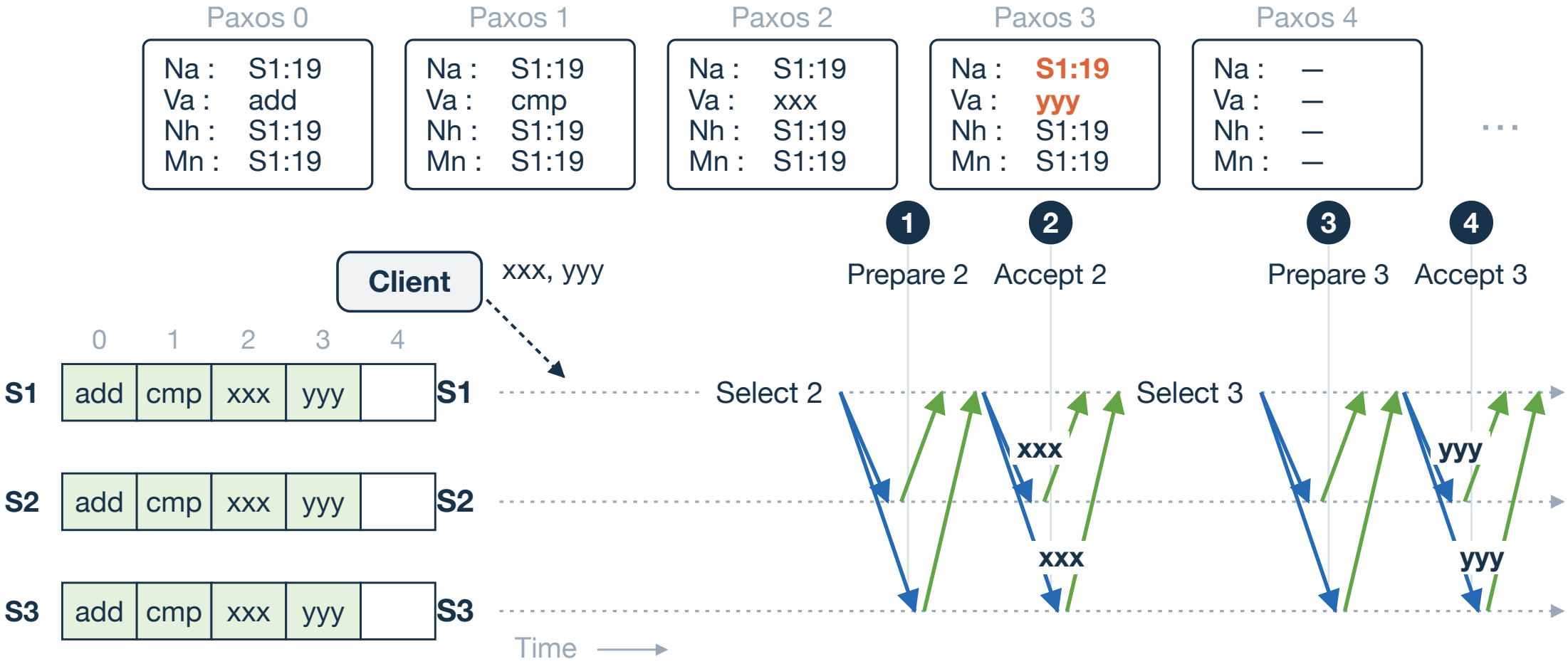
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



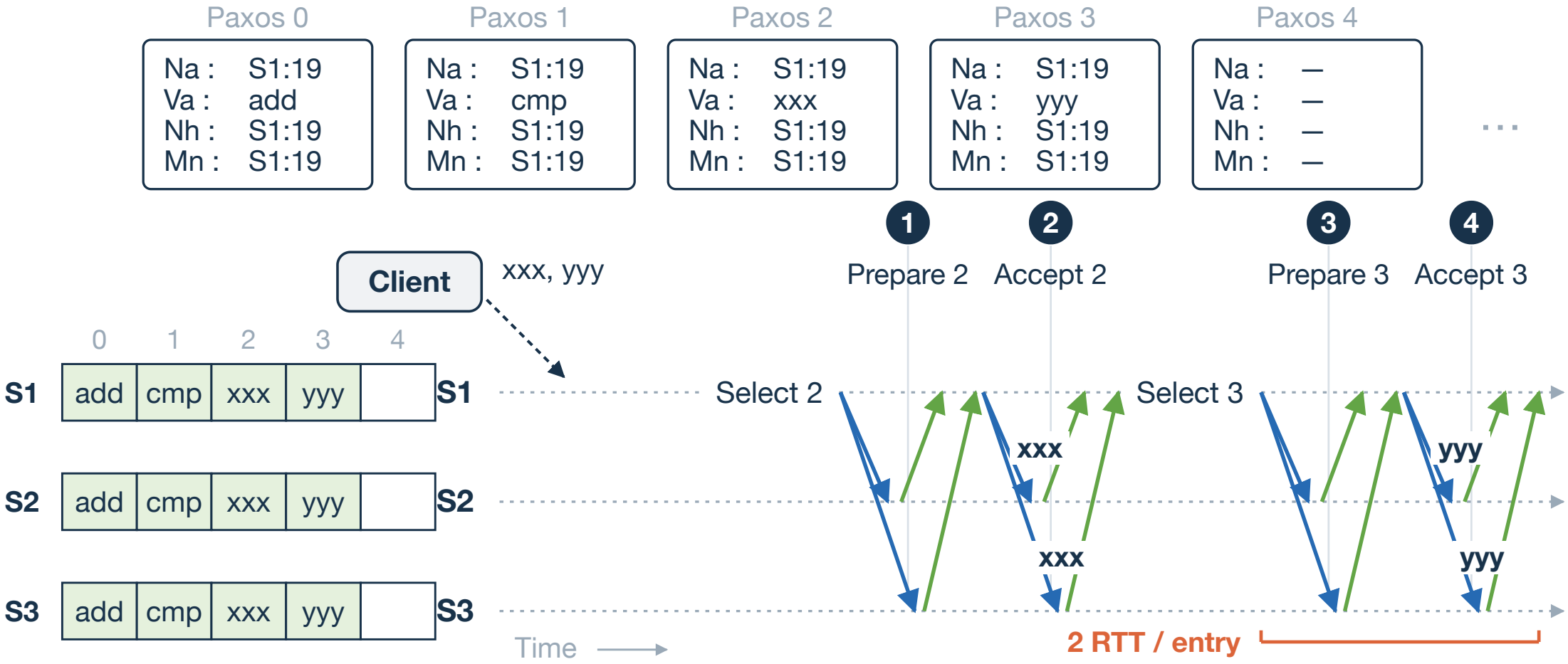
Benefits of a leader: first, without batching

Suppose S1 is the leader; each command runs a full single-decree Paxos



Benefits of a leader: first, without batching

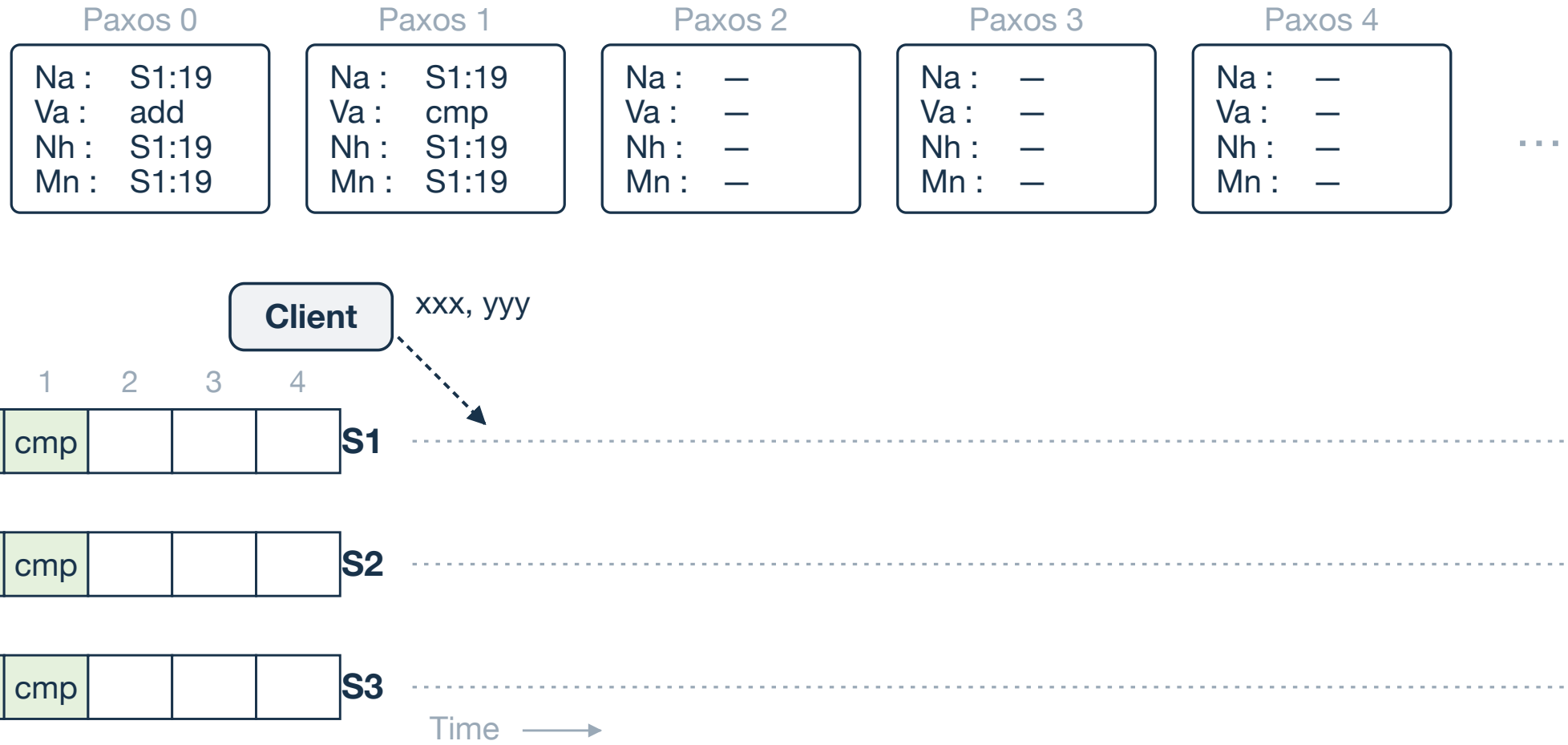
Suppose S1 is the leader; each command runs a full single-decree Paxos



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

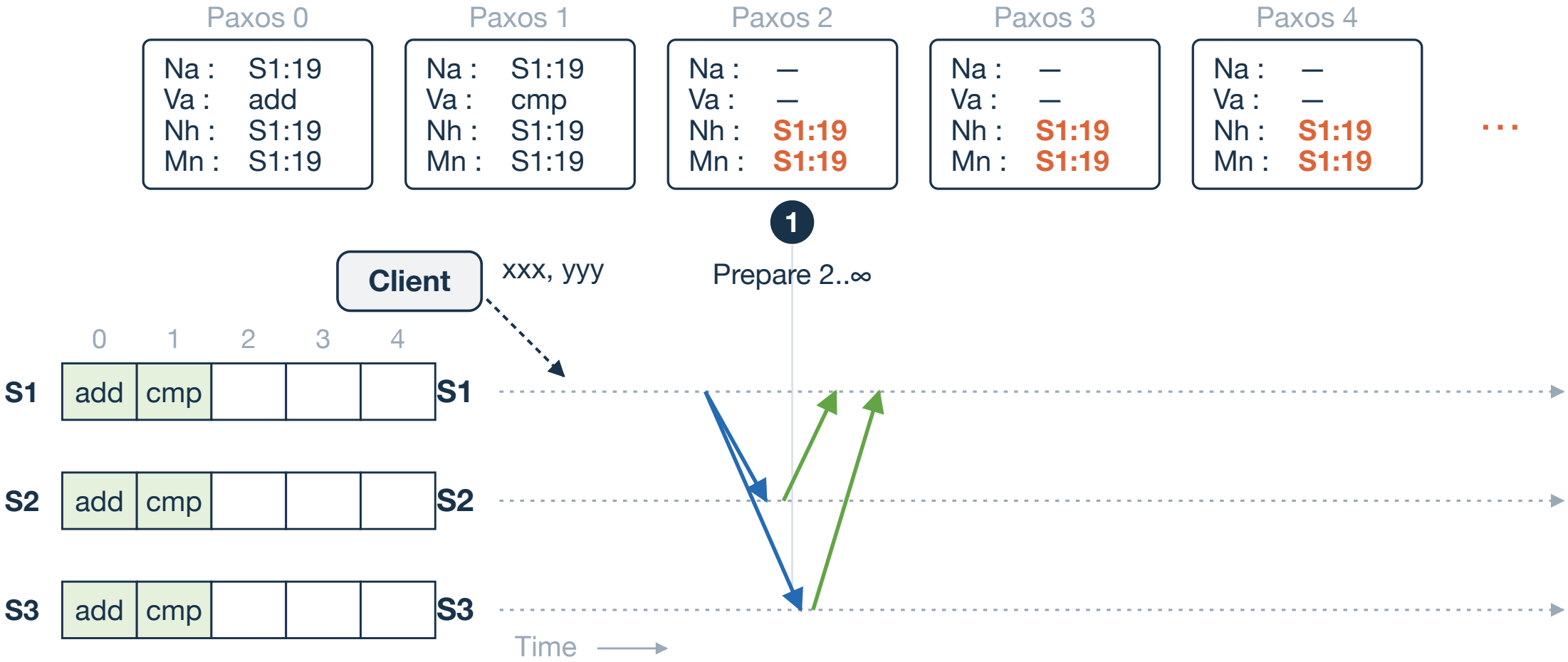
- So each later command only costs an Accept



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

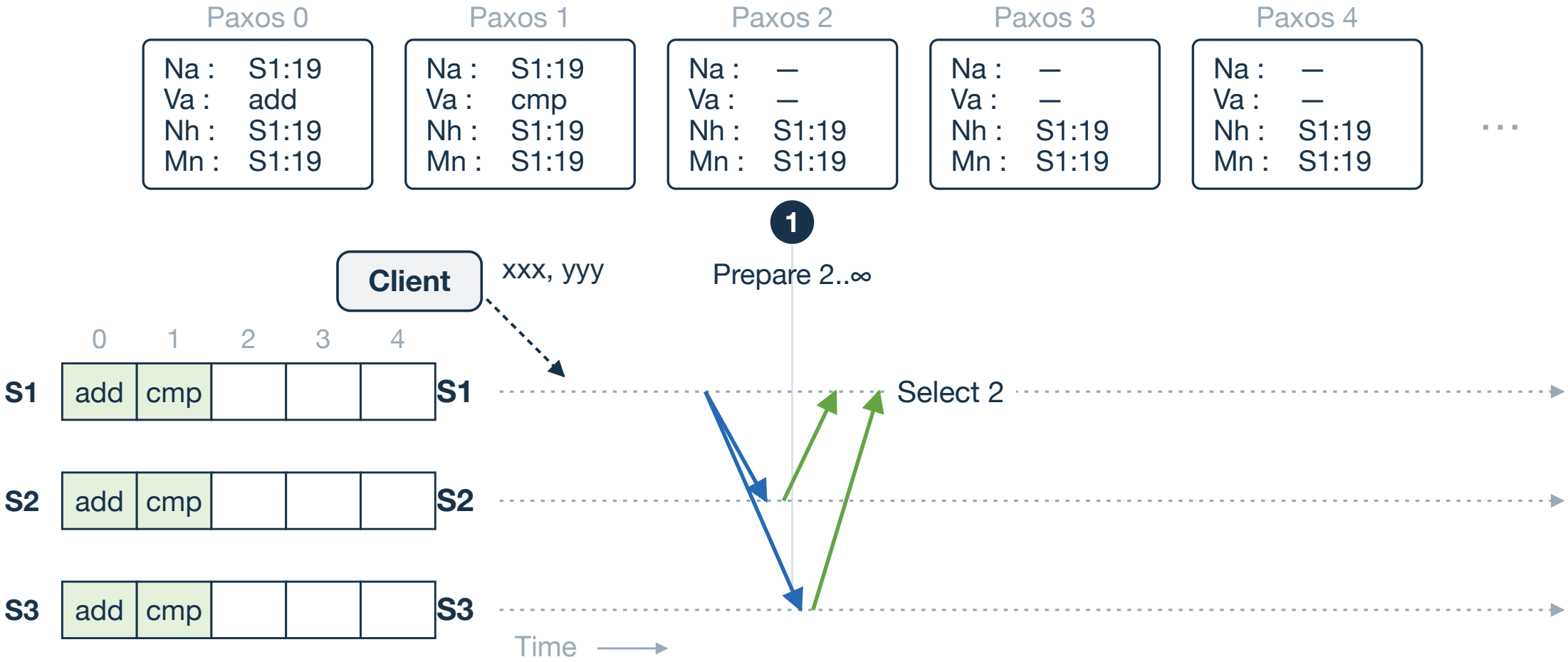
- So each later command only costs an Accept



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

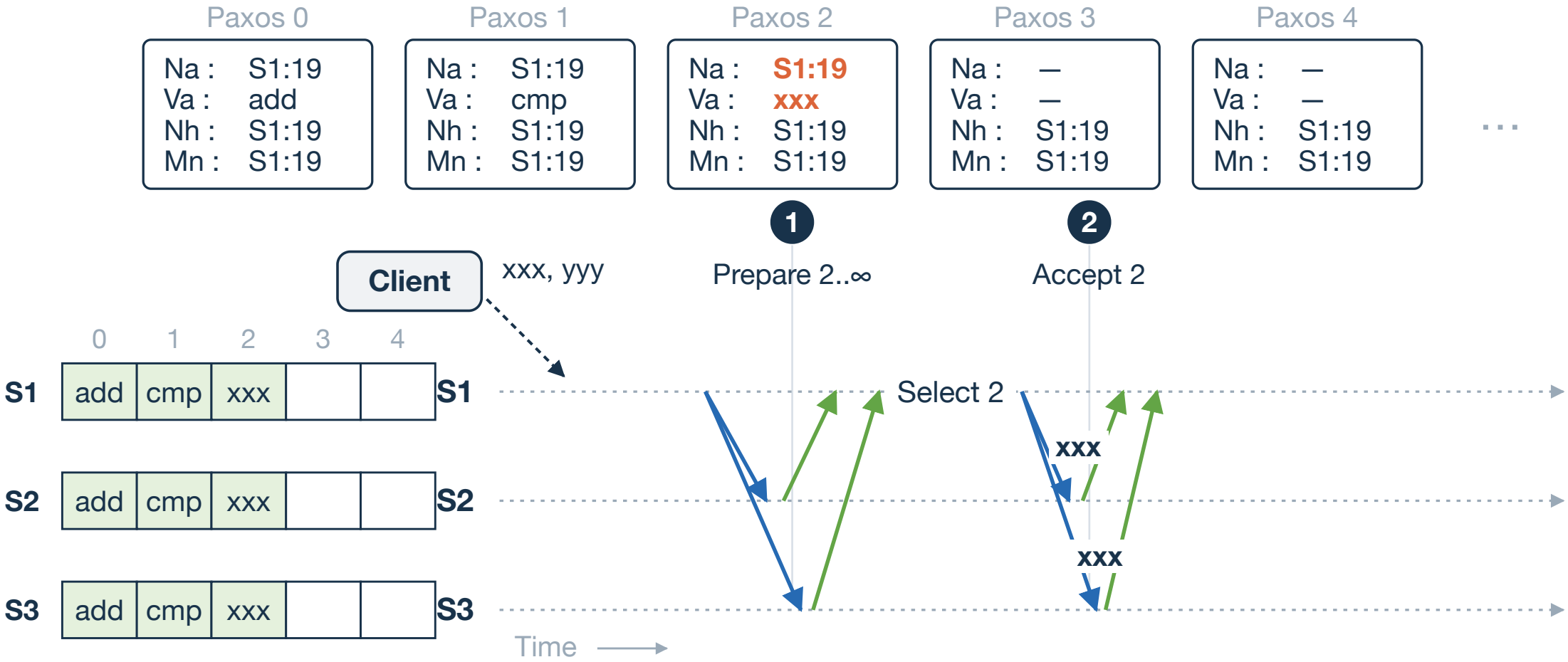
- So each later command only costs an Accept



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

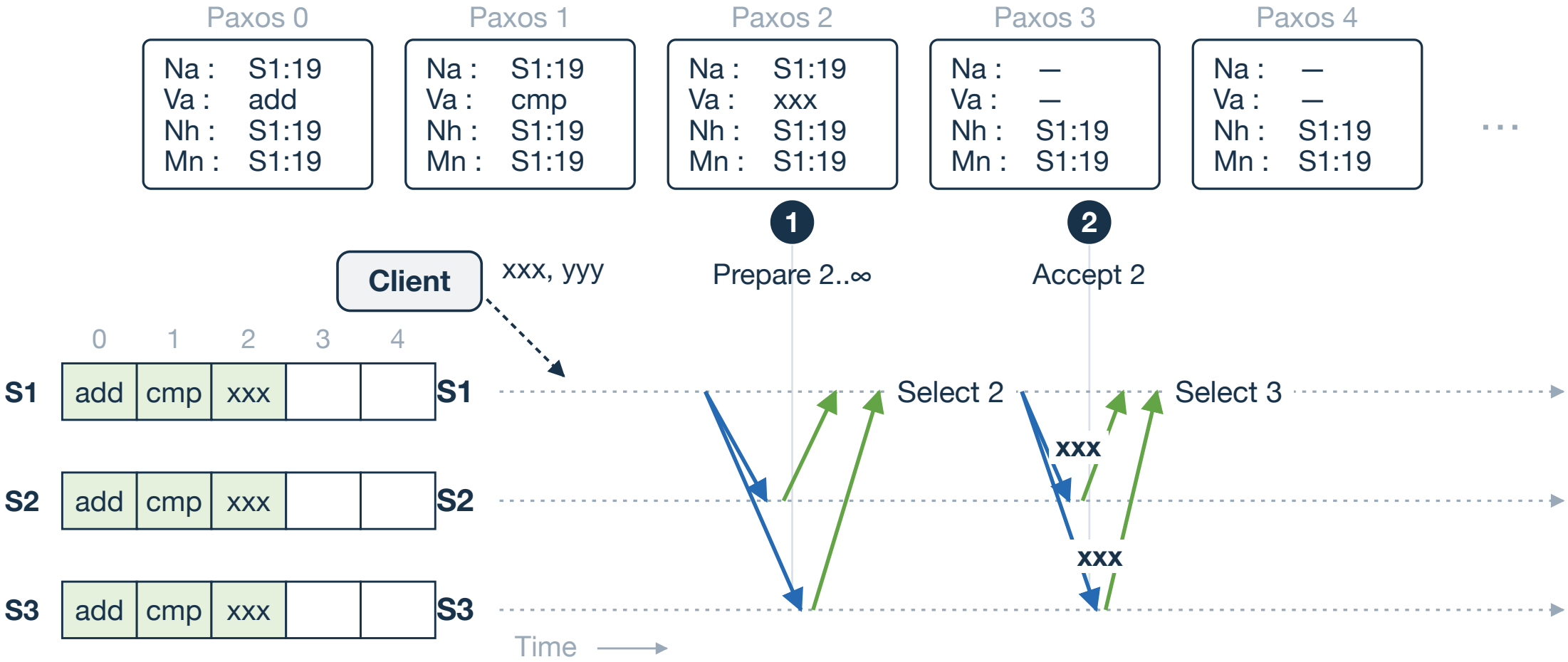
- So each later command only costs an Accept



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

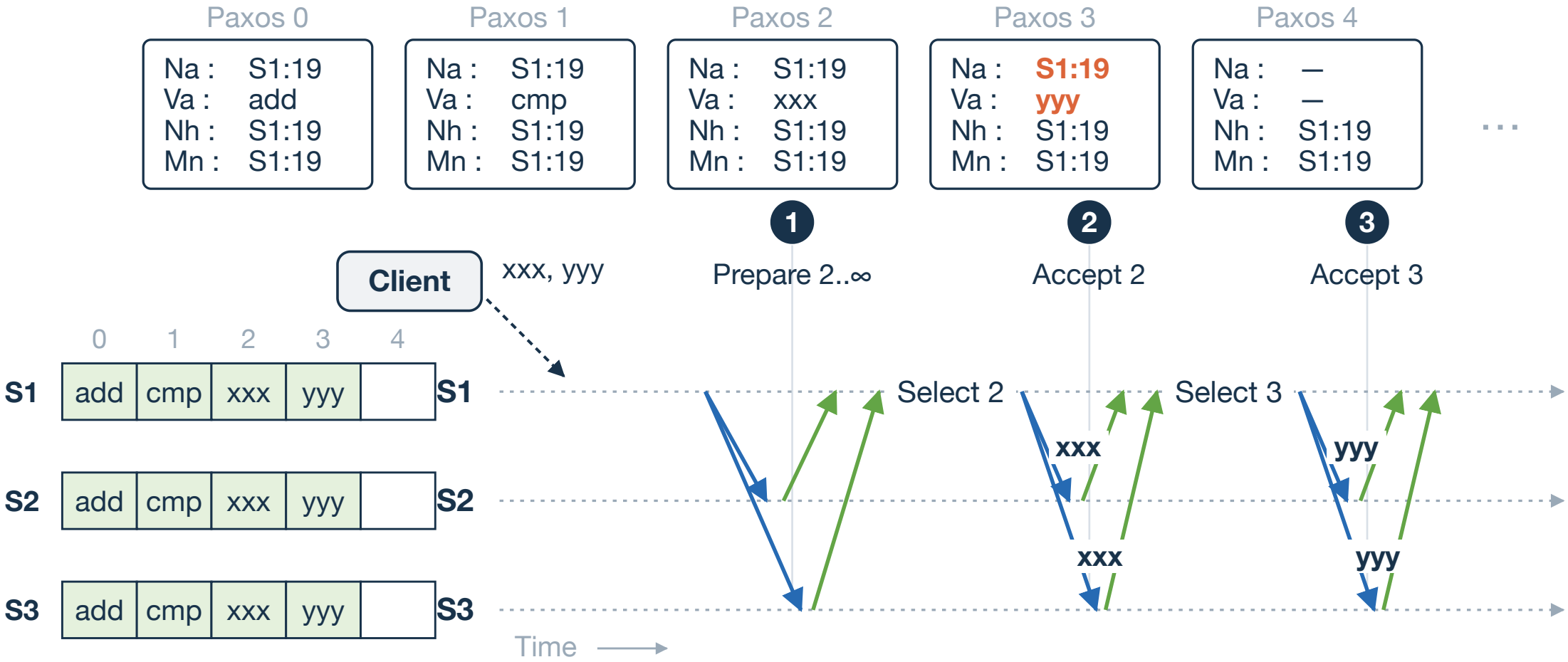
- So each later command only costs an Accept



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

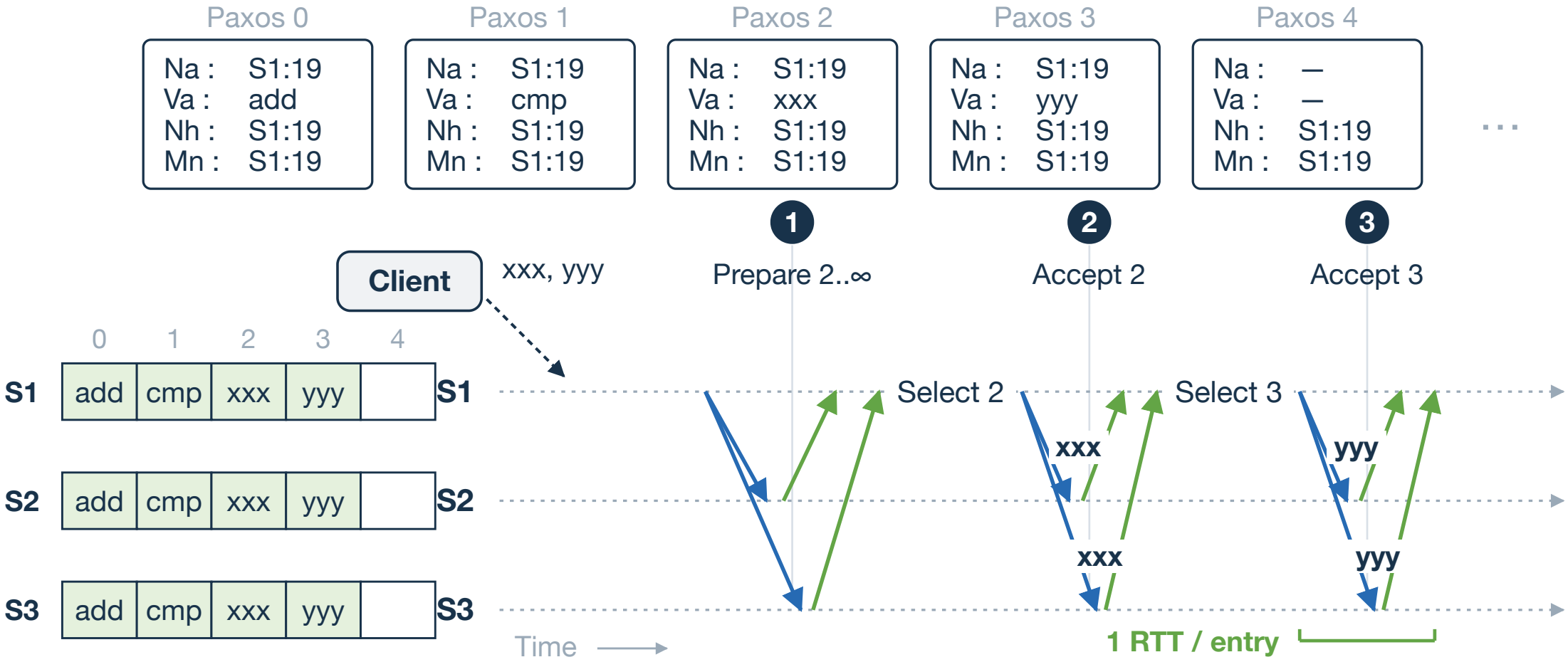
- So each later command only costs an Accept



Benefits of a leader: Prepare message batching

One Prepare with `S1:19` claims every instance from 2 onward, not just the next entry

- So each later command only costs an Accept



Summary: Why Multi-Paxos uses a distinguished proposer

The distinguished proposer is the leader

- The only one that issues proposals
  - i.e., no proposer conflicts in the optimal case

Clients only send commands to the leader

- The leader decides the position of each command

Question: is a single leader necessary for Multi-Paxos?

- No: if two or more servers act as proposers at the same time, the protocol still works correctly

Paxos is challenging to implement

The Raft authors discuss a few:[1]

📖 **Two drawbacks:** *Paxos is exceptionally difficult to understand.* The full explanation is *notoriously opaque … it does not provide a good foundation for building practical implementations.*

📖 **On Multi-Paxos in particular:** *There is no widely agreed-upon algorithm for multi-Paxos. Lamport's descriptions are mostly about single-decree Paxos; he sketched possible approaches to multi-Paxos, but many details are missing.*

📖 **The Chubby implementers, quoted there:** *There are significant gaps between the description of the Paxos algorithm and the needs of a real-world system. . . . the final system will be based on an unproven protocol.*

[1] From In Search of an Understandable Consensus Algorithm §3

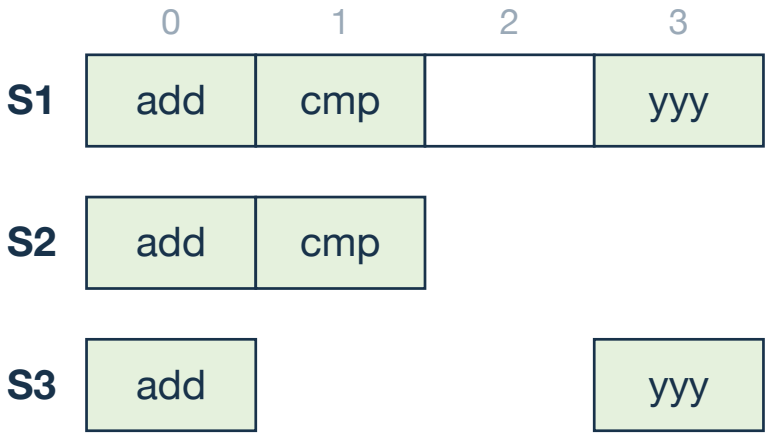
Two case studies of why implementing Paxos is challenging

1. The leader: the protocol assumes one, but never says how to manage it

- No election, and no way to find out the leader has failed
- Two leaders at once: still safe, but with possible performance issues [1]

2. The log: there is no explicit discussion of how to maintain the log, which leads to the "hole" issue that needs to be fixed by the developers

- Entry 2 stays empty, so entry 3 can never be executed — the application fills it with a no-op



[1] *A reliable algorithm for electing a proposer must use either randomness or real time — for example, by using timeouts* (Paxos Made Simple §2.4)

Raft: a different approach for consensus

Paxos' approach is bottom-up

- Solve single-decree consensus first: agree on one value
- Then replicate a sequence of values (e.g., log in the replicated state machine) using it

Raft's approach is top-down

- Directly solve the log replication problem

Why learn Raft? (Instead of Paxos)

Raft is described in a concrete form that favors system design & implementation

Widely implemented and deployed nowadays, especially in open-source projects













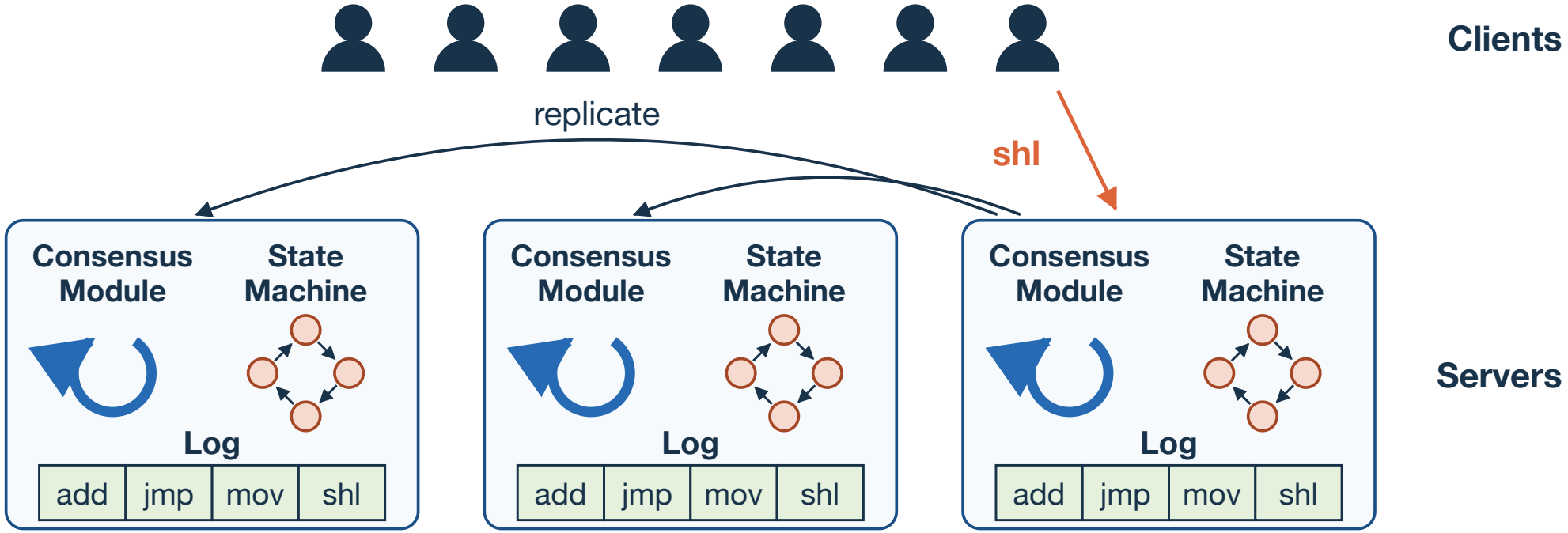
Raft was designed for understandability

- The paper lays out each server's state, the two RPCs, and the rules in one table — you can implement directly from it

Raft's abstraction: a replicated log

Replicated log ⇒ replicated state machine

- All servers execute the same commands in the same order (consistency)
- Commands are deterministic: same input, same result
- The Raft protocol (Consensus module) ensures the logs are the same



Raft's high-level approach: problem decomposition

Leader election

- Select one server as the leader — similar to picking a primary in primary-backup replication
- Detect leader crashes, choose a new leader to keep the service on
- No view server

Log replication: the normal operation

- Leader accepts commands from clients, appends them to its log
- Leader replicates its log to other servers, overwriting inconsistencies

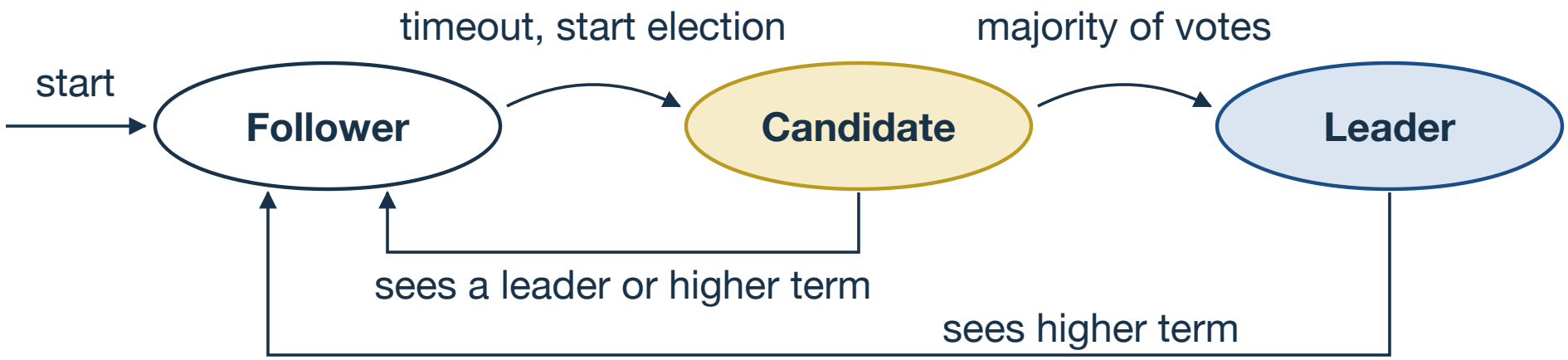
Safety (quite tricky due to overwrites)

- Keep logs consistent across servers

The corresponding Raft server states

At any time, each server is in exactly one of three states

- Leader: receives client requests and then replicates the corresponding log entries to the other servers
- Follower: maintains replicated logs according to the leader
- Candidate: if a follower suspects that the leader has failed, it enters the candidate state trying to become the new leader



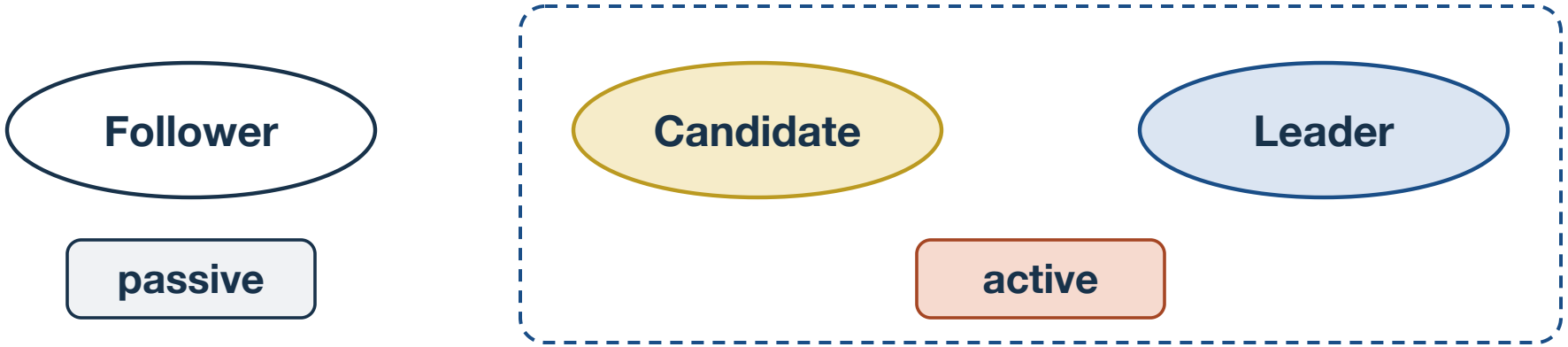
Simplification (and decomposition): followers are passive

Followers are passive

- They only respond to requests from leaders or candidates

The leader/candidate is the active side

- Leader: Receives client requests
- Leader: Replicates its log to the followers
- Candidate: trying to become the new leader



Questions: how to determine the one and only leader? Terms

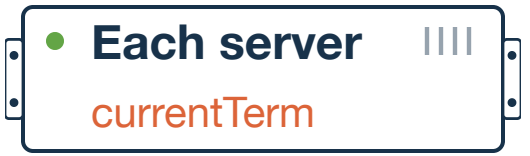
Raft divides time into terms of arbitrary length

- Each term starts with an election, and ends with one leader — or with no leader

Thus, each server must maintain a `currentTerm`

A new term starts when the previous one cannot go on

- The leader goes silent, or the election picks nobody

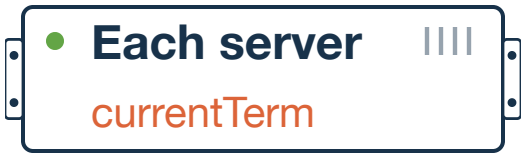




Questions: how to determine the one and only leader? Terms

Raft divides time into terms of arbitrary length

- Each term starts with an election, and ends with one leader — or with no leader



Thus, each server must maintain a `currentTerm`

A new term starts when the previous one cannot go on

- The leader goes silent, or the election picks nobody



Questions: how to determine the one and only leader? Terms

Raft divides time into terms of arbitrary length

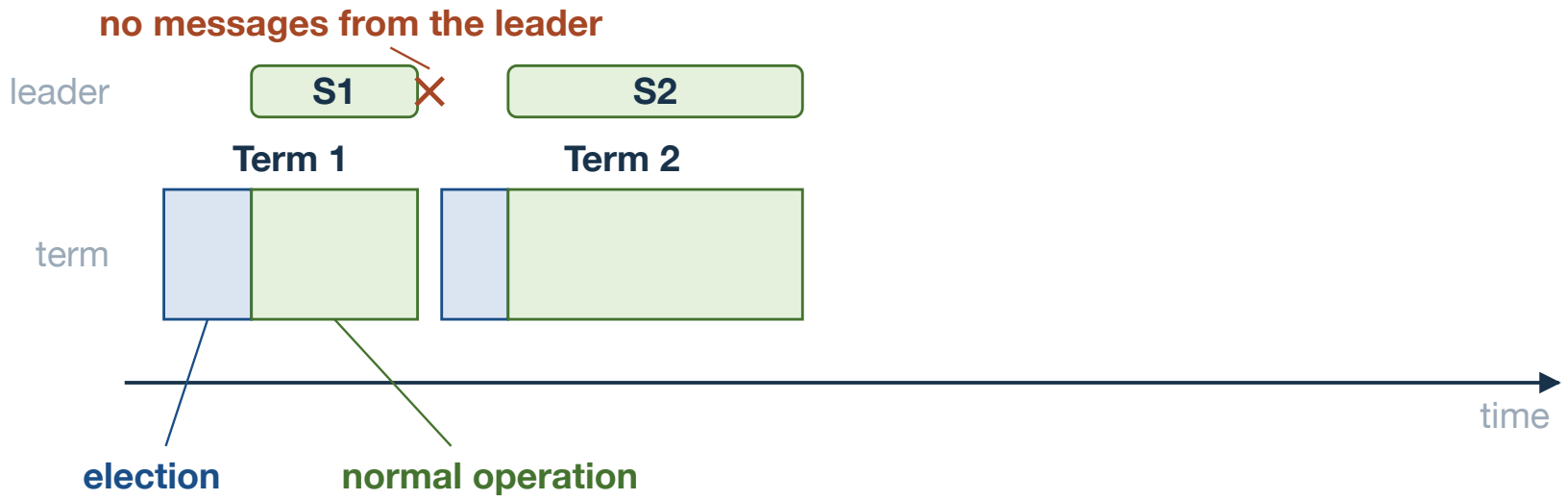
- Each term starts with an election, and ends with one leader — or with no leader

- **Each server** IIII
  - currentTerm

Thus, each server must maintain a `currentTerm`

A new term starts when the previous one cannot go on

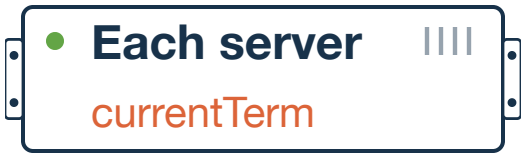
- The leader goes silent, or the election picks nobody



Questions: how to determine the one and only leader? Terms

Raft divides time into terms of arbitrary length

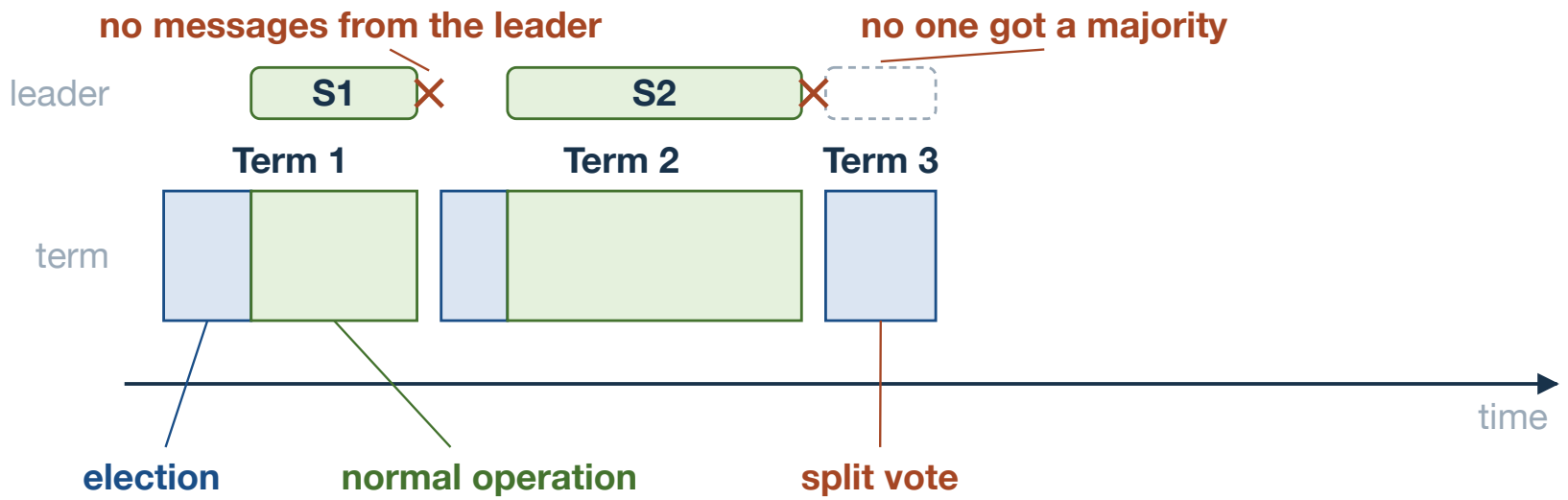
- Each term starts with an election, and ends with one leader — or with no leader



Thus, each server must maintain a `currentTerm`

A new term starts when the previous one cannot go on

- The leader goes silent, or the election picks nobody



Questions: how to determine the one and only leader? Terms

Raft divides time into terms of arbitrary length

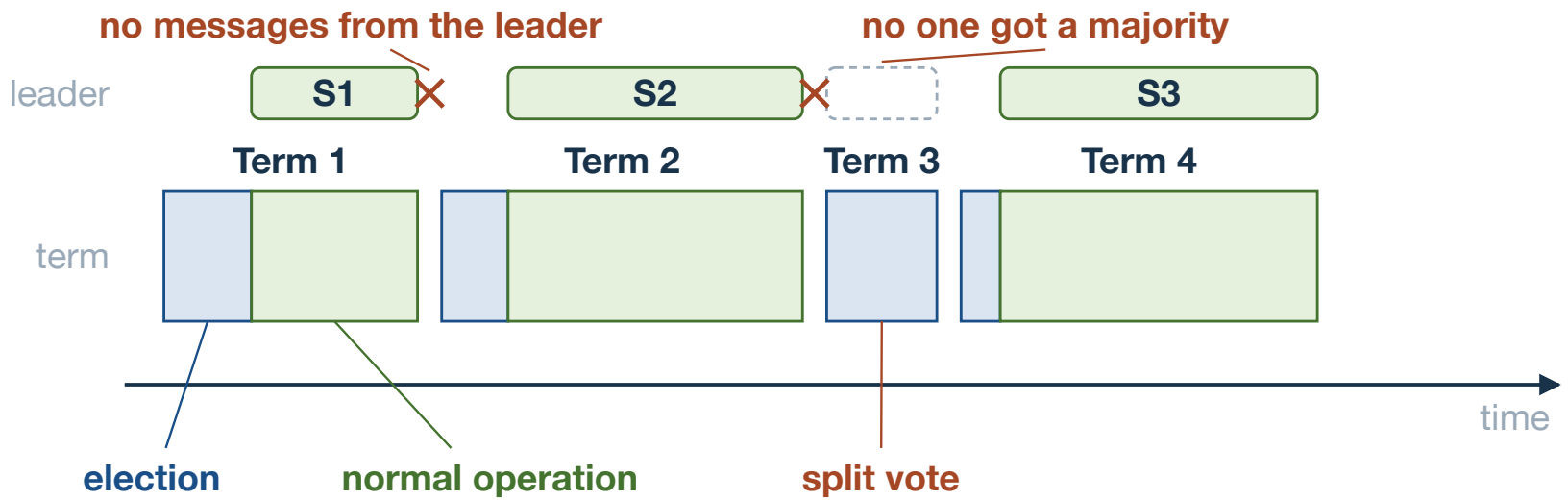
- Each term starts with an election, and ends with one leader — or with no leader

- **Each server** IIII
  - currentTerm

Thus, each server must maintain a `currentTerm`

A new term starts when the previous one cannot go on

- The leader goes silent, or the election picks nobody



Questions: how to determine the one and only leader? Terms

Raft divides time into terms of arbitrary length

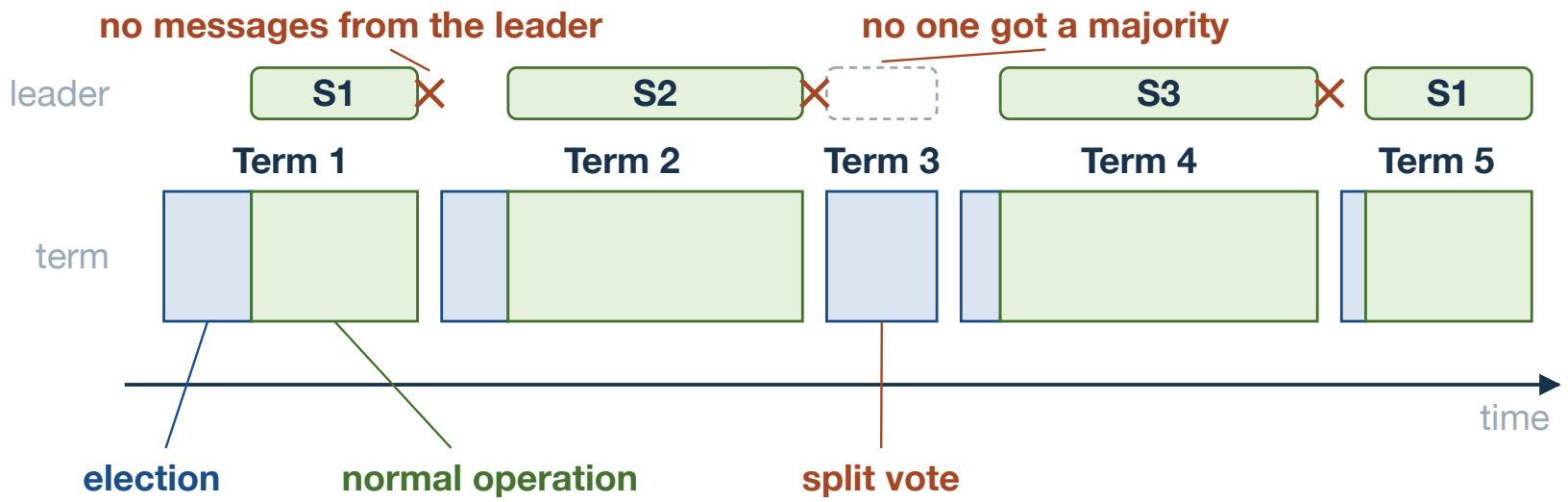
- Each term starts with an election, and ends with one leader — or with no leader

- **Each server** IIII
  - currentTerm

Thus, each server must maintain a `currentTerm`

A new term starts when the previous one cannot go on

- The leader goes silent, or the election picks nobody

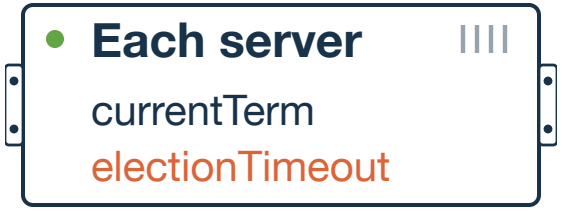


Raft's key invariant: at most one leader per term!

Heartbeats & timeouts: Detect leader failures for starting a new term

Servers start as followers

- A follower expects to keep receiving RPCs from a leader or candidate



The leader must send heartbeats to maintain its authority in a term

If an election timeout elapses with no RPCs

- The follower assumes the leader has crashed, and starts a new election
- Timeouts are typically `100–500 ms`

For simplicity, we omit the timeout logic in this lecture

Election basics: For at most one leader per term

A follower that starts an election does three things first

1. Change its state to candidate
2. Increment `currentTerm`
3. Vote for itself

It then sends `RequestVote RPCs` to all other servers, retrying until it either

- receives votes from a majority of servers => becomes the leader!
- receives an RPC from a valid leader => returns to the follower state

When does a follower grant its vote?

- The candidate's term ≥ its own, and it has not voted in this term yet

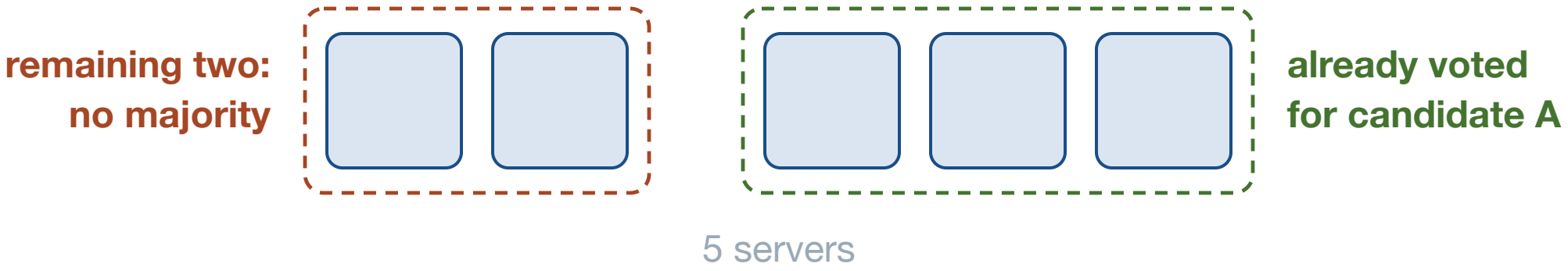
Election requirement #1: safety

At most one winner per term

- Each server gives only one vote per term, persisted on disk
- A majority ensures at most one leader — similar to propose of Paxos

Different candidates can exist in the same term, so each server must remember `votedFor`

- whom this term's vote went to



Election requirement #2: liveness

Some candidate must eventually win

- If multiple candidates start together, none will win

Question: can multiple candidates votes for the same time?

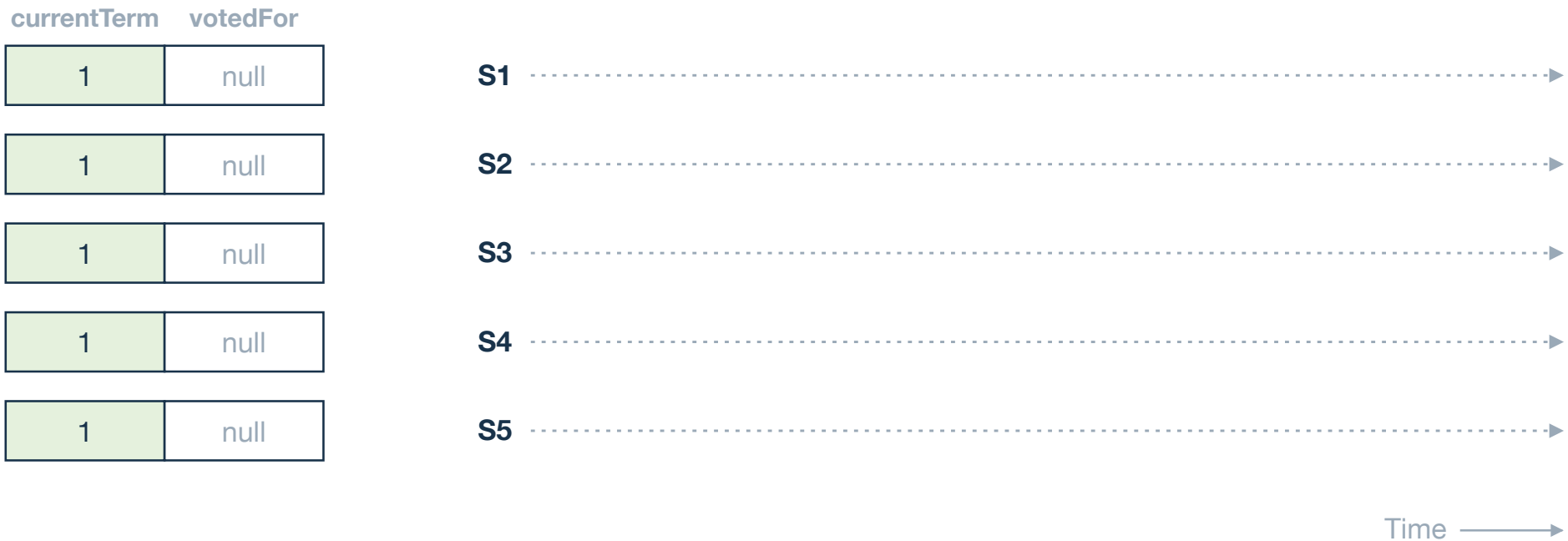
Raft's key idea to prevent so: use a random timeout before the retry

- Typically a random value in [T, 2T], where T is the election timeout

Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

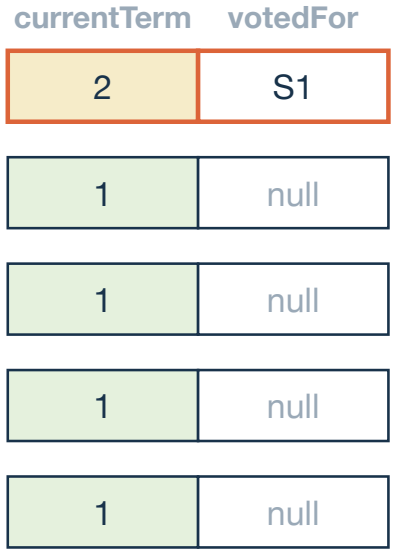
Question: which of one should be persisted?

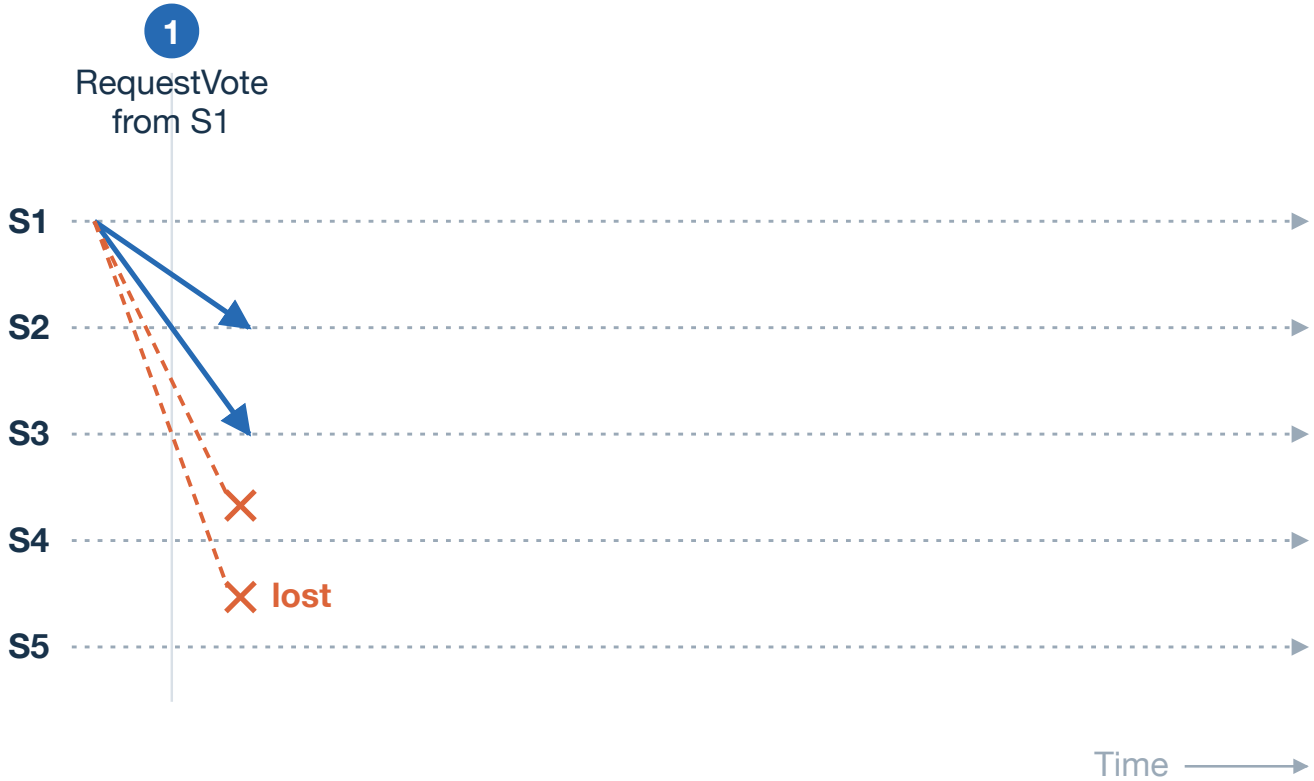


Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?

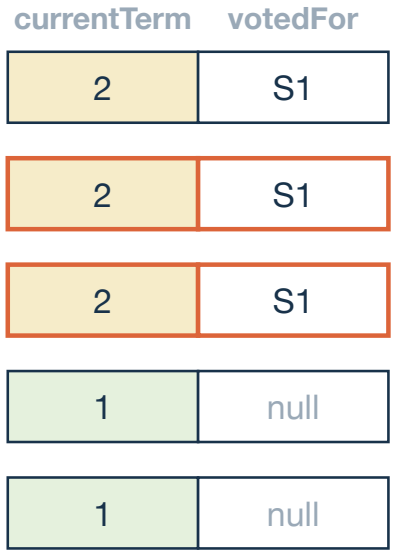


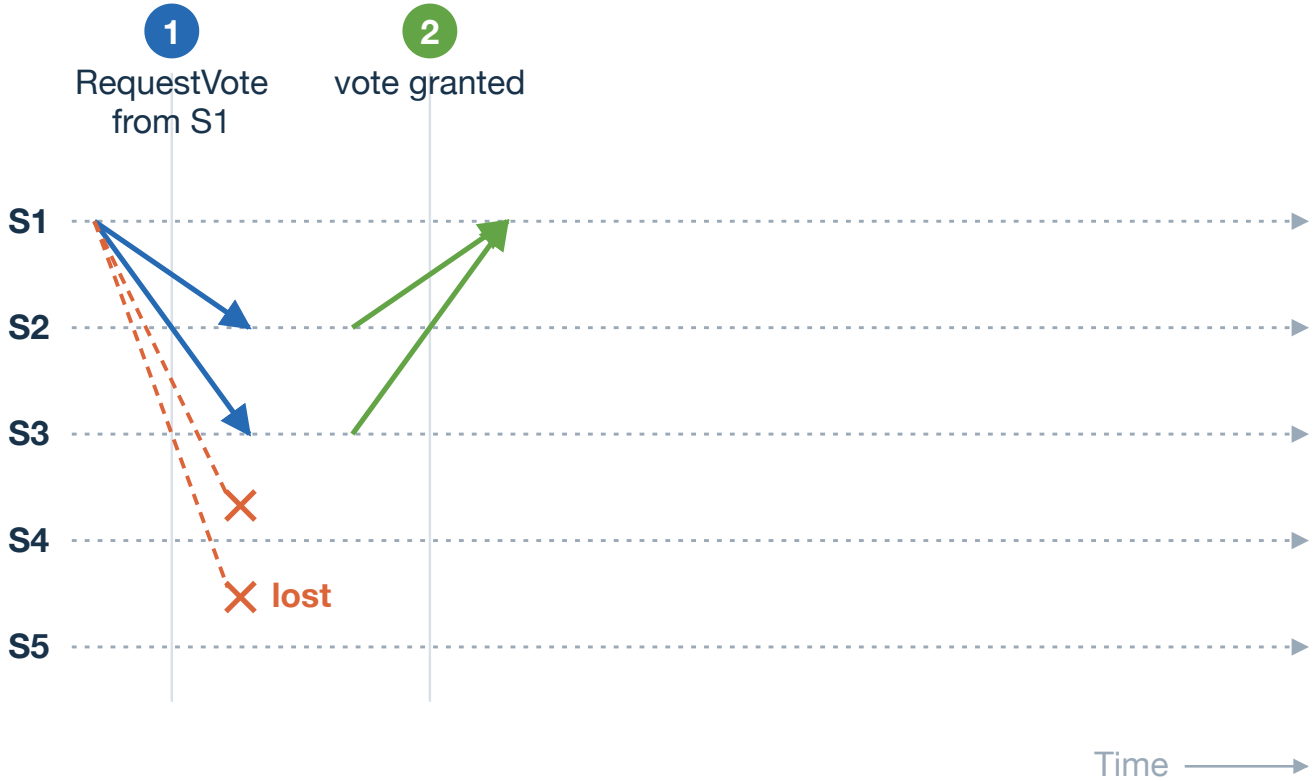


Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?



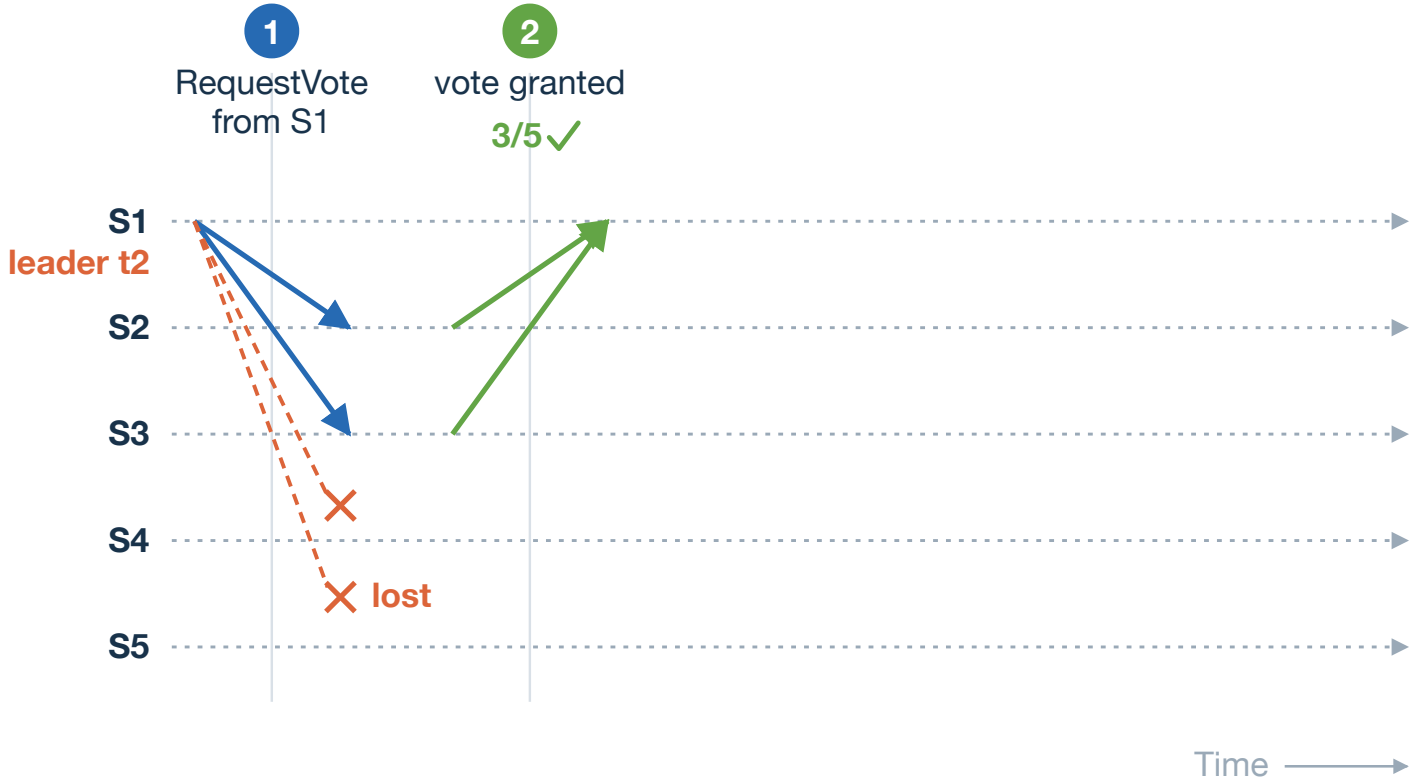


Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?

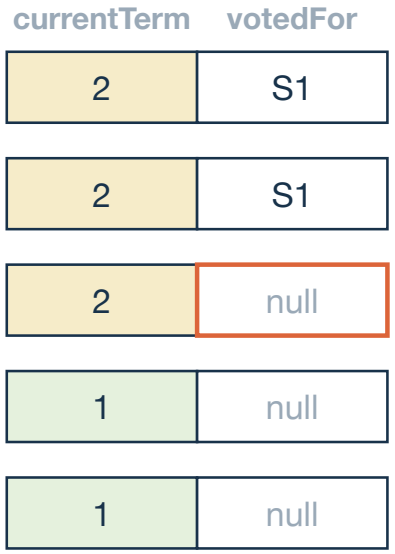
currentTerm	votedFor
2	S1
2	S1
2	S1
1	null
1	null

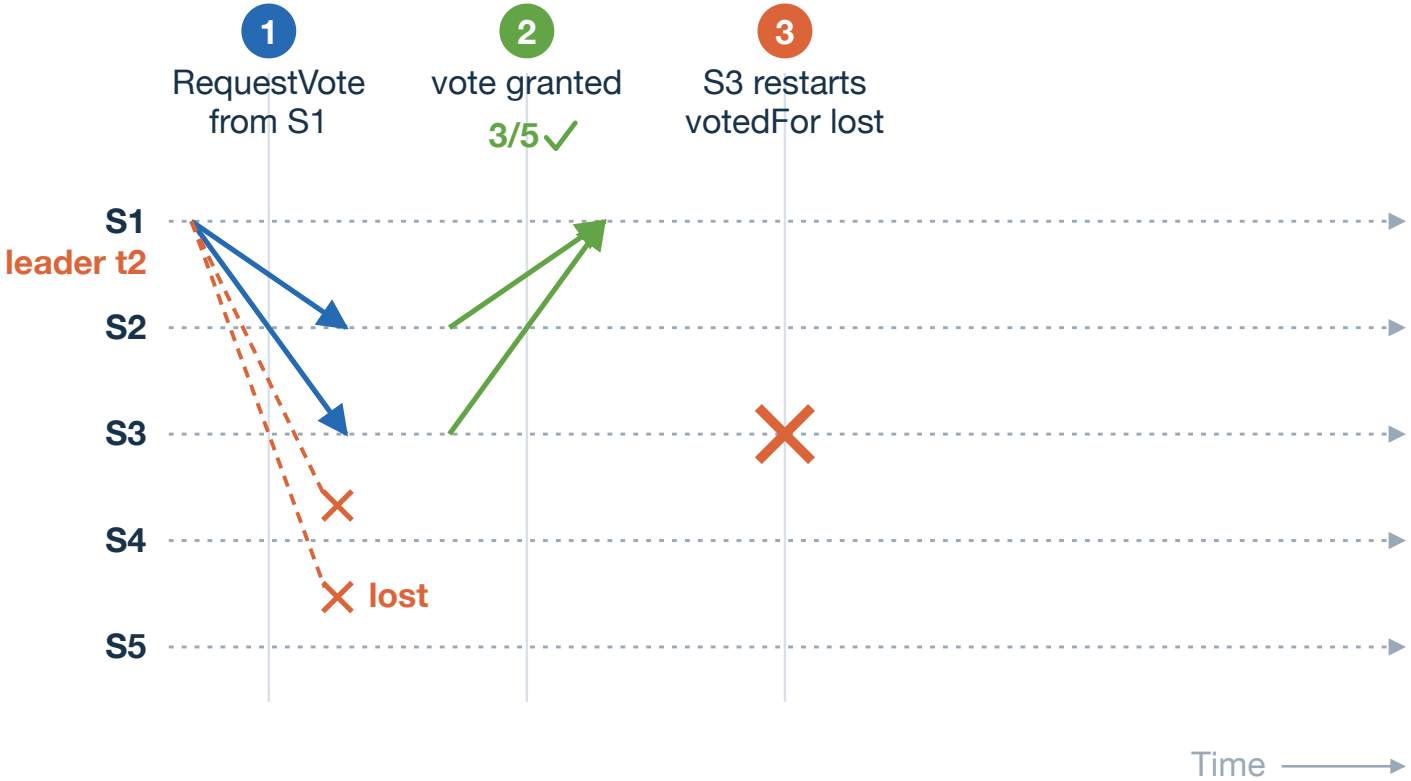


Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?

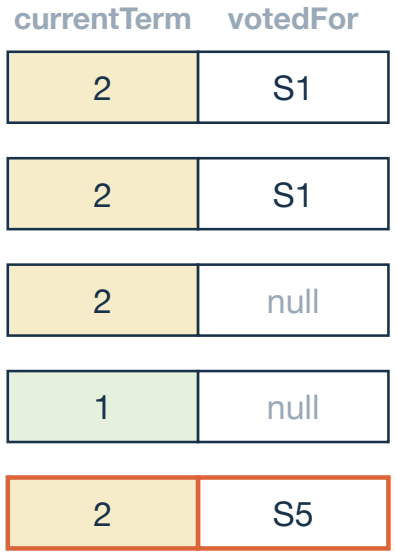


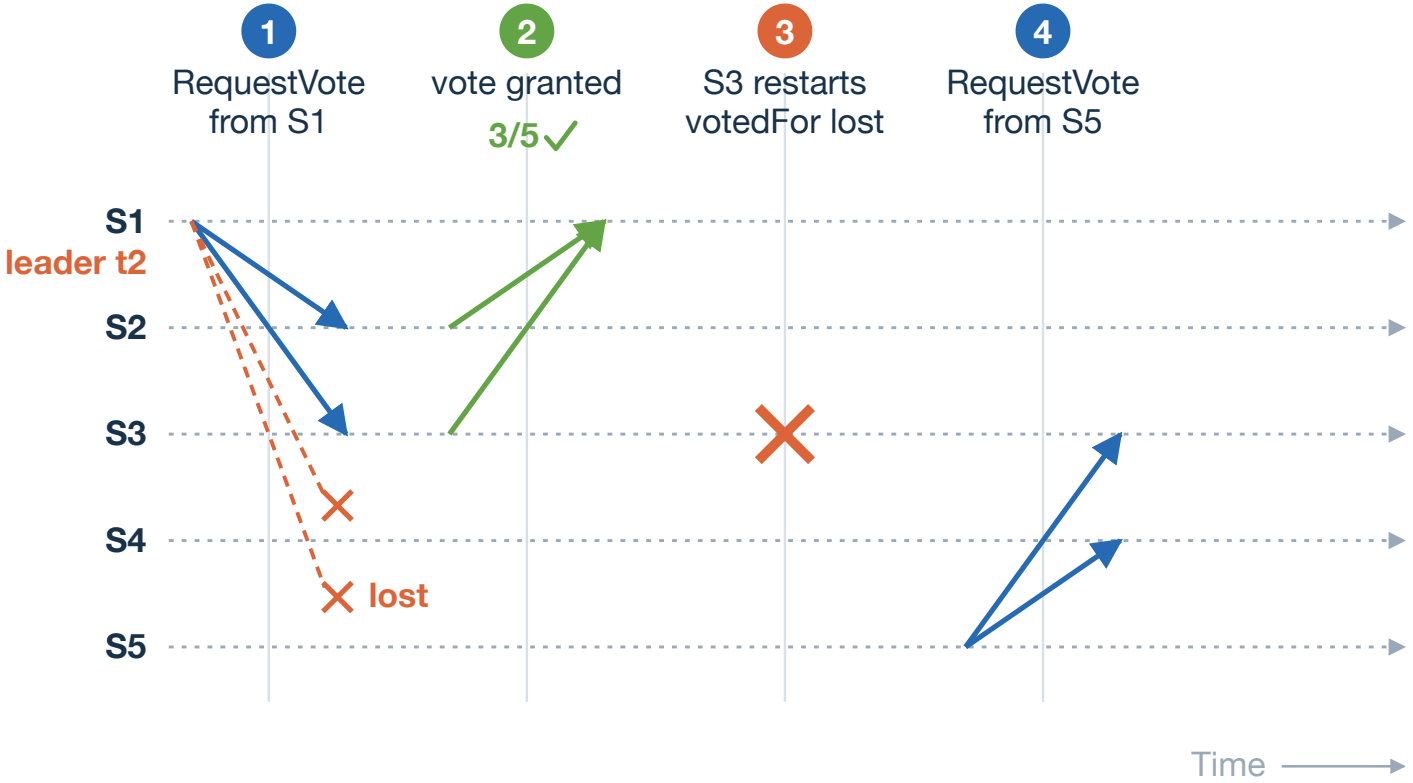


Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?

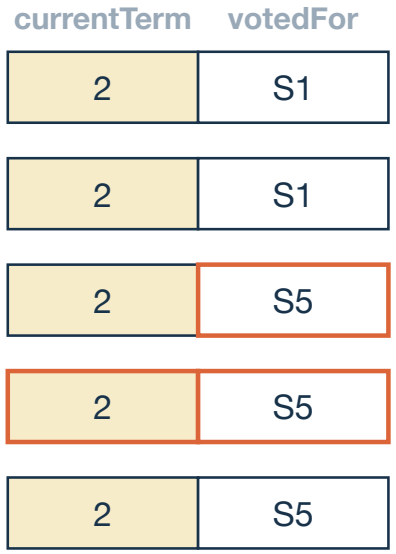


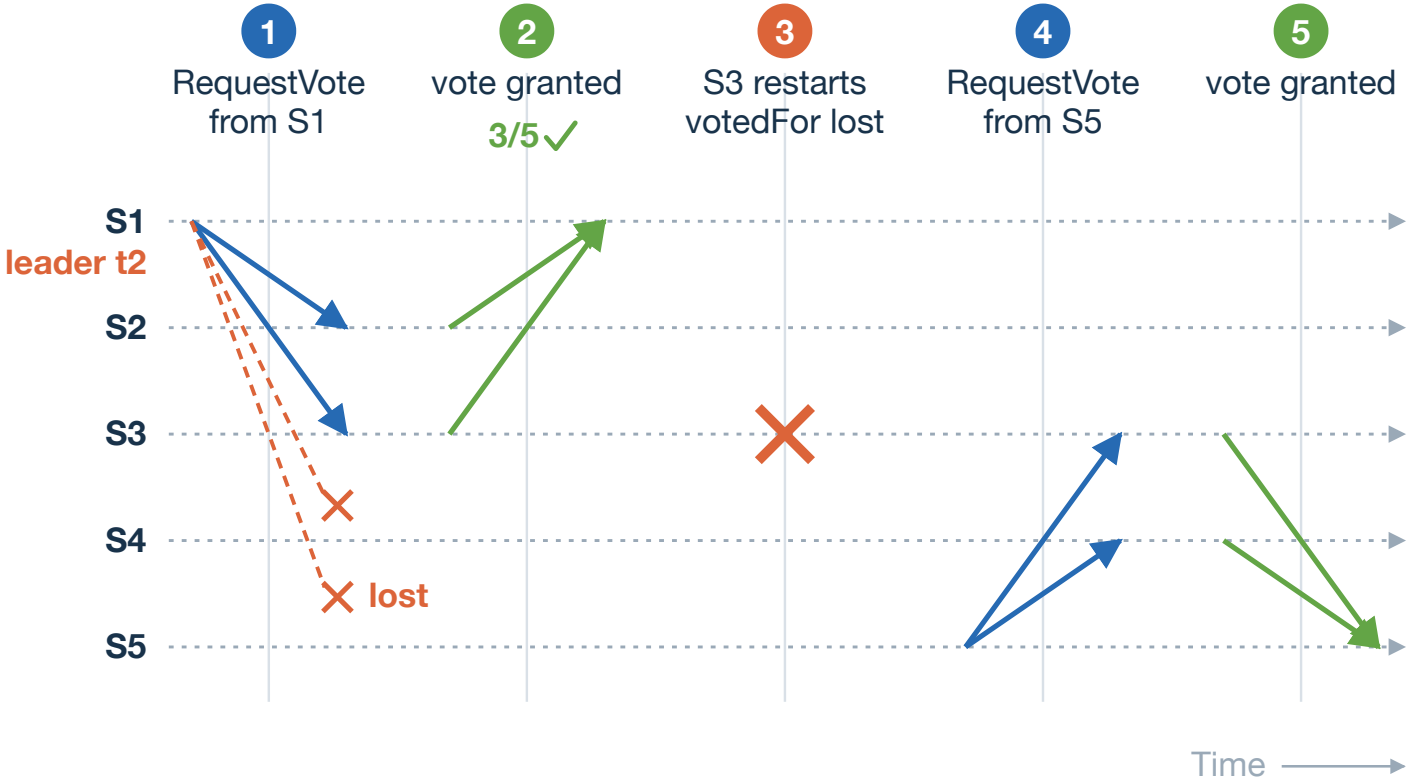


Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?

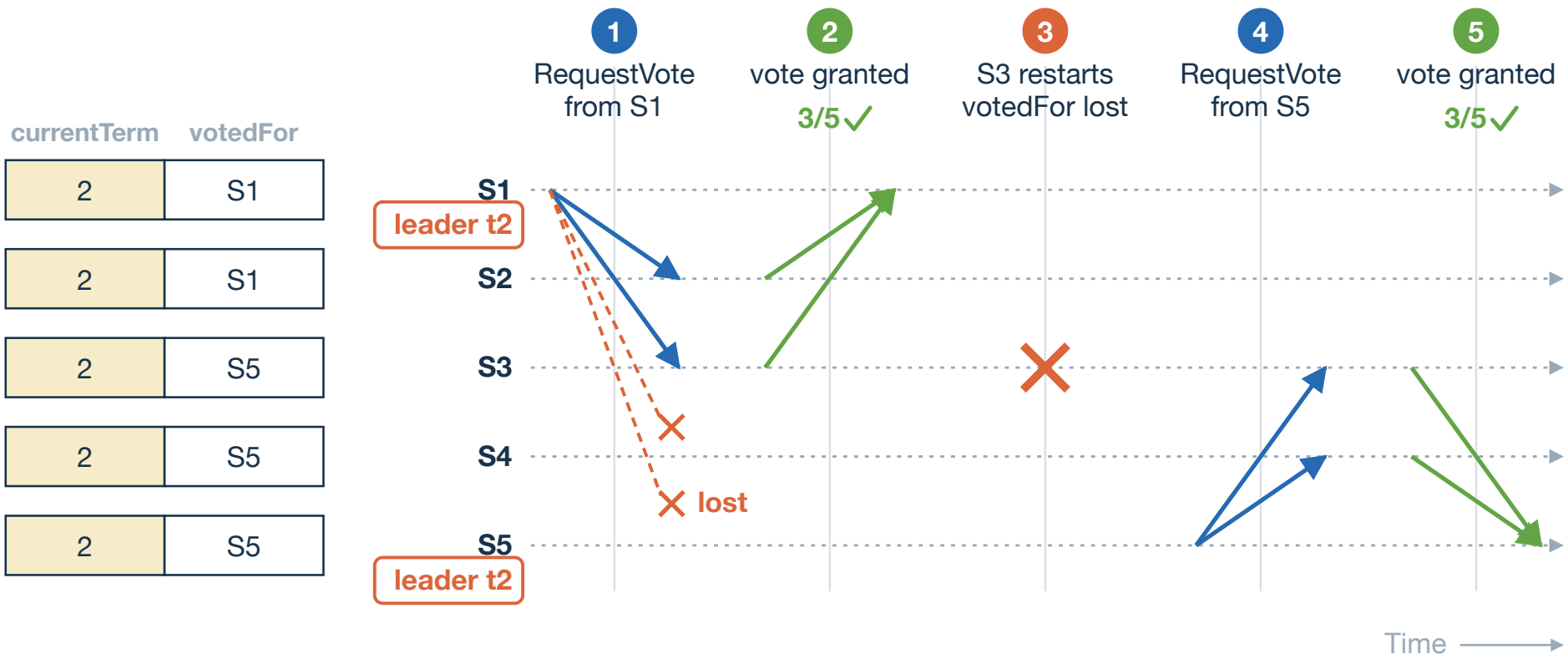




Put it all together: the Raft election process

`currentTerm`	latest term the server has seen; 0 on first boot
`votedFor`	candidate that received this term's vote; null if none

Question: which of one should be persisted?



The code: RequestVote RPC

Invoked by candidates to gather votes

```
Arguments:
  term
  candidateId

Results:
  term
  voteGranted

Implementation:
  if term > currentTerm
      currentTerm = term
  if term == currentTerm and votedFor in (null, candidateId)
      votedFor = candidateId
      voteGranted = true
```

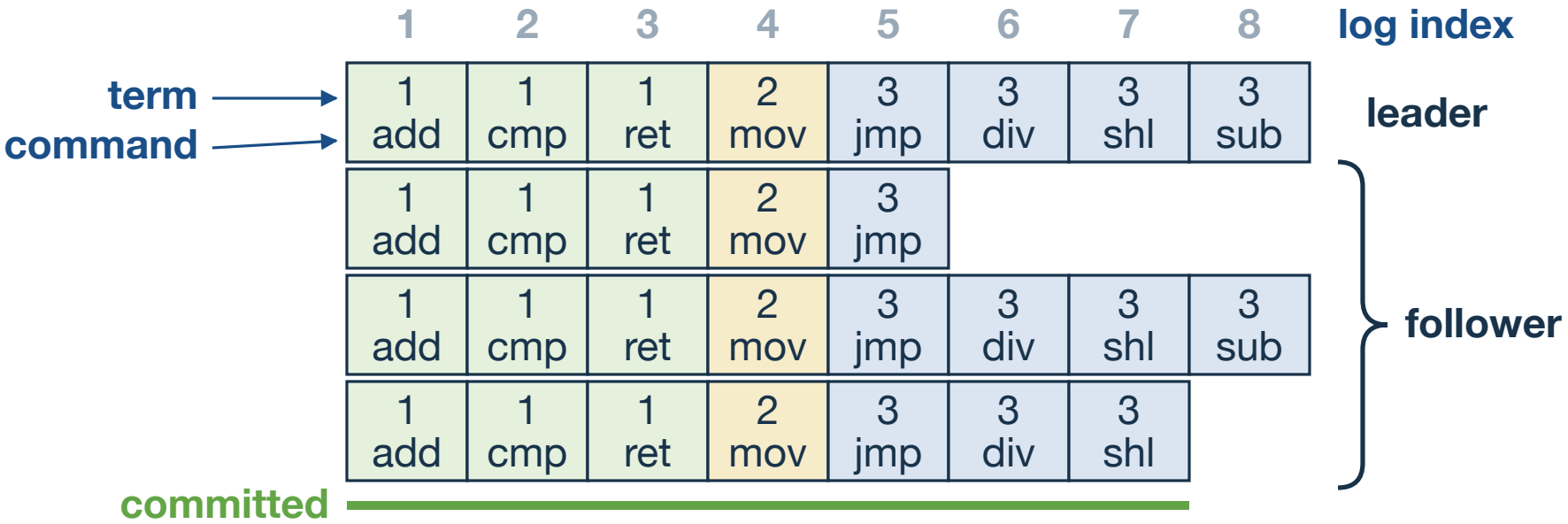
Part two of the decomposition: log replication

Log replication is the normal-operation path

- The leader appends the commands it gets from clients to its log, then replicates that log to the other servers

A log entry = `index` + `term` + `command`, stored on disk to tolerate failures

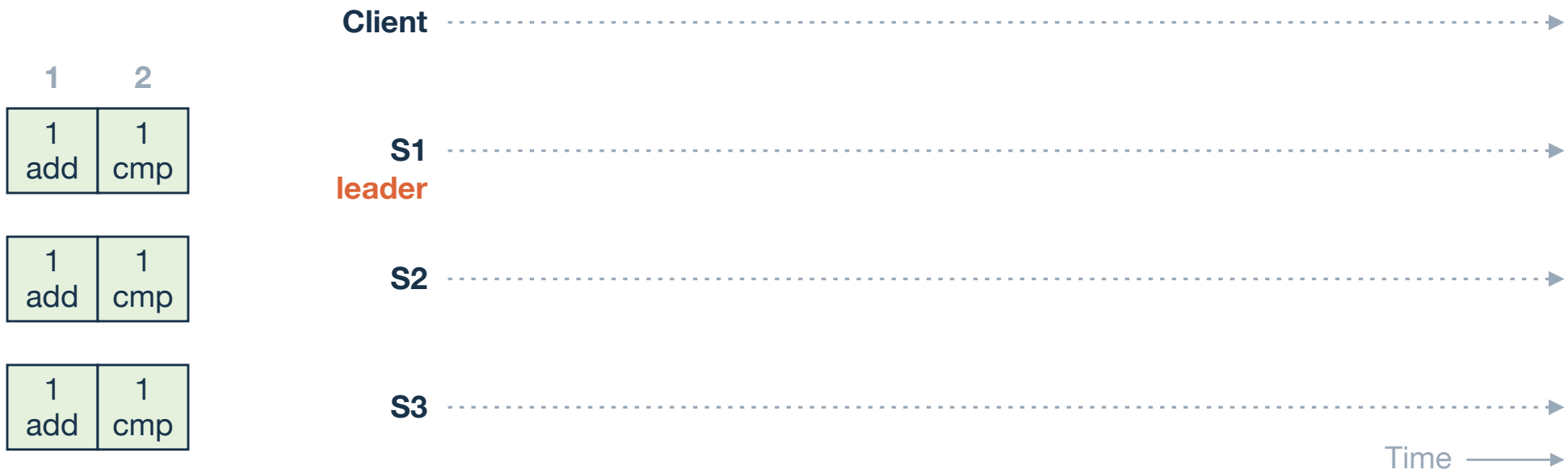
An entry is committed if it will eventually be stored on all the servers with the same value



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

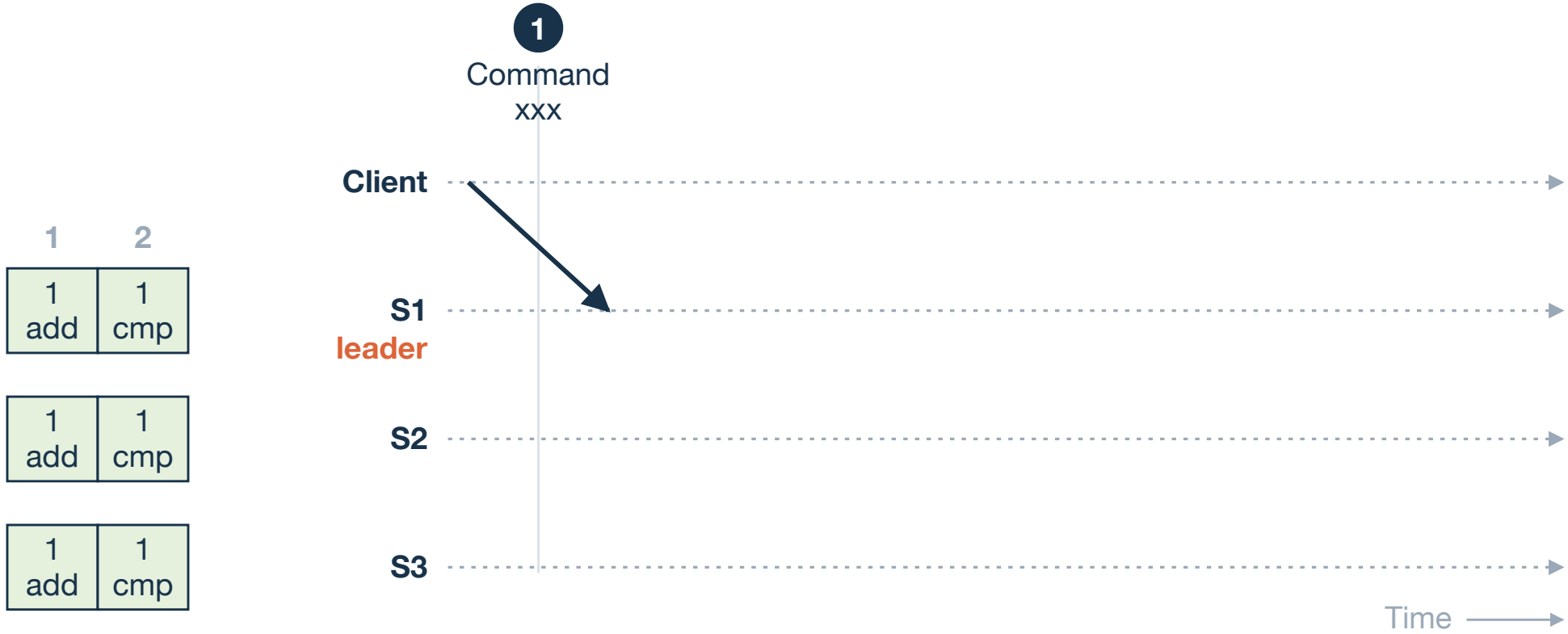
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

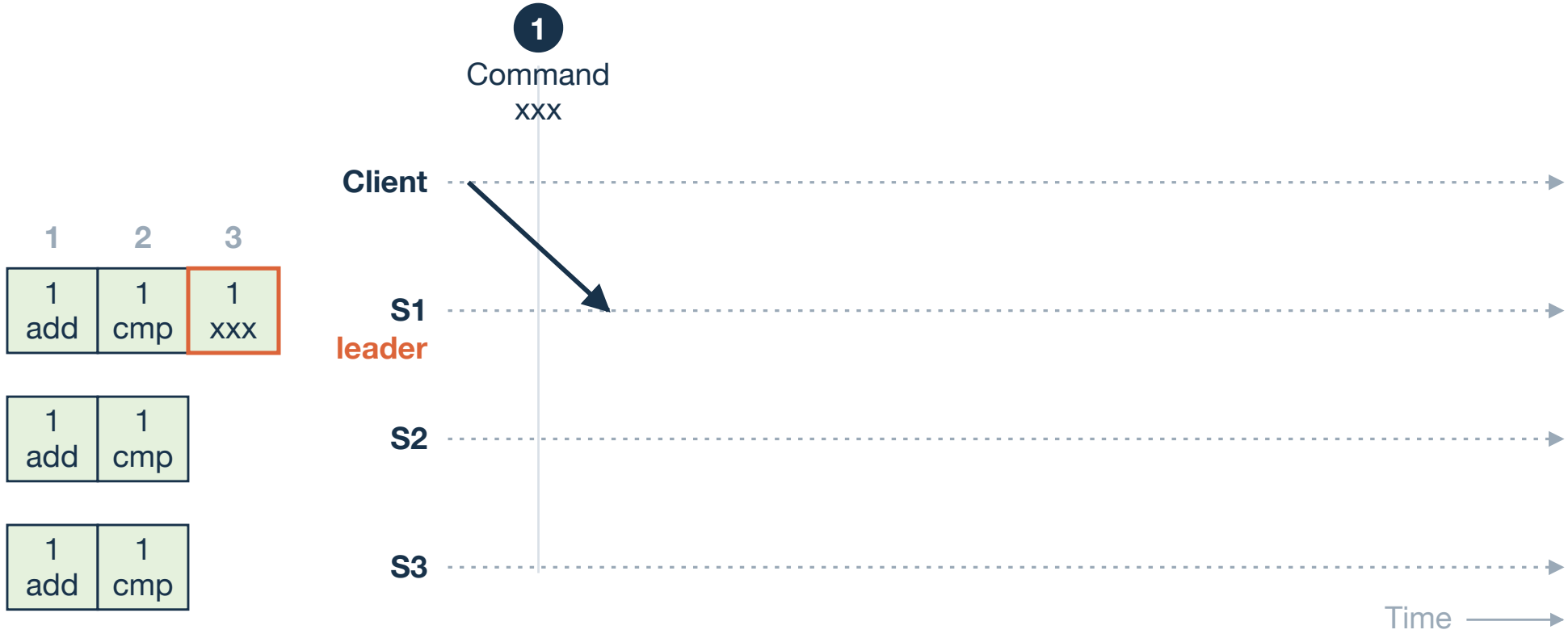
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

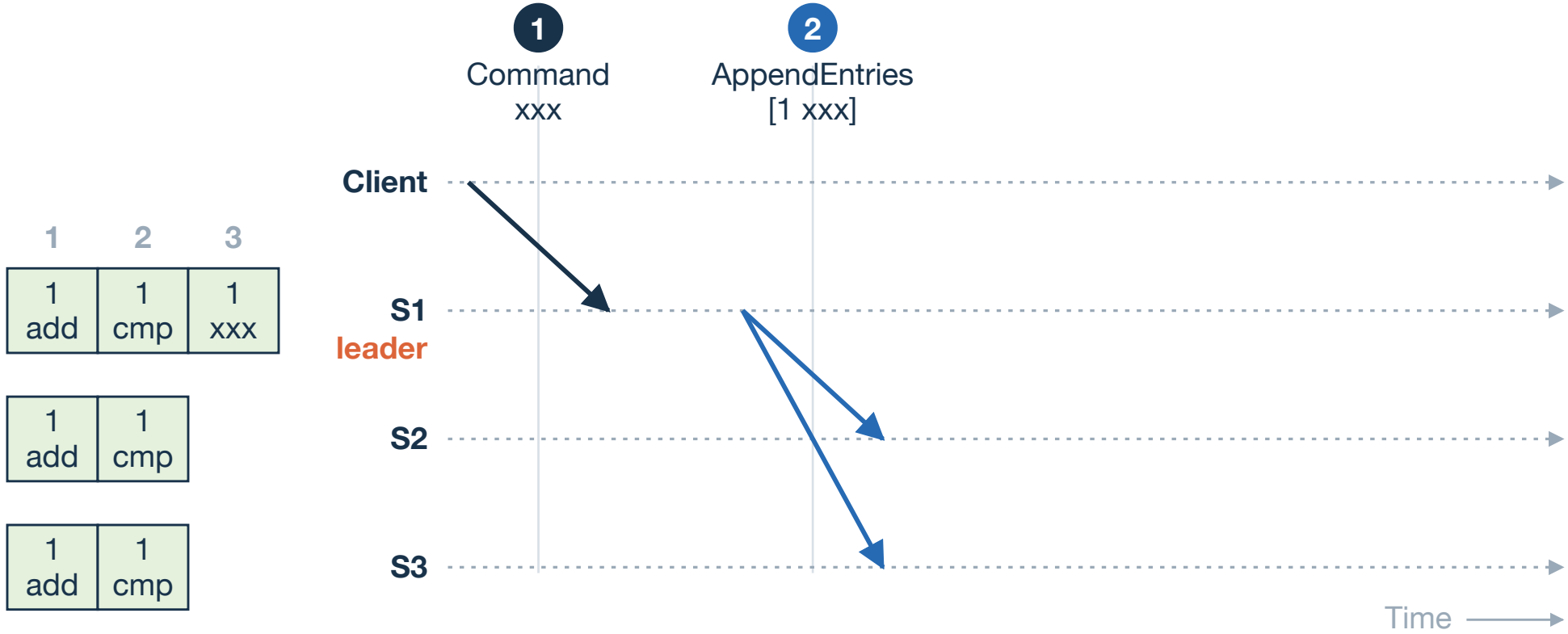
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

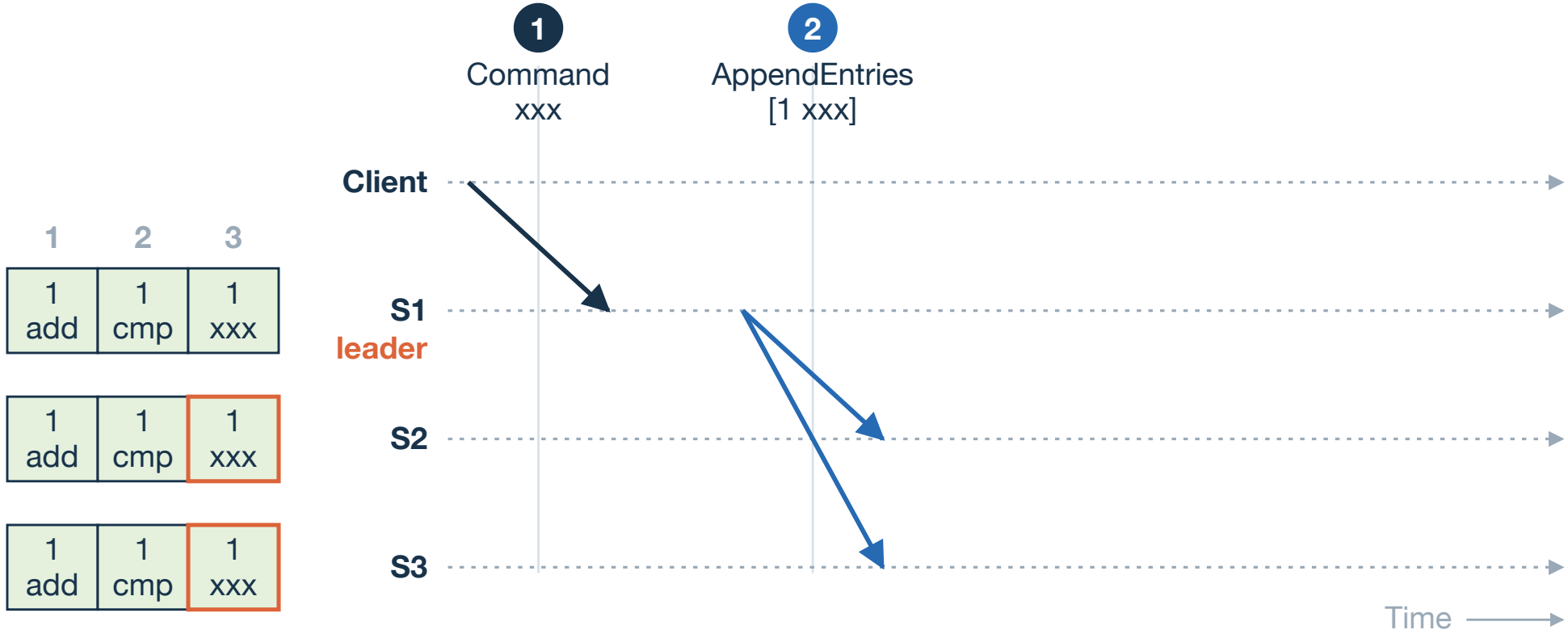
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

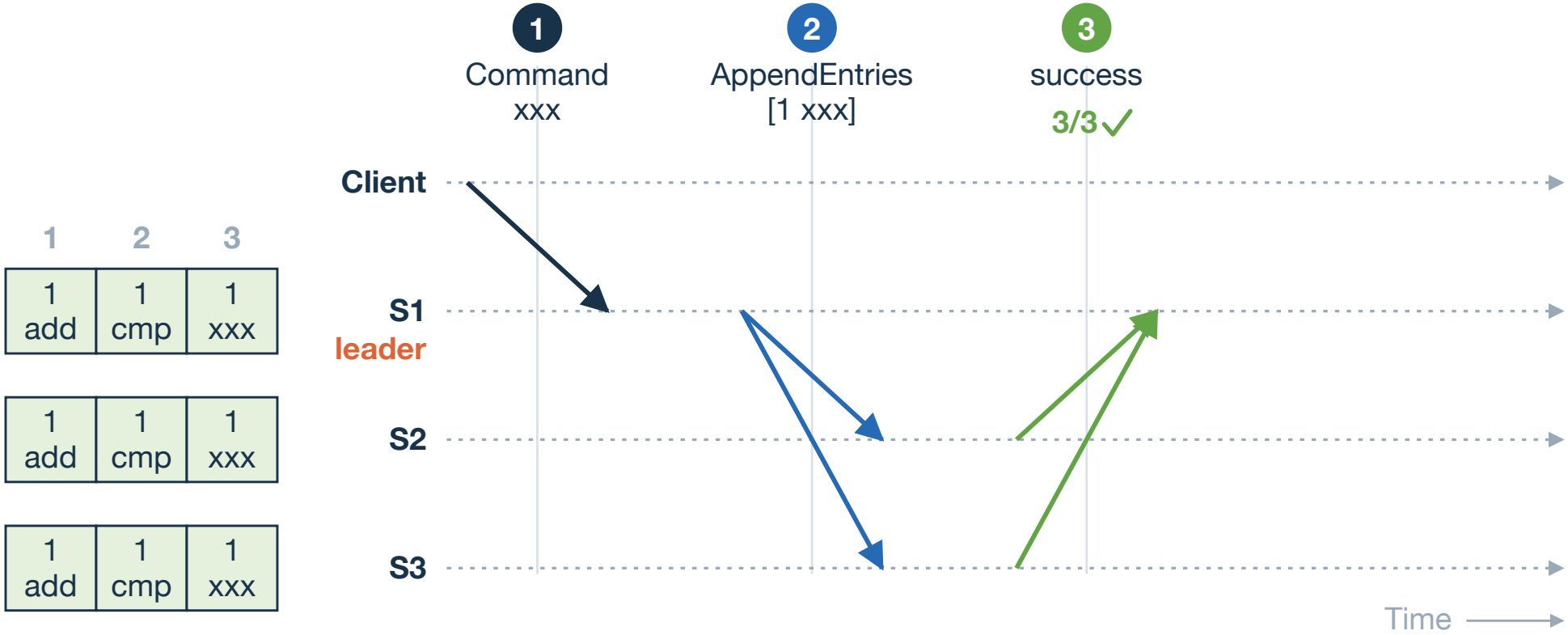
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

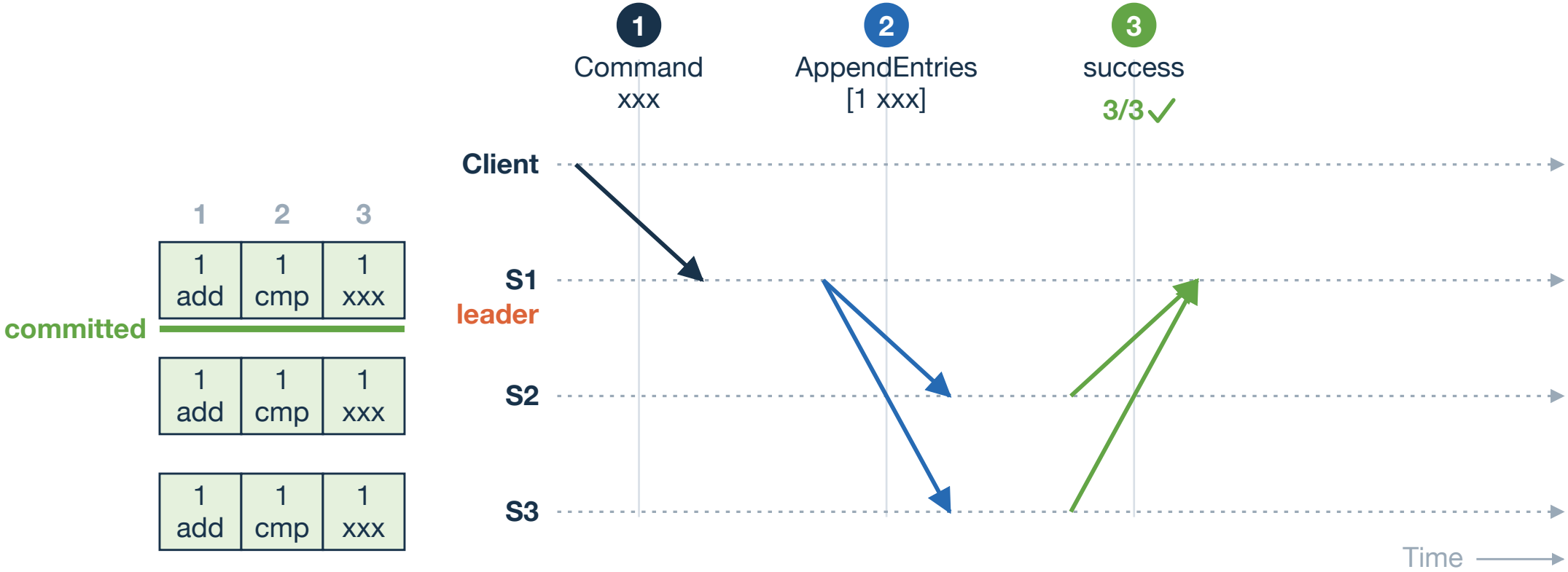
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

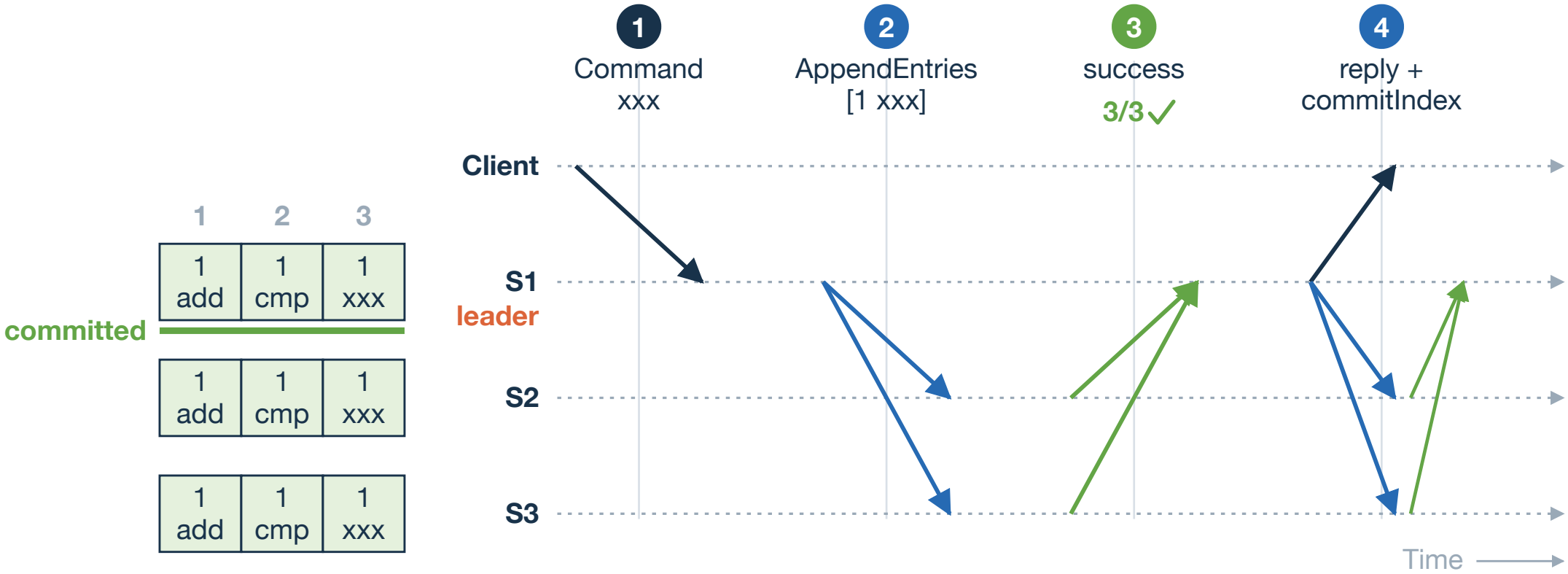
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

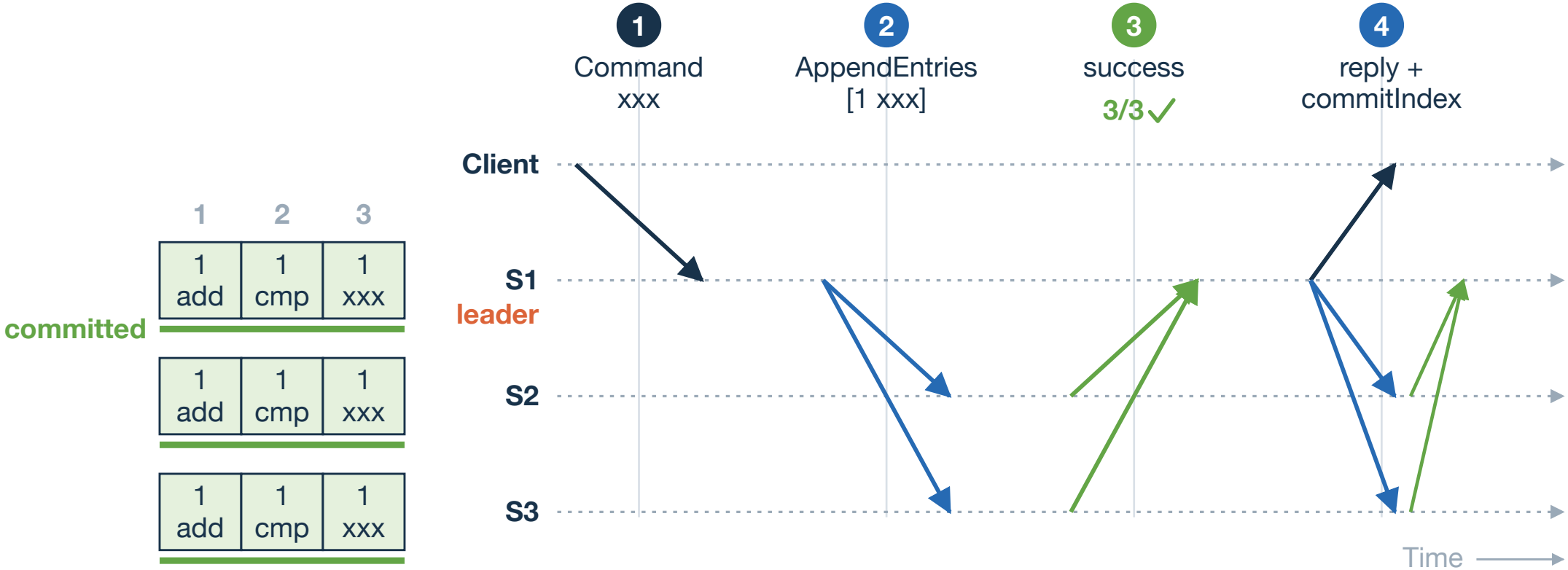
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: the path of one log entry replication

A normal write: append -> AppendEntries -> committed once a majority responds

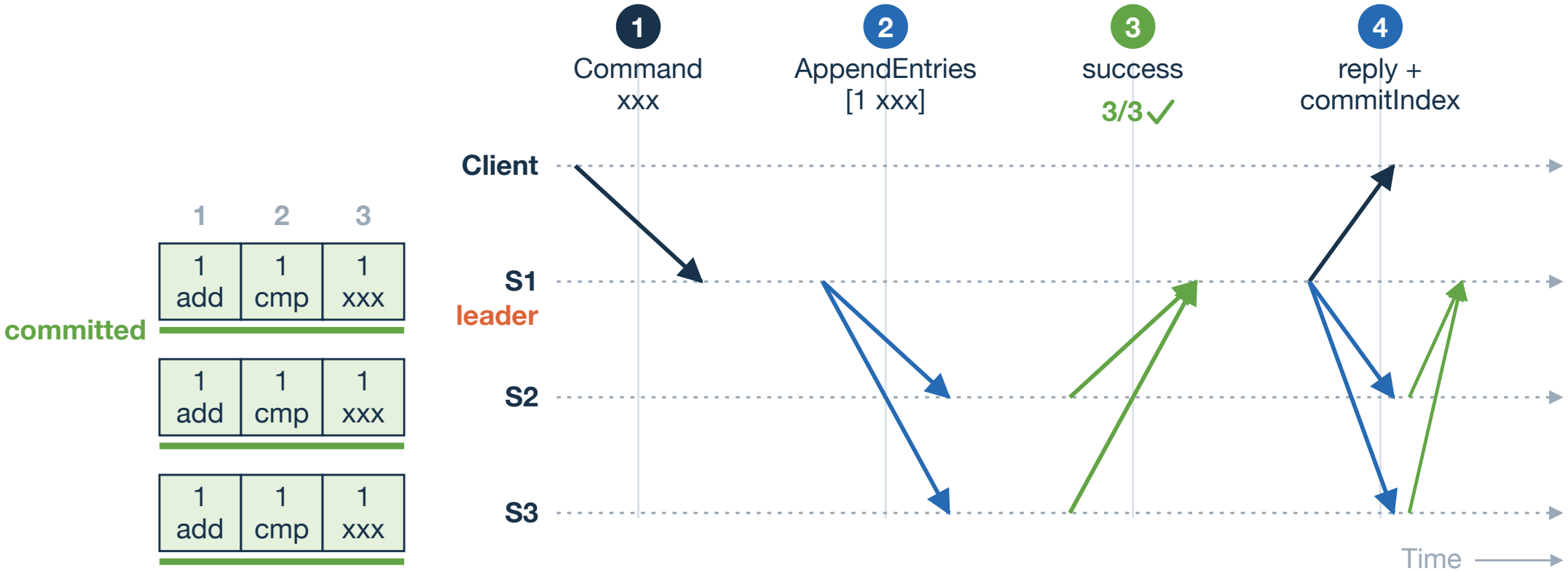
- Only then does the leader execute and reply, piggybacking the commit point



Normal operation: crashed or slow followers

A slow or crashed follower does not hold up the commit

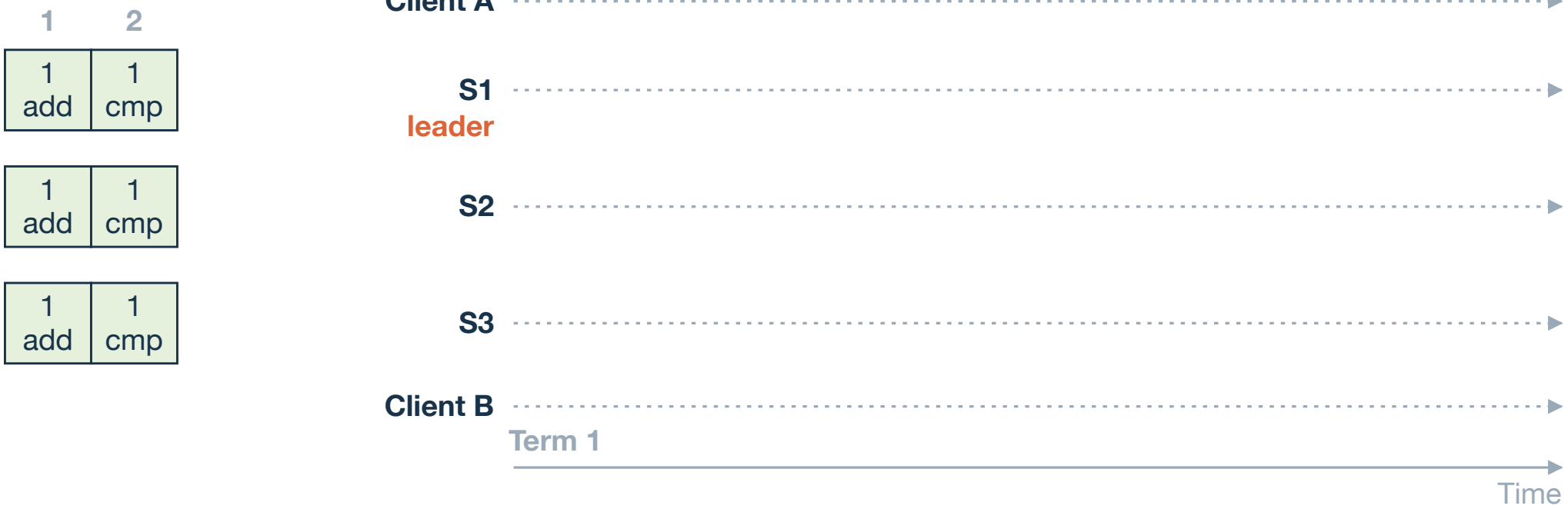
- The leader retries the RPC until it succeeds: `AppendEntries` is at-least-once



The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

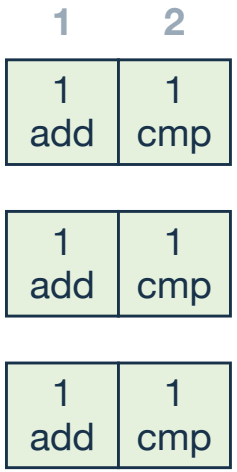
- Yet clients can still reach it, and it still appends to its own log

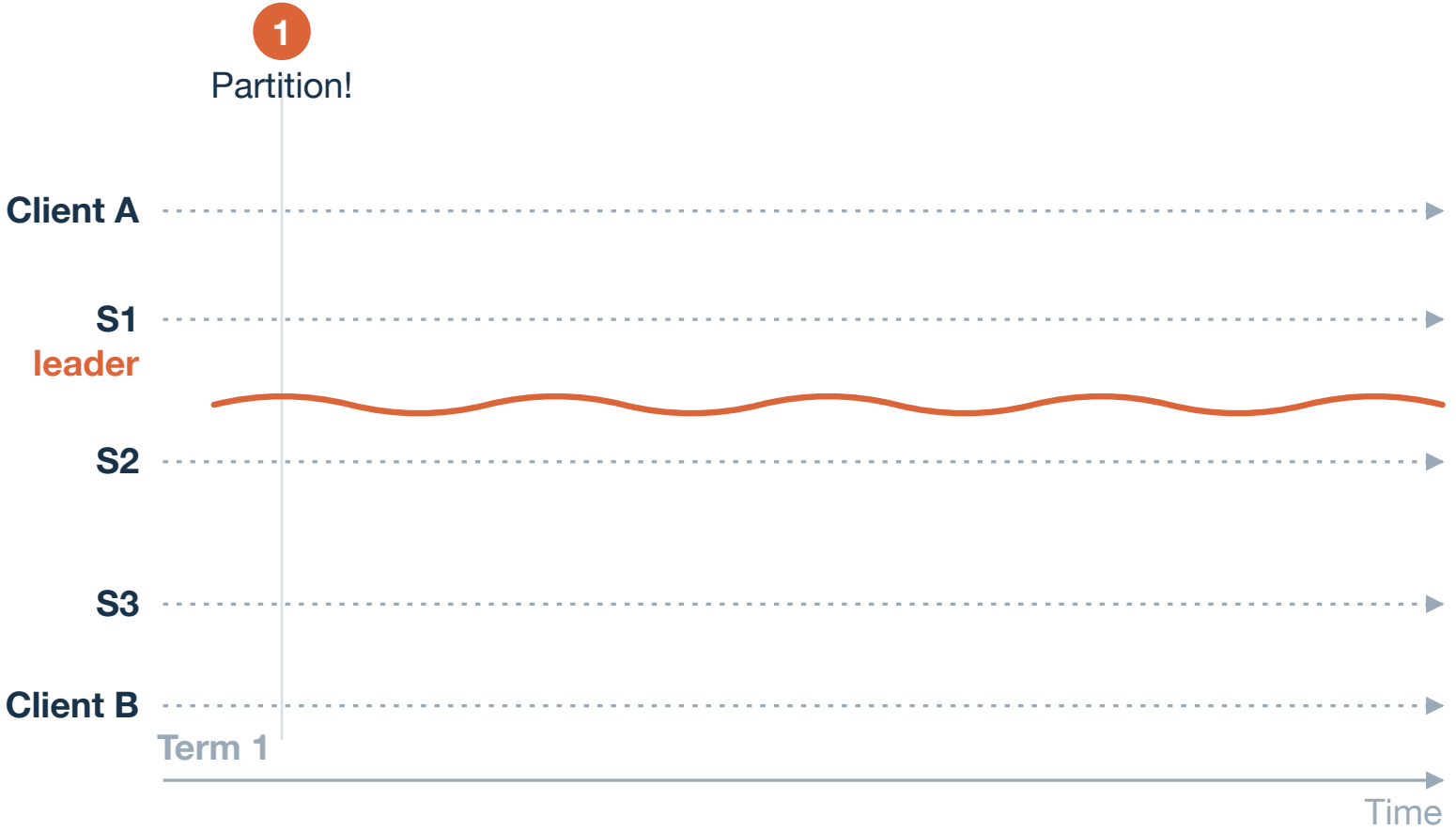


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

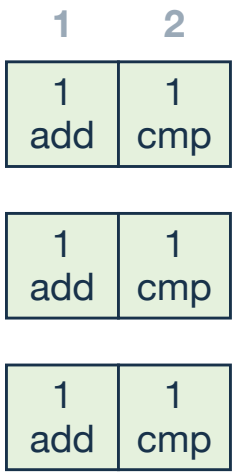


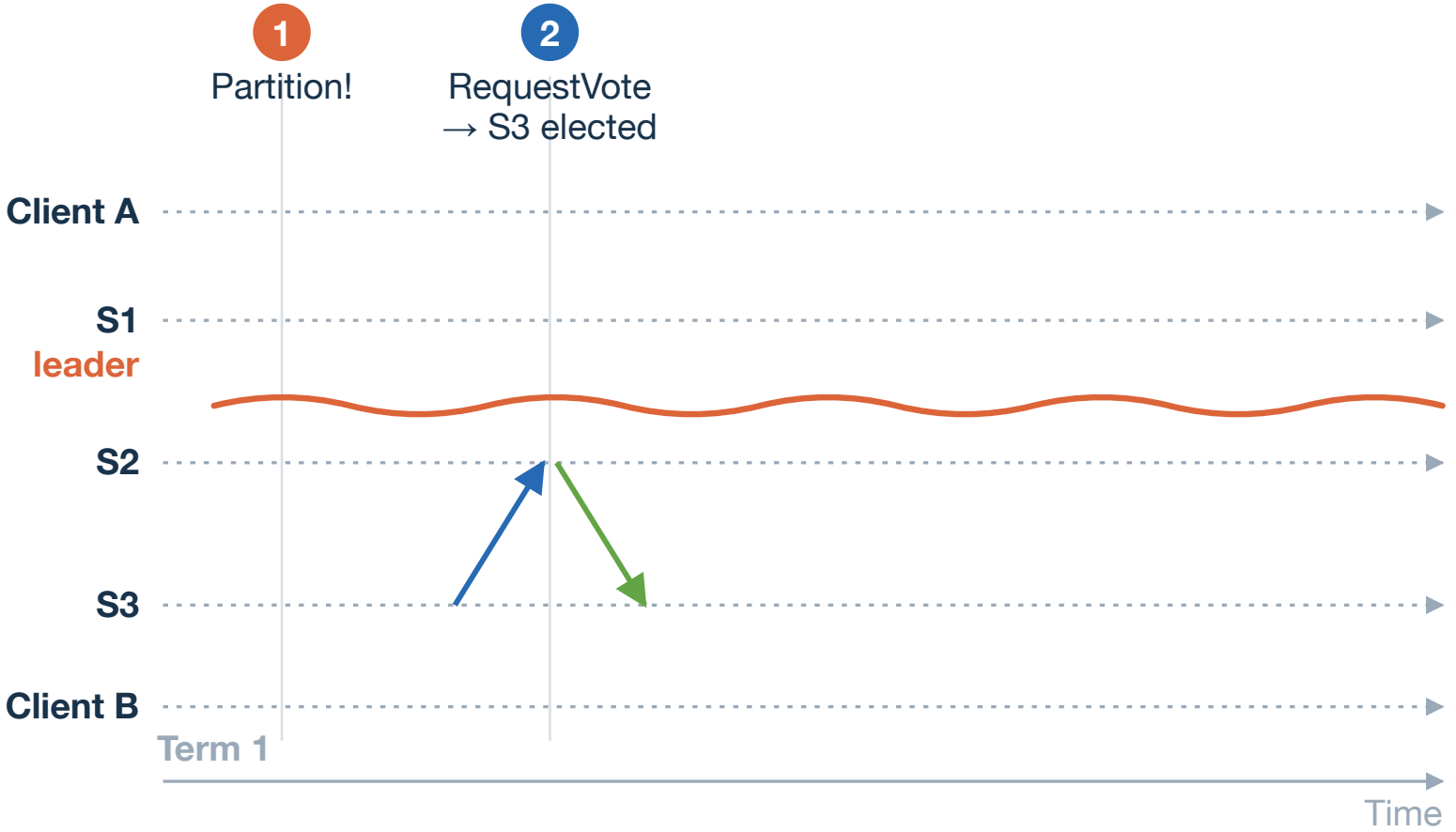


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

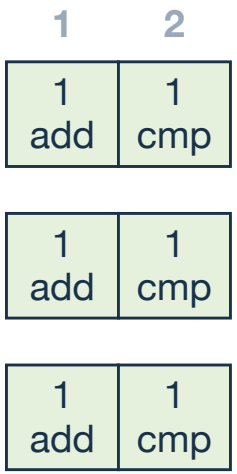


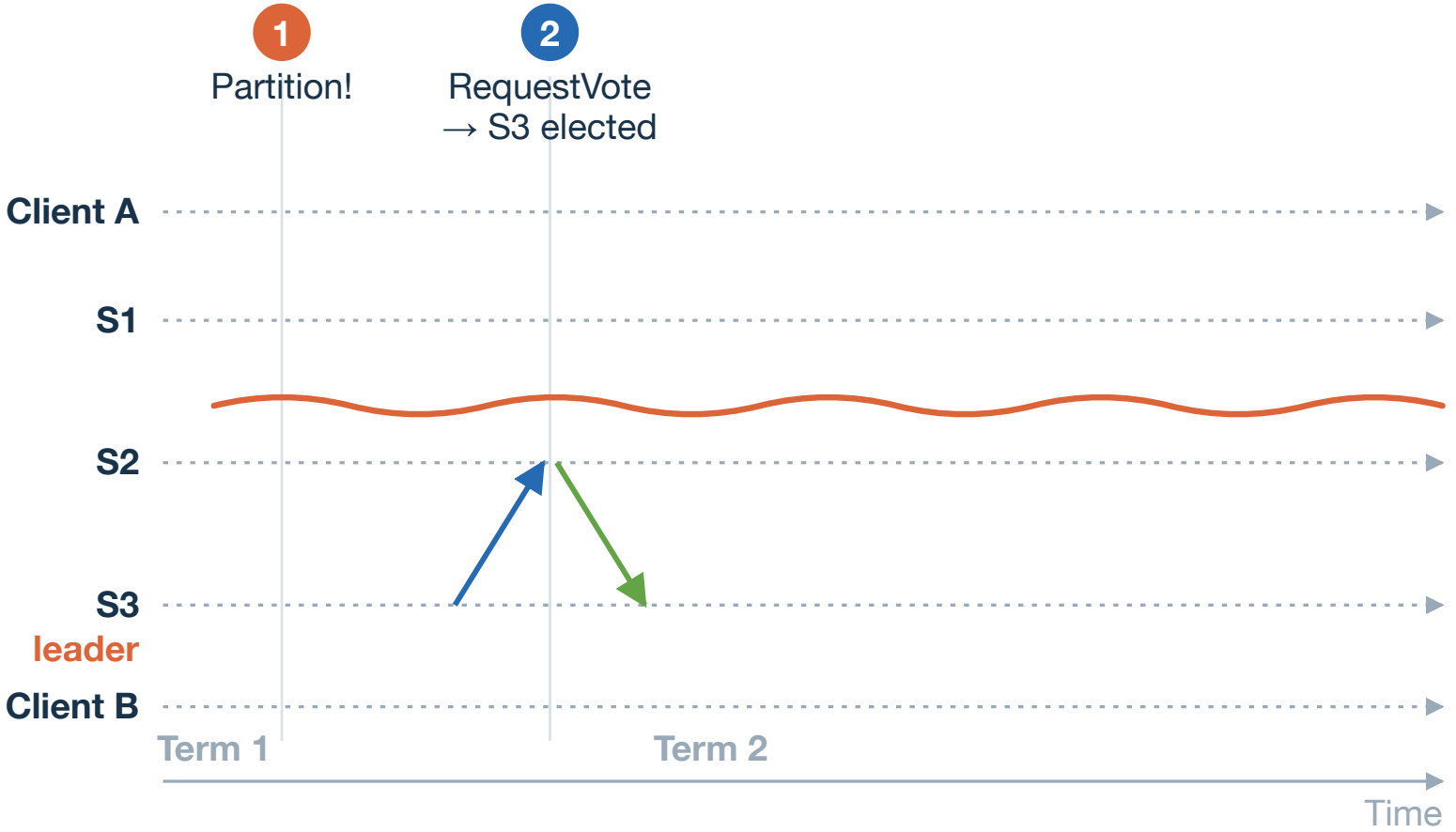


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

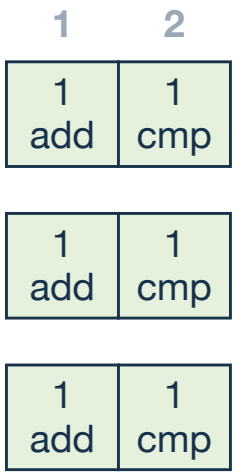


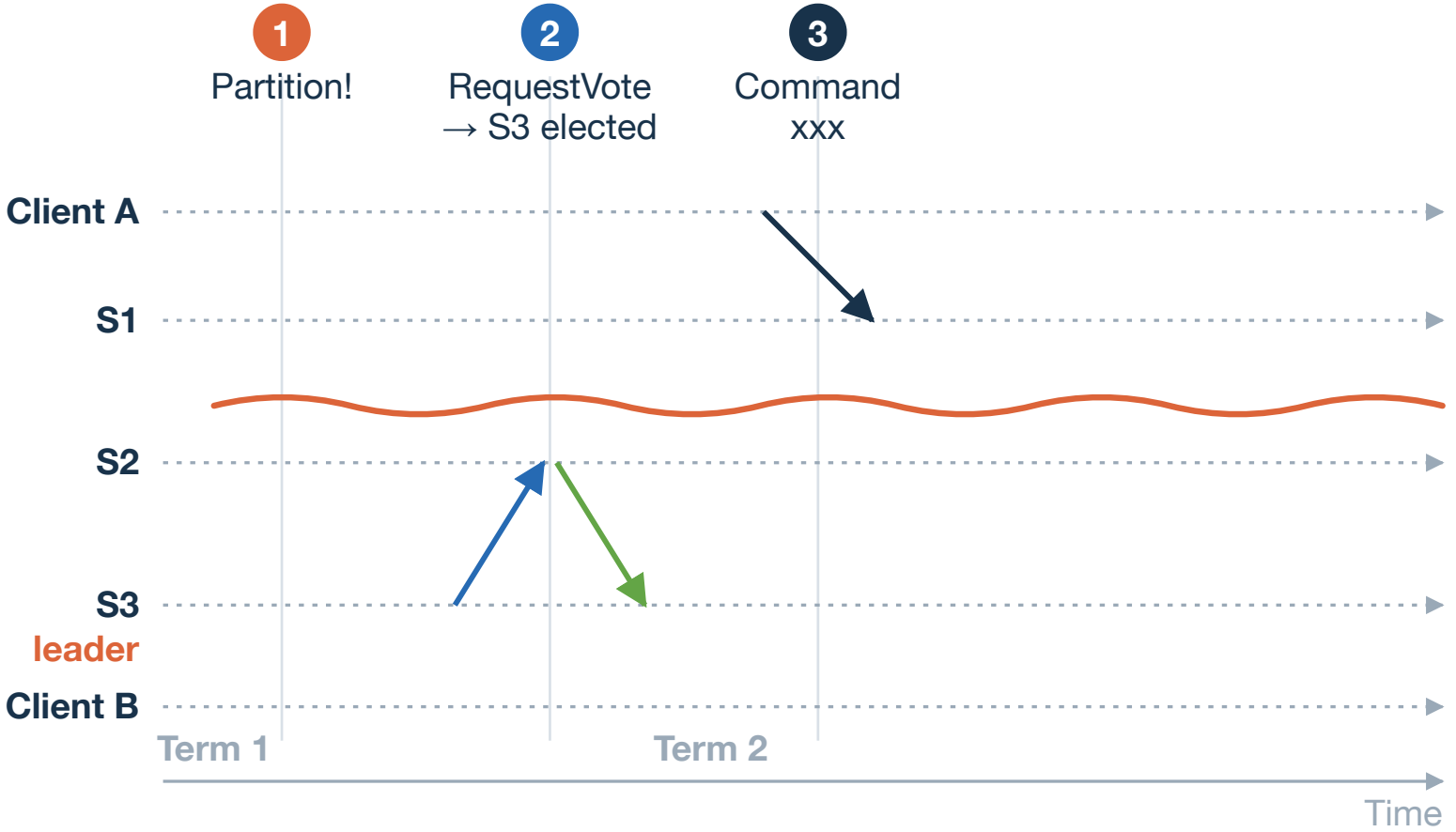


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

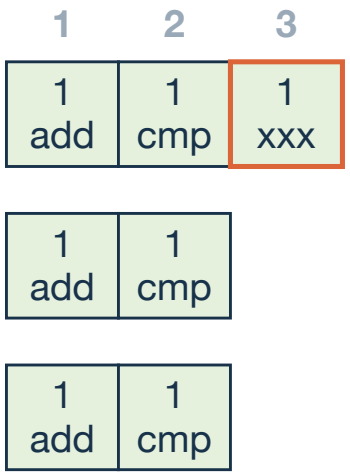


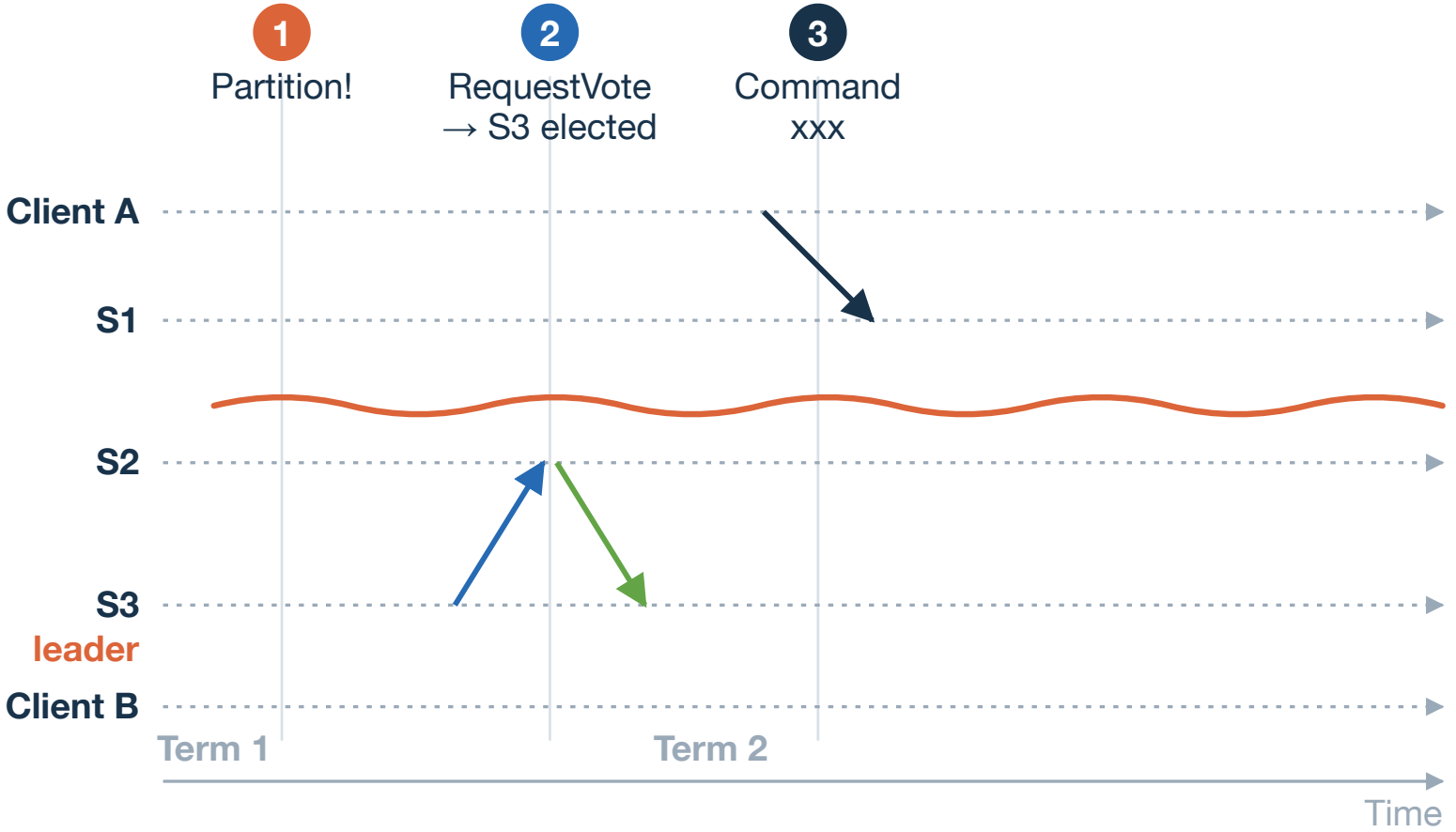


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

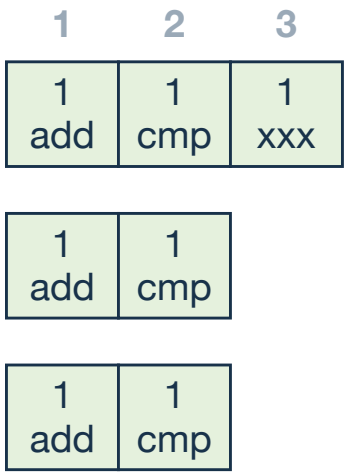


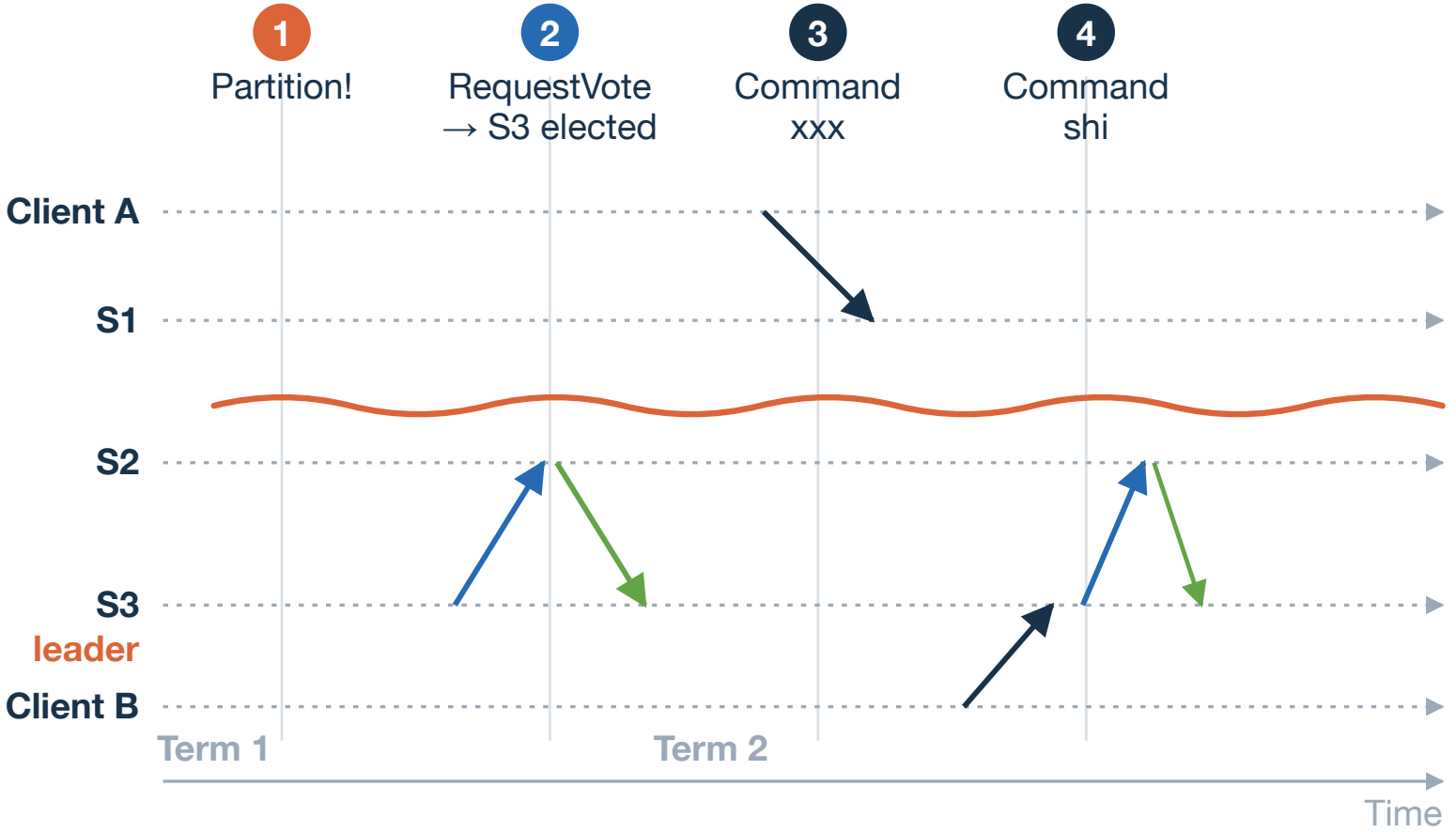


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

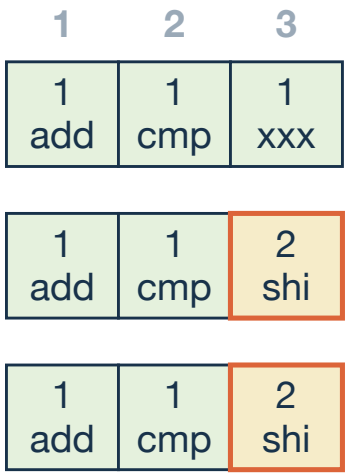


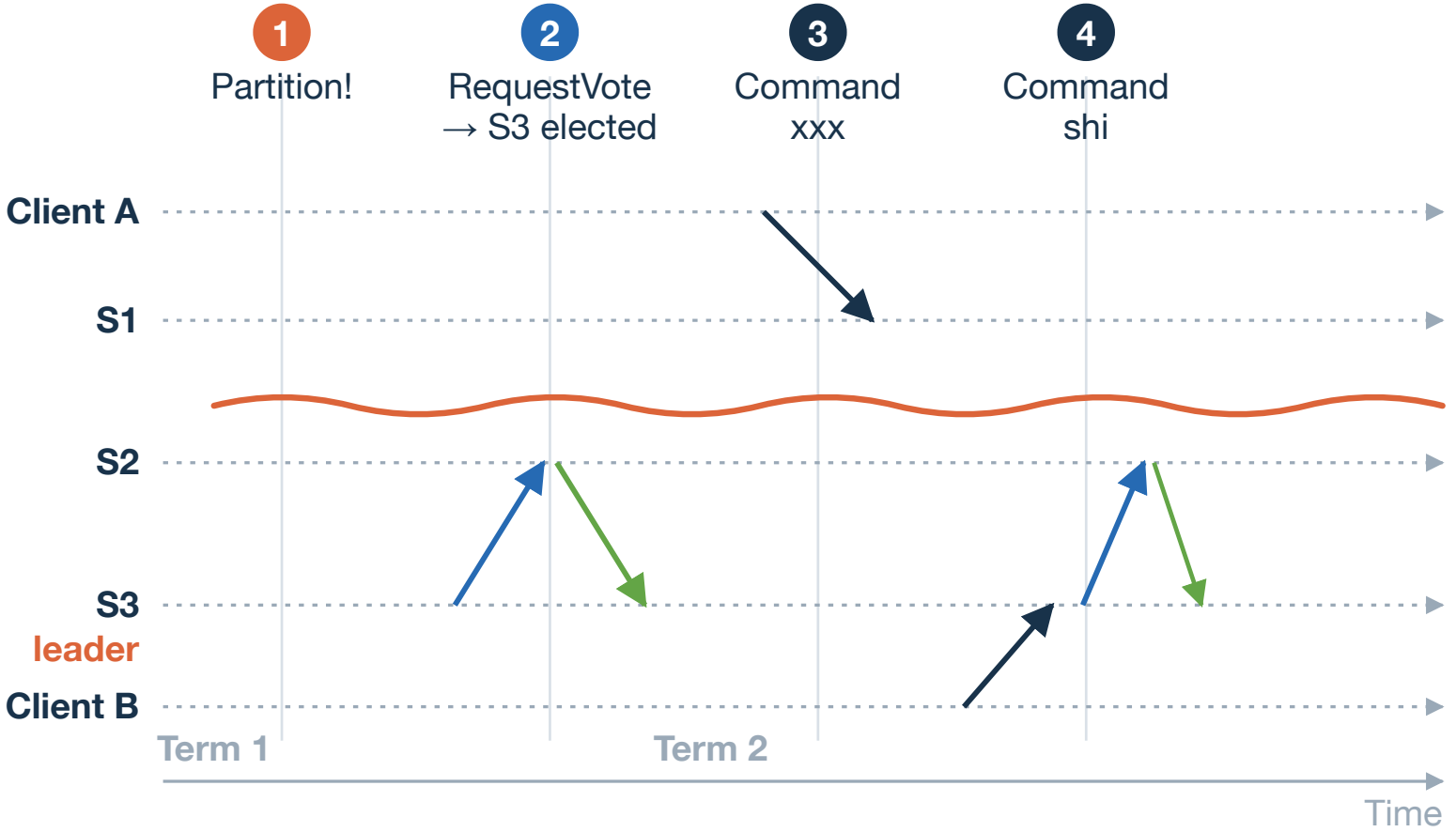


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

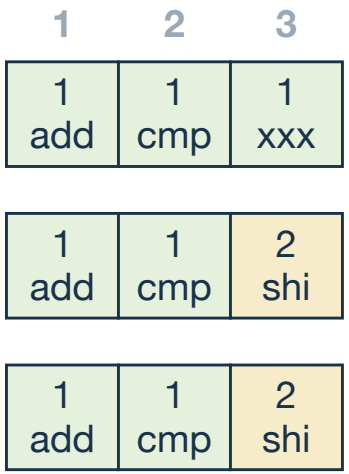


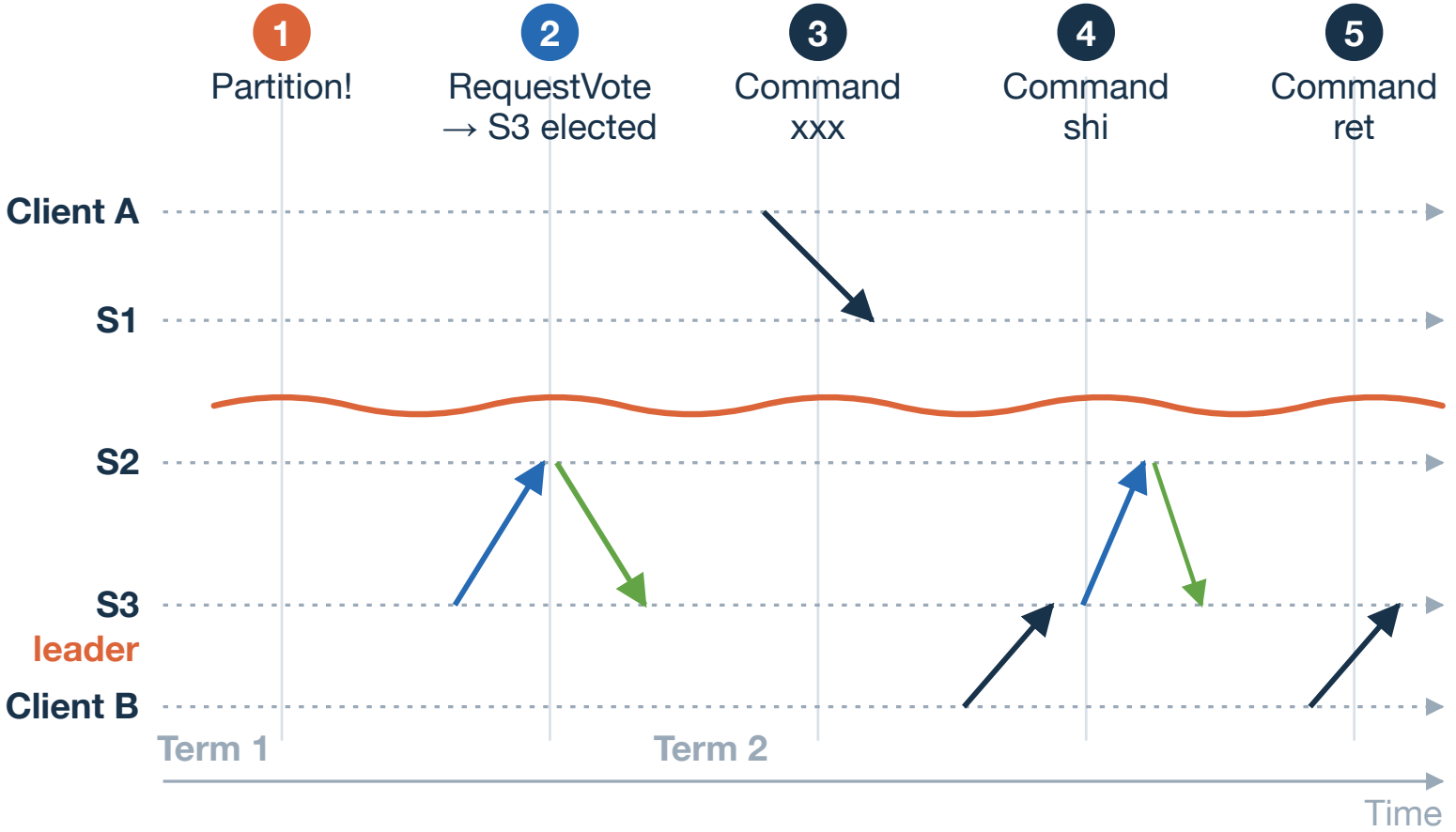


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log

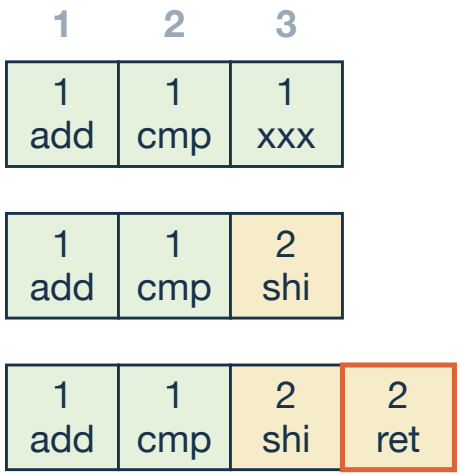


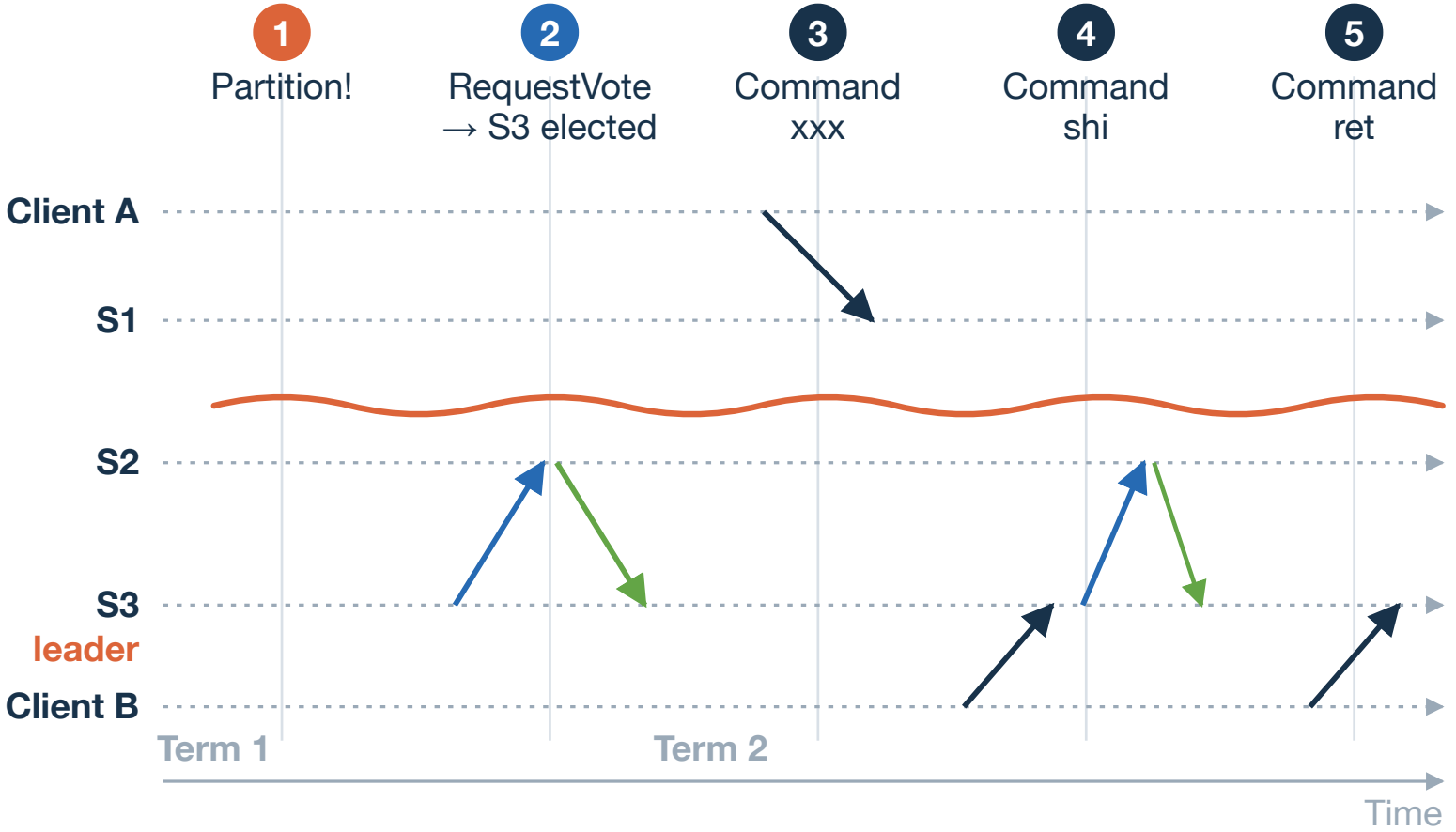


The problem: crashes and partitions make logs diverge

An old leader can be unaware of the new leader: a partition cut it off from the majority

- Yet clients can still reach it, and it still appends to its own log





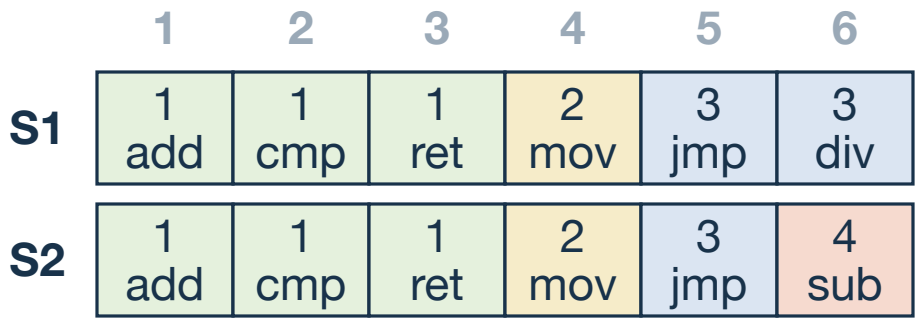
The coherency Raft maintains

Raft minimizes special code by repairing inconsistencies

- Leaders assume their own log is correct; normal operation will repair all inconsistencies

The consistency guarantee: If log entries on different servers have the same index & term

- They store the same command
- And the logs are identical in all preceding entries

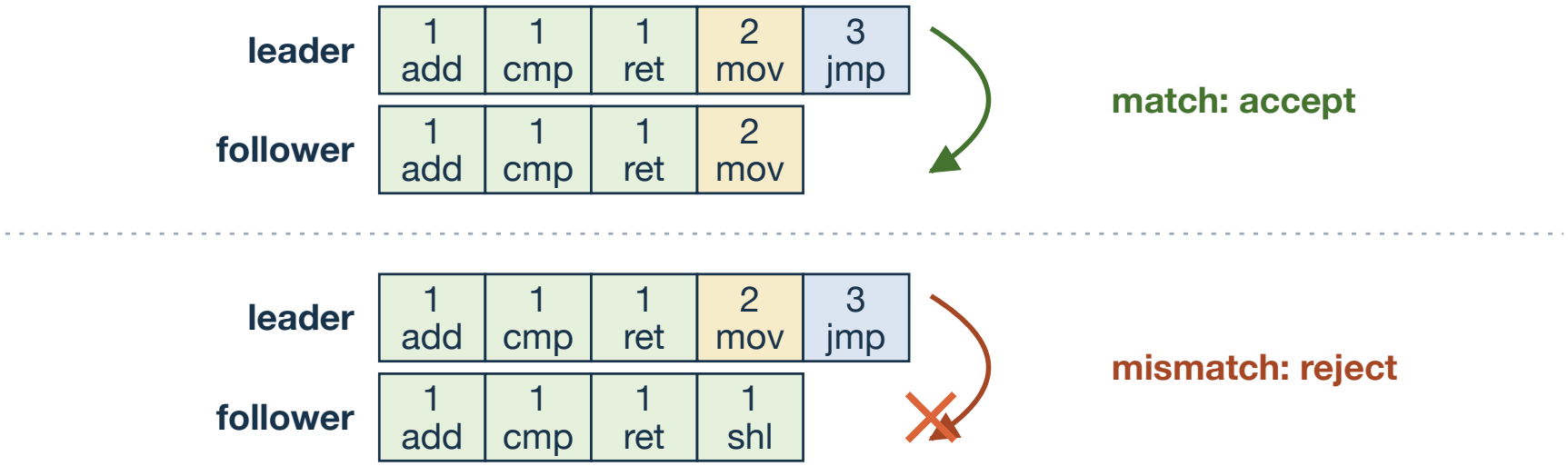


Benefit: If a given entry is committed, all preceding entries are also committed

Realizing coherence: AppendEntries consistency check

Each `AppendEntries` RPC carries the index and term of the entry preceding the new ones

The follower accepts only if it has that matching entry; otherwise it rejects the request

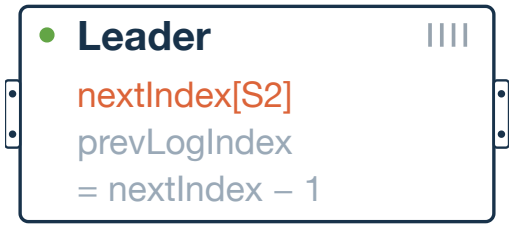


This check is the ==induction== step of coherency

Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

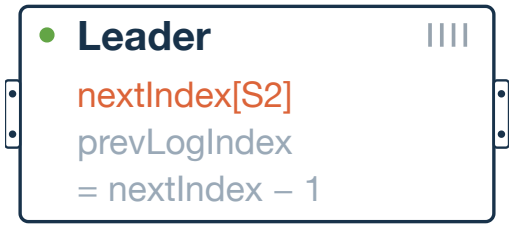


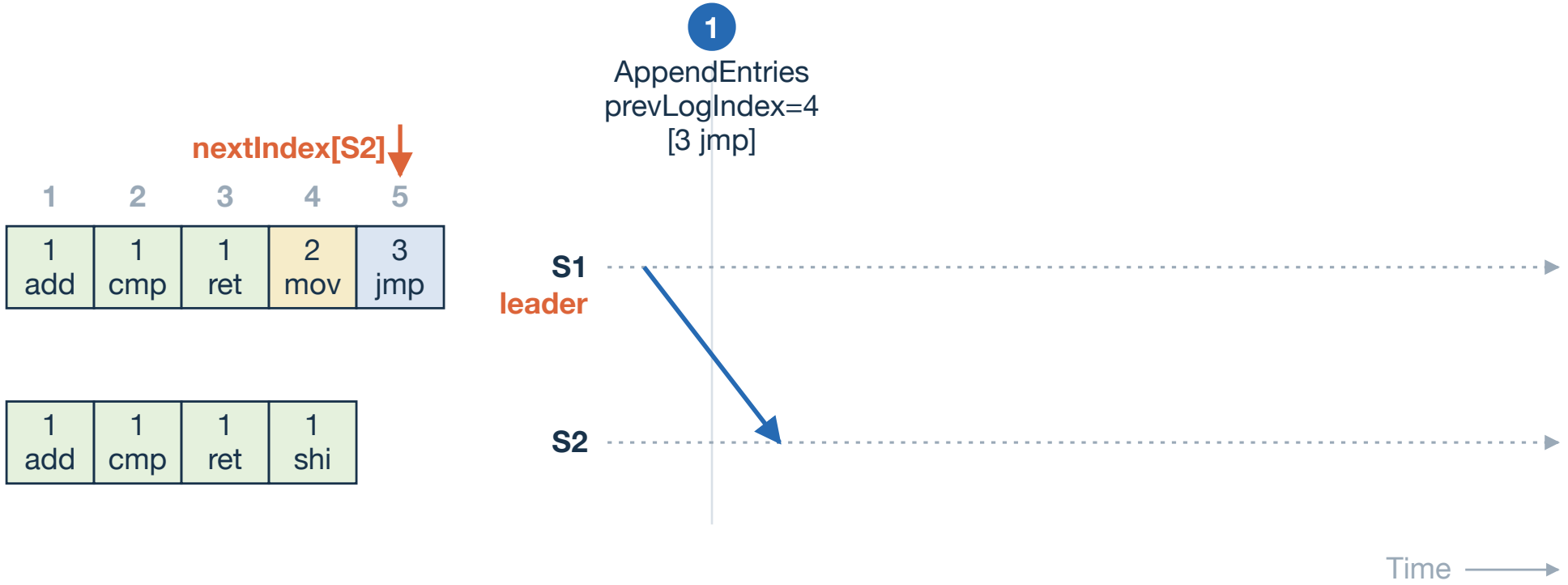


Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

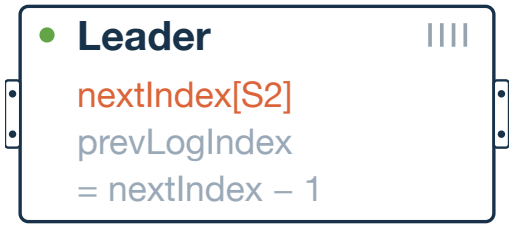


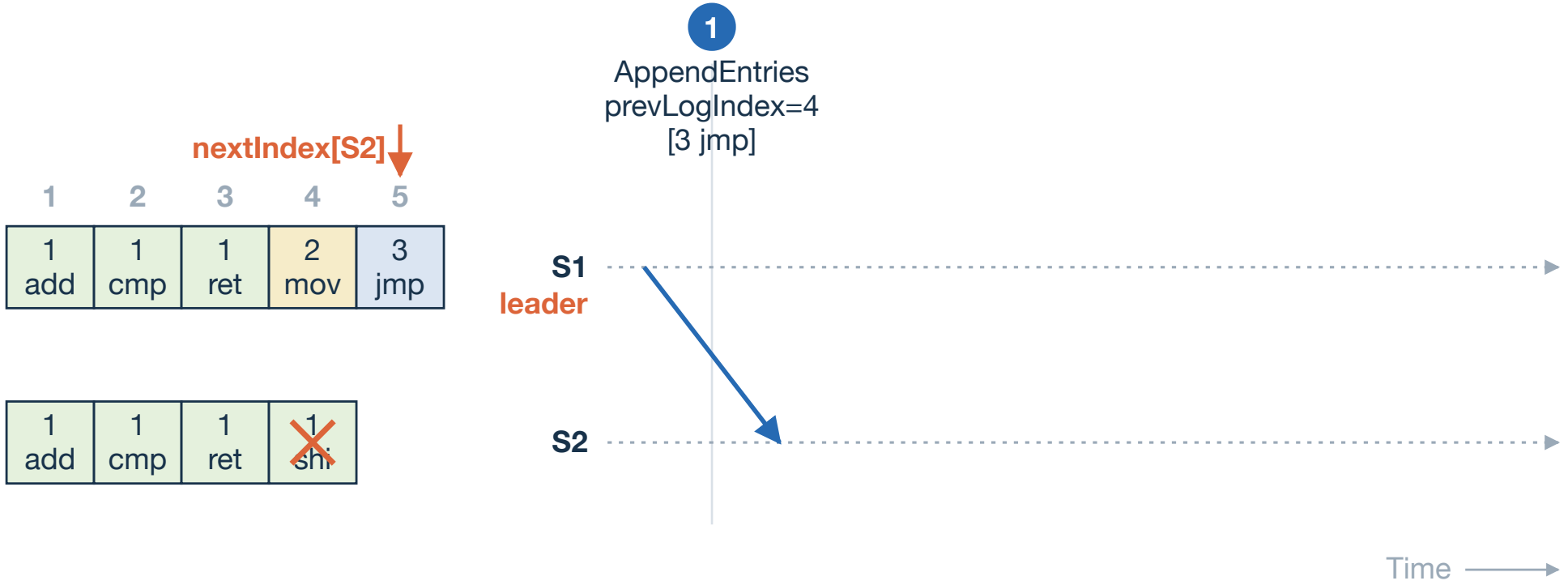


Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

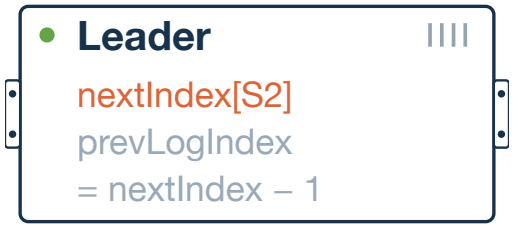


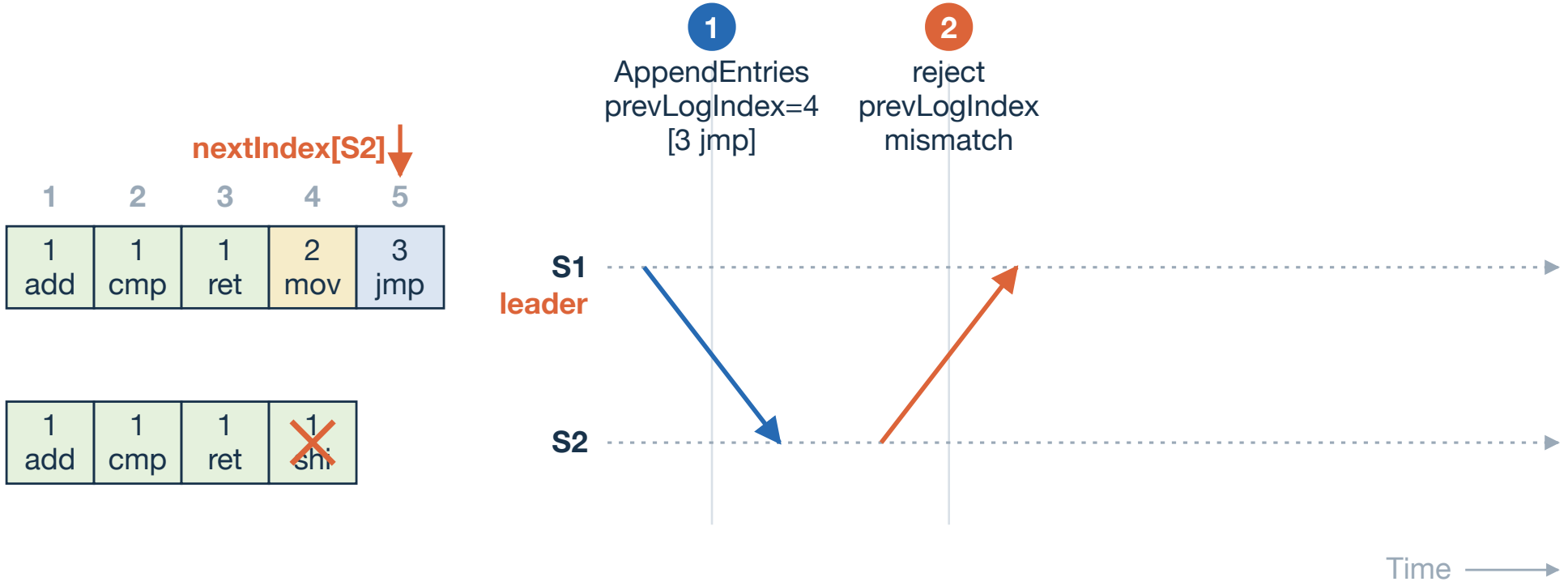


Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

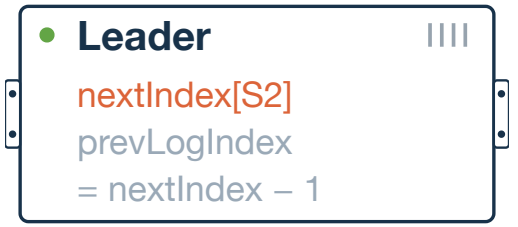


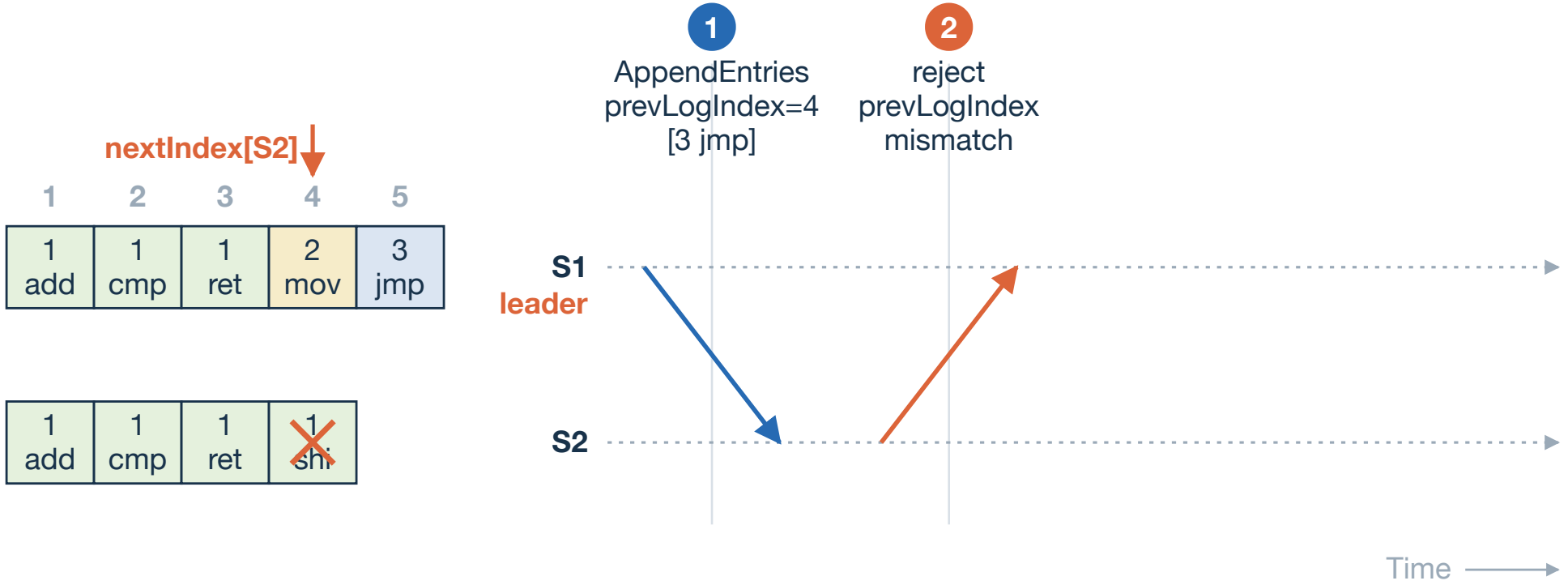


Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

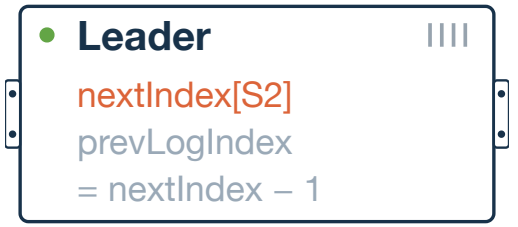


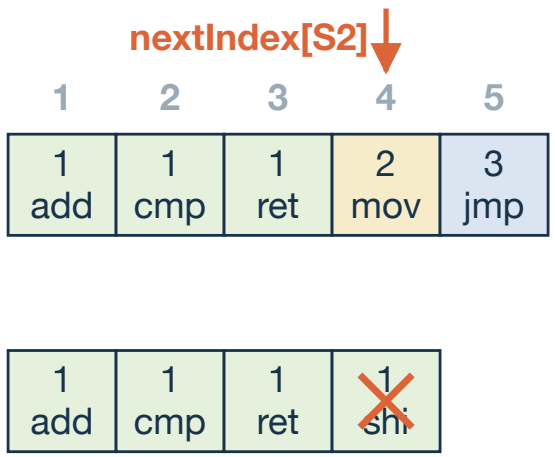


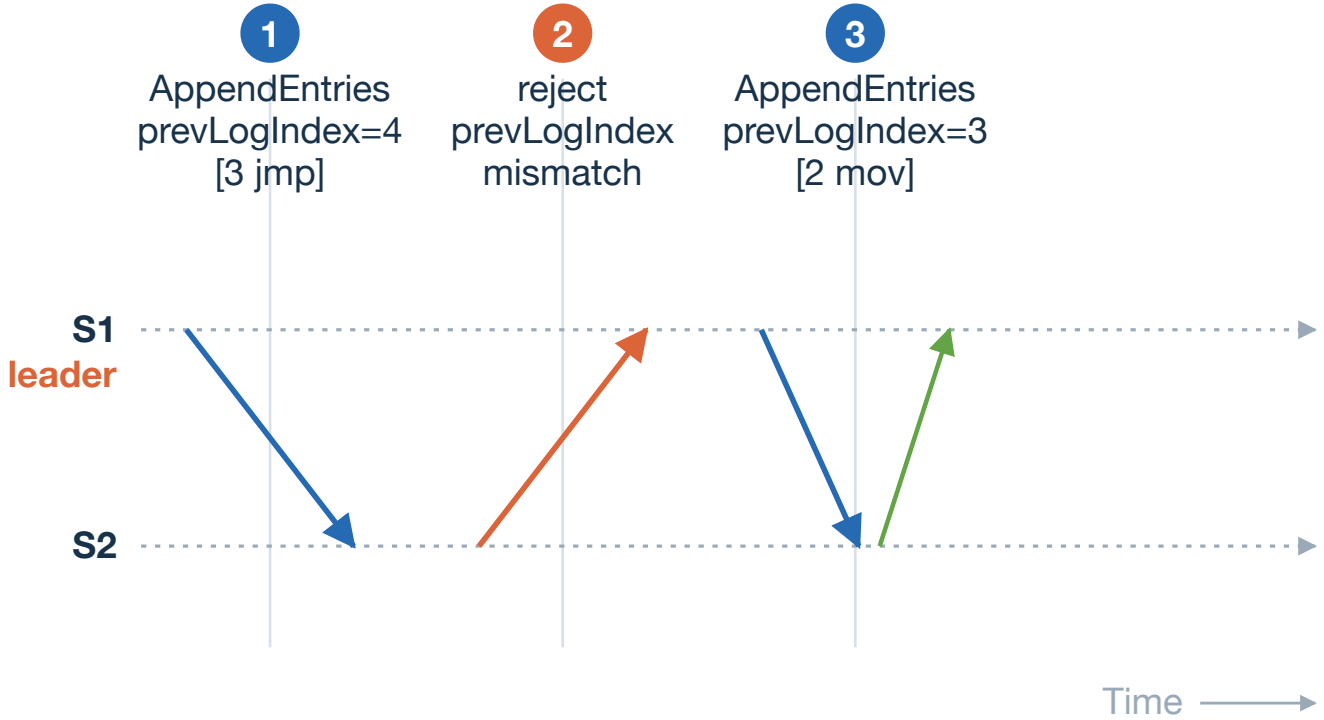
Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part



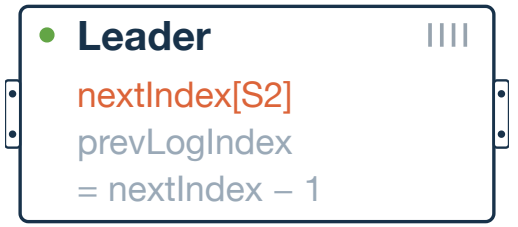


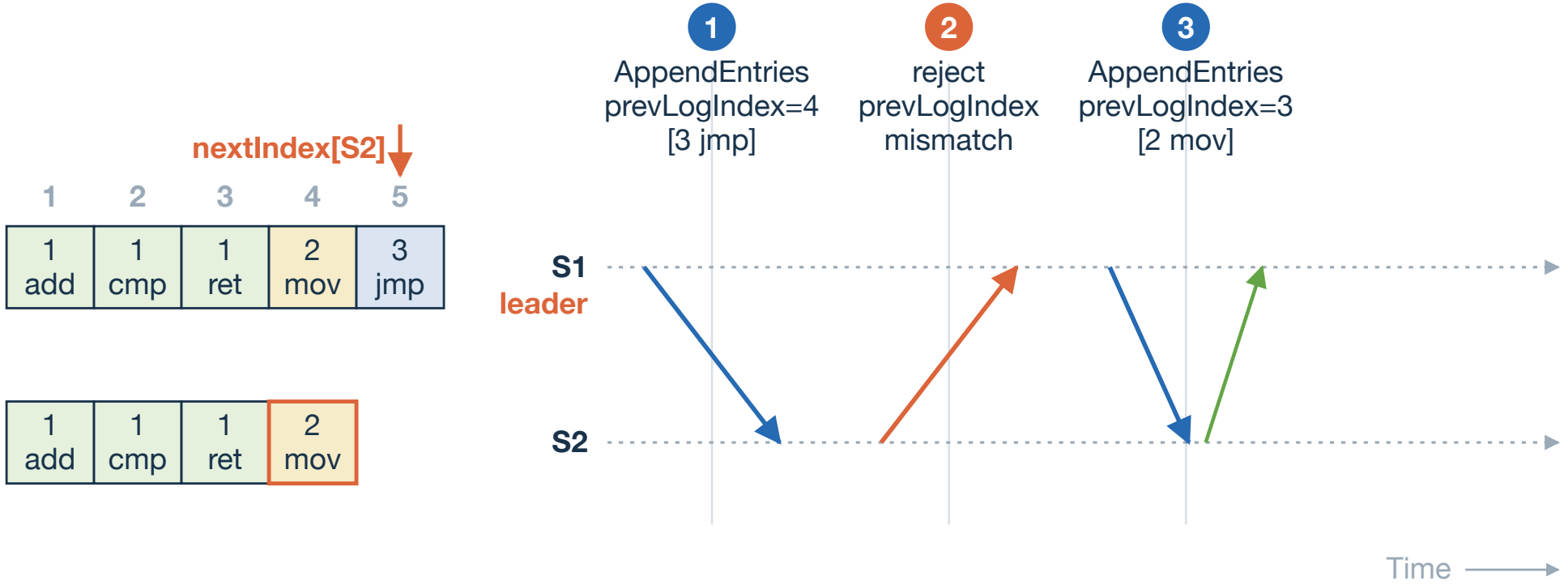


Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

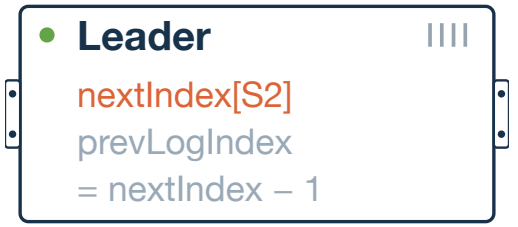


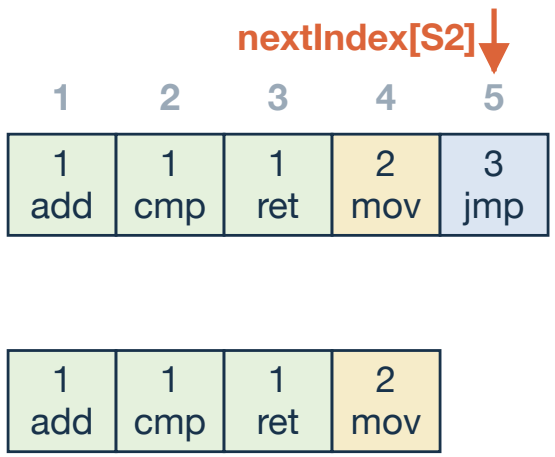


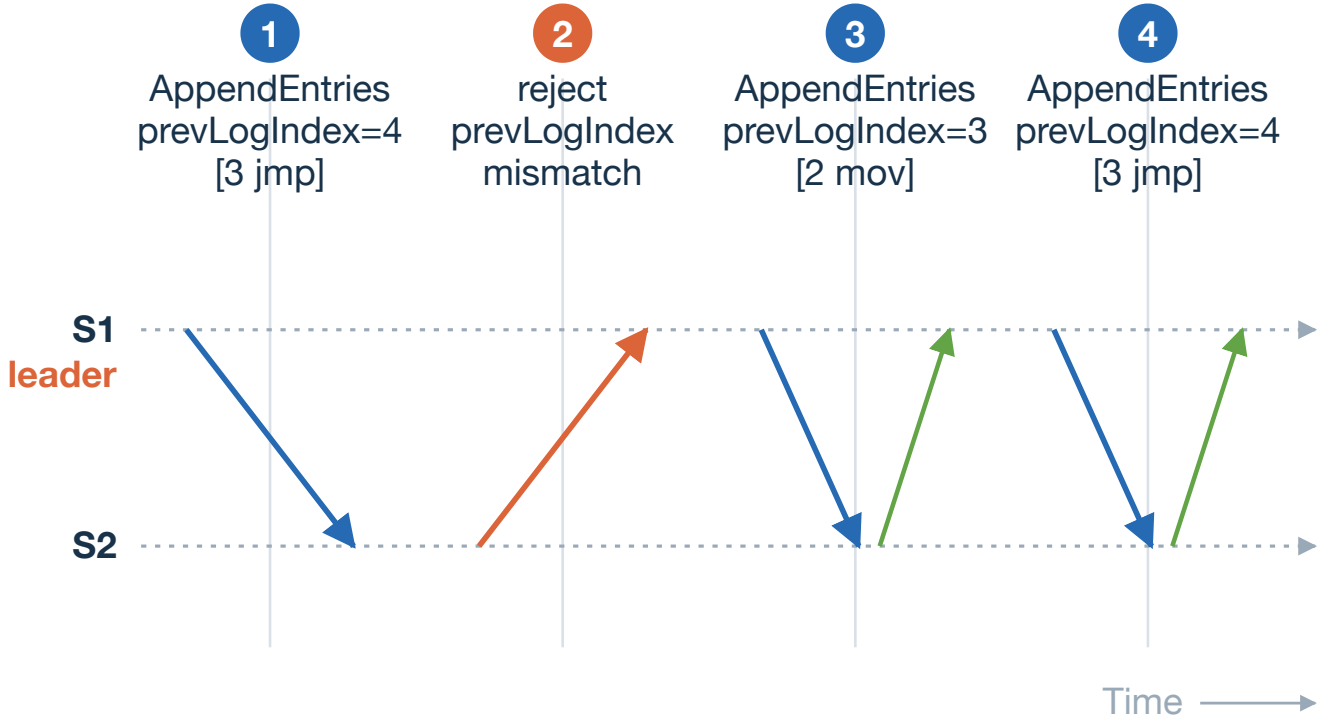
Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part



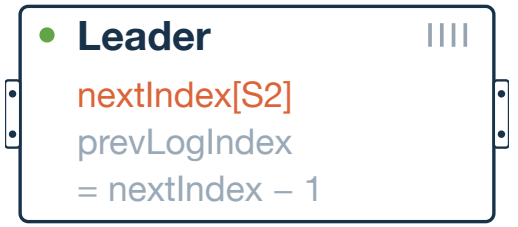


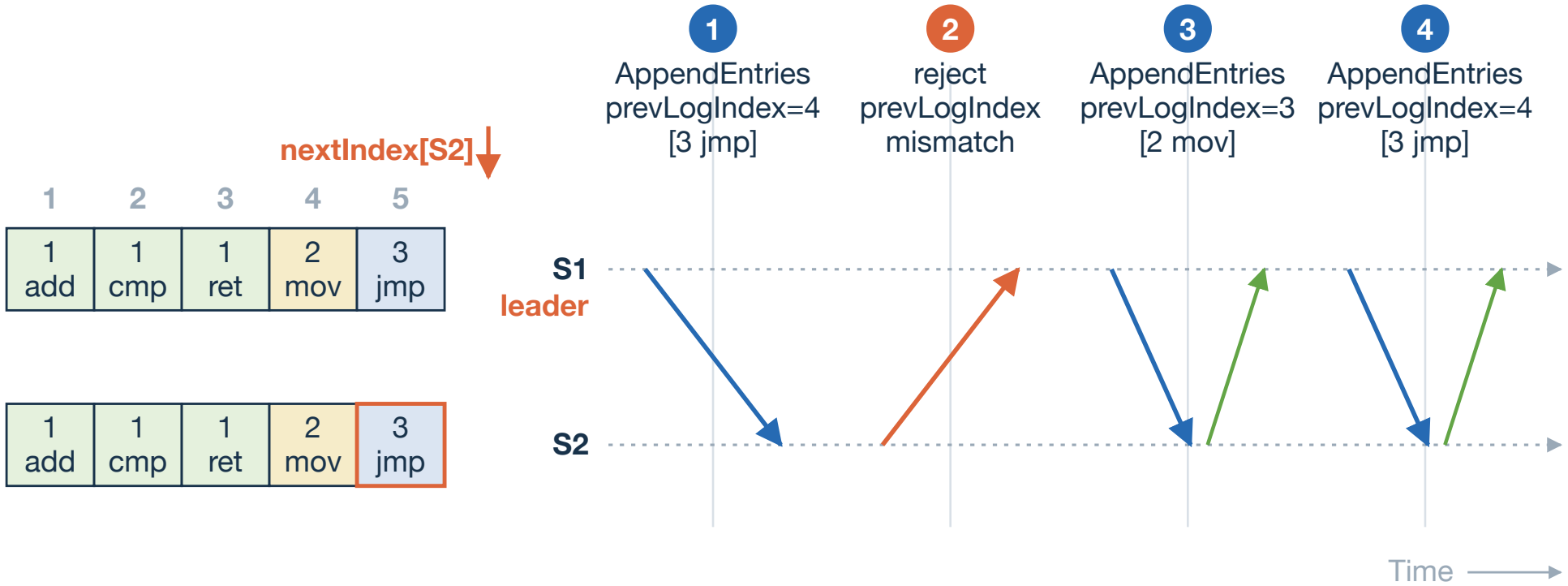


Handling rejections

We need to ensure the progress of the system

- The leader do so by rewriting the inconsistency part

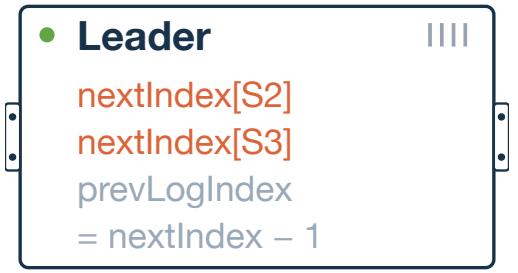


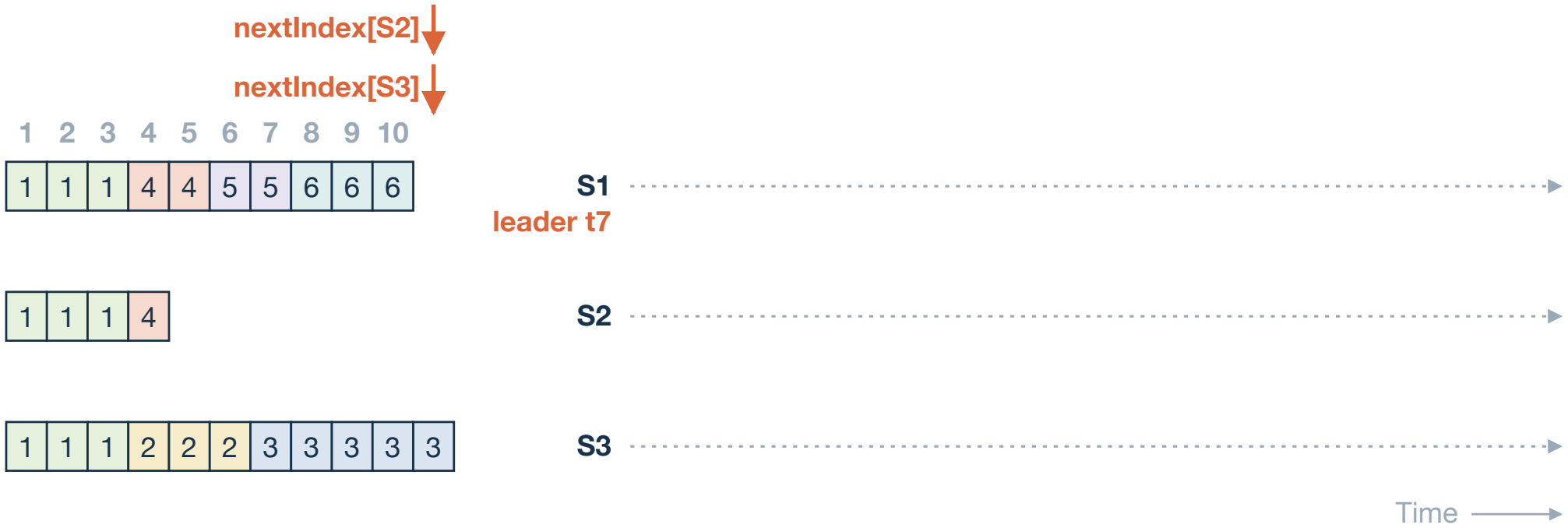


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones

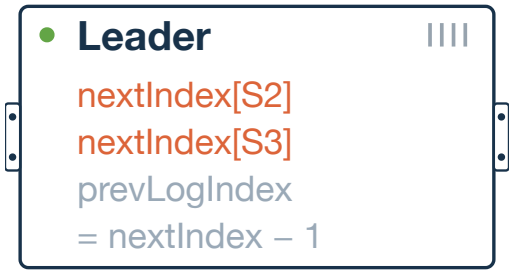


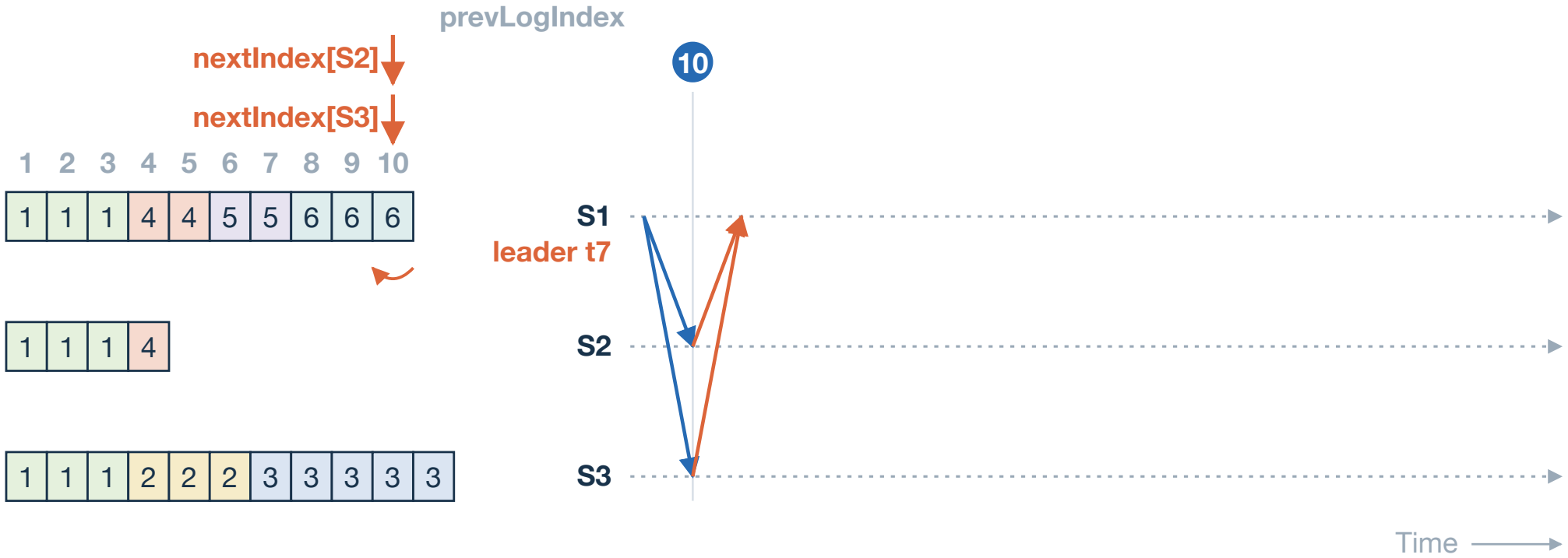


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones

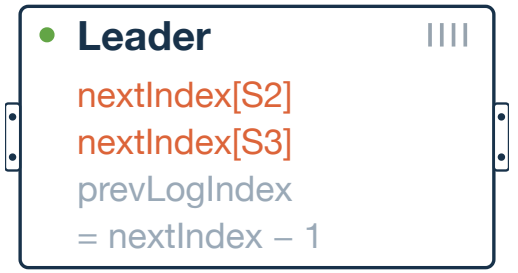


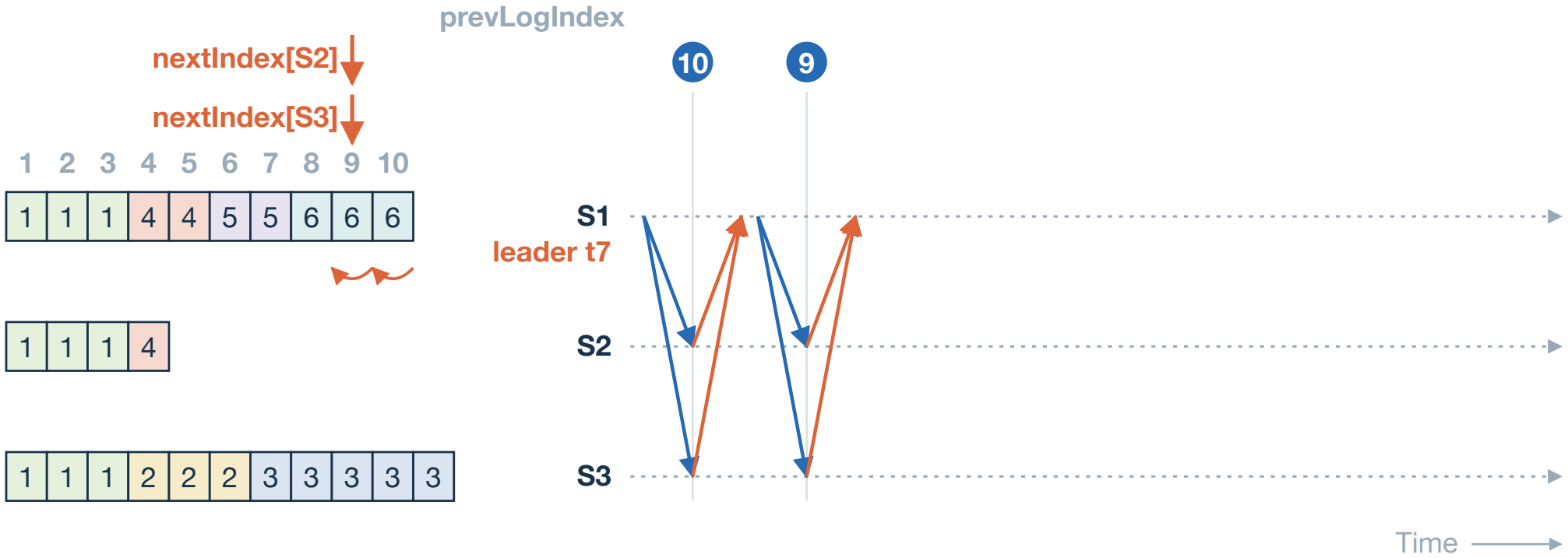


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones

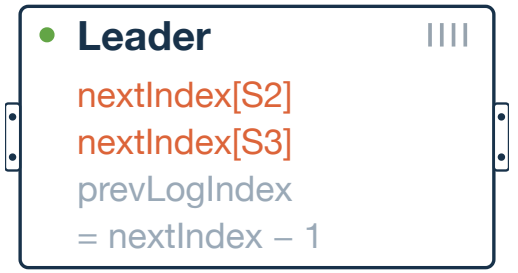


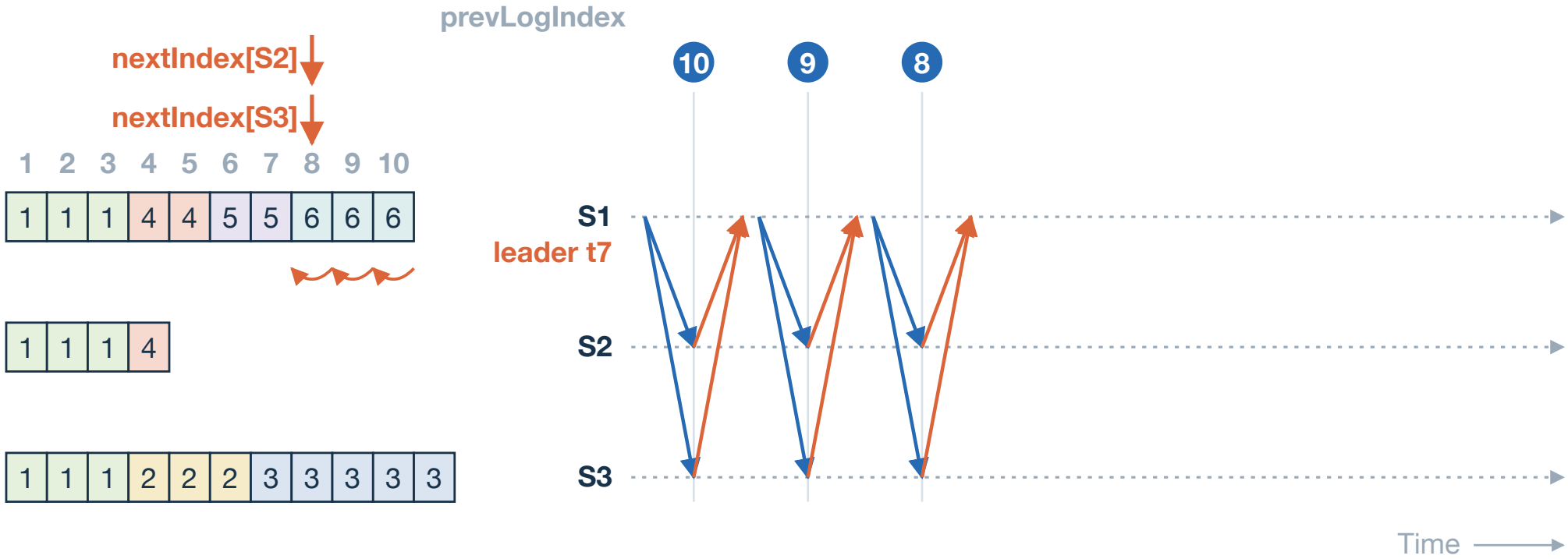


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones

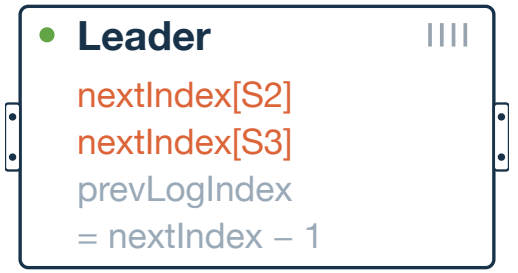


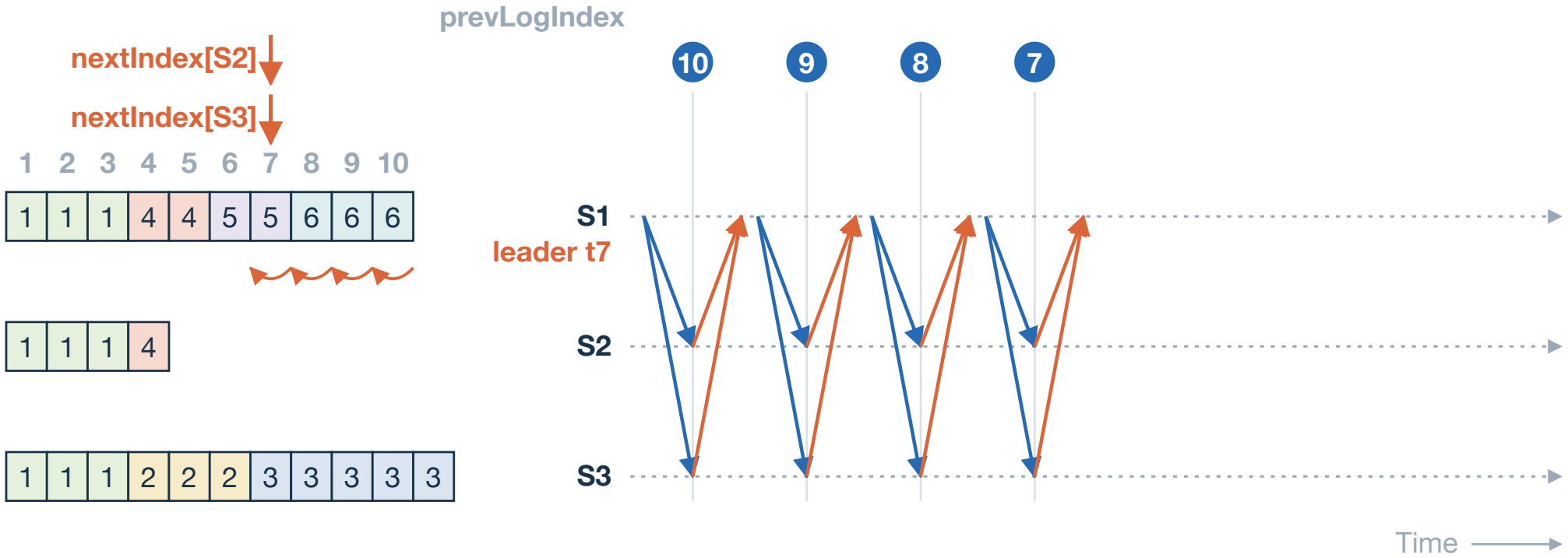


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones

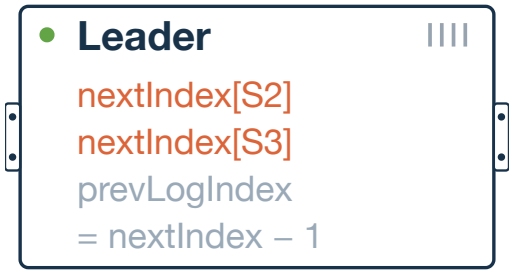


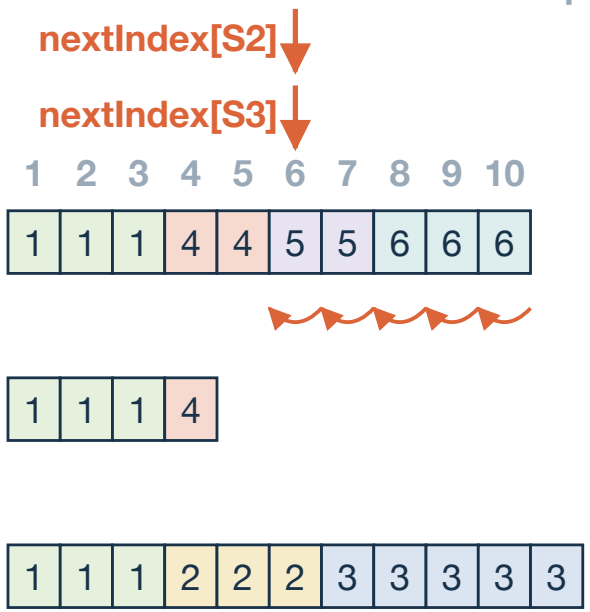


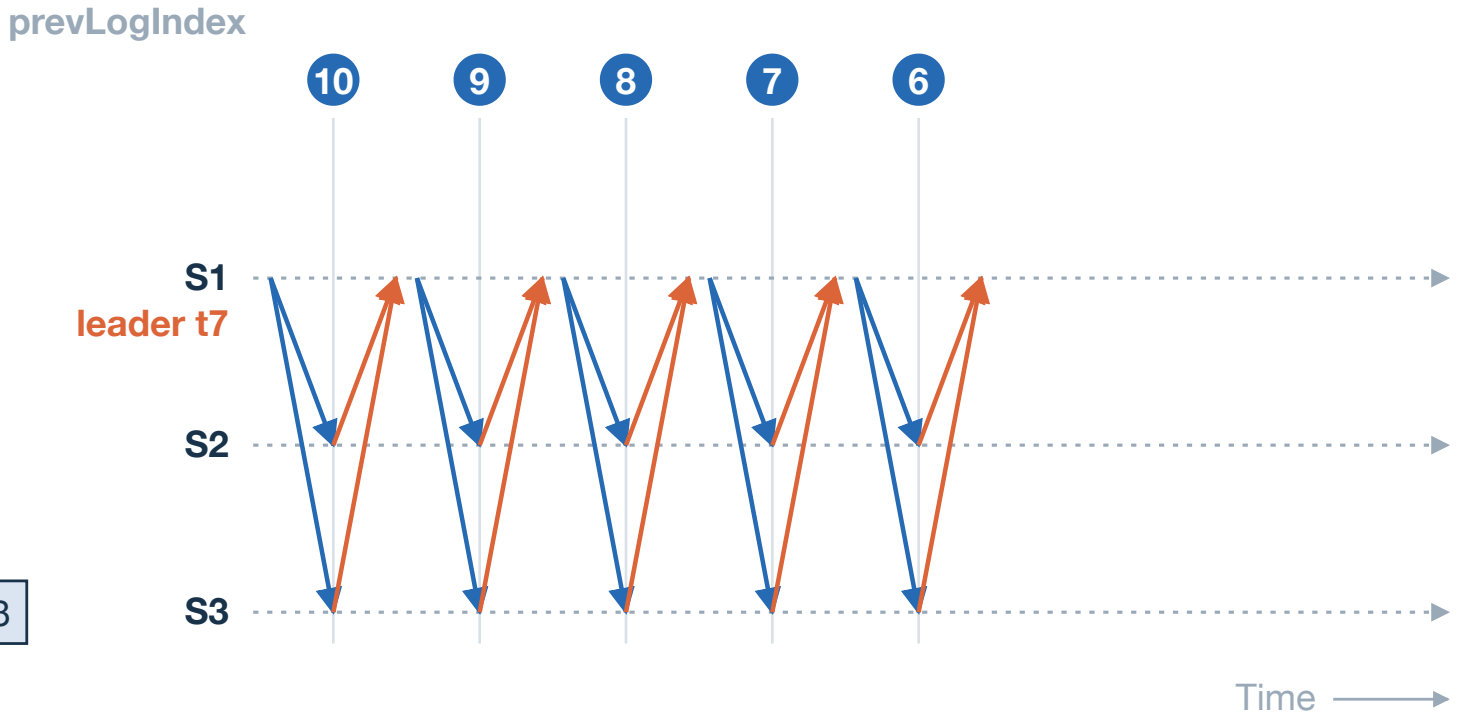
nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones



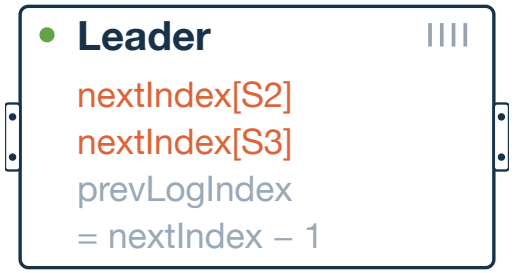


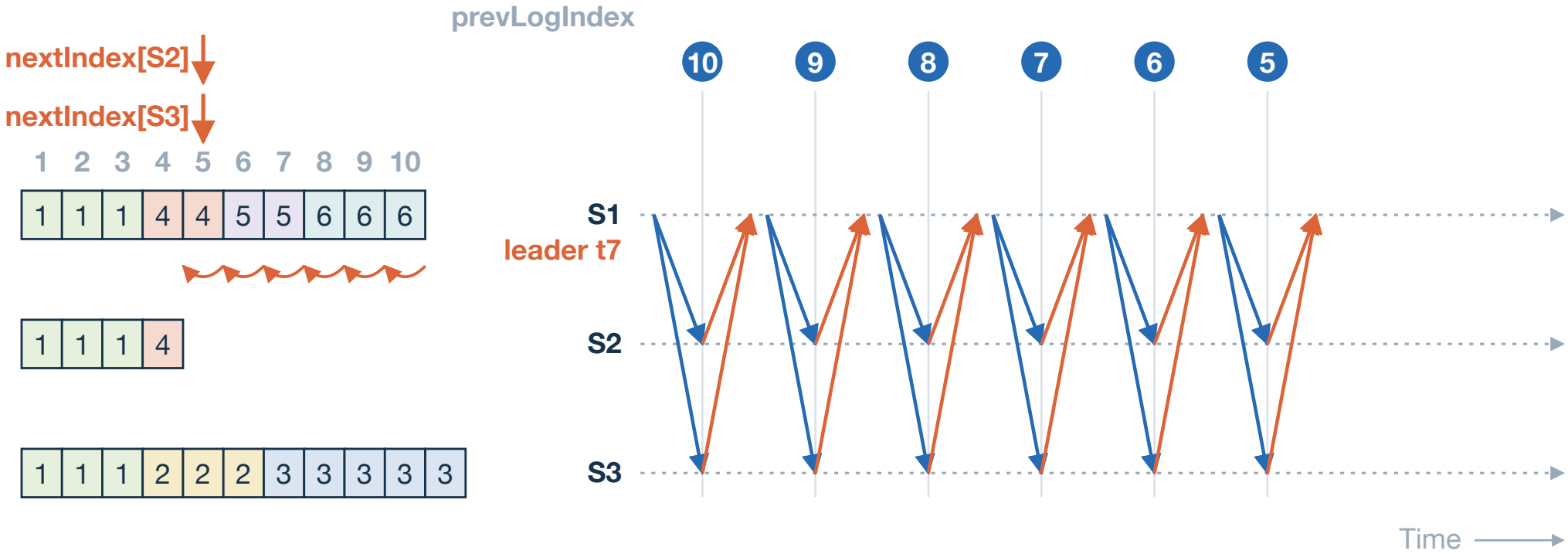


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones

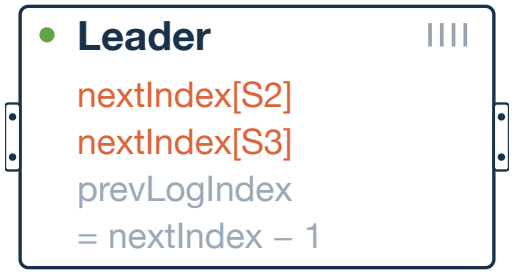


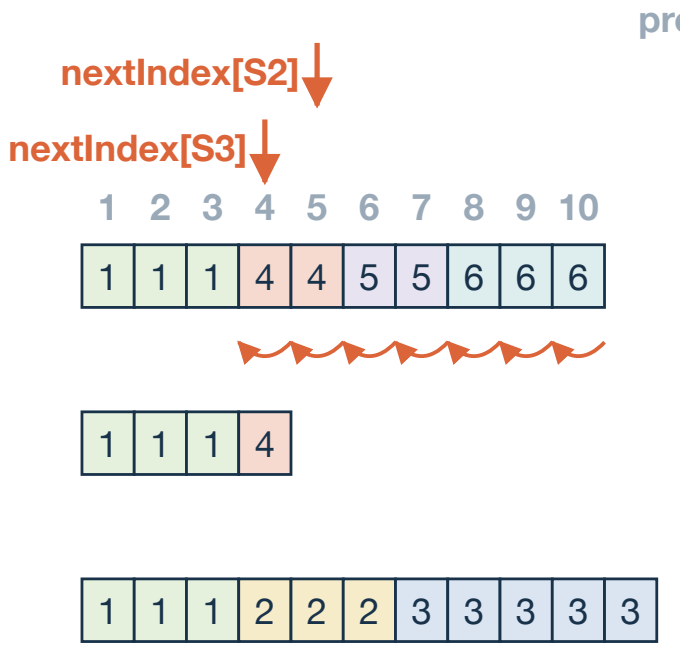


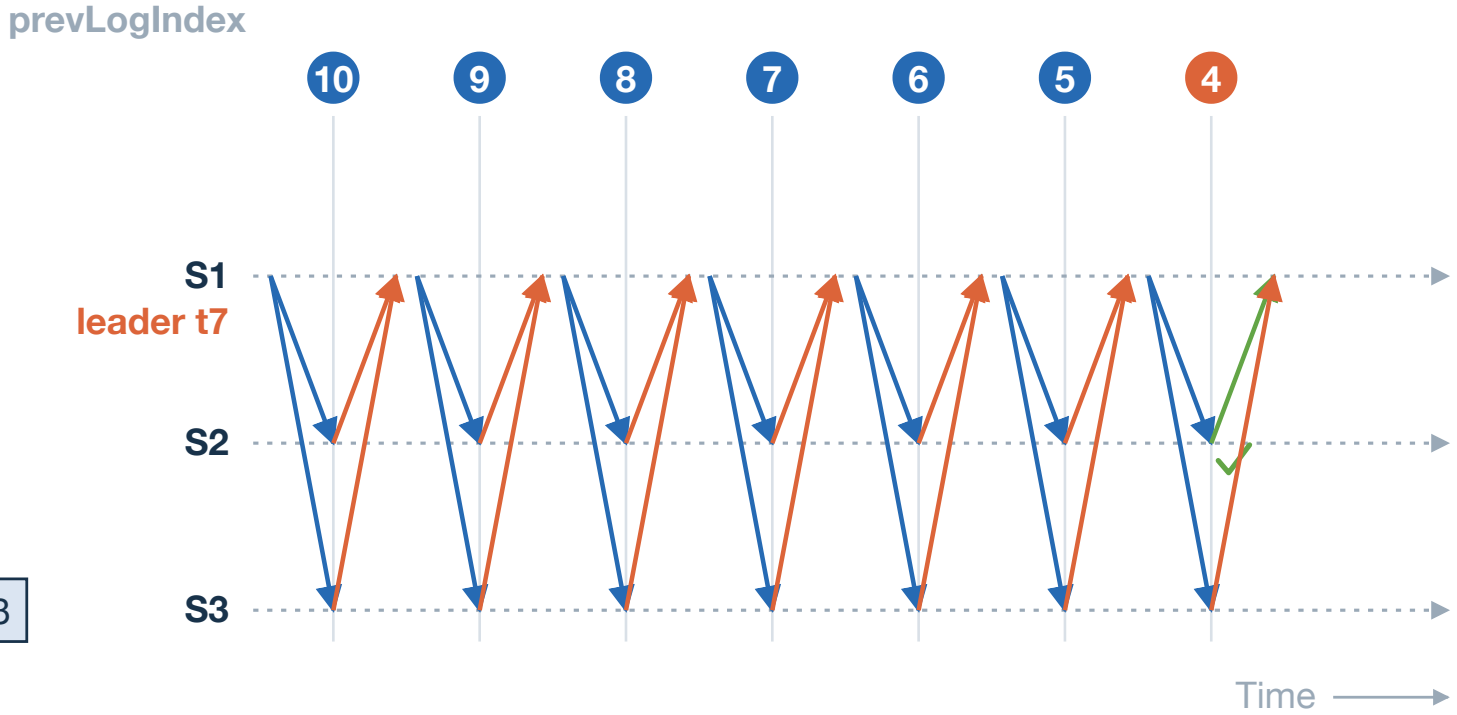
nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones



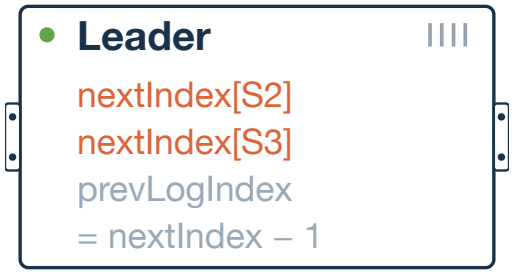


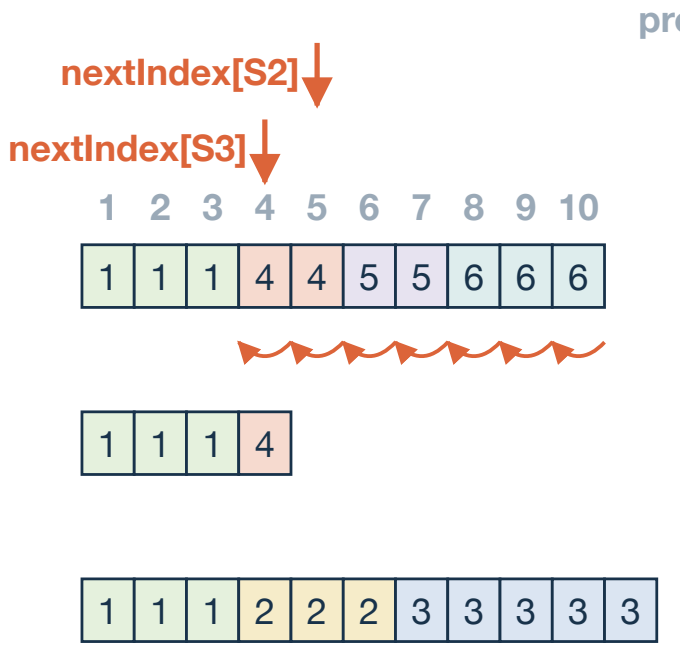


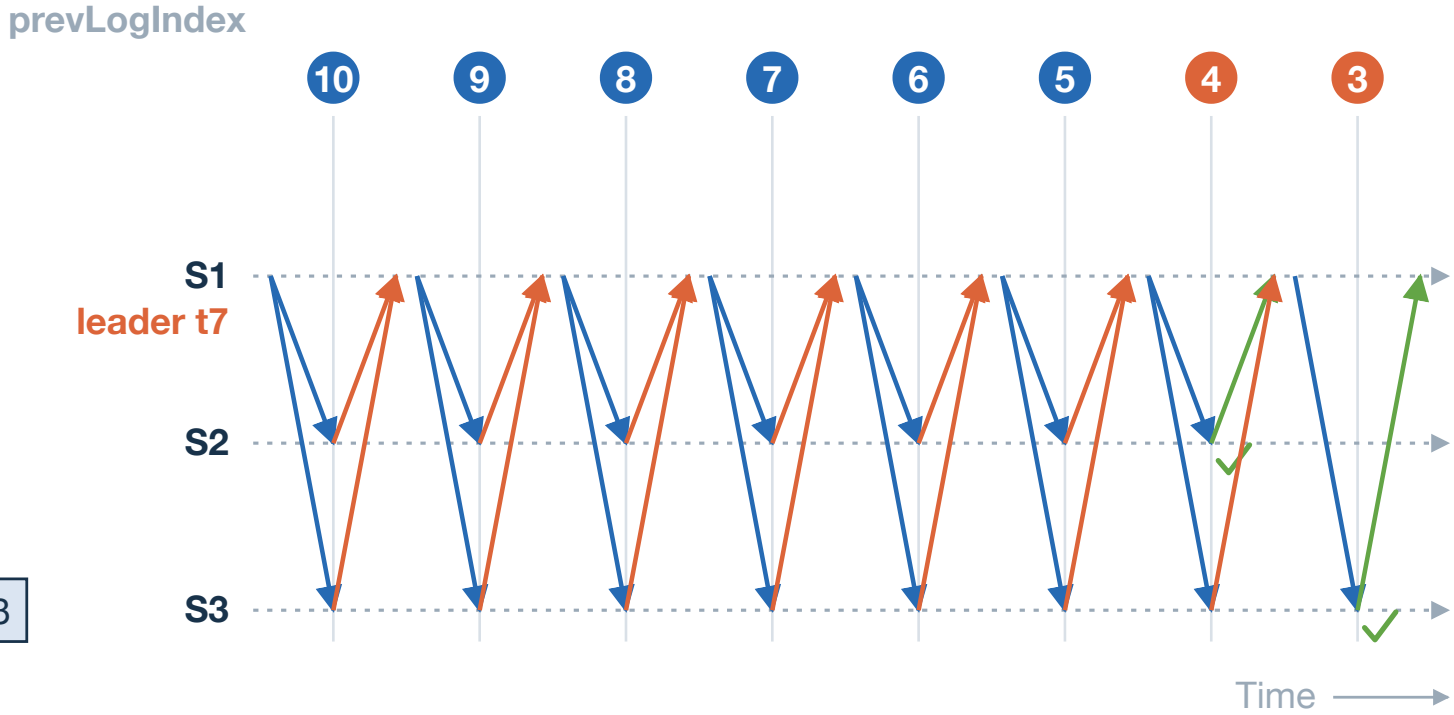
nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones



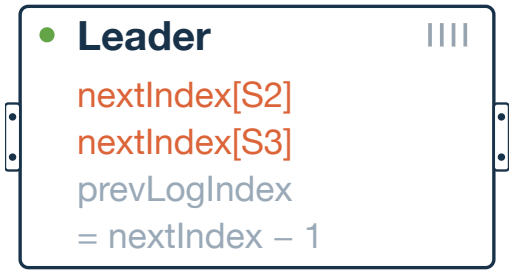


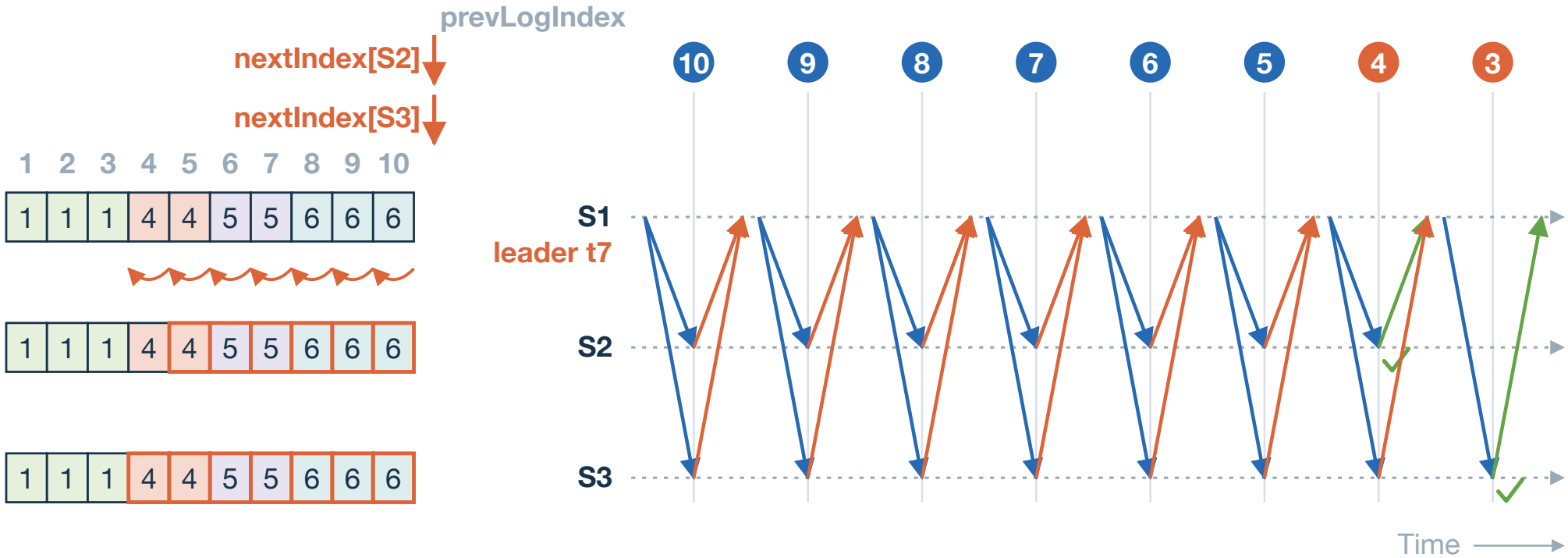


nextIndex: step back one entry per rejection

The leader keeps a `nextIndex` for each follower: the index of the next log entry to send it

- Each rejection steps it back one, until the logs match — then delete extraneous entries and fill in missing ones





AppendEntries RPC

Invoked by the leader to replicate its log to followers

```
Arguments:
  term
  prevLogIndex
  prevLogTerm
  entries

Results:
  term
  success

Implementation:
  if term < currentTerm
      success = false
  if no entry at prevLogIndex whose term is prevLogTerm
      success = false
  else
      delete the conflicting entry and all that follow it
      append entries
      success = true
```

Overwriting repairs the log — but when can we commit?

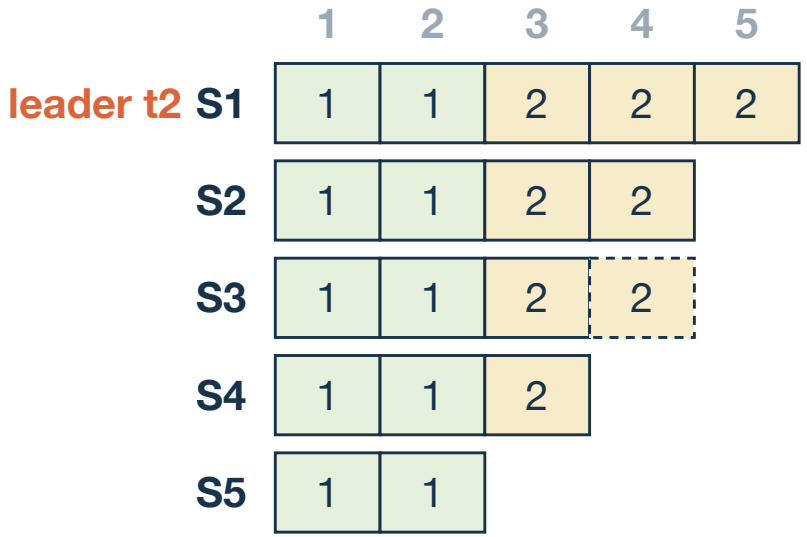
The leader pulls everyone's log back in line by overwriting followers

- The price: an entry already appended may later be overwritten

The intuitive answer: once an entry is replicated on a majority, it can be applied to the state machine

- Not always true on the Raft we have so far

Question: after the append reaches S3, can index 4 still be overwritten?



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

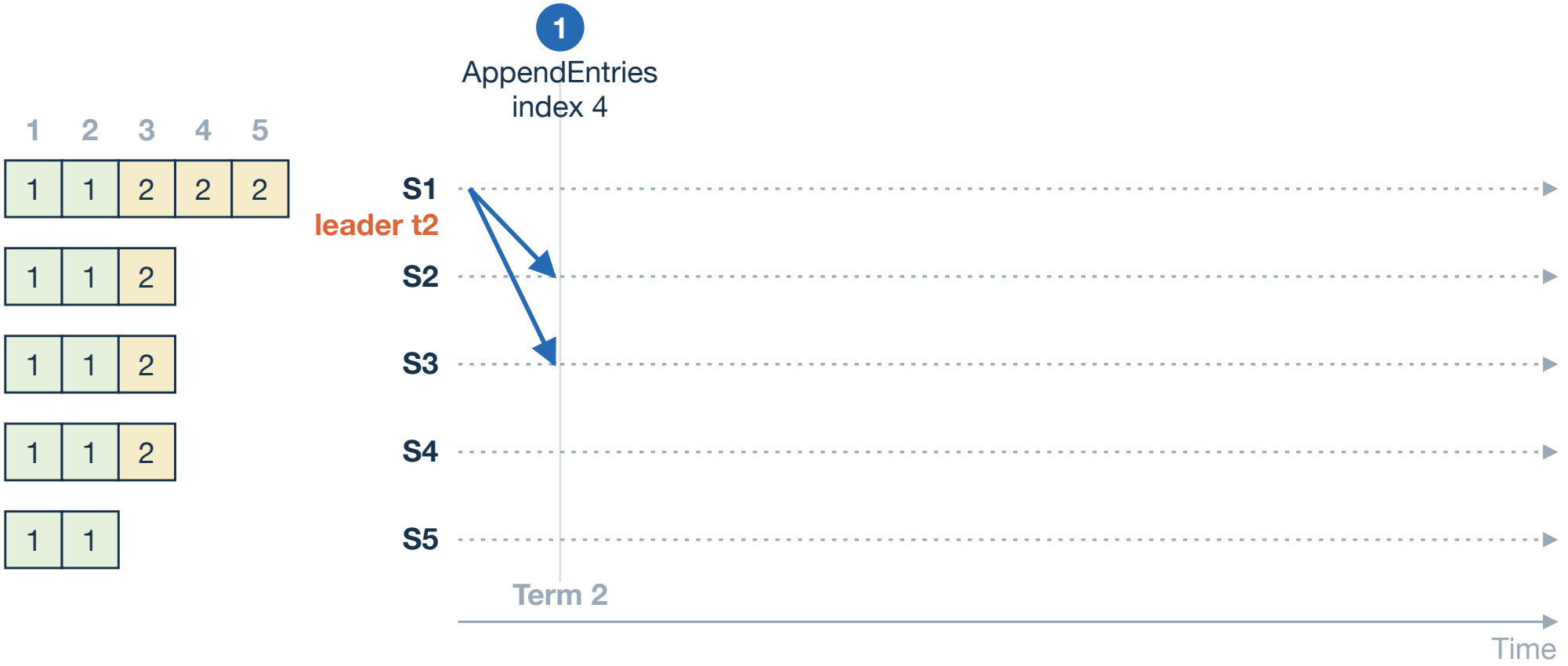
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

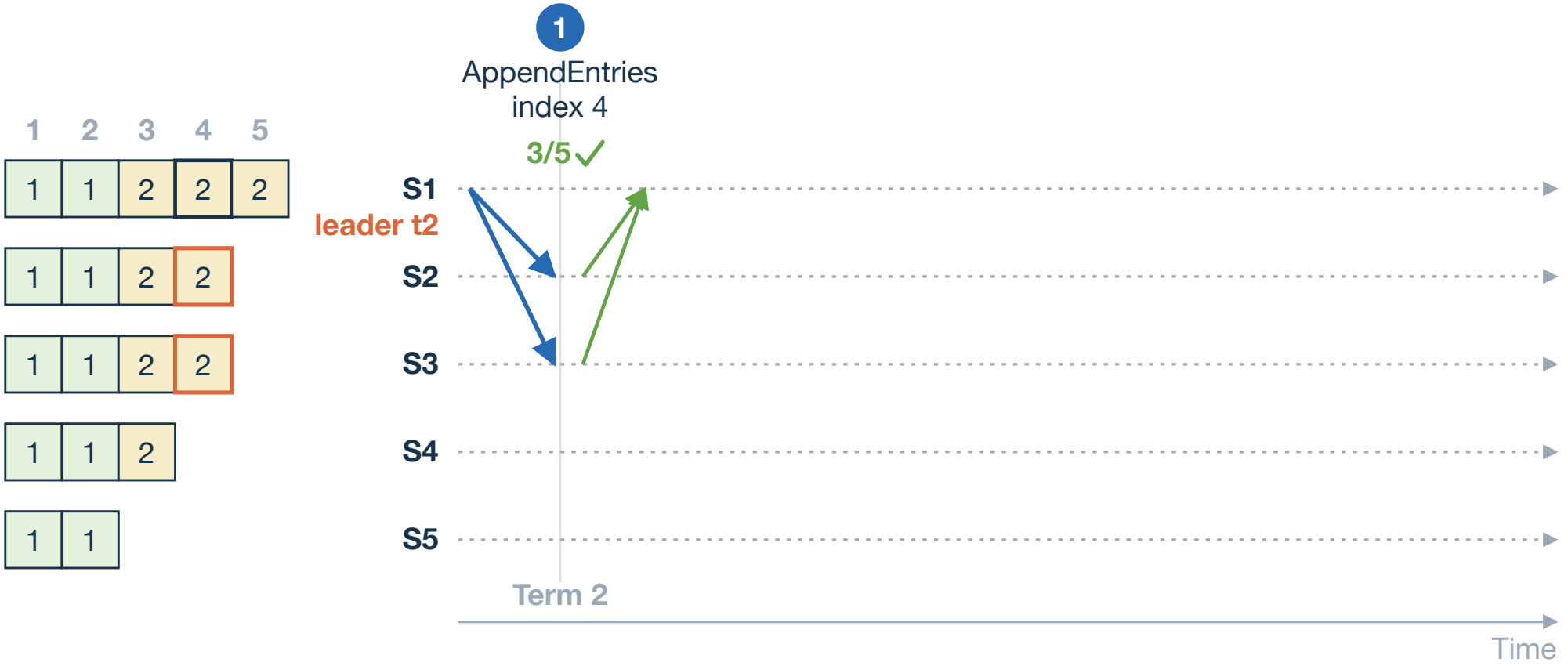
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

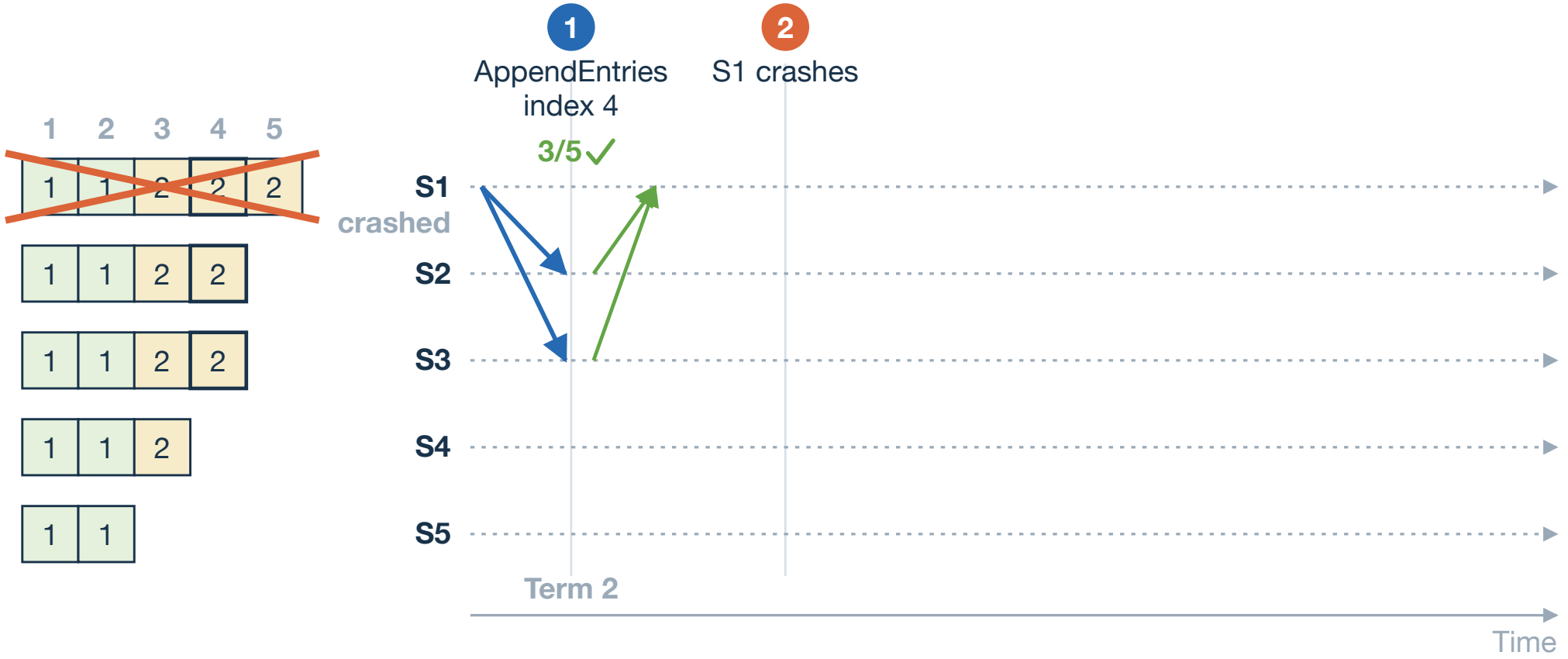
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

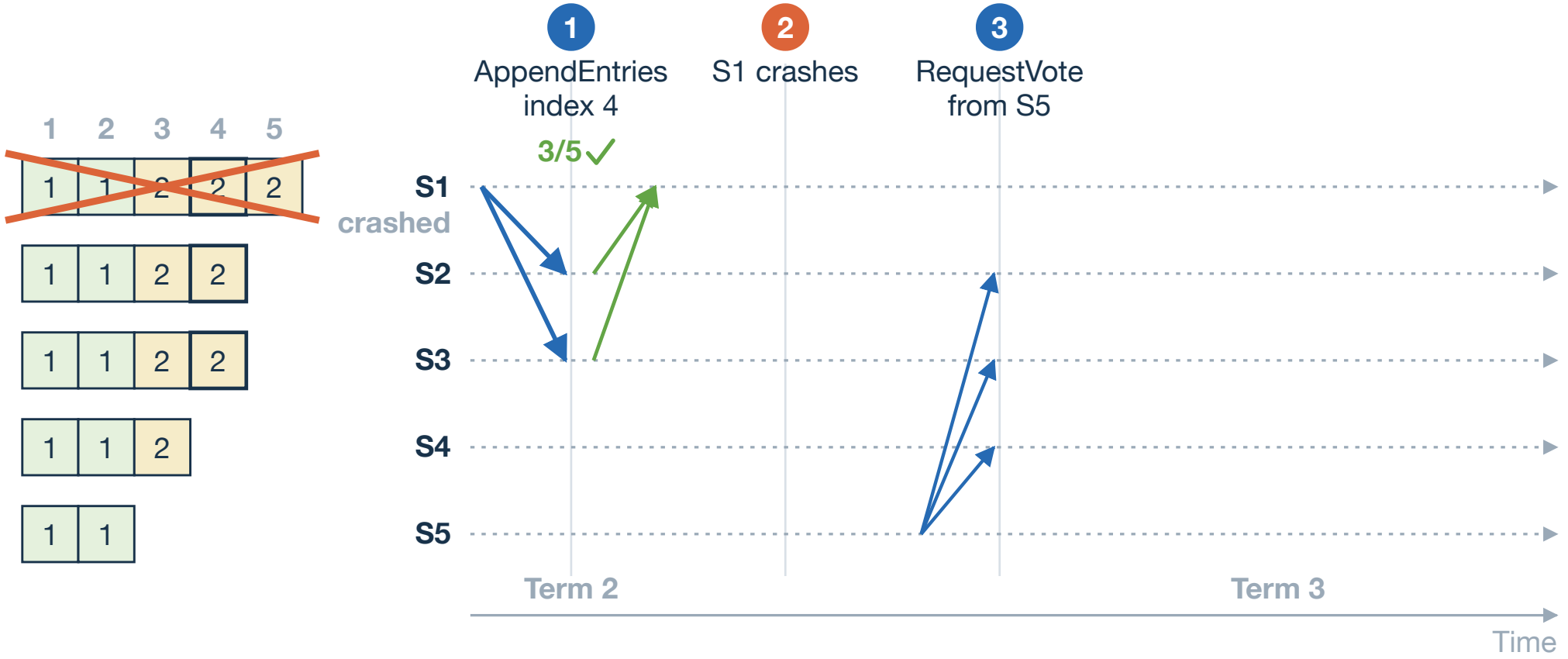
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

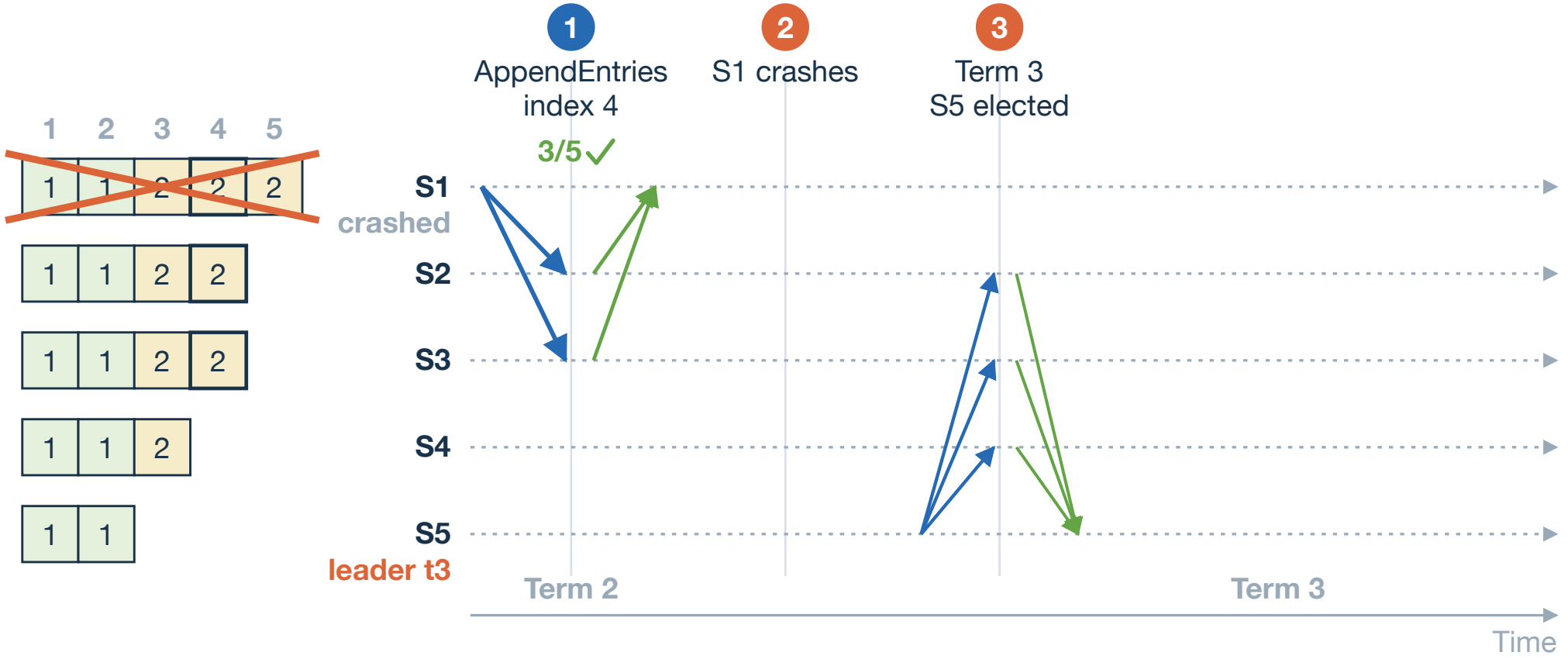
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

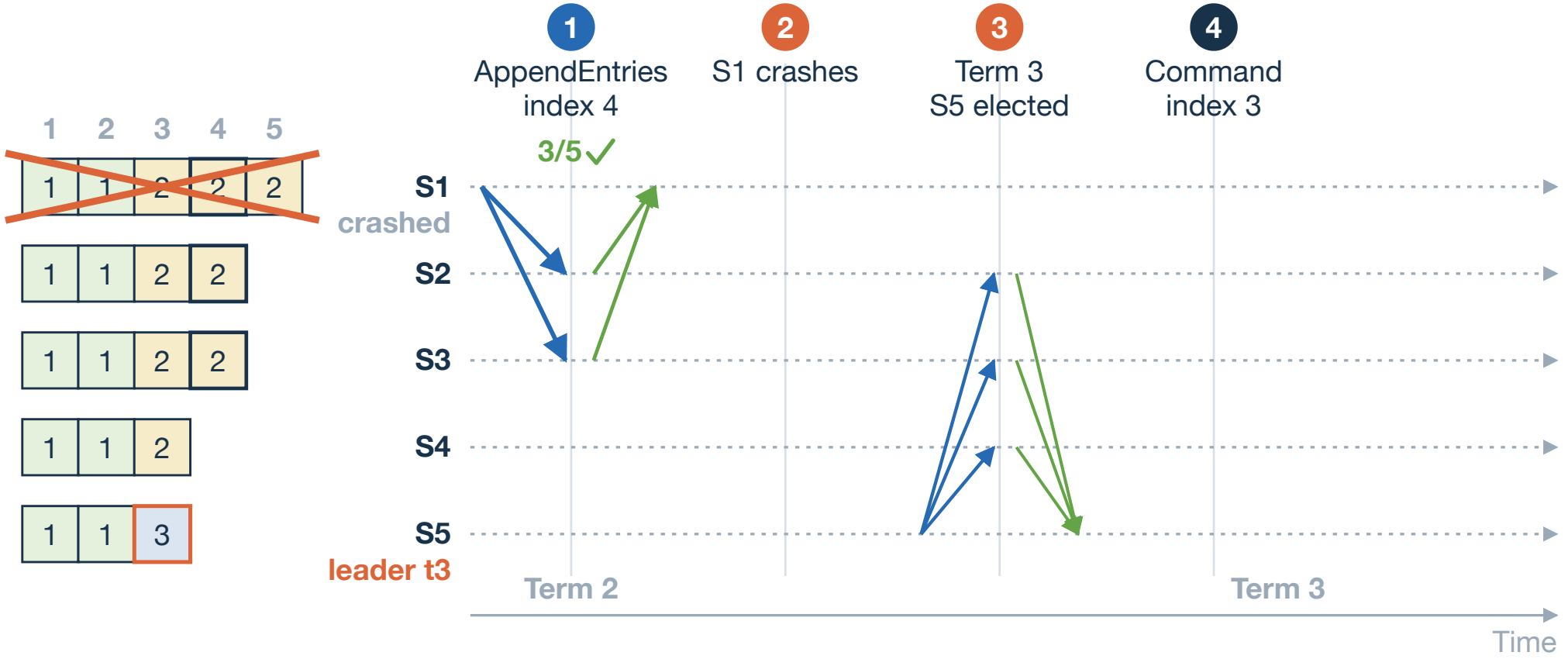
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

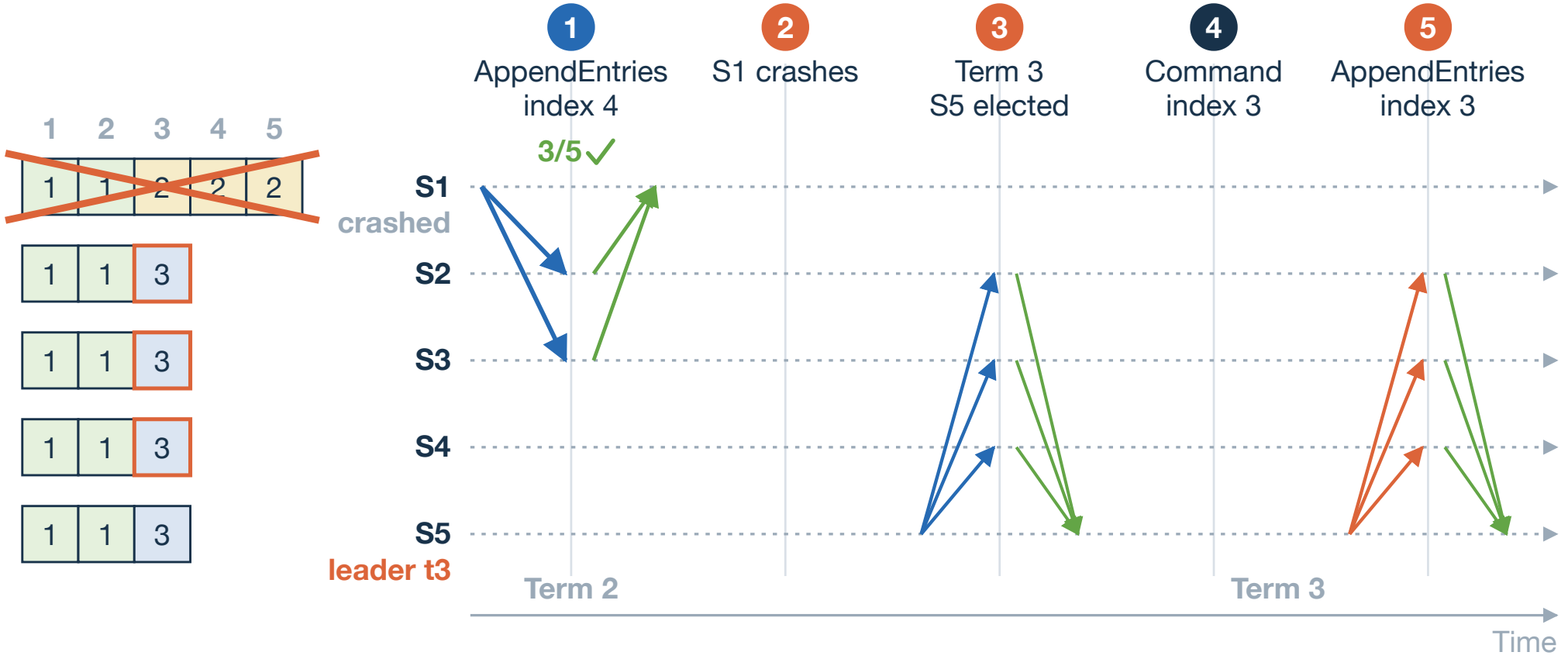
- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Case study: an entry on a majority can be overwritten

Leader S1 of term 2 replicated index 4 to S1, S2, S3 — a majority

- S1 then crashes; S5 has the shortest log, but still wins term 3 — index 4 is gone



Safety requirement for commit

Once a leader has decided that an entry is committed, that entry will be present in the logs of all future leaders

- Leaders never overwrite entries in their own logs
- Only entries in the leader's log can be committed

But we need to prevent future leaders' from overwriting

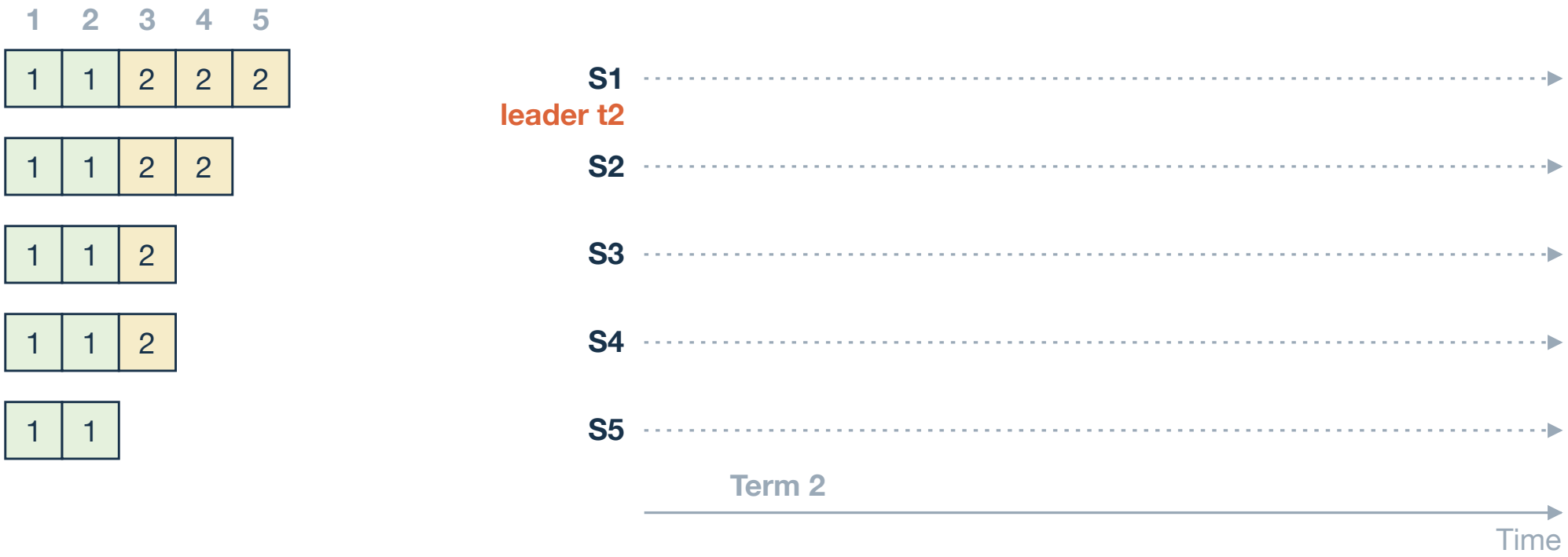
Goal: once an entry is replicated to a majority of followers, it should be able to commit

- Similar to Paxos

Case #1/2: committing an entry from the current term

The leader decides an entry from its current term is committed — the danger is S4 and S5

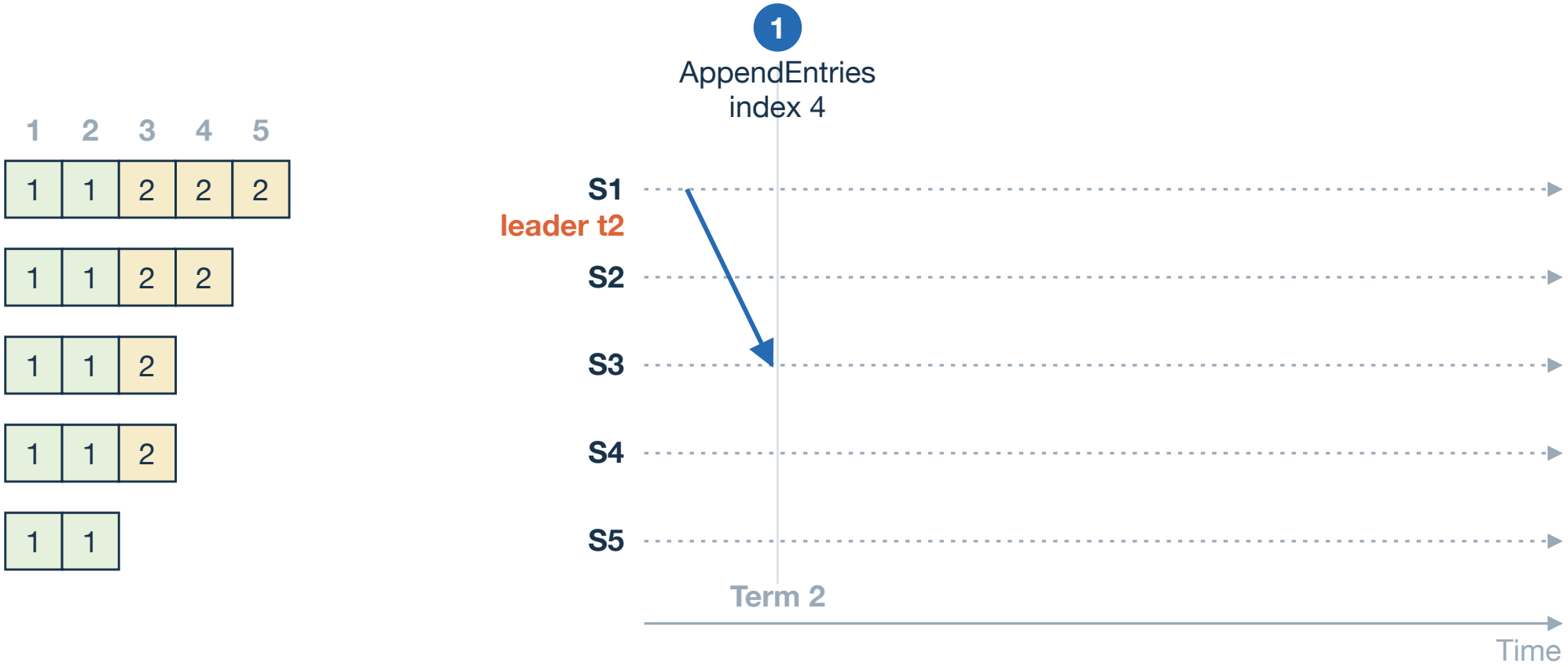
Question: how do we prevent index 4 from being overwritten?



Case #1/2: committing an entry from the current term

The leader decides an entry from its current term is committed — the danger is S4 and S5

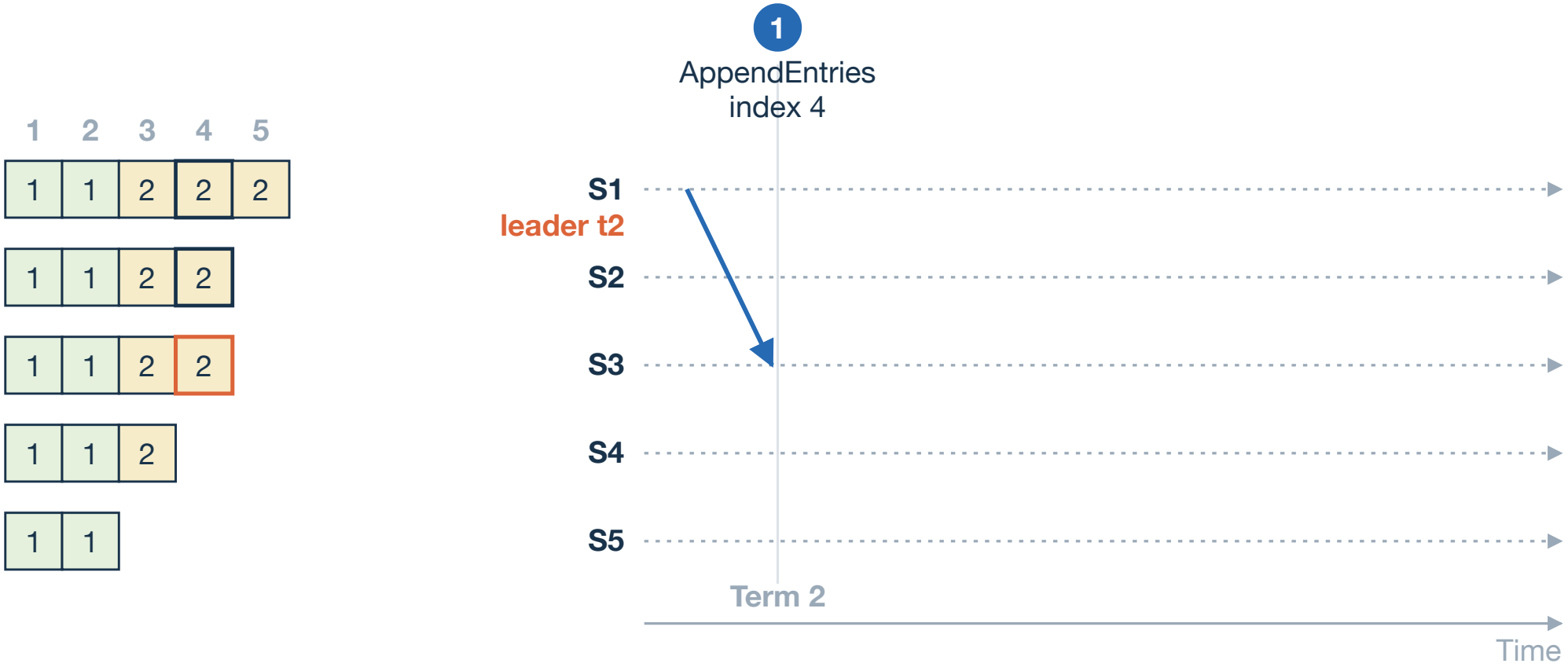
Question: how do we prevent index 4 from being overwritten?



Case #1/2: committing an entry from the current term

The leader decides an entry from its current term is committed — the danger is S4 and S5

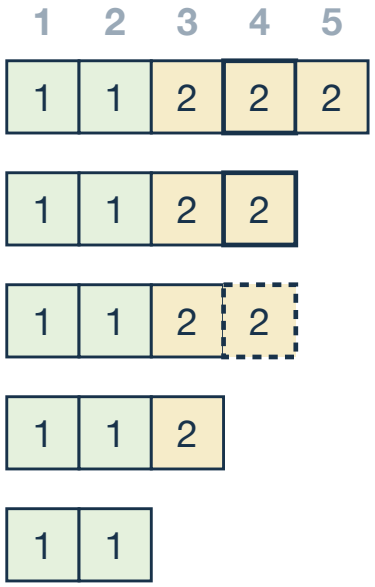
Question: how do we prevent index 4 from being overwritten?

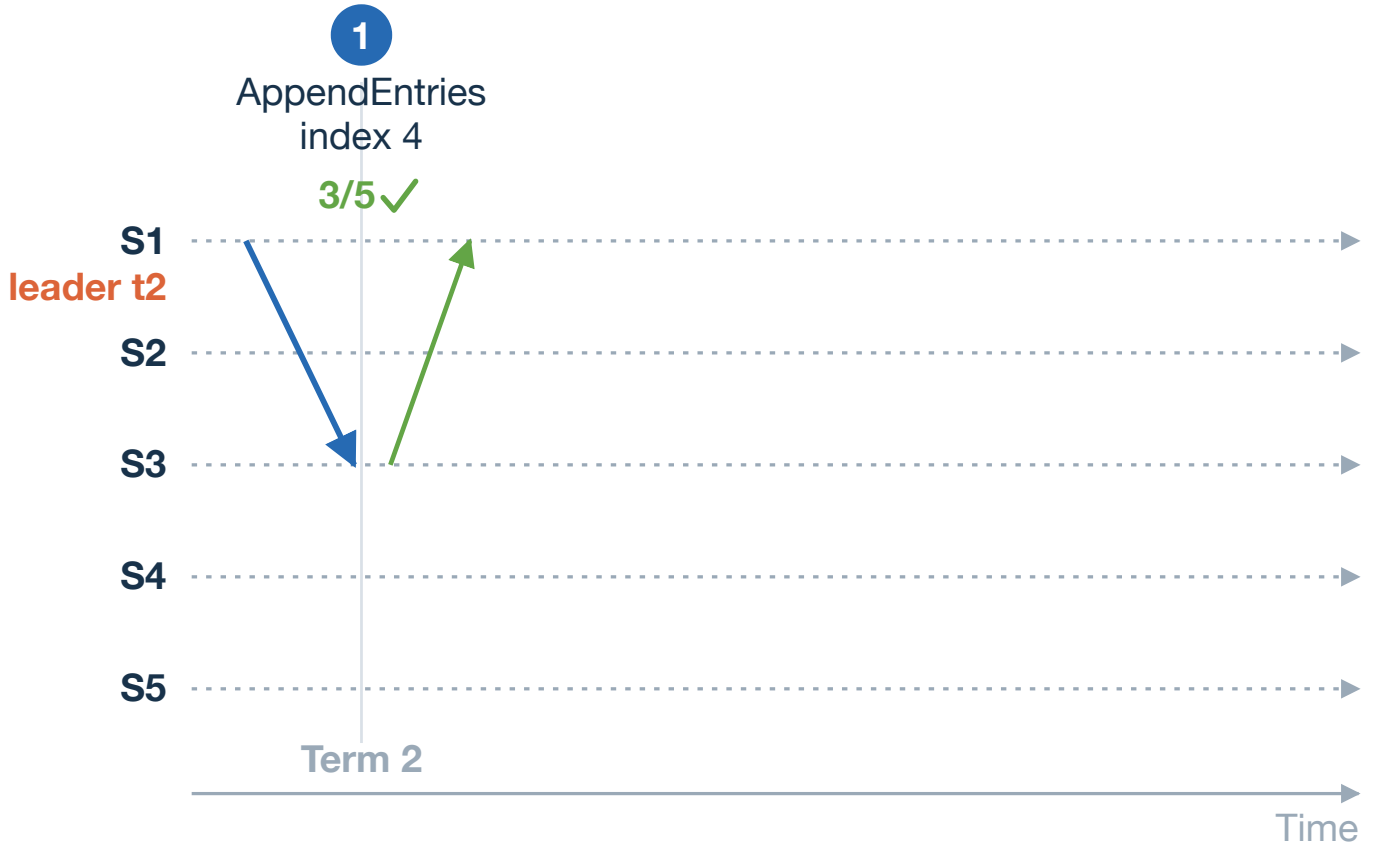


Case #1/2: committing an entry from the current term

The leader decides an entry from its current term is committed — the danger is S4 and S5

Question: how do we prevent index 4 from being overwritten?

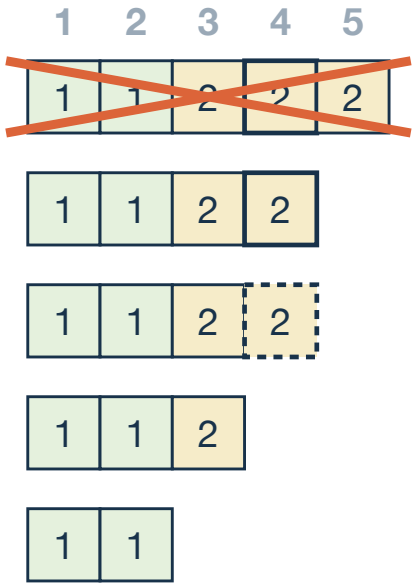


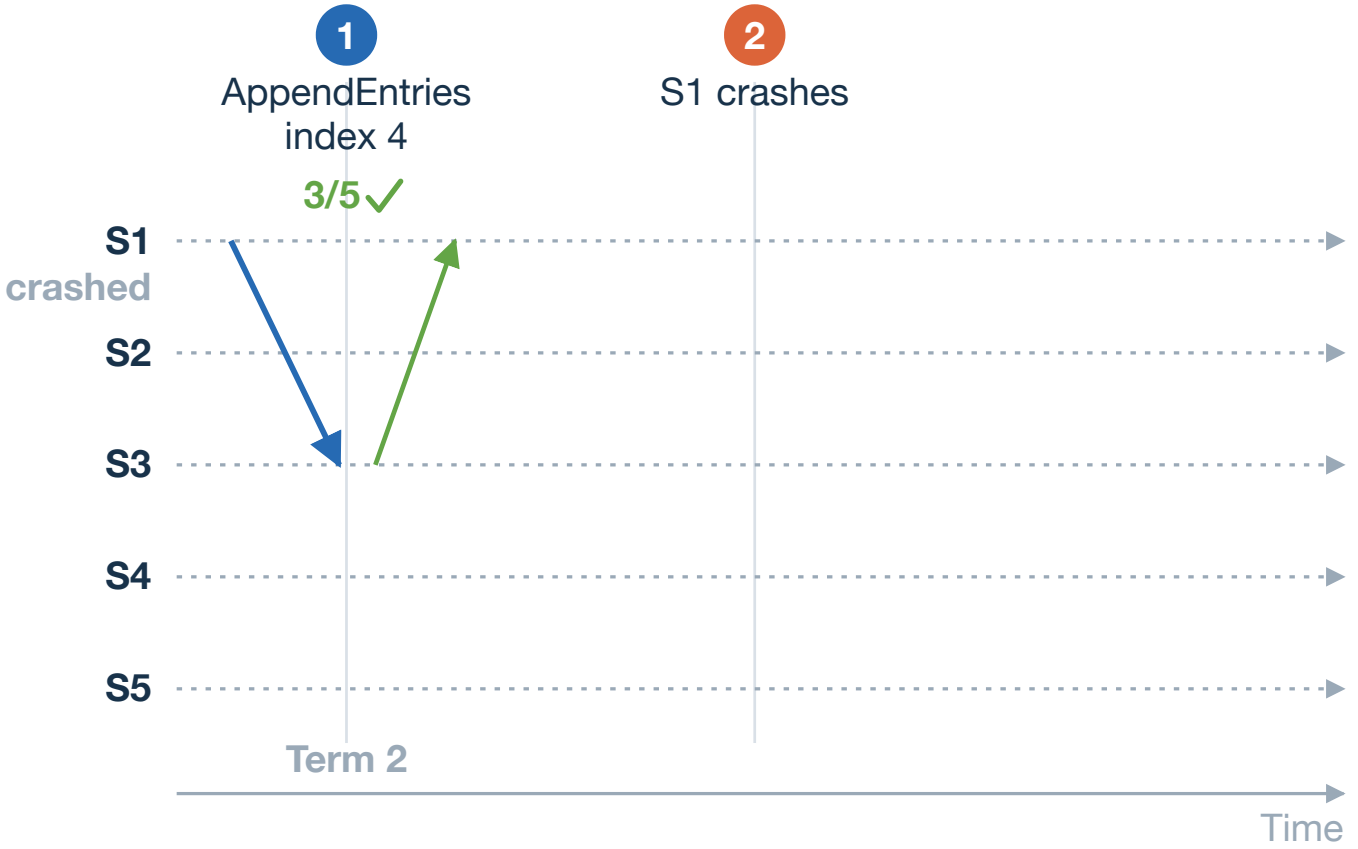


Case #1/2: committing an entry from the current term

The leader decides an entry from its current term is committed — the danger is S4 and S5

Question: how do we prevent index 4 from being overwritten?

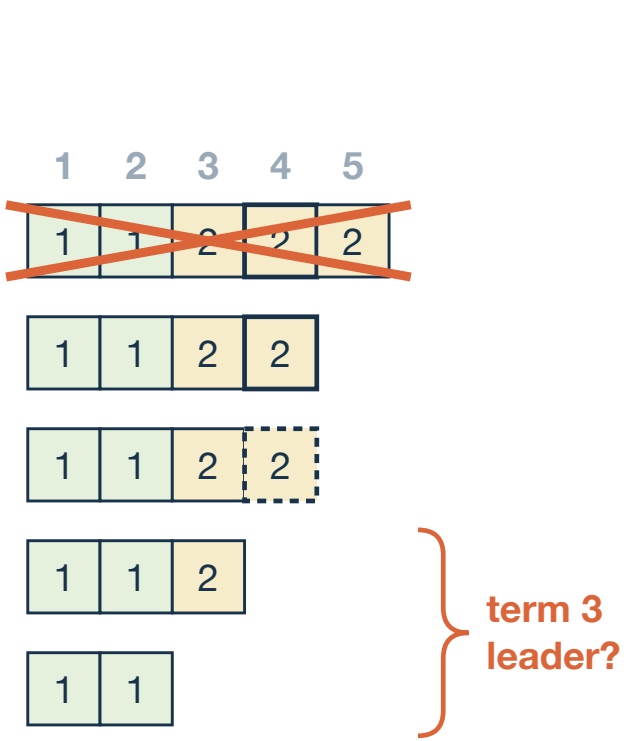


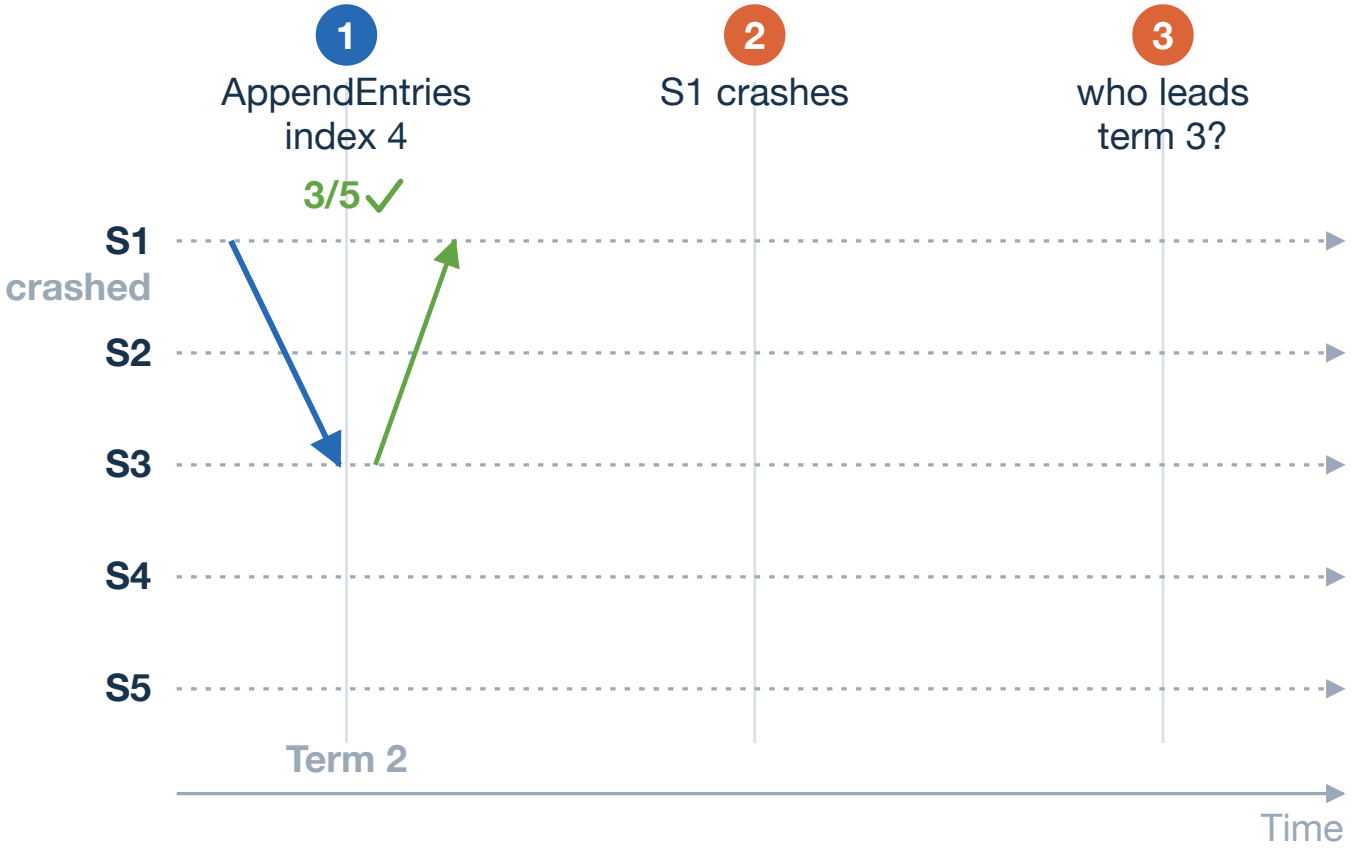


Case #1/2: committing an entry from the current term

The leader decides an entry from its current term is committed — the danger is S4 and S5

Question: how do we prevent index 4 from being overwritten?





Picking the best leader

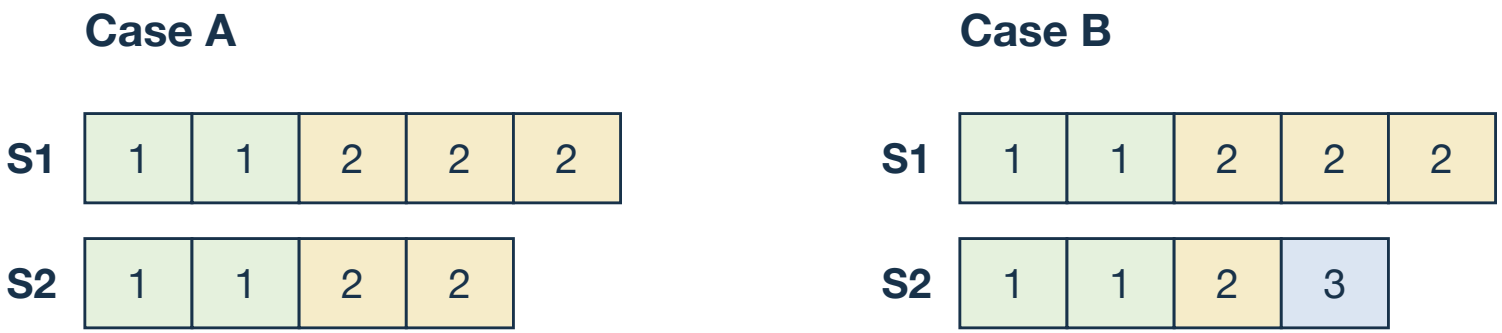
During elections, choose the candidate whose log is most likely to contain all committed entries

- Candidates include their log info in RequestVote RPCs: the index & term of the last log entry

The voting server V denies the vote if its own log is more complete

- `lastTermV > lastTermC`, or the terms are equal and `lastIndexV > lastIndexC`

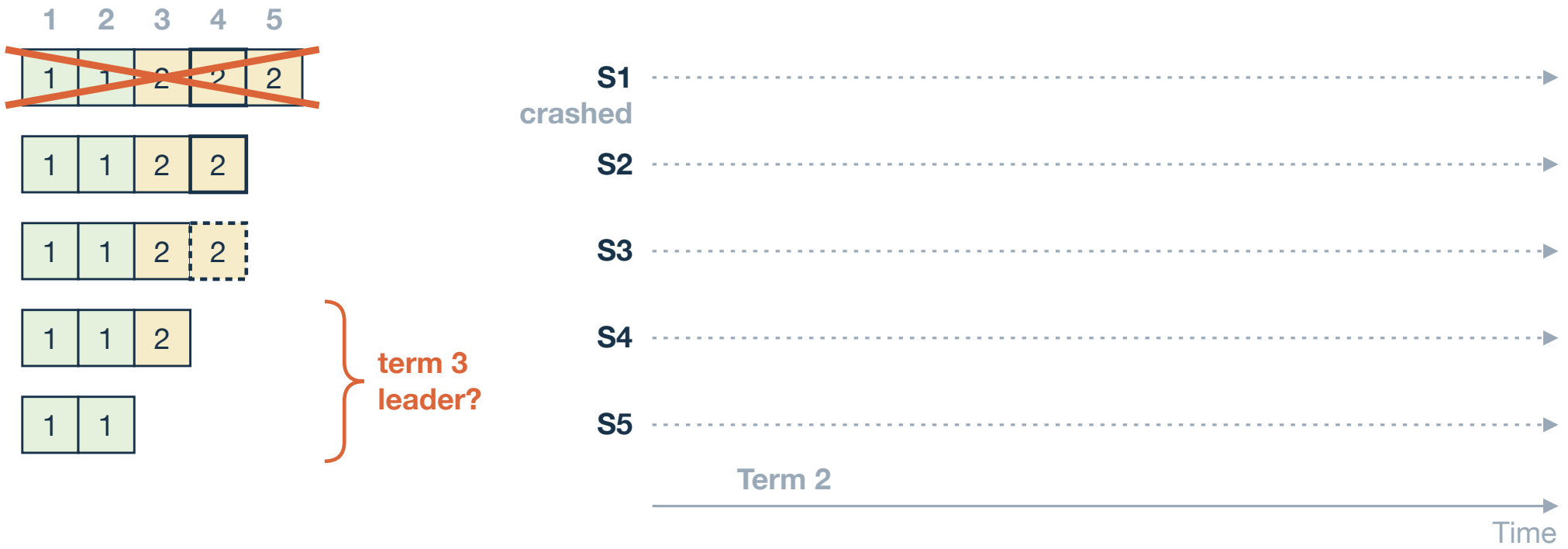
Question: in each pair, which server's log is more complete?



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

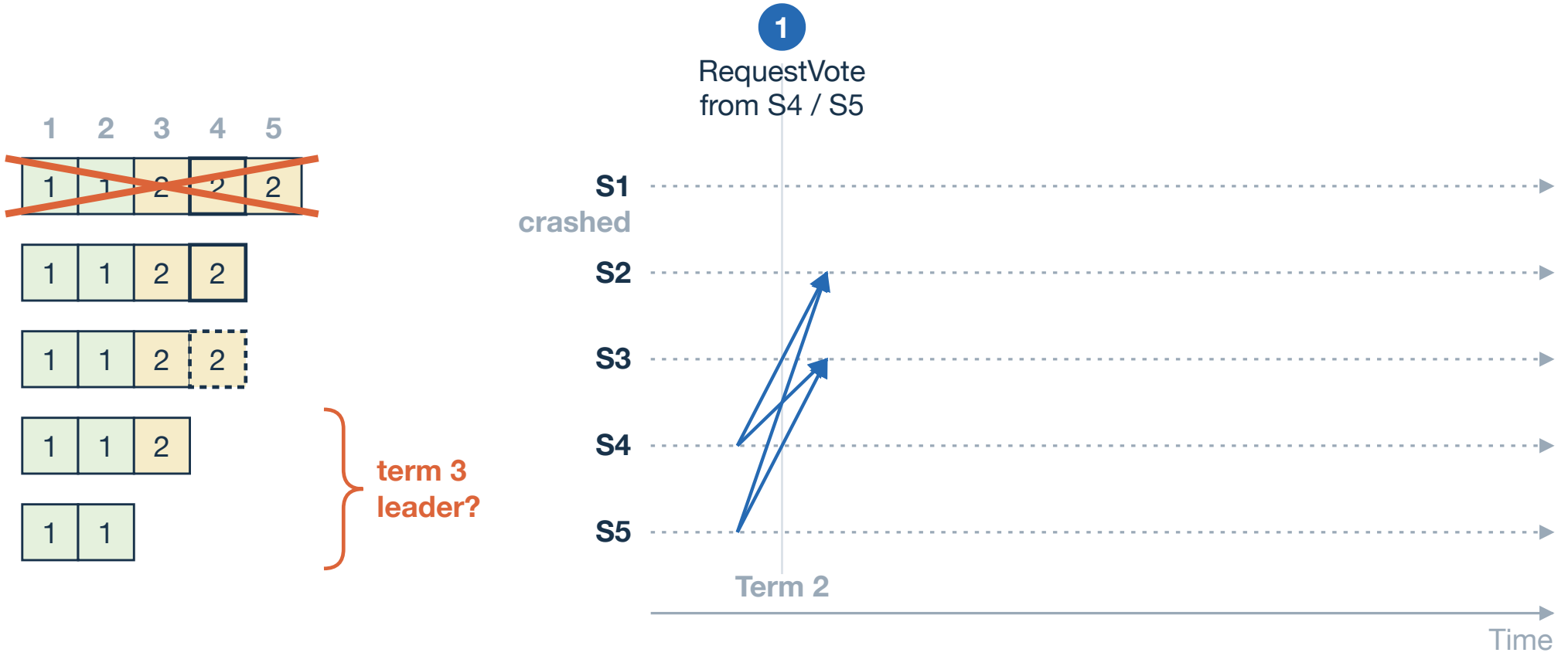
- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

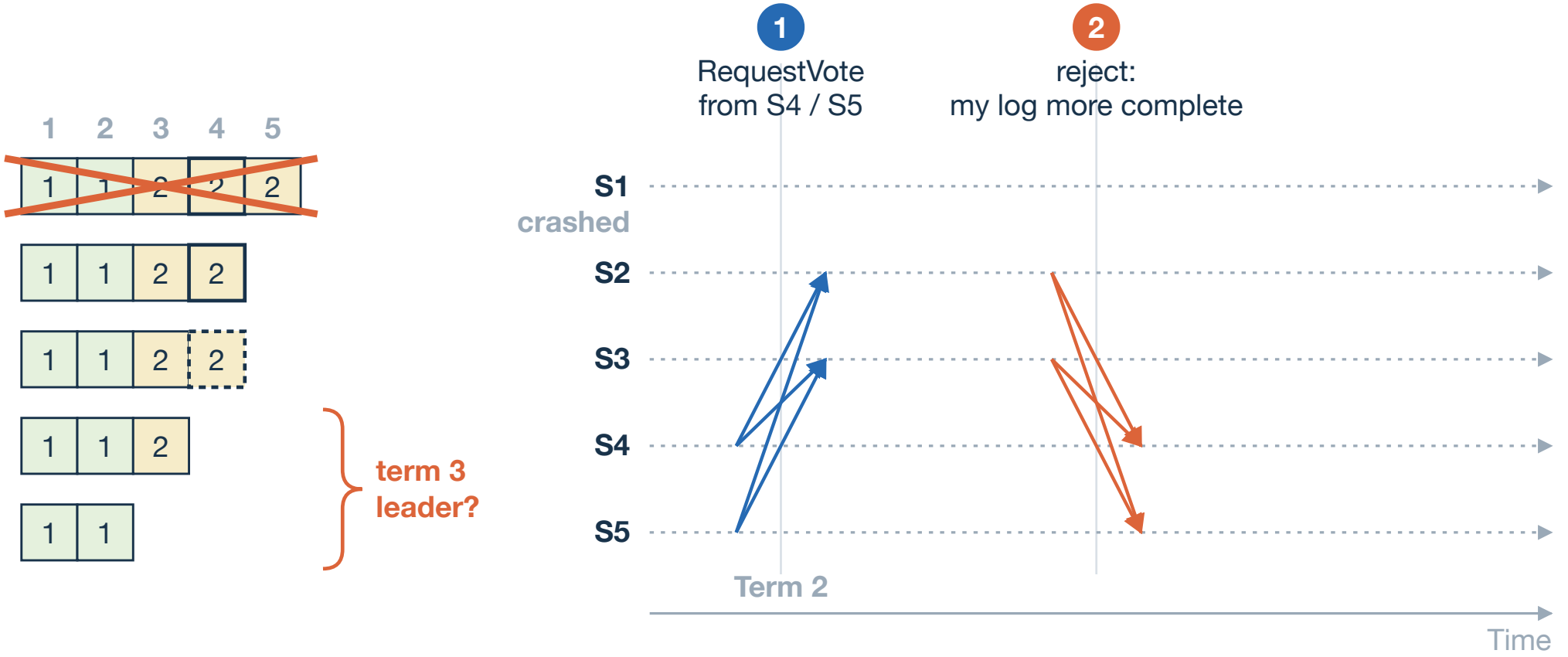
- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

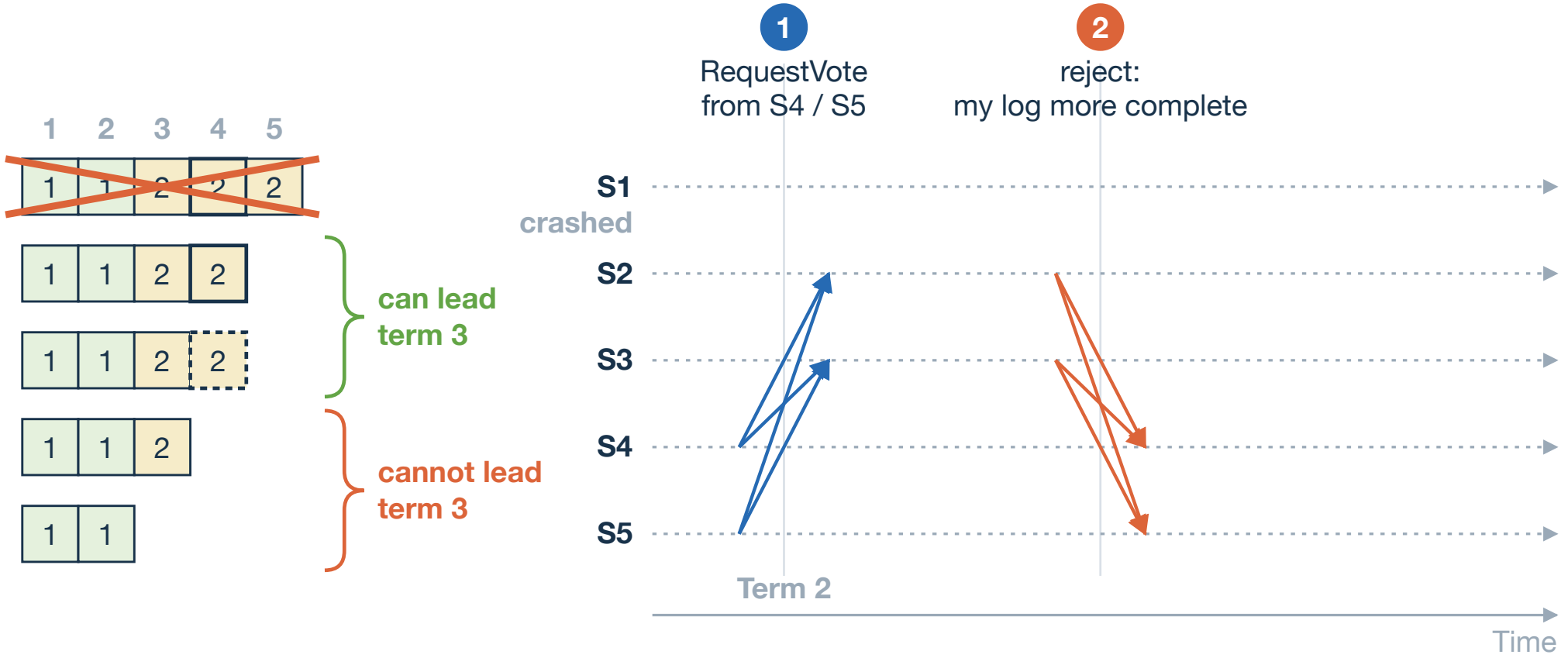
- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

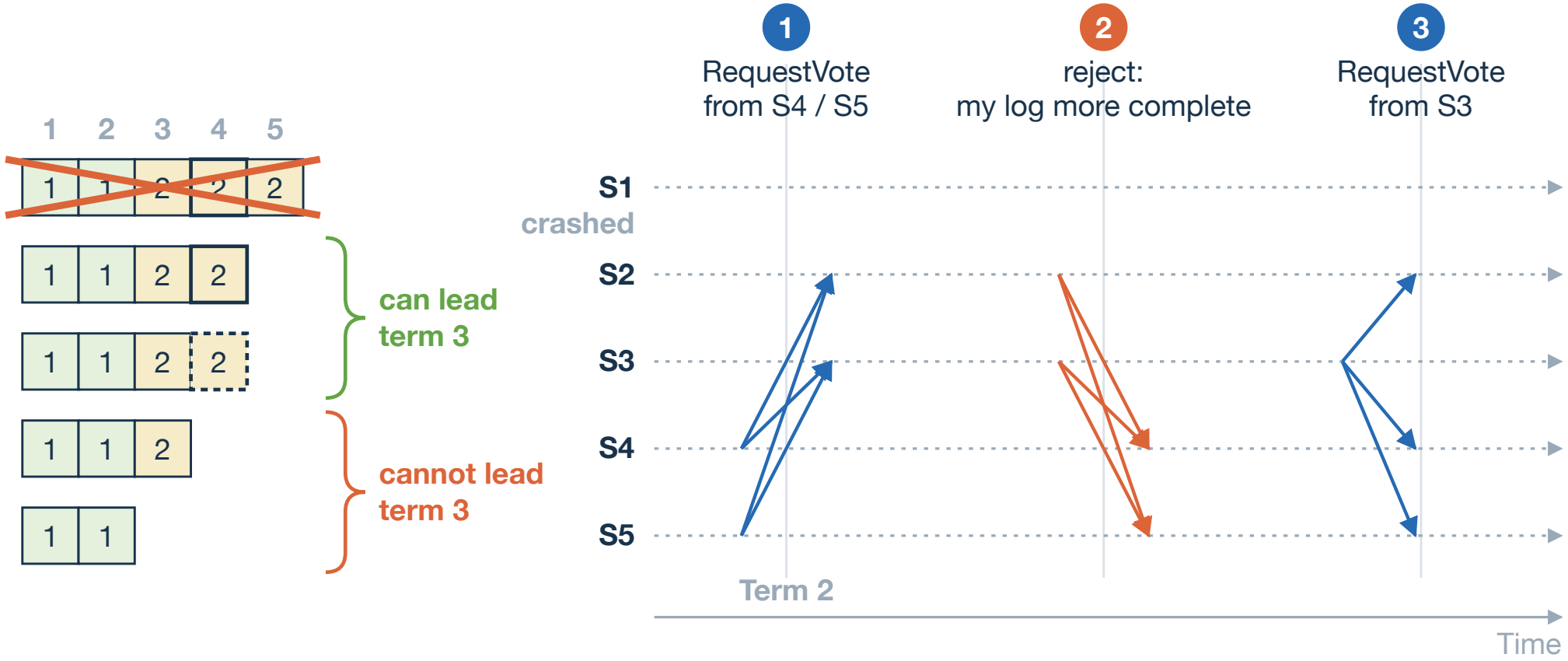
- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

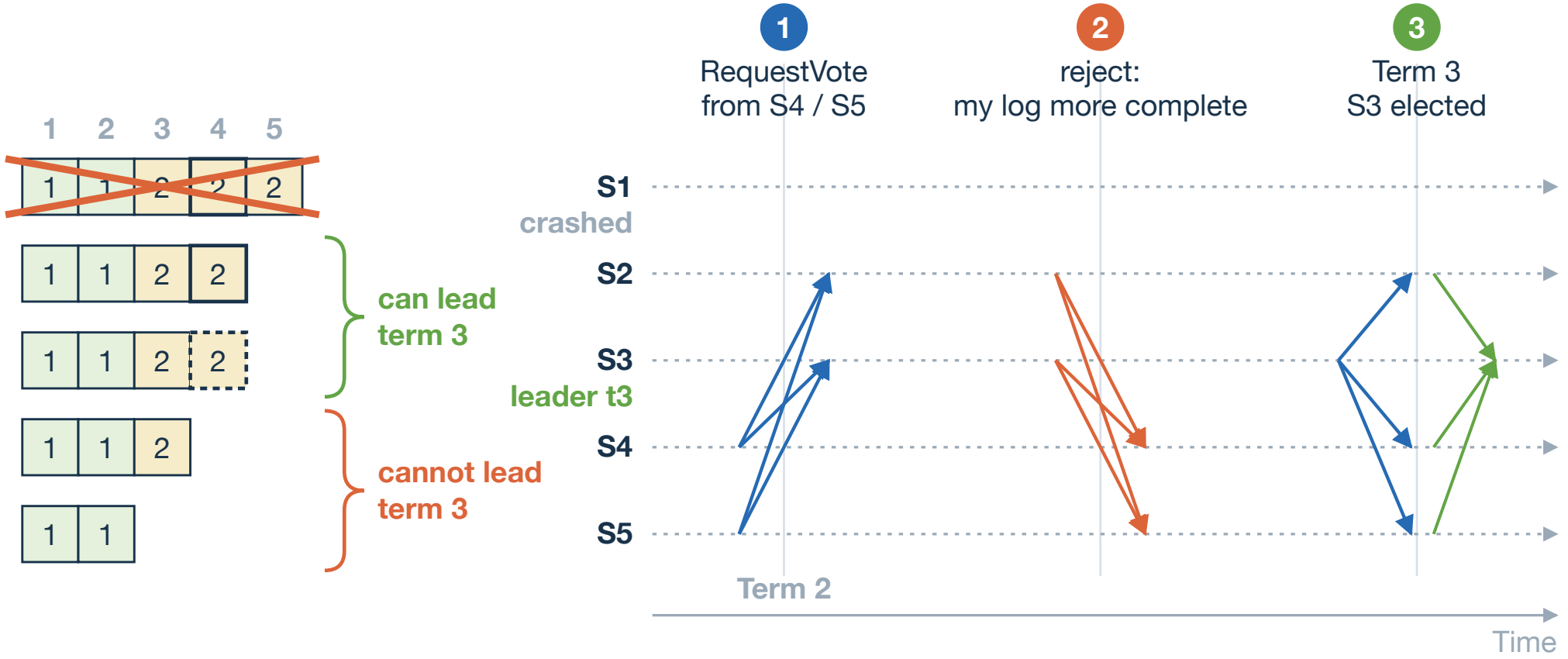
- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

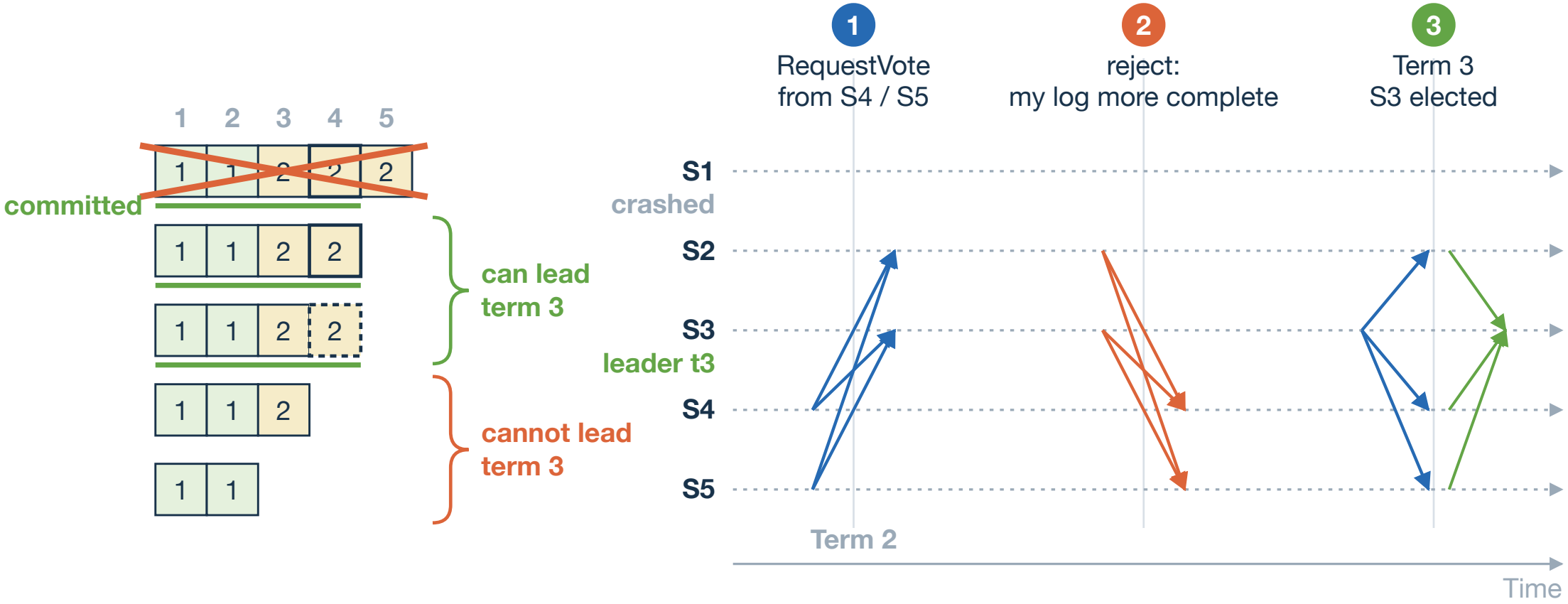
- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



Case #1/2: current-term entries are safe

With the election rule, the leader can decide a current-term entry is committed

- S4 and S5 can't get elected; S2 and S3, who can, both hold index 4



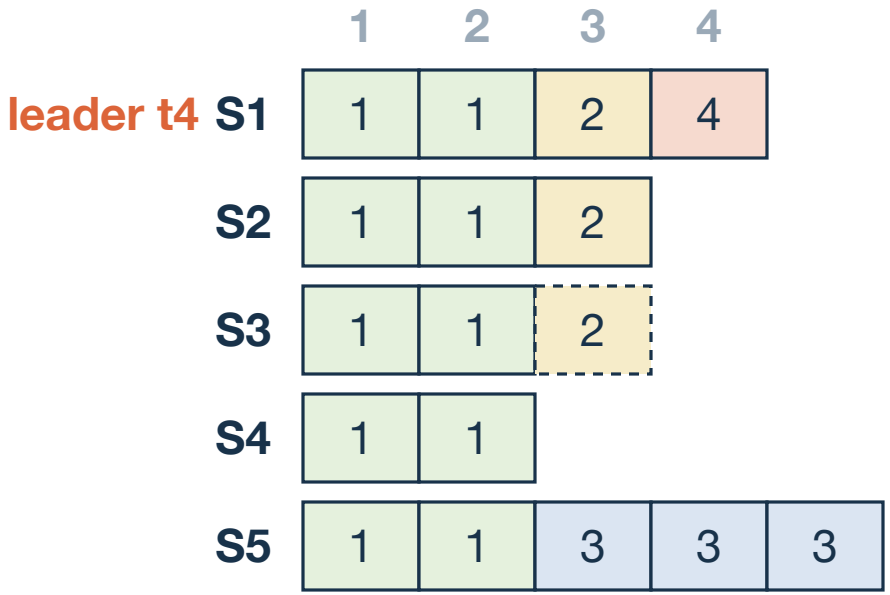
The commit rule so far

A log entry from the leader's term, replicated to a majority of followers

- can be treated as committed, because every later leader must contain it

But what about replicating an entry from a previous term?

- The previous term's entry may have failed to reach a majority



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

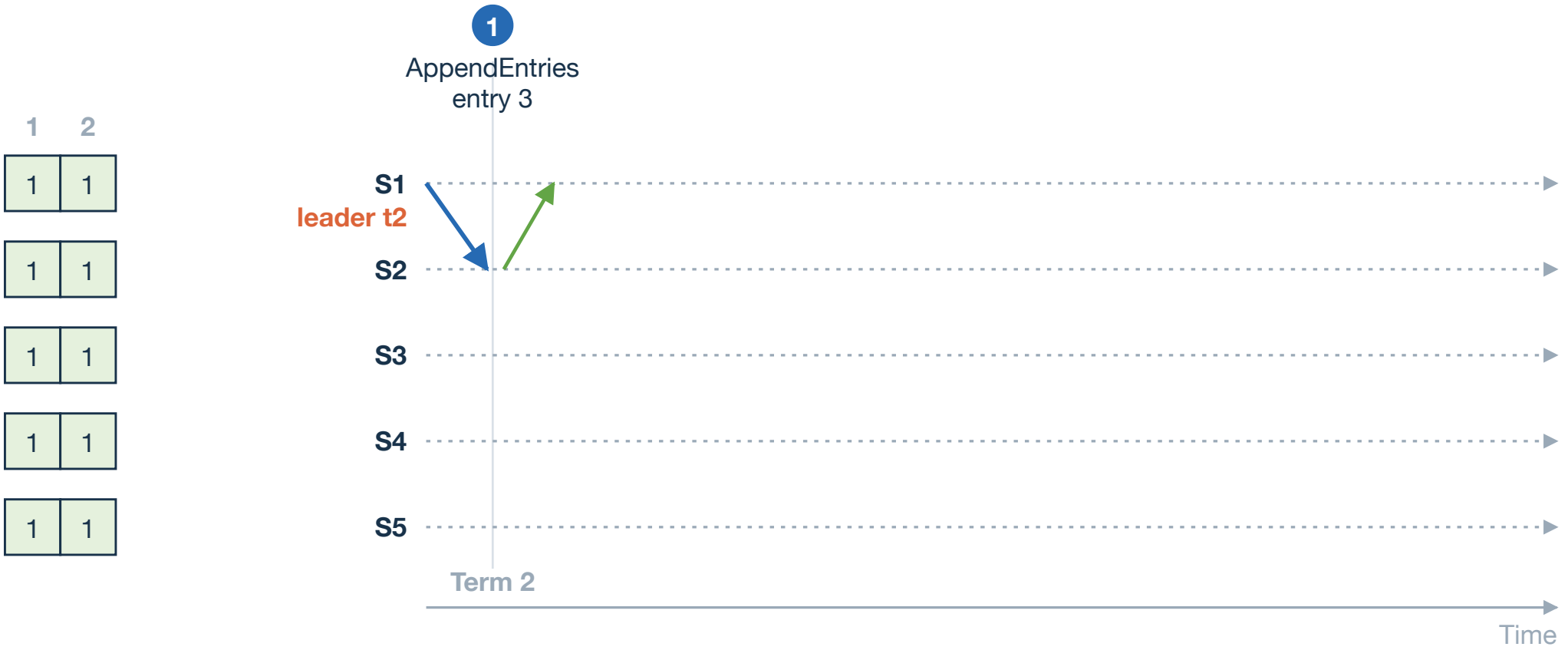
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

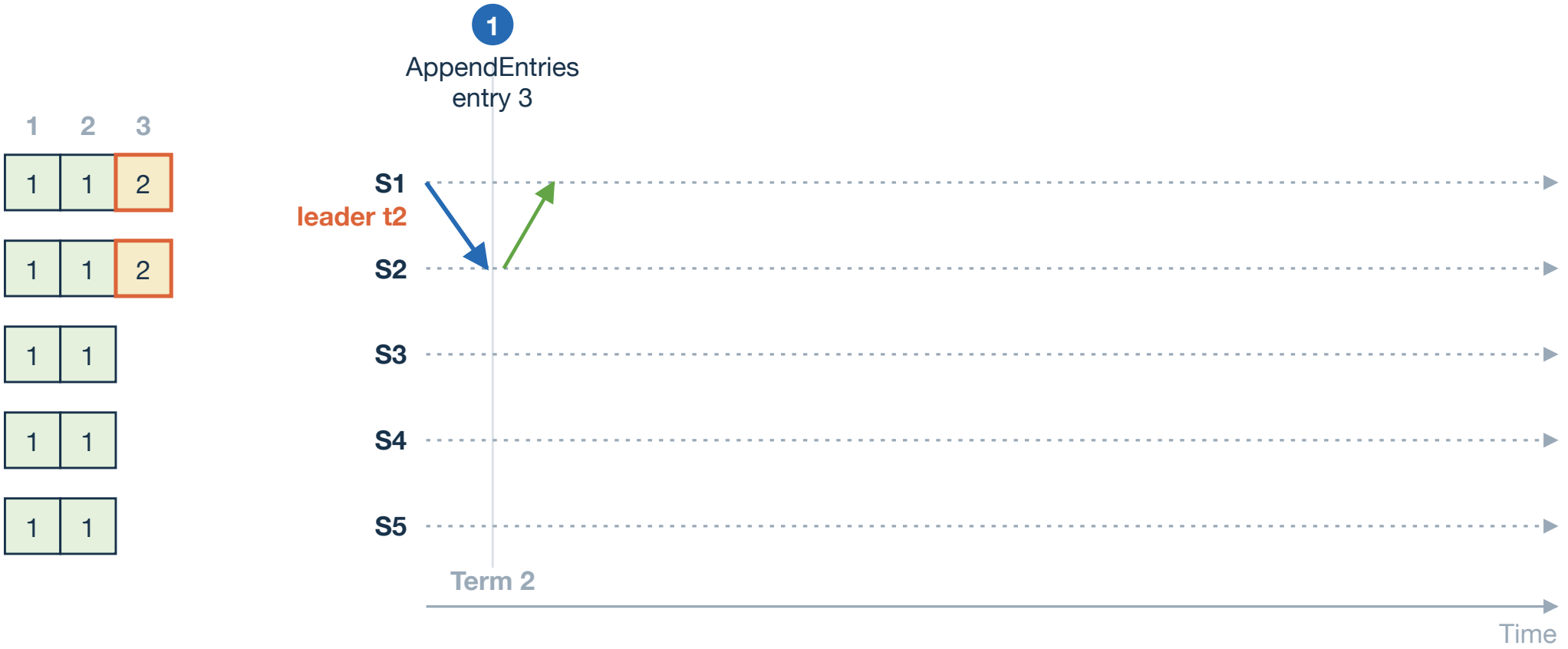
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

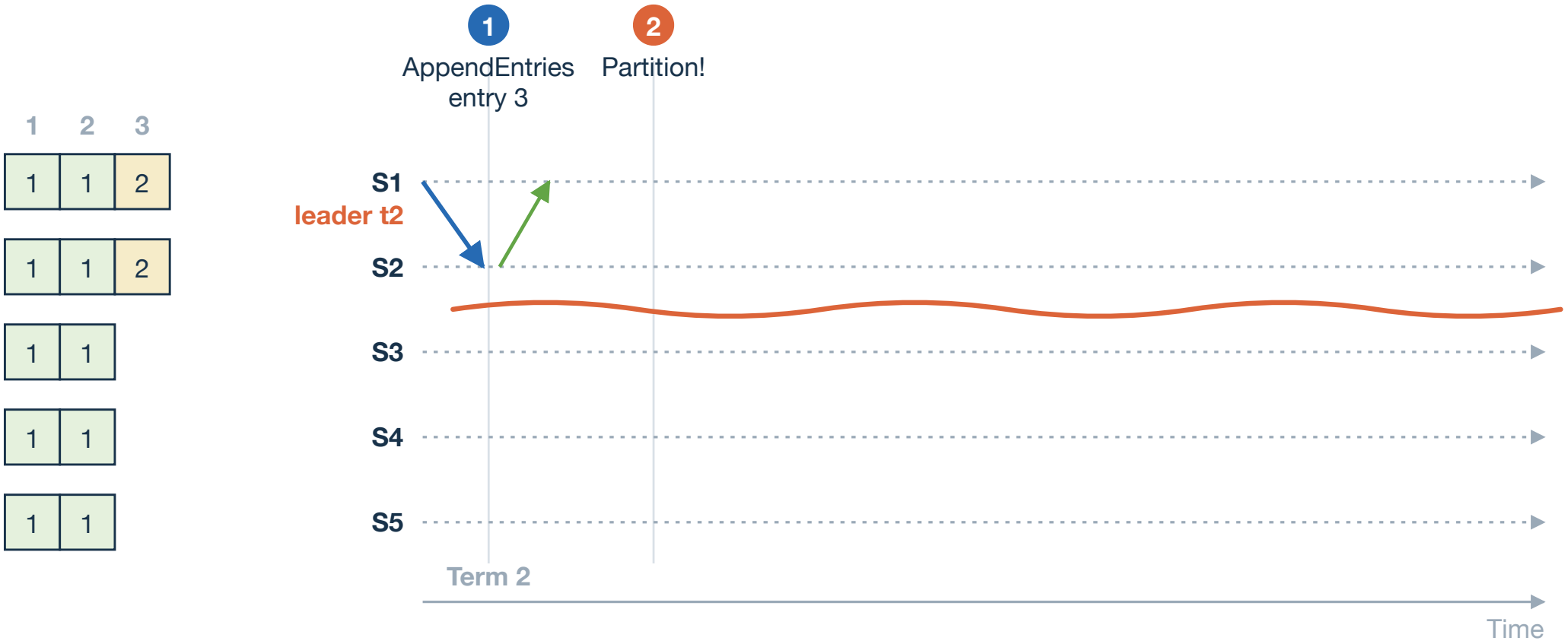
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

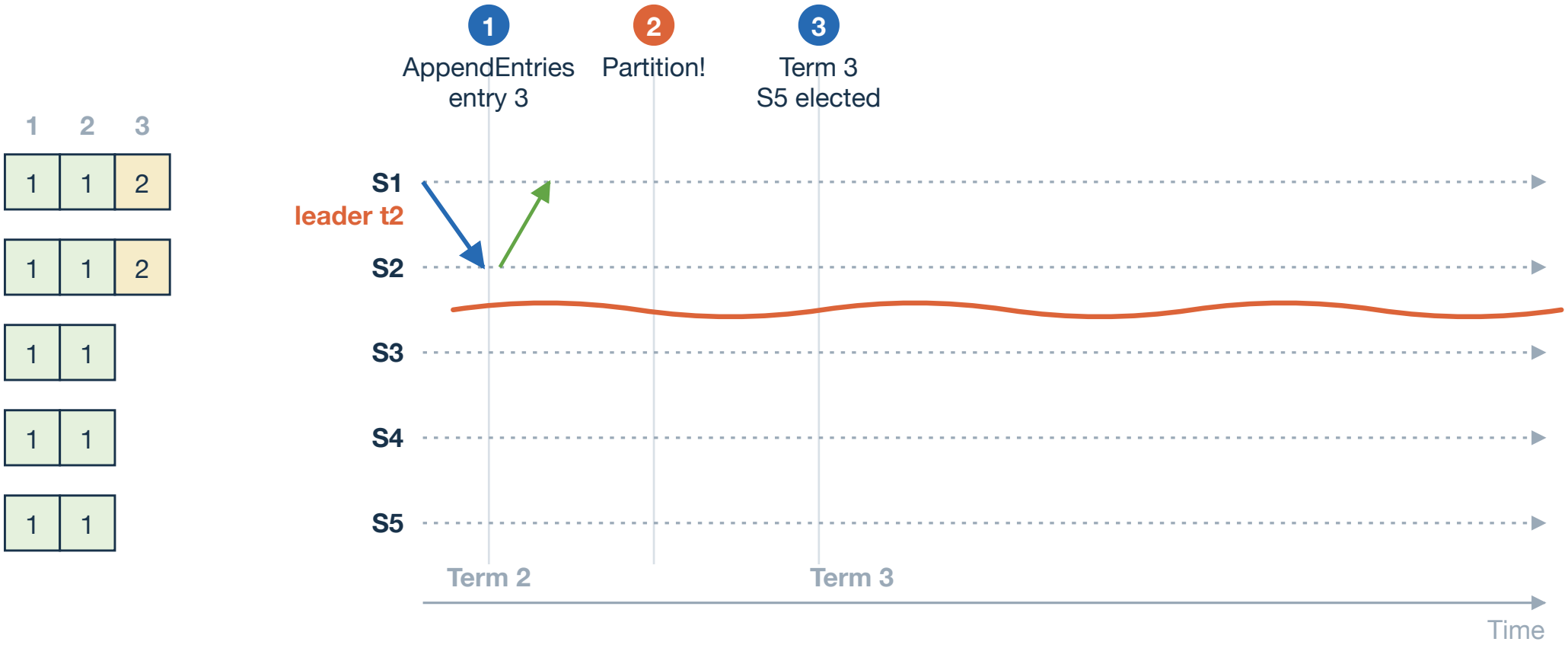
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

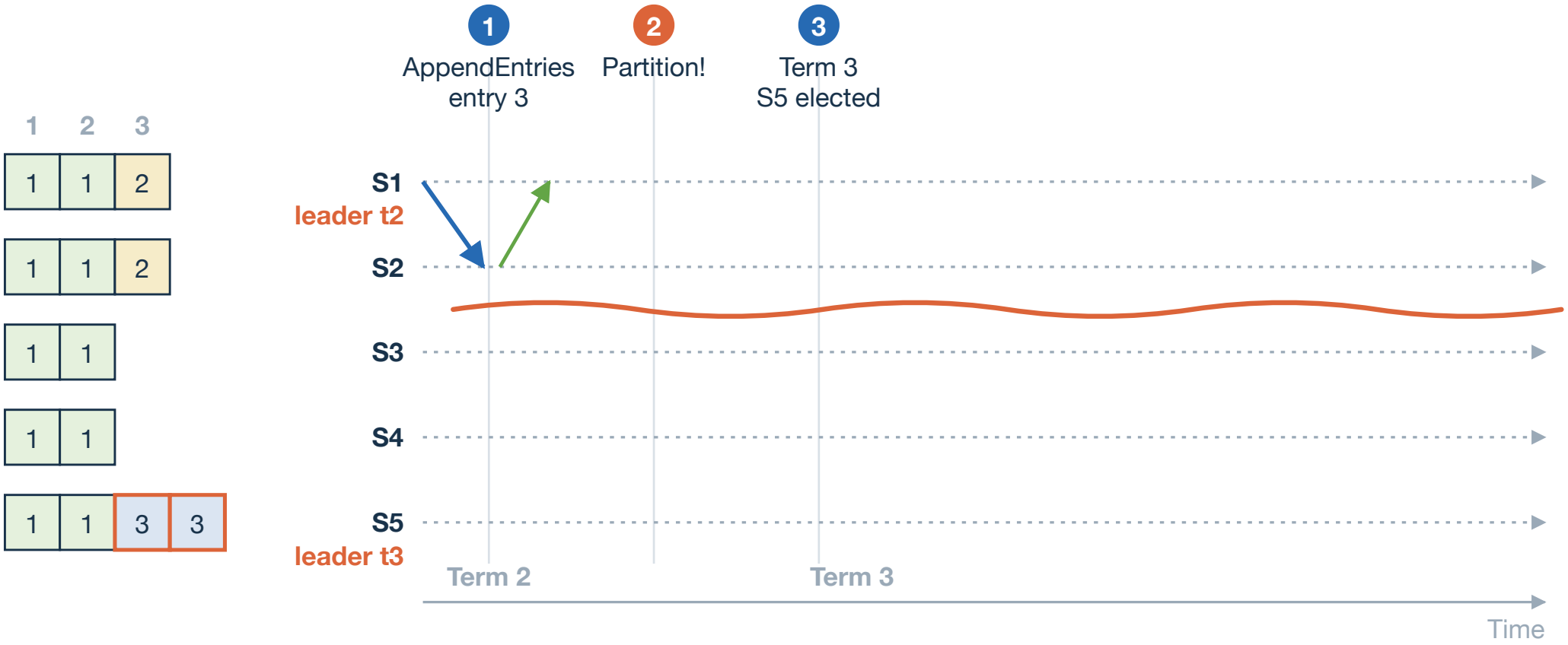
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

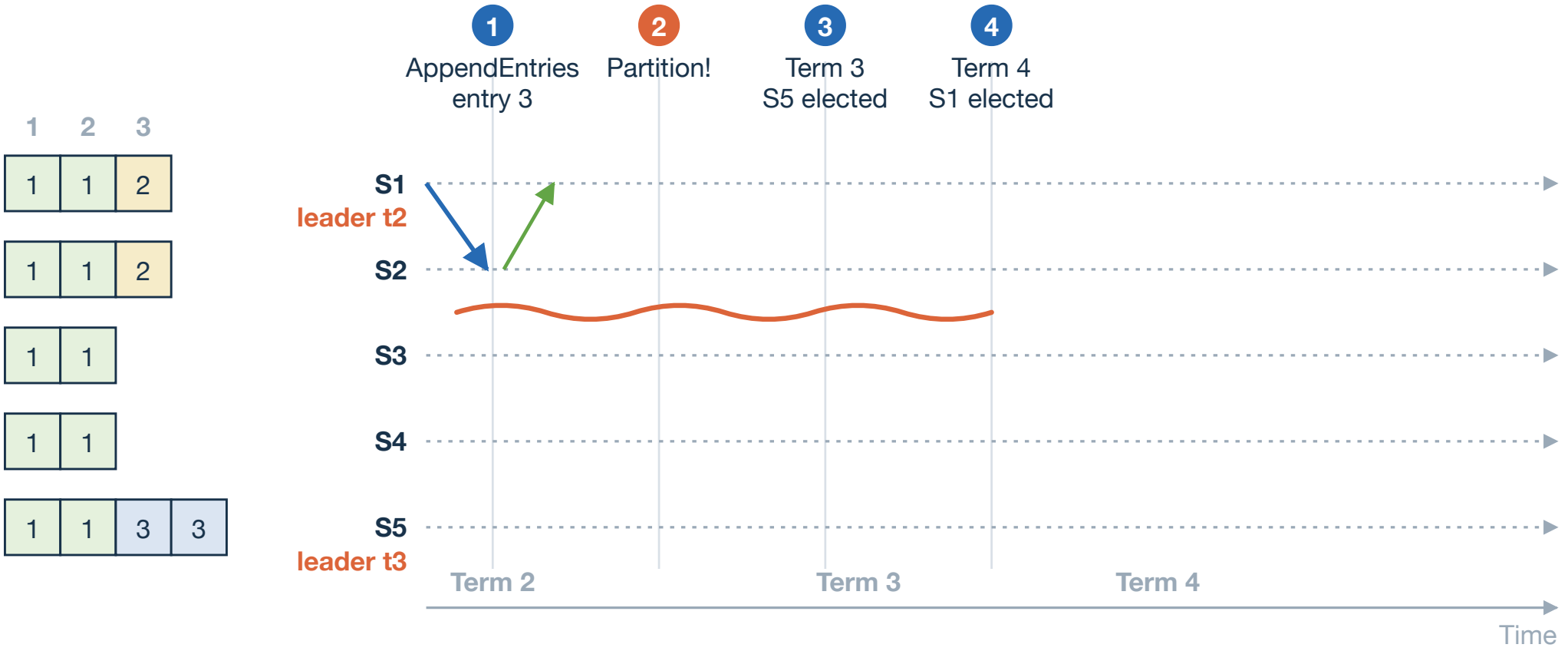
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

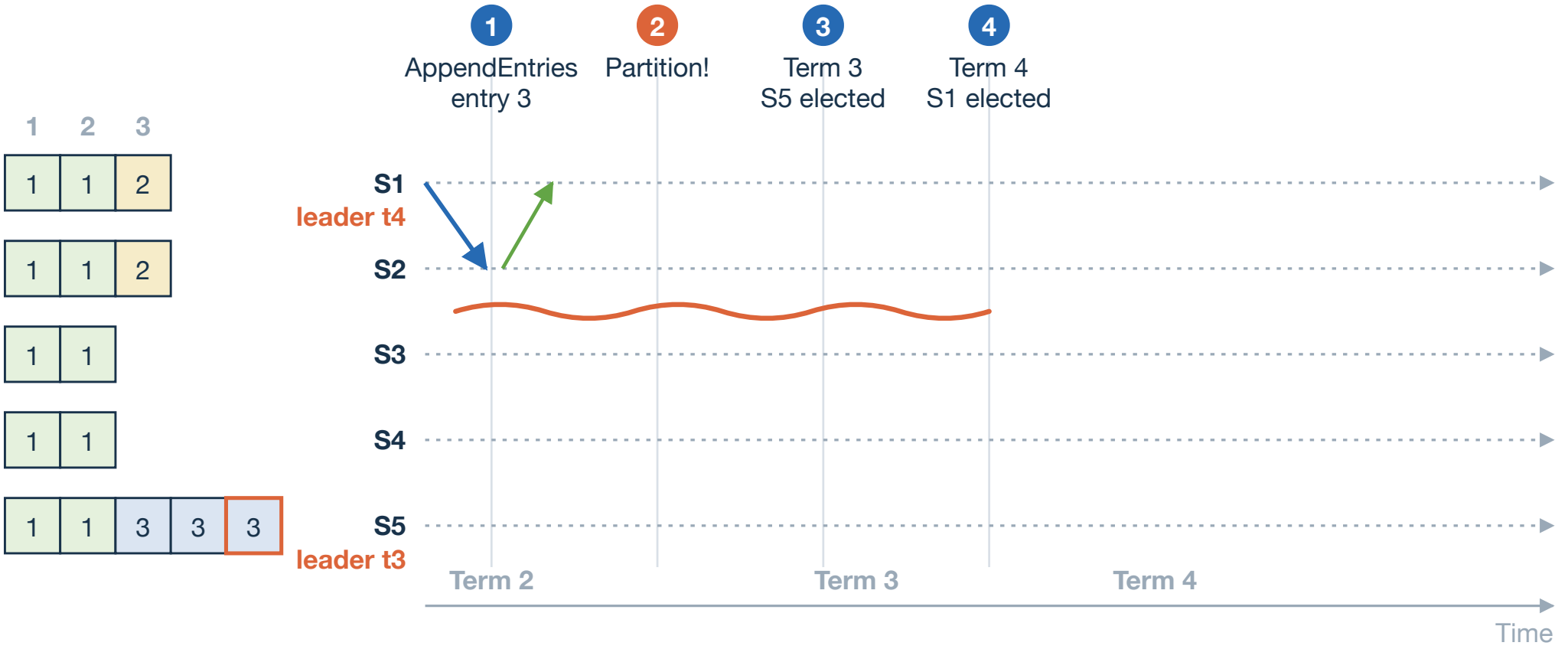
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

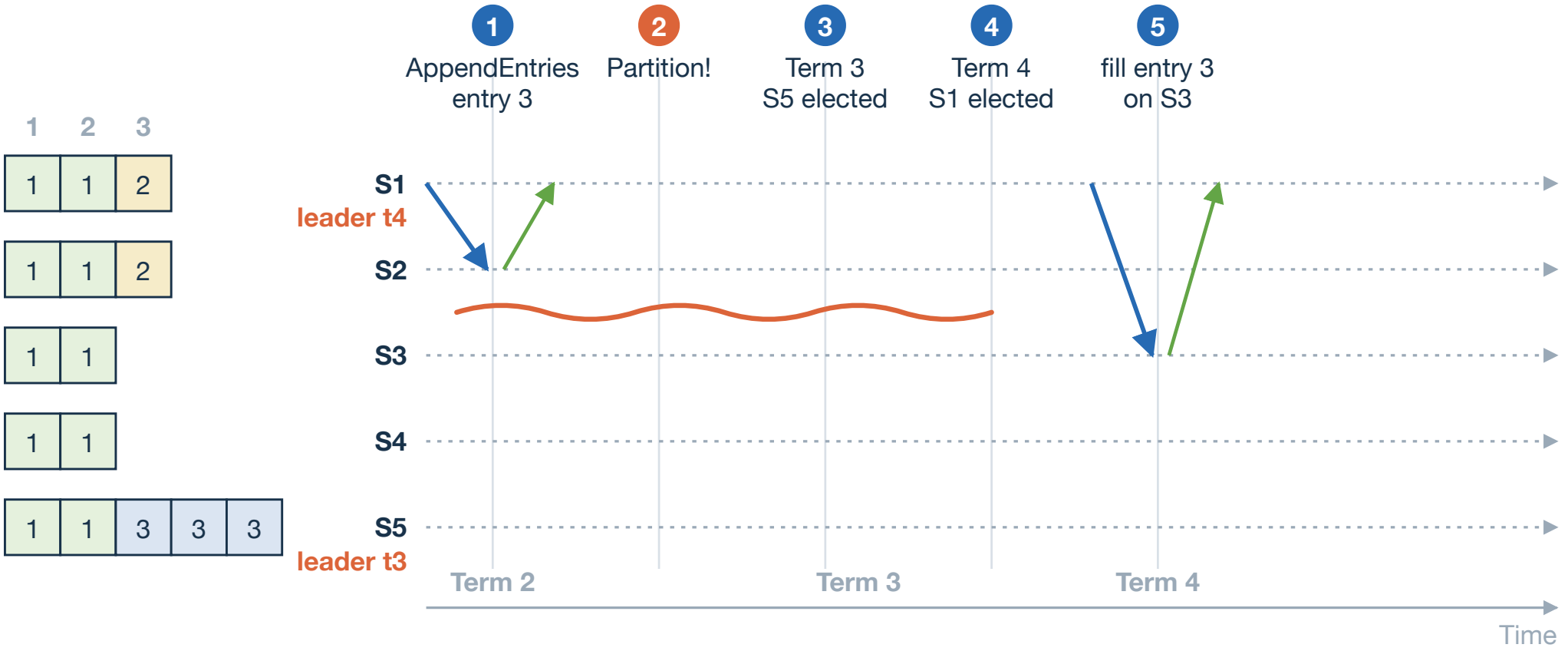
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

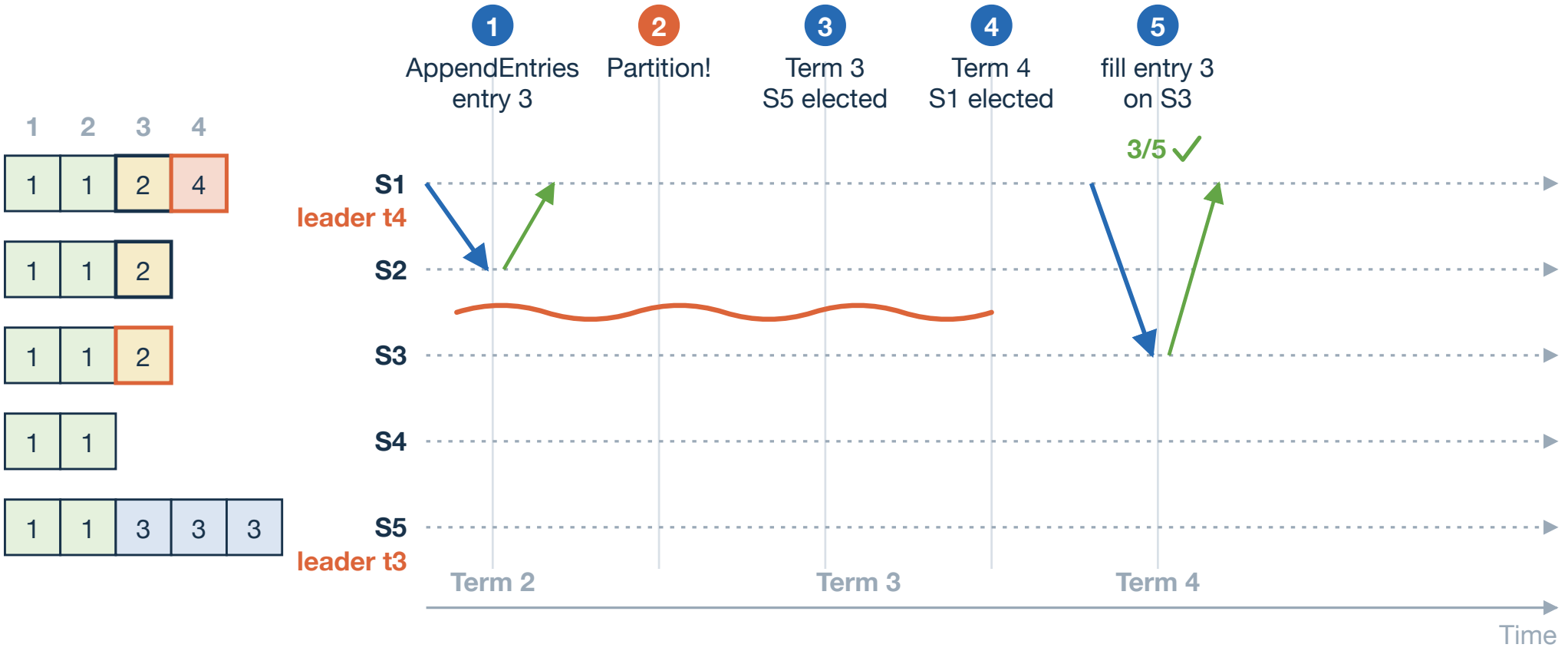
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

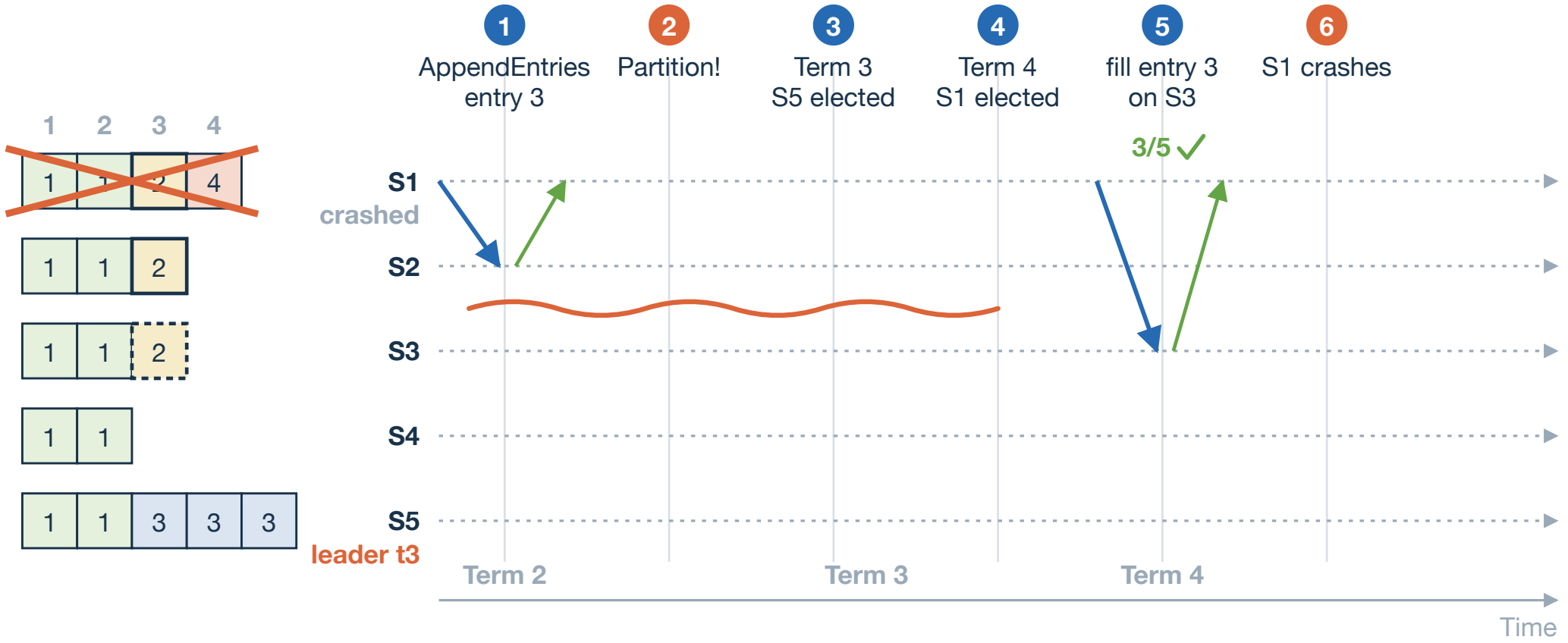
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

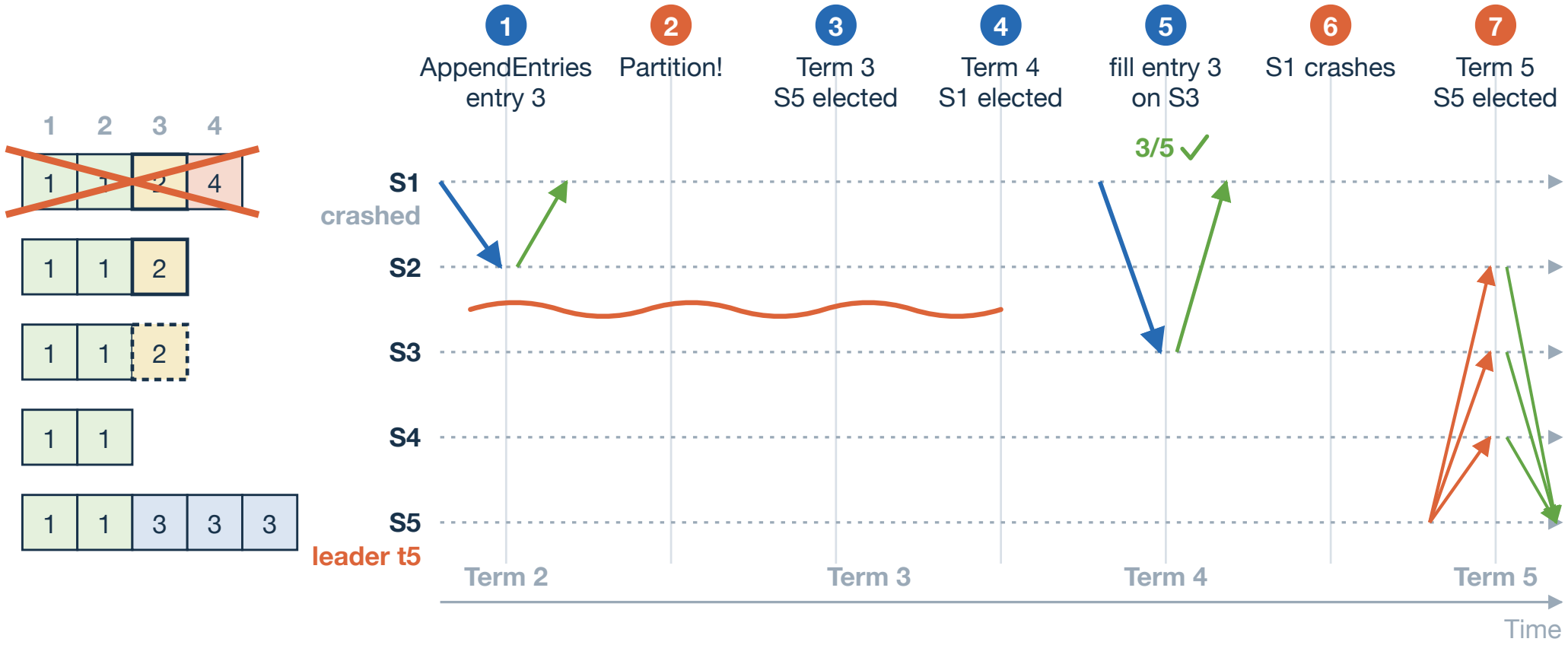
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

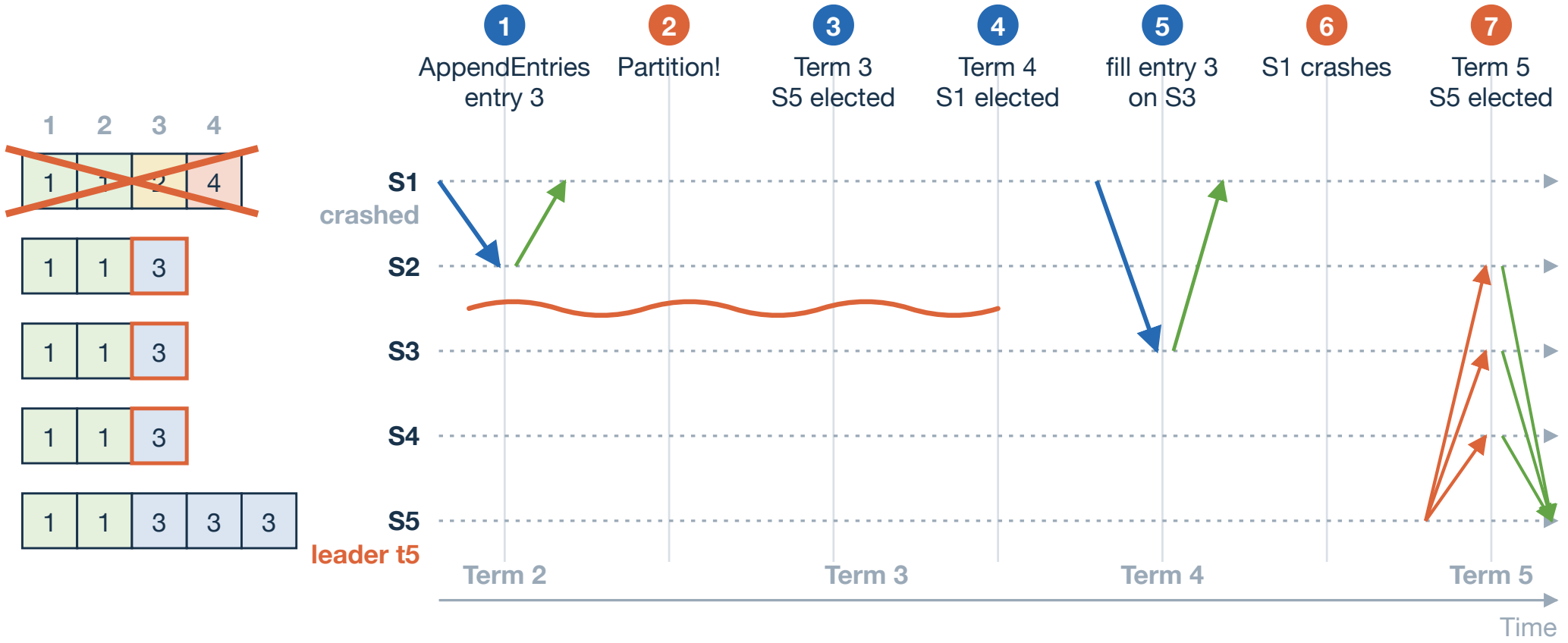
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Case study: an old entry on a majority still gets overwritten

Entry 3 of term 2 reached only S1 and S2; term-4 leader S1 fills it in on a majority

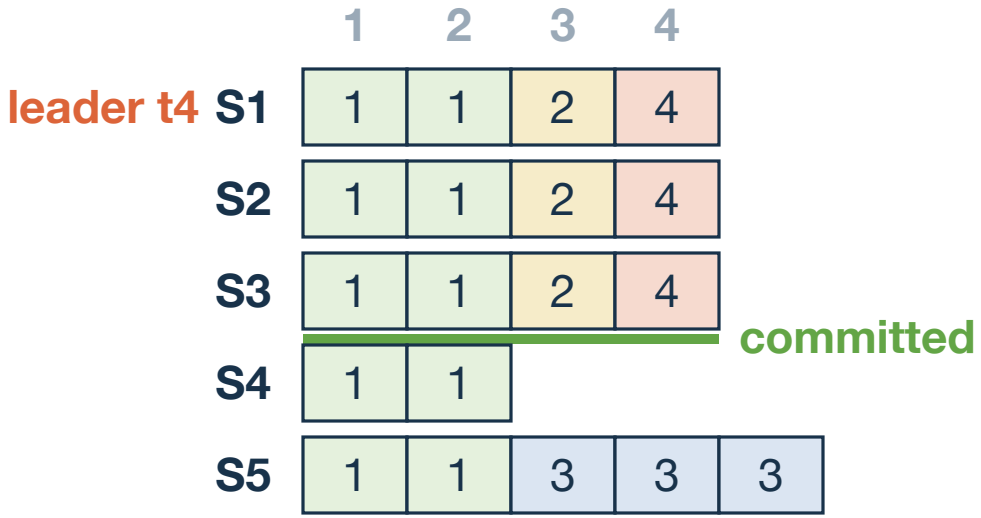
- If S1 crashes again, S5 can still win term 5 and overwrite entry 3



Raft's commit rules

For a leader to decide an entry from an earlier term is committed

- the entry must be stored on a majority of servers
- at least one new entry from the **leader's term** must also be stored on a majority



Once entry 4 is committed, S5 cannot be elected leader for term 5 — entries 3 and 4 are both safe

Election rule + commit rule, together, make Raft safe

Paxos and Raft

Consensus is a key requirement for distributed systems

Today we have covered two approaches:

- Paxos's approach
- Raft's approach

Both are widely used nowadays

参考文献

[1] Paxos Made Simple — Lamport, Leslie, 2001. ACM SIGACT News 32(4) (P3)

[2] The Part-Time Parliament — Lamport, Leslie, 1998. ACM TOCS 16(2) (P4)

[3] In Search of an Understandable Consensus Algorithm (Extended Version) — Ongaro, Diego and Ousterhout, John, 2014. USENIX ATC (P33, P35)

[4] Raft Consensus Algorithm (author's lecture slides) — Ongaro, Diego and Ousterhout, John, 2013 (P35)